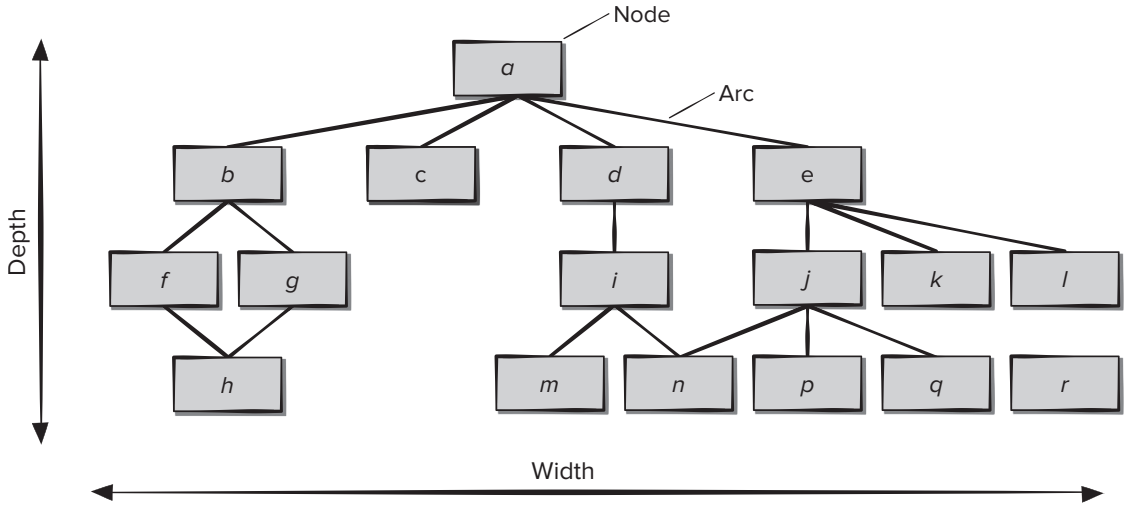
FIGURE 23.1 Morphology metrics



to derive a *design structure quality index* (DSQI) that ranges from 0 to 1 (see: [USA87] and [Cha89] for details).

23.3.3 Design Metrics for Object-Oriented Software

There is much about object-oriented design that is subjective—an experienced designer "knows" how to characterize an OO system so that it will effectively implement customer requirements. But, as an OO design model grows in size and complexity, a more objective view of the characteristics of the design can benefit both the experienced designer (who gains additional insight) and the novice (who obtains an indication of quality that would otherwise be unavailable).

In a detailed treatment of software metrics for OO systems, Whitmire [Whi97] describes nine distinct and measurable characteristics of an OO design. *Size* is defined by taking a static count of OO entities such as classes or operations, coupled with the depth of an inheritance tree. *Complexity* is defined in terms of structural characteristics by examining how classes of an OO design are interrelated to one another. *Coupling* is measured by counting physical connections between elements of the OO design (e.g., the number of collaborations between classes or the number of messages passed between objects). *Sufficiency* is "the degree to which an abstraction [class] possesses the features required of it . . ." [Whi97]. *Completeness* determines whether a class delivers the set of properties that fully reflect the needs of the problem domain. *Cohesion* is determined be examining whether all operations work together to achieve a single, well-defined purpose. *Primitiveness* is the degree to which an operation is atomic—that is, the operation cannot be constructed out of a sequence of other operations contained within a class. Similarity determines the degree to which two or more classes are similar in terms of their structure, function, behavior, or purpose. Volatility measures the likelihood that a change will occur.

In reality, product metrics for OO systems can be applied not only to the design model, but also to the requirements model. In the remainder of this section, we discuss metrics that provide an indication of quality at the OO class level and the operation level. In addition, metrics applicable for project management and testing are also explored.

Chidamber and Kemerer (CK) have proposed one of the most widely referenced sets of OO software metrics [Chi94].[4] Often referred to as the *CK metrics suite,* the authors have proposed six class-based design metrics for OO systems.[5]

Weighted Methods per Class (WMC). Assume that n methods of complexity c_1, c_2, . . . , c_n are defined for a class **C.** The specific complexity metric that is chosen (e.g., cyclomatic complexity) should be normalized so that nominal complexity for a method takes on a value of 1.0.

$$\text{WMC} = \Sigma c_i$$

for $i = 1$ to n. The number of methods and their complexity are reasonable indicators of the amount of effort required to implement and test a class. In addition, the larger the number of methods, the more complex is the inheritance tree (all subclasses inherit the methods of their parents). Finally, as the number of methods grows for a given class, it is likely to become more and more application specific, thereby limiting potential reuse. For all of these reasons, WMC should be kept as low as is reasonable.

Depth of the Inheritance Tree (DIT). This metric is "the maximum length from the node to the root of the tree" [Chi94]. Referring to Figure 23.2, the value of DIT for the class hierarchy shown is 4. As DIT grows, it is likely that lower-level classes will inherit many methods. This leads to potential difficulties when attempting to predict the behavior of a class. A deep class hierarchy (DIT is large) also leads to greater design complexity. On the positive side, large DIT values imply that many methods may be reused.

Number of Children (NOC). The subclasses that are immediately subordinate to a class in the class hierarchy are termed its children. Referring to Figure 23.2, class $\mathbf{C}_2$ has three children—subclasses $\mathbf{C}_{21}$, $\mathbf{C}_{22}$, and $\mathbf{C}_{23}$. As the number of children grows, reuse increases, but also, as NOC increases, the abstraction represented by the parent class can be diluted if some of the children are not appropriate members of the parent class. As NOC increases, the amount of testing (required to exercise each child in its operational context) will also increase.

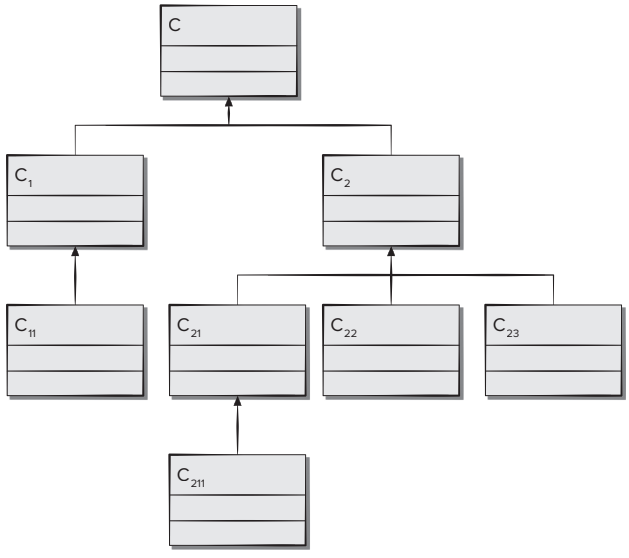
Coupling Between Object Classes (CBO). The CRC model (Chapter 8) may be used to determine the value for CBO. In essence, CBO is the number of collaborations listed for a class on its CRC index card.[6] As CBO increases, it is likely that the

4 An alternative suite of OO metrics has been proposed by Harrison, Counsell, and Nithi [Har98b]. Interested readers are urged to examine their work.

5 Chidamber and Kemerer use the term *methods* rather than *operations*. Their usage of the term is reflected in this section.

6 If CRC index cards are developed manually, completeness and consistency must be assessed before CBO can be determined reliably.

FIGURE 23.2
A class hierarchy



reusability of a class will decrease. High values of CBO also complicate modifications and the testing that ensues when modifications are made. In general, the CBO values for each class should be kept as low as is reasonable. This is consistent with the general guideline to reduce coupling in conventional software.

Response for a Class (RFC). The response set of a class is "a set of methods that can potentially be executed in response to a message received by an object of that class" [Chi94]. RFC is the number of methods in the response set. As RFC increases, the effort required for testing also increases because the test sequence (Chapter 20) grows. It also follows that, as RFC increases, the overall design complexity of the class increases.

Lack of Cohesion in Methods (LCOM). Each method within a class **C** accesses one or more attributes (also called instance variables). LCOM is the number of methods that access one or more of the same attributes.[7] If no methods access the same attributes, then LCOM = 0. To illustrate the case where LCOM ≠ 0, consider a class with six methods. Four of the methods have one or more attributes in common (i.e., they access common attributes). Therefore, LCOM = 4. If LCOM is high, methods may be coupled to one another via attributes. This increases the complexity of the class design. Although there are cases in which a high value for LCOM is justifiable, it is desirable to keep cohesion high, that is, keep LCOM low.[8]

7 The formal definition is a bit more complex. See [Chi94] for details.

8 The LCOM metric provides useful insight in some situations, but it can be misleading in others. For example, keeping coupling encapsulated within a class increases the cohesion of the system as a whole. Therefore, in at least one important sense, higher LCOM actually suggests that a class may have higher cohesion, not lower.

SAFEHOME



Applying CK Metrics

The scene: Vinod's cubicle.

The players: Vinod, Jamie, Shakira, and Ed, members of the *SafeHome* software engineering team who are continuing to work on component-level design and test-case design.

The conversation:

Vinod: Did you guys get a chance to read the description of the CK metrics suite I sent you on Wednesday and make those measurements?

Shakira: Wasn't too complicated. I went back to my UML class and sequence diagrams, like you suggested, and got rough counts for DIT, RFC, and LCOM. I couldn't find the CRC model, so I didn't count CBO.

Jamie (smiling): You couldn't find the CRC model because I had it.

Shakira: That's what I love about this team, superb communication.

Vinod: I did my counts . . . did you guys develop numbers for the CK metrics?

(Jamie and Ed nod in the affirmative.)

Jamie: Since I had the CRC cards, I took a look at CBO, and it looked pretty uniform across most of the classes. There was one exception, which I noted.

Ed: There are a few classes where RFC is pretty high, compared with the averages . . . maybe we should take a look at simplifying them.

Jamie: Maybe yes, maybe no. I'm still concerned about time, and I don't want to fix stuff that isn't really broken.

Vinod: I agree with that. Maybe we should look for classes that have bad numbers in at least two or more of the CK metrics. Kind of two strikes and you're modified.

Shakira (looking over Ed's list of classes with high RFC): Look, see this class. It's got a high LCOM as well as a high RFC. Two strikes?

Vinod: Yeah I think so . . . it'll be difficult to implement because of complexity and difficult to test for the same reason. Probably worth designing two separate classes to achieve the same behavior.

Jamie: You think modifying it'll save us time?

Vinod: Over the long haul, yes.

23.3.4 User Interface Design Metrics

Although there is significant literature on the design of human-computer interfaces (Chapter 12), relatively little information has been published on metrics that would provide insight into the quality and usability of the interface. Although UI metrics may be useful in some cases, the final arbiter should be user input based on GUI prototypes. Nielsen and Levy [Nie94] report that "one has a reasonably large chance of success if one chooses between interface [designs] based solely on users' opinions. Users' average task performance and their subjective satisfaction with a GUI are highly correlated."

In the paragraphs that follow, we present a representative sampling of design metrics that may have application for websites, browser-based applications, and mobile applications. Many of these metrics are applicable to all user interfaces. It is important to note, however, that many of these metrics have not as yet been validated and should be used judiciously.

Interface Metrics. For WebApps, the following interface measures can be considered:

Suggested Metric	*Description*
Layout appropriateness	The relative position of entities within the interface
Layout complexity	Number of distinct regions[9] defined for an interface
Layout region complexity	Average number of distinct links per region
Recognition complexity	Average number of distinct items the user must look at before making a navigation or data input decision
Recognition time	Average time (in seconds) that it takes a user to select the appropriate action for a given task
Typing effort	Average number of keystrokes required for a specific function
Mouse pick effort	Average number of mouse picks per function
Selection complexity	Average number of links that can be selected per page
Content acquisition time	Average number of words of text per Web page
Memory load	Average number of distinct data items that the user must remember to achieve a specific objective

Aesthetic (Graphic Design) Metrics. By its nature, aesthetic design relies on qualitative judgment and is not generally amenable to measurement and metrics. However, Ivory and her colleagues [Ivo01] propose a set of measures that may be useful in assessing the impact of aesthetic design:

Suggested Metric	*Description*
Word count	Total number of words that appear on a page
Body text percentage	Percentage of words that are body versus display text (e.g., headers)
Emphasized body text percentage	Portion of body text that is emphasized (e.g., bold, capitalized)
Text positioning count	Changes in text position from flush left
Text cluster count	Text areas highlighted with color, bordered regions, rules, or lists
Link count	Total links on a page
Page size	Total bytes for the page as well as elements, graphics, and style sheets
Graphic percentage	Percentage of page bytes that are for graphics
Graphics count	Total graphics on a page (not including graphics specified in scripts, applets, and objects)
Color count	Total colors employed
Font count	Total fonts employed (i.e., face + size + bold + italic)

9 A distinct region is an area within the layout display that accomplishes some specific set of related functions (e.g., a menu bar, a static graphical display, a content area, an animated display).

Content Metrics. Metrics in this category focus on content complexity and on clusters of content objects that are organized into pages [Men01].

Suggested Metric	Description
Page wait	Average time required for a page to download at different connection speeds
Page complexity	Average number of different types of media used on page, not including text
Graphic complexity	Average number of graphics media per page
Audio complexity	Average number of audio media per page
Video complexity	Average number of video media per page
Animation complexity	Average number of animations per page
Scanned image complexity	Average number of scanned images per page

Navigation Metrics. Metrics in this category address the complexity of the navigational flow [Men01]. In general, they are applicable only for static Web applications, which don't include dynamically generated links and pages.

Suggested Metric	Description
Page-linking complexity	Number of links per page
Connectivity	Total number of internal links, not including dynamically generated links
Connectivity density	Connectivity divided by page count

Using a subset of the metrics suggested, it may be possible to derive empirical relations that allow a WebApp development team to assess technical quality and predict effort based on projected estimates of complexity. Further work remains to be accomplished in this area.

23.3.5 Metrics for Source Code

Halstead's theory of "software science" [Hal77] proposed the first analytical "laws" for computer software.[10] Halstead assigned quantitative laws to the development of computer software, using a set of primitive measures that may be derived after code is generated or estimated once design is complete. The measures are:

n_1 = number of distinct operators that appear in a program

n_2 = number of distinct operands that appear in a program

N_1 = total number of operator occurrences

N_2 = total number of operand occurrences

10 It should be noted that Halstead's "laws" have generated substantial controversy, and many believe that the underlying theory has flaws. However, experimental verification for selected programming languages has been performed (e.g., [Fel89]).

Halstead uses these primitive measures to develop expressions for the overall program length, potential minimum volume for an algorithm, the actual volume (number of bits required to specify a program), the program level (a measure of software complexity), the language level (a constant for a given language), and other features such as development effort, development time, and even the projected number of faults in the software.

Halstead shows that length N can be estimated as

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

and program volume may be defined as

$$V = N \log_2 (n_1 + n_2)$$

It should be noted that V will vary with programming language and represents the volume of information (in bits) required to specify a program.

Theoretically, a minimum volume must exist for a particular algorithm. Halstead defines a volume ratio L as the ratio of volume of the most compact form of a program to the volume of the actual program. In actuality, L must always be less than 1. In terms of primitive measures, the volume ratio may be expressed as

$$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$$

Halstead's work is amenable to experimental verification, and a large body of research has been conducted to investigate software science. A discussion of this work is beyond the scope of this book. For further information, see [Zus90], [Fen91], and [Zus97].

23.4 METRICS FOR TESTING

Testing metrics fall into two broad categories: (1) metrics that attempt to predict the likely number of tests required at various testing levels, and (2) metrics that focus on test coverage for a given component. The majority of metrics proposed for testing focus on the process of testing, not the technical characteristics of the tests themselves. In general, testers must rely on analysis, design, and code metrics to guide them in the design and execution of test cases.

Architectural design metrics provide information on the ease or difficulty associated with integration testing and the need for specialized testing software (e.g., stubs and drivers). Cyclomatic complexity (a component-level design metric) lies at the core of basis path testing, a test-case design method presented in Chapter 19. In addition, cyclomatic complexity can be used to target modules as candidates for extensive unit testing. Modules with high cyclomatic complexity are more likely to be error prone than modules whose cyclomatic complexity is lower. For this reason, you should expend above-average effort to uncover errors in such modules before they are integrated in a system.

Testing effort can be estimated using metrics derived from Halstead measures (Section 23.3.5). Using the definitions for program volume V and program level PL, Halstead effort e can be computed as

$$PL = \frac{1}{(n_1/2)(N_2/n_2)}$$

$$e = \frac{V}{PL}$$

The percentage of overall testing effort to be allocated to a module k can be estimated using the following relationship:

$$\text{Percentage of testing effort } (k) = \frac{e(k)}{\Sigma\, e(i)}$$

where $e(k)$ is computed for module k and the summation in the denominator is the sum of Halstead effort across all modules of the system.

OO testing can be quite complex. Metrics can assist you in targeting testing resources at threads, scenarios, and packages of classes that are "suspect" based on measured characteristics. The OO design metrics noted in Section 23.3.3 provide an indication of design quality. They also provide a general indication of the amount of testing effort required to exercise an OO system.

Binder [Bin94b] suggests a broad array of design metrics that have a direct influence on the "testability" of an OO system. The metrics consider aspects of encapsulation and inheritance.

Lack of cohesion in methods (LCOM).[11] The higher the value of LCOM, the more states must be tested to ensure that methods do not generate side effects.

Percent public and protected (PAP). Public attributes are inherited from other classes and therefore are visible to those classes. Protected attributes are accessible to methods in subclasses. This metric indicates the percentage of class attributes that are public or protected. High values for PAP increase the likelihood of side effects among classes because public and protected attributes lead to high potential for coupling.[12] Tests must be designed to ensure that such side effects are uncovered.

Public access to data members (PAD). This metric indicates the number of classes (or methods) that can access another class's attributes, a violation of encapsulation. High values for PAD lead to the potential for side effects among classes. Tests must be designed to ensure that such side effects are uncovered.

Number of root classes (NOR). This metric is a count of the distinct class hierarchies that are described in the design model. Test suites for each root class and the corresponding class hierarchy must be developed. As NOR increases, testing effort also increases.

Fan-in (FIN). When used in the OO context, fan-in in the inheritance hierarchy is an indication of multiple inheritance. FIN > 1 indicates that a class inherits its attributes and operations from more than one root class. FIN > 1 should be avoided when possible.

Number of children (NOC) and depth of the inheritance tree (DIT).[13] As we mentioned in Chapter 18, superclass methods will have to be retested for each subclass.

11 See Section 23.3.3 for a description of LCOM.

12 Some people promote designs with none of the attributes being public or private, that is, PAP = 0. This implies that all attributes must be accessed in other classes via methods.

13 See Section 23.3.3 for a description of NOC and DIT.

23.5 METRICS FOR MAINTENANCE

All the software metrics introduced in this chapter can be used for the development of new software and the maintenance of existing software. However, metrics designed explicitly for maintenance activities have been proposed.

IEEE Std. 982.1-2005 [IEE05] suggests a *software maturity index* (SMI) that provides an indication of the stability of a software product (based on changes that occur for each release of the product). The following information is determined:

M_T = number of modules in the current release

F_c = number of modules in the current release that have been changed

F_a = number of modules in the current release that have been added

F_d = number of modules from the preceding release that were deleted in the current release

The software maturity index is computed in the following manner:

$$\text{SMI} = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

As SMI approaches 1.0, the product begins to stabilize. SMI may also be used as a metric for planning software maintenance activities. The mean time to produce a release of a software product can be correlated with SMI, and empirical models for maintenance effort can be developed.

23.6 PROCESS AND PROJECT METRICS

Process metrics are collected across all projects and over long periods of time. Their intent is to provide a set of process indicators that lead to long-term software process improvement (Chapter 28). *Project metrics* enable a software project manager to (1) assess the status of an ongoing project, (2) track potential risks, (3) uncover problem areas before they go "critical," (4) adjust work flow or tasks, and (5) evaluate the project team's ability to control quality of software work products.

Measures that are collected by a project team and converted into metrics for use during a project can also be transmitted to those with responsibility for software process improvement. For this reason, many of the same metrics are used in both the process and project domains.

Unlike software process metrics that are used for strategic purposes, software project measures are tactical. That is, project metrics and the indicators derived from them are used by a project manager and a software team to adapt project work flow and technical activities.

The only rational way to improve any process is to measure specific attributes of the process, develop a set of meaningful metrics based on these attributes, and then use the metrics to provide indicators that will lead to a strategy for improvement (Chapter 28). But before we discuss software metrics and their impact on software process improvement, it is important to note that process is only one of a number of

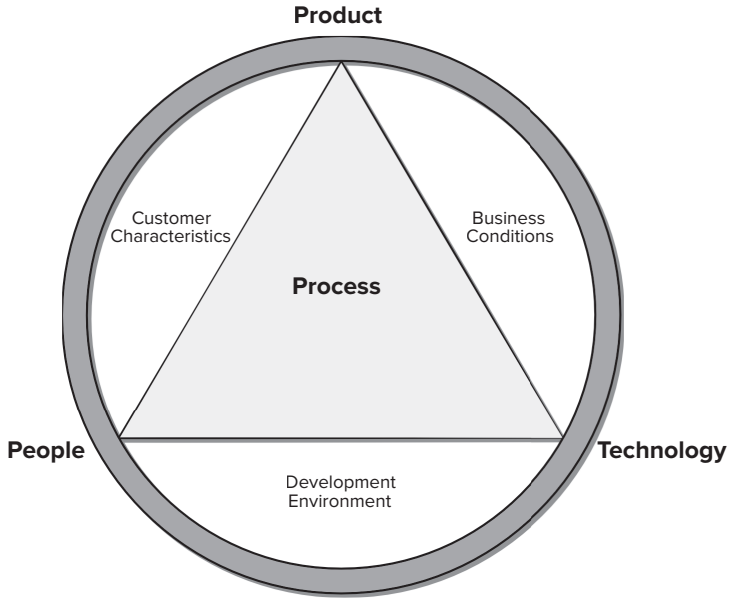


FIGURE 23.3
Determinants for software quality and organizational effectiveness

"controllable factors in improving software quality and organizational performance" [Pau94].

Referring to Figure 23.3, process sits at the center of a triangle connecting three factors that have a profound influence on software quality and organizational performance. The skill and motivation of people have been shown [Boe81] to be the most influential factors in quality and performance. The complexity of the product can have a substantial impact on quality and team performance. The technology (i.e., the software engineering methods and tools) that populates the process also has an impact.

In addition, the process triangle exists within a circle of environmental conditions that include the development environment (e.g., integrated software tools), business conditions (e.g., deadlines, business rules), and customer characteristics (e.g., ease of communication and collaboration).

You can only measure the efficacy of a software process indirectly. That is, you derive a set of metrics based on the outcomes that can be derived from the process. Outcomes include measures of errors uncovered before release of the software, defects delivered to and reported by end users, work products delivered (productivity), human effort expended, calendar time used, schedule conformance, and other measures. You can also derive process metrics by measuring the characteristics of specific software engineering tasks. For example, you might measure the effort and time spent performing the umbrella activities and the generic software engineering activities described in Chapter 1.

The first application of project metrics on most software projects occurs during estimation. Metrics collected from past projects are used as a basis from which effort and time estimates are made for current software work. As a project proceeds,

measures of effort and calendar time expended are compared to original estimates (and the project schedule). The project manager uses these data to monitor and control progress.

As technical work commences, other project metrics begin to have significance. Production rates represented in terms of models created, review hours, function points, and delivered source lines are measured. In addition, errors uncovered during each software engineering task are tracked. As the software evolves from requirements into design, technical metrics are collected to assess design quality and to provide indicators that will influence the approach taken to code generation and testing.

The intent of project metrics is twofold. First, these metrics are used to minimize the development schedule by making the adjustments necessary to avoid delays and mitigate potential problems and risks. Second, project metrics are used to assess product quality on an ongoing basis and, when necessary, modify the technical approach to improve quality.

As quality improves, defects are minimized, and as the defect count goes down, the amount of rework required during the project is also reduced. This leads to a reduction in overall project cost.

Software process metrics can provide significant benefits as an organization works to improve its overall level of process maturity. However, like all metrics, these can be misused, creating more problems than they solve. Grady [Gra92] suggests a "software metrics etiquette" that is appropriate for both managers and practitioners as they institute a process metrics program:

- Use common sense and organizational sensitivity when interpreting metrics data.
- Provide regular feedback to the individuals and teams who collect measures and metrics.
- Don't use metrics to appraise individuals.
- Work with practitioners and teams to set clear goals and metrics that will be used to achieve them.
- Never use metrics to threaten individuals or teams.
- Metrics data that indicate a problem area should not be considered "negative." These data are merely an indicator for process improvement.
- Don't obsess on a single metric to the exclusion of other important metrics.

As an organization becomes more comfortable with the collection and use of process metrics, the derivation of simple indicators gives way to a more rigorous approach called *statistical software process improvement* (SSPI). In essence, SSPI uses software failure analysis to collect information about all errors and defects[14] encountered as an application, system, or product is developed and used.

14 In this book, an *error* is defined as some flaw in a software engineering work product that is uncovered *before* the software is delivered to the end user. A *defect* is a flaw that is uncovered *after* delivery to the end user. It should be noted that others do not make this distinction.

SAFEHOME



Establishing a Metrics Approach

The scene: Doug Miller's office as the *SafeHome* software project is about to begin.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman and Jamie Lazar, members of the product software engineering team.

The conversation:

Doug: Before we start work on this project, I'd like you guys to define and collect a set of simple metrics. To start, you'll have to define your goals.

Vinod (frowning): We've never done that before, and . . .

Jamie (interrupting): And based on the time line management has been talking about, we'll never have the time. What good are metrics anyway?

Doug (raising his hand to stop the onslaught): Slow down and take a breath, guys. The fact that we've never done it before is all the more reason to start now, and the metrics work I'm talking about shouldn't take much time at all . . . in fact, it just might save us time.

Vinod: How?

Doug: Look, we're going to be doing a lot more in-house software engineering as our products get more intelligent, become context aware, mobile, all that . . . and we need to understand the process we use to build software . . . and improve it so we can build software better. The only way to do that is to measure.

Jamie: But we're under time pressure, Doug. I'm not in favor of more paper pushing . . . we need the time to do our work, not collect data.

Doug (calmly): Jamie, an engineer's work involves collecting data, evaluating it, and using the results to improve the product and the process. Am I wrong?

Jamie: No, but . . .

Doug: What if we hold the number of measures we collect to no more than five or six and focus on quality?

Vinod: No one can argue against high quality . . .

Jamie: True . . . but, I don't know. I still think this isn't necessary.

Doug: I'm going to ask you to humor me on this one. How much do you guys know about software metrics?

Jamie (looking at Vinod): Not much.

Doug: Here are some Web refs . . . spend a few hours getting up to speed.

Jamie (smiling): I thought you said this wouldn't take any time.

Doug: Time you spend learning is never wasted . . . go do it, and then we'll establish some goals, ask a few questions, and define the metrics we need to collect.

23.7 SOFTWARE MEASUREMENT

Measurements in the physical world can be categorized in two ways: direct measures (e.g., the length of a bolt) and indirect measures (e.g., the "quality" of bolts produced, measured by counting rejects). Software metrics can be categorized similarly.

Direct measures of the software process include cost and effort applied. Direct measures of the product include lines of code (LOC) produced, execution speed, memory size, and defects reported over some set period of time. Indirect measures of the product include functionality, quality, complexity, efficiency, reliability, maintainability, and many other "–abilities" that are discussed in Chapter 15.

The cost and effort required to build software, the number of lines of code produced, and other direct measures are relatively easy to collect, as long as specific conventions for measurement are established in advance. However, the quality and functionality of software or its efficiency or maintainability are more difficult to assess and can be measured only indirectly.

We have partitioned the software metrics domain into process, project, and product metrics and noted that product metrics that are private to an individual are often combined to develop project metrics that are public to a software team. Project metrics are then consolidated to create process metrics that are public to the software organization as a whole. But how does an organization combine metrics that come from different individuals or projects?

To illustrate, consider a simple example. Individuals on two different project teams record and categorize all errors that they find during the software process. Individual measures are then combined to develop team measures. Team A found 342 errors during the software process prior to release. Team B found 184 errors. All other things being equal, which team is more effective in uncovering errors throughout the process? Because you do not know the size or complexity of the projects, you cannot answer this question. However, if the measures are normalized, it is possible to create software metrics that enable comparison to broader organizational averages.

Size-oriented software metrics are derived by normalizing quality and/or productivity measures by considering the *size* of the software that has been produced. If a software organization maintains simple records, a table of size-oriented measures, such as the one shown in Figure 23.4, can be created. The table lists each software development project that has been completed over the past few years and corresponding measures for that project. Referring to the table entry (Figure 23.4) for project *alpha*: 12,100 lines of code were developed with 24 person-months of effort at a cost of $168,000. It should be noted that the effort and cost recorded in the table represent all software engineering activities (analysis, design, code, and test), not just coding. Further information for project *alpha* indicates that 365 pages of documentation were developed, 134 errors were recorded before the software was released, and 29 defects were encountered after release to the customer within the first year of operation. Three people worked on the development of software for project alpha.

To develop metrics that can be assimilated with similar metrics from other projects, you can choose lines of code as a normalization value. From the rudimentary data

FIGURE 23.4 Size-oriented metrics

Project	LOC	Effort	$(000)	Pp. doc.	Errors	Defects	Category
alpha	12,100	24	168	365	134	29	3
beta	27,200	62	440	1224	321	86	5
gamma	20,200	43	314	1050	256	64	6
⋮	⋮	⋮	⋮	⋮	⋮		

contained in the table, a set of simple size-oriented metrics can be developed for each project:

- Errors per KLOC (thousand lines of code)
- Defects per KLOC
- $ per KLOC
- Pages of documentation per KLOC

In addition, other interesting metrics can be computed:

- Errors per person-month
- KLOC per person-month
- $ per page of documentation

Size-oriented metrics are not universally accepted as the best way to measure the software process. Most of the controversy swirls around the use of lines of code as a key measure. Proponents of the LOC measure claim that LOC is an "artifact" of all software development projects that can be easily counted, that many existing software estimation models use LOC or KLOC as a key input, and that a large body of literature and data predicated on LOC already exists. On the other hand, opponents argue that LOC measures are programming language dependent, that when productivity is considered, they penalize well-designed but shorter programs; that they cannot easily accommodate nonprocedural languages; and that their use in estimation requires a level of detail that may be difficult to achieve (i.e., the planner must estimate the LOC to be produced long before analysis and design have been completed).

Similar arguments, pro and con, can be made for function-oriented metrics such as function points (FP) or use case points (both are discussed in Chapter 25). Function-oriented software metrics use a measure of the functionality delivered by the application as a normalization value. Computation of a function-oriented metric is based on characteristics of the software's information domain and complexity.

The function point, like the LOC measure, is controversial. Proponents claim that FP is programming language–independent, making it ideal for applications using conventional and nonprocedural languages, and that it is based on data that are more likely to be known early in the evolution of a project, making FP more attractive as an estimation approach. Opponents claim that the method requires some "sleight of hand" in that computation is based on subjective rather than objective data, that counts of the information domain (and other dimensions) can be difficult to collect after the fact, and that FP has no direct physical meaning—it's just a number.

Function points and LOC-based metrics have been found to be relatively accurate predictors of software development effort and cost. However, to use LOC and FP for estimation (Chapter 25), an historical baseline of information must be established. It is this historical data that over time will let you judge the value of a particular metric on future projects.

Size-oriented measures (e.g., LOC) and function-oriented measures are often used to derive productivity metrics. This invariably leads to a debate about the use of such data. Should the LOC/person-month (or FP/person-month) of one group be compared to similar data from another? Should managers appraise the performance of individuals by using these metrics? The answer to these questions is an emphatic no! The reason

for this response is that many factors influence productivity, making for "apples and oranges" comparisons that can be easily misinterpreted.

Within the context of process and project metrics, you should be concerned primarily with productivity and quality—measures of software development "output" as a function of effort and time applied and measures of the "fitness for use" of the work products that are produced.

For process improvement and project planning purposes, your interest is historical. What was software development productivity on past projects? What was the quality of the software that was produced? How can past productivity and quality data be extrapolated to the present? How can it help us improve the process and plan new projects more accurately?

23.8 Metrics for Software Quality

The quality of a system, application, or product is only as good as the requirements that describe the problem, the design that models the solution, the code that leads to an executable program, and the tests that exercise the software to uncover errors. Software is a complex entity. Therefore, errors are to be expected as work products are developed. Process metrics are intended to improve the software process so that errors are uncovered in the most effective manner.

You can use measurement to assess the quality of the requirements and design models, the source code, and the test cases that have been created as the software is engineered. To accomplish this real-time assessment, you apply product metrics to evaluate the quality of software engineering work products in objective rather than subjective ways.

A project manager must also evaluate quality as the project progresses. Private metrics collected by individual software engineers are combined to provide project-level results. Although many quality measures can be collected, the primary thrust at the project level is to measure errors and defects. Metrics derived from these measures provide an indication of the effectiveness of individual and group software quality assurance and control activities.

Metrics such as work product errors per function point, errors uncovered per review hour, and errors uncovered per testing hour provide insight into the efficacy of each of the activities implied by the metric. Error data can also be used to compute the *defect removal efficiency* (DRE) for each process framework activity. DRE is discussed later in this section.

Although there are many measures of software quality, correctness, maintainability, integrity, and usability provide useful indicators for the project team. Gilb [Gil88] suggests definitions and measures for each.

> **Correctness.** Correctness is the degree to which the software performs its required function. Defects (lack of correctness) are those problems reported by a user of the program after the program has been released for general use. For quality assessment purposes, defects are counted over a standard period of time, typically one year. The most common measure for correctness is defects per KLOC, where a defect is defined as a verified lack of conformance to requirements.

Maintainability. Maintainability is the ease with which a program can be corrected if an error is encountered, adapted if its environment changes, or enhanced if the customer desires a change in requirements. There is no way to measure maintainability directly; therefore, we must use indirect measures. A simple time-oriented metric is *mean time to change* (MTTC), the time it takes to analyze the change request, design an appropriate modification, implement the change, test it, and distribute the change to all users.

Integrity. This attribute measures a system's ability to withstand attacks (both accidental and intentional) to its security. To measure integrity, two additional attributes must be defined: threat and security. *Threat* is the probability (which can be estimated or derived from empirical evidence) that an attack of a specific type will occur within a given time. *Security* is the probability (which can be estimated or derived from empirical evidence) that the attack of a specific type will be repelled. The integrity of a system can then be defined as:

$$\text{Integrity} = \Sigma[1 - (\text{threat} \times (1 - \text{security}))]$$

For example, if threat (the probability that an attack will occur) is 0.25 and security (the likelihood of repelling an attack) is 0.95, the integrity of the system is 0.99 (very high). If, on the other hand, the threat probability is 0.50 and the likelihood of repelling an attack is only 0.25, the integrity of the system is 0.63 (unacceptably low).

Usability. Usability is an attempt to quantify ease of use and can be measured in terms of the characteristics presented in Chapter 12.

These four factors are only a sampling of those that have been proposed as measures for software quality.

A quality metric that provides benefit at both the project and process level is *defect removal efficiency* (DRE). In essence, DRE is a measure of the filtering ability of quality assurance and control actions as they are applied throughout all process framework activities.

When considered for a project as a whole, DRE is defined in the following manner:

$$\text{DRE} = \frac{E}{E + D}$$

where E is the number of errors found before delivery of the software to the end user and D is the number of defects found after delivery.

The ideal value for DRE is 1. That is, no defects are found in the software. Realistically, D will be greater than 0, but the value of DRE can still approach 1. As E increases (for a given value of D), the overall value of DRE begins to approach 1. In fact, as E increases, it is likely that the final value of D will decrease (errors are filtered out before they become defects). If used as a metric that provides an indicator of the filtering ability of quality control and assurance activities, DRE encourages a software project team to institute techniques for finding as many errors as possible before delivery.

DRE can also be used within the project to assess a team's ability to find errors before they are passed to the next framework activity or software engineering task.

For example, requirements analysis produces a requirements model that can be reviewed to find and correct errors. Those errors that are not found during the review of the requirements model are passed on to design (where they may or may not be found). When used in this context, we redefine DRE as

$$\text{DRE}_i = \frac{E_i}{E_i + E_{i+1}}$$

where E_i is the number of errors found during software engineering action i and E_{i+1} is the number of errors found during software engineering action $i + 1$ that are traceable to errors that were not discovered in software engineering action i.

A quality objective for a software team (or an individual software engineer) is to achieve a DRE_i that approaches 1. That is, errors should be filtered out before they are passed on to the next activity or action. If DRE is low as you move through analysis and design, spend some time improving the way you conduct formal technical reviews.

SAFEHOME



A Metrics-Based Quality Approach

The scene: Doug Miller's office 2 days after initial meeting on software metrics.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman and Jamie Lazar, members of the product software engineering team.

The conversation:

Doug: Did you both have a chance to learn a little about process and project metrics?

Vinod and Jamie: (Both nod.)

Doug: It's always a good idea to establish goals when you adopt any metrics. What are yours?

Vinod: Our metrics should focus on quality. In fact, our overall goal is to keep the number of errors we pass on from one software engineering activity to the next to an absolute minimum.

Doug: And be very sure you keep the number of defects released with the product to as close to zero as possible.

Vinod (nodding): Of course.

Jamie: I like DRE as a metric, and I think we can use it for the entire project, but also as we move from one framework activity to the next. It'll encourage us to find errors at each step.

Vinod: I'd also like to collect the number of hours we spend on reviews.

Jamie: And the overall effort we spend on each software engineering task.

Doug: You can compute a review-to-development ratio . . . might be interesting.

Jamie: I'd like to track some use case data as well. Like the amount of effort required to develop a use case, the amount of effort required to build software to implement a use case, and . . .

Doug (smiling): I thought we were going to keep this simple.

Vinod: We should, but once you get into this metrics stuff, there's a lot of interesting things to look at.

Doug: I agree, but let's walk before we run and stick to our goal. Limit data to be collected to five or six items, and we're ready to go.

23.9 ESTABLISHING SOFTWARE METRICS PROGRAMS

The Software Engineering Institute has developed a comprehensive guidebook [Par96b] for establishing a "goal-driven" software metrics program. The guidebook suggests the following steps: (1) identify your business goals, (2) identify what you want to know or learn, (3) identify your subgoals, (4) identify the entities and attributes related to your subgoals, (5) formalize your measurement goals, (6) identify quantifiable questions and the related indicators that you will use to help you achieve your measurement goals, (7) identify the data elements that you will collect to construct the indicators, (8) identify the measures to be used, and make these definitions operational, (9) identify the actions that you will take to implement the measures, and (10) prepare a plan for implementing the measures. A detailed discussion of these steps is best left to the SEI's guidebook. However, a brief overview of key points is illustrated by the following example.

Because software supports business functions, differentiates computer-based systems or products, or acts as a product in itself, goals defined for the business can almost always be traced downward to specific goals at the software engineering level. For example, consider the *SafeHome* product. Working as a team, software engineering and business managers develop a list of prioritized business goals:

1. Improve our customers' satisfaction with our products.
2. Make our products easier to use.
3. Reduce the time it takes us to get a new product to market.
4. Make support for our products easier.
5. Improve our overall profitability.

The software organization examines each business goal and asks: "What activities do we manage, execute, or support and what do we want to improve within these activities?" To answer these questions, the SEI recommends the creation of an "entity-question list" in which all things (entities) within the software process that are managed or influenced by the software organization are noted. Examples of entities include development resources, work products, source code, test cases, change requests, software engineering tasks, and schedules. For each entity listed, software people develop a set of questions that assess quantitative characteristics of the entity (e.g., size, cost, time to develop). The questions derived as a consequence of the creation of an entity-question list lead to the derivation of a set of subgoals that relate directly to the entities created and the activities performed as part of the software process.

Consider the fourth goal: "Make support for our products easier." The following list of questions might be derived for this goal [Par96b]:

- Do customer change requests contain the information we require to adequately evaluate the change and then implement it in a timely manner?
- How large is the change request backlog?
- Is our response time for fixing bugs acceptable based on customer need?
- Is our change control process (Chapter 22) followed?
- Are high-priority changes implemented in a timely manner?

Based on these questions, the software organization can derive the following subgoal: *Improve the performance of the change management process.* The software process entities and attributes that are relevant to the subgoal are identified, and the measurement goals associated with them are delineated.

The SEI [Par96b] provides detailed guidance for steps 6 through 10 of its goal-driven measurement approach. In essence, you refine measurement goals into questions that are further refined into entities and attributes that are then refined into metrics.

The vast majority of software development organizations have fewer than 20 software people. It is unreasonable, and in most cases unrealistic, to expect that such organizations will develop comprehensive software metrics programs. However, it is reasonable to suggest that software organizations[15] of all sizes measure and then use the resultant metrics to help improve their local software process and the quality and timeliness of the products they produce.

A small organization can begin by focusing not on measurement but rather on results. The software group is polled to define a single objective that requires improvement. For example, "reduce the time to evaluate and implement change requests." A small organization might select the following set of easily collected measures:

- Time (hours or days) elapsed from the time a request is made until evaluation is complete, t_{queue}.
- Effort (person-hours) to perform the evaluation, W_{eval}.
- Time (hours or days) elapsed from completion of evaluation to assignment of change order to personnel, t_{eval}.
- Effort (person-hours) required to make the change, W_{change}.
- Time required (hours or days) to make the change, t_{change}.
- Errors uncovered during work to make change, E_{change}.
- Defects uncovered after change is released to the customer base, D_{change}.

Once these measures have been collected for a number of change requests, it is possible to compute the total elapsed time from change request to implementation of the change and the percentage of elapsed time absorbed by initial queuing, evaluation and change assignment, and change implementation. Similarly, the percentage of effort required for evaluation and implementation can be determined. These metrics can be assessed in the context of quality data, E_{change} and D_{change}. The percentages provide insight into where the change request process slows down and may lead to process improvement steps to reduce t_{queue}, W_{eval}, t_{eval}, W_{change}, and/or E_{change}. In addition, the defect removal efficiency can be computed as

$$\text{DRE} = \frac{E_{change}}{E_{change} + D_{change}}$$

DRE can be compared to elapsed time and total effort to determine the impact of quality assurance activities on the time and effort required to make a change.

The majority of software developers still do not measure, and sadly, most have little desire to begin. As we noted previously in this chapter, the problem is cultural.

15 This discussion is equally relevant to software teams that have adopted an agile software development process (Chapter 3).

Attempting to collect measures where none have been collected in the past often precipitates resistance. "Why do we need to do this?" asks a harried project manager. "I don't see the point," complains an overworked practitioner. Why is it so important to measure the process of software engineering and the product (software) that it produces? The answer is relatively obvious. If you do not measure, there is no real way of determining whether you are improving. And if you are not improving, you are lost.

23.10 SUMMARY

Measurement enables managers and practitioners to improve the software process; assist in the planning, tracking, and control of software projects; and assess the quality of the product (software) that is produced. Measures of specific attributes of the process, project, and product are used to compute software metrics. These metrics can be analyzed to provide indicators that guide management and technical actions.

Process metrics enable an organization to take a strategic view by providing insight into the effectiveness of a software process. Project metrics are tactical. They enable a project manager to adapt project work flow and technical approach in a real-time manner.

Measurement results in cultural change. Data collection, metrics computation, and metrics analysis are the three steps that must be implemented to begin a metrics program. In general, a goal-driven approach helps an organization focus on the right metrics for its business.

Both size- and function-oriented metrics are used throughout the industry. Size-oriented metrics use the line of code as a normalizing factor for other measures such as person-months or defects. Few product metrics have been proposed for direct use in software testing and maintenance. However, many other product metrics can be used to guide the testing process and as a mechanism for assessing the maintainability of a computer program.

Software metrics provide a quantitative way to assess the quality of internal product attributes, thereby enabling you to assess quality before the product is built. Metrics provide the insight necessary to create effective requirements and design models, solid code, and thorough tests.

Software quality metrics, like productivity metrics, focus on the process, the project, and the product. By developing and analyzing a metrics baseline for quality, an organization can correct those areas of the software process that are the cause of software defects.

To be useful in a real-world context, a software metric must be simple and computable, persuasive, consistent, and objective. It should be programming language independent and provide you with effective feedback.

PROBLEMS AND POINTS TO PONDER

23.1. Software for System X has 24 individual functional requirements and 14 nonfunctional requirements. What is the specificity of the requirements? The completeness?

23.2. A major information system has 1140 modules. There are 96 modules that perform control and coordination functions and 490 modules whose function depends on prior processing. The system processes approximately 220 data objects that each has an average of three attributes.

There are 140 unique database items and 90 different database segments. Finally, 600 modules have single entry and exit points. Compute the DSQI for this system.

23.3. A class **X** has 12 operations. Cyclomatic complexity has been computed for all operations in the OO system, and the average value of module complexity is 4. For class **X,** the complexity for operations 1 to 12 is 5, 4, 3, 3, 6, 8, 2, 2, 5, 5, 4, 4, respectively. Compute the weighted methods per class.

23.4. A legacy system has 940 modules. The latest release required that 90 of these modules be changed. In addition, 40 new modules were added and 12 old modules were removed. Compute the software maturity index for the system.

23.5. Why should some software metrics be kept "private"? Provide examples of three metrics that should be private. Provide examples of three metrics that should be public.

23.6. Team A found 342 errors during the software engineering process prior to release. Team B found 184 errors. What additional measures would have to be made for projects A and B to determine which of the teams eliminated errors more efficiently? What metrics would you propose to help in making the determination? What historical data might be useful?

23.7. A Web engineering team has built an e-commerce WebApp that contains 145 individual pages. Of these pages, 65 are dynamic; that is, they are internally generated based on end-user input. What is the customization index for this application?

23.8. A WebApp and its support environment have not been fully fortified against attack. Web engineers estimate that the likelihood of repelling an attack is only 30 percent. The system does not contain sensitive or controversial information, so the threat probability is 25 percent. What is the integrity of the WebApp?

23.9. At the conclusion of a project, it has been determined that 30 errors were found during the modeling phase and 12 errors were found during the construction phase that were traceable to errors not discovered in the modeling phase. What is the DRE for these two phases?

23.10. A software team delivers a software increment to end users. The users uncover eight defects during the first month of use. Prior to delivery, the software team found 242 errors during formal technical reviews and all testing tasks. What is the overall DRE for the project after 1 month's usage?

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

Four

MANAGING SOFTWARE PROJECTS

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn the management techniques required to plan, organize, monitor, and control software projects. These questions are addressed in the chapters that follow:

- How must people, process, and problem be managed during a software project?
- How can software metrics be used to manage a software project and the software process?
- How does a software team generate reliable estimates of effort, cost, and project duration?
- What techniques can be used to systematically assess the risks that can have an impact on project success?
- How does a software project manager select the set of software engineering work tasks?
- How is a project schedule created?
- Why are maintenance and support so important for both software engineering managers and practitioners?

Once these questions are answered, you'll be better prepared to manage software projects in a way that will lead to timely delivery of a high-quality product constrained by the available resources.

CHAPTER

24 PROJECT MANAGEMENT CONCEPTS

In the preface to his book on software project management, Meiler Page-Jones [Pag85] comments on software projects that are not going well, "I've watched in horror as . . . managers futilely struggled through nightmarish projects, squirmed under impossible deadlines, or delivered systems that outraged their users and went on to devour huge chunks of maintenance time."

KEY CONCEPTS

agile teams 495
coordination and communication 496
critical practices 502
people 491
problem decomposition 497
product. 491
project 492
software scope 497
software team 494
stakeholders 493
team leaders 493
W^{5}HH principle 501

QUICK LOOK

What is it? Although many of us (in our darker moments) take Dilbert's[1] view of "management," it remains a very necessary activity when computer-based systems and products are built. Project management involves the planning, monitoring, and coordinating of people, processes, and events that occur as software evolves from a preliminary concept to full operational deployment.

Who does it? Everyone "manages" to some extent, but the scope of management activities varies among people involved in a software project.

Why is it important? Building computer software is a complex undertaking, particularly if it involves many people working over a relatively long time. That's why software projects need to be managed.

What are the steps? Understand the four Ps—people, product, process, and project. People must be organized to perform software work effectively. Product scope and requirements must be understood. A process that is appropriate for the people and the product should be selected. The project must be planned by estimating effort and calendar time to accomplish work tasks. This is true even for agile projects management.

What is the work product? A project plan is created and evolves as project activities commence. The plan is a living document that defines the process and tasks to be conducted, the people who will do the work, and the mechanisms for assessing risks, controlling change, and evaluating quality.

How do I ensure that I've done it right? You're never completely sure that the project plan is right until the team has delivered a high-quality product on time and within budget. However, a team leader does it right when she encourages software people to work together as an effective team, focusing their attention on customer needs and product quality.

1 Try searching for the term *management* on the Dilbert website: http://dilbert.com/.

What Page-Jones describes are symptoms that result from an array of management and technical problems. However, if a postmortem were to be conducted for every project, it is very likely that a consistent theme would be encountered: project management was weak or nonexistent.

In this chapter and Chapters 25 through 27, we'll present the key concepts that lead to effective software project management. This chapter considers basic software project management concepts and principles. Chapter 25 discusses the techniques that are used to estimate costs and create realistic (but flexible) schedules. Chapter 26 presents the management activities that lead to effective risk monitoring, mitigation, and management. Chapter 27 considers product support concerns and discusses the management issues that you'll encounter when dealing with maintenance of deployed systems. Finally, Chapter 28 discusses techniques for studying and improving your team's software engineering processes.

24.1 The Management Spectrum

Effective software project management focuses on the four Ps: people, product, process, and project. The order is not arbitrary. The manager who forgets that software engineering work is an intensely human endeavor will never have success in project management. A manager who fails to encourage comprehensive stakeholder communication early in the evolution of a product risks building an elegant solution for the wrong problem. The manager who pays little attention to the process runs the risk of inserting competent technical methods and tools into a vacuum. The manager who begins work without a solid plan jeopardizes the success of the project. The manager who is not ready to revise the plan when changes arise is doomed to fail.

24.1.1 The People

The cultivation of motivated, highly skilled software people has been discussed since the 1960s. In fact, the "people factor" is so important that the Software Engineering Institute has developed a *People Capability Maturity Model* (People-CMM), in recognition of the fact that "every organization needs to continually improve its ability to attract, develop, motivate, organize, and retain the workforce needed to accomplish its strategic business objectives" [Cur09].

The people capability maturity model defines the following key practice areas for software people: staffing, communication and coordination, work environment, performance management, training, compensation, competency analysis and development, career development, workgroup development, and team/culture development, and others. Organizations that achieve high levels of People-CMM maturity have a higher likelihood of implementing effective software project management practices.

24.1.2 The Product

Before a project can be planned, product objectives and scope should be established, alternative solutions should be considered, and technical and management constraints should be identified. Without this information, it is impossible to define reasonable (and accurate) estimates of the cost, an effective assessment of risk, a realistic breakdown of project tasks, or a manageable project schedule that provides a meaningful indication of progress.

As a software developer, you and other stakeholders must meet to define product objectives and scope. In many cases, this activity begins as part of the system engineering or business process engineering and continues as the first step in software requirements engineering (Chapter 7). Objectives identify the overall goals for the product (from the stakeholders' points of view) without considering how these goals will be achieved. These often take the form of user stories and formal use cases. Scope identifies the primary data, functions, and behaviors that characterize the product, and more important, attempts to bound these characteristics in a quantitative manner.

Once the product objectives and scope are understood, alternative solutions are considered. Although very little detail is discussed, the alternatives enable managers and practitioners to select a "best" approach, given the constraints imposed by delivery deadlines, budgetary restrictions, personnel availability, technical interfaces, and myriad other factors.

24.1.3 The Process

A software process (Chapters 2 through 4) provides the framework from which a comprehensive plan for software development can be established. A small number of framework activities are applicable to all software projects, regardless of their size or complexity. Even agile developers follow a change-friendly process (Chapter 3) to impose some discipline on their software engineering work. A number of task sets—tasks, milestones, work products, and quality assurance points—enable the framework activities to be adapted to the characteristics of the software project and the requirements of the project team. Finally, umbrella activities—such as software quality assurance, software configuration management, and measurement—overlay the process model. Umbrella activities are independent of any one framework activity and occur throughout the process.

24.1.4 The Project

We conduct planned and controlled software projects for one primary reason—it is the only known way to manage complexity. And yet, software teams still struggle. In a study of 250 large software projects between 1998 and 2004, Capers Jones [Jon04] found that "about 25 were deemed successful in that they achieved their schedule, cost, and quality objectives. About 50 had delays or overruns below 35 percent, while about 175 experienced major delays and overruns, or were terminated without completion." Although the success rate for present-day software projects may have improved somewhat, our project failure rate remains much higher than it should be.[2]

To avoid project failure, a software project manager and the software engineers who build the product must avoid a set of common warning signs, understand the critical success factors that lead to good project management, and develop a commonsense approach for planning, monitoring, and controlling the project [Gha14]. Each of these issues is discussed in Section 24.5 and in the chapters that follow.

---

2 Given these statistics, it's reasonable to ask how the impact of computers continues to grow exponentially. Part of the answer, we think, is that a substantial number of these "failed" projects are ill conceived in the first place. Customers lose interest quickly (because what they've requested wasn't really as important as they first thought), and the projects are cancelled.

24.2 PEOPLE

People build computer software, and projects succeed because well-trained, motivated people get things done. All of us, from senior engineering vice presidents to the lowliest practitioner, often take people for granted. Managers argue that people are primary, but their actions sometimes belie their words. In this section, we examine the stakeholders who participate in the software process and the manner in which they are organized to perform effective software engineering.

24.2.1 The Stakeholders

The software process (and every software project) is populated by stakeholders who can be categorized into one of five constituencies:

1. **Senior managers (product owners)** who define the business issues that often have a significant influence on the project.
2. **Project (technical) managers (Scrum masters or team leads)** who must plan, motivate, organize, and coordinate the practitioners who do software work.
3. **Practitioners** who deliver the technical skills that are necessary to engineer a product or application.
4. **Customers** who specify the requirements for the software to be engineered and other stakeholders who have a peripheral interest in the outcome.
5. **End users** who interact with the software once it is released for production use.

Every software project is populated by people who fall within this taxonomy.[3] To be effective, the project team must be organized in a way that maximizes each person's skills and abilities. And that's the job of the team leader.

24.2.2 Team Leaders

Project management is a people-intensive activity, and for this reason, competent practitioners often make poor team leaders. They simply don't have the right mix of people skills. And yet, as Edgemon states: "Unfortunately and all too frequently it seems, individuals just fall into a project manager role and become accidental project managers" [Edg95]. Shared leadership often helps teams perform better, but team leaders often monopolize decision-making authority and fail to provide team members with the levels of autonomy needed to complete their tasks [Hoe16].

James Kouzes has been writing about effective leadership in various technical areas for many years. He lists five practices found in exemplary technology leaders [Kou14]:

> **Model the way.** Leaders must practice what they preach. They demonstrate commitment to the team and project through shared sacrifice (e.g., by being the last one to go home each night or taking the time to become an expert on the software application).
>
> **Inspire and shared vision.** Leaders recognize that they cannot lead without followers. It is important to motivate team members to tie their personal

3 When WebApps, MobileApps, or games are developed, other nontechnical people may be involved in content creation.

aspirations to the team goals. This means involving stakeholders early in the goal-setting process.

Challenge the process. Leaders must take the initiative to look for innovative ways to improve their own work and the work of their teams. Encourage team members to experiment and take risks by helping them generate frequent small successes while learning from their failures.

Enable others to act. Foster the team's collaborative abilities by building trust and facilitating relationships. Increase the team's sense of competence through sharing decision making and goal setting.

Encourage the heart. Celebrate the accomplishments of individuals. Build community (team) spirit by celebrating shared goals and victories, both inside and outside the team.

Another way of looking at successful project leaders might be to suggest that they adopt a problem-solving management style. A software project manager should concentrate on understanding the problem to be solved, coordinate the flow of ideas from stakeholders, and let everyone on the team know (by words and, far more important, by actions) that quality begins with each one of them and that their input and contributions are valued.

24.2.3 The Software Team

There are almost as many human organizational structures for software development as there are organizations that develop software. For better or worse, organizational structure cannot be easily modified. Concerns with the practical and political consequences of organizational change are not within the software project manager's scope of responsibility. However, the organization of the people directly involved in a new software project is within the project manager's purview.

The "best" team structure depends on the management style of your organization, the number of people who will populate the team and their skill levels, and the overall problem difficulty. Mantei [Man81] describes seven project factors that should be considered when planning the structure of software engineering teams: (1) difficulty of the problem to be solved, (2) "size" of the resultant program(s) in lines of code or function points, (3) time that the team will stay together (team lifetime), (4) degree to which the problem can be modularized, (5) quality and reliability of the system to be built, (6) rigidity of the delivery date, and (7) degree of sociability (communication) required for the project.

Regardless of team organization, the objective for every project manager is to help create a team that exhibits cohesiveness. In their book *Peopleware,* DeMarco and Lister [DeM98] look for teams that "jell." They write:

> A jelled team is a group of people so strongly knit that the whole is greater than the sum of the parts . . .
>
> Once a team begins to jell, the probability of success goes way up. The team can become unstoppable, a juggernaut for success . . . They don't need to be managed in the traditional way, and they certainly don't need to be motivated. They've got momentum.

DeMarco and Lister contend that members of jelled teams are significantly more productive and more motivated than average. They share a common goal, a common culture, and in many cases, a "sense of eliteness" that makes them unique.

But not all teams jell. In fact, many teams suffer from what Jackman [Jac98] calls "team toxicity." She defines five factors that "foster a potentially toxic team environment": (1) a frenzied work atmosphere, (2) high frustration that causes friction among team members, (3) a "fragmented or poorly coordinated" software process, (4) an unclear definition of roles on the software team, and (5) "continuous and repeated exposure to failure."

To avoid a frenzied work environment, the project manager should be certain that the team has access to all information required to do the job and that major goals and objectives, once defined, should not be modified unless absolutely necessary. A software team can avoid frustration if it is given as much responsibility for decision making as possible. An inappropriate process (e.g., unnecessary or burdensome work tasks or poorly chosen work products) can be avoided by understanding the product to be built and the people doing the work and by allowing the team to select the process model. The team itself should establish its own mechanisms for accountability (technical reviews[4] are an excellent way to accomplish this) and define a series of corrective approaches when a member of the team fails to perform. And finally, the key to avoiding an atmosphere of failure is to establish team-based techniques for feedback and problem solving.

Many software organizations advocate agile software development (Chapter 3) as an antidote to many of the problems that have plagued software project work. To review, the agile philosophy encourages customer satisfaction and early incremental delivery of software; small, highly motivated project teams; informal methods; minimal software engineering work products; and overall development simplicity.

The small, highly motivated project team, also called an *agile team,* adopts many of the characteristics of successful software project teams discussed in the preceding section and avoids many of the toxins that create problems [Hoe16]. However, the agile philosophy stresses individual (team member) competency coupled with group collaboration as critical success factors for the team. Cockburn and Highsmith [Coc01a] note this when they write:

> If the people on the project are good enough, they can use almost any process and accomplish their assignment. If they are not good enough, no process will repair their inadequacy—"people trump process" is one way to say this. However, lack of user and executive support can kill a project—"politics trump people." Inadequate support can keep even good people from accomplishing the job . . .

To make effective use of the competencies of each team member and to foster effective collaboration through a software project, agile teams are *self-organizing.*

Many agile process models (e.g., Scrum) give the agile team significant autonomy to make the project management and technical decisions required to get the job done. Planning is kept to a minimum, and the team is allowed to select its own approach (e.g., process, methods, tools), constrained only by business requirements and organizational standards. As the project proceeds, the team self-organizes to focus individual competency in a way that is most beneficial to the project at a given point in time.

To accomplish this, an agile team might conduct daily team meetings to coordinate and synchronize the work that must be accomplished for that day. Based on information obtained during these meetings, the team adapts its approach in a way that accomplishes an increment of work. As each day passes, continual self-organization and collaboration move the team toward a completed software increment.

4 Technical reviews are discussed in detail in Chapter 16.

24.2.4 Coordination and Communication Issues

There are many reasons that software projects get into trouble. The scale of many development efforts is large, leading to complexity, confusion, and significant difficulties in coordinating team members. Uncertainty is common, resulting in a continuing stream of changes that ratchets the project team. Interoperability has become a key characteristic of many systems. New software must communicate with existing software and conform to predefined constraints imposed by the system or product.

These characteristics of modern software—scale, uncertainty, and interoperability—are facts of life. To deal with them effectively, you must establish effective methods for coordinating the people who do the work. To accomplish this, mechanisms for formal and informal communication among team members and between multiple teams must be established. Formal communication is accomplished through "writing, structured meetings, and other relatively non-interactive and impersonal communication channels" [Kra95]. Informal communication is more personal. Members of a software team share ideas on an ad hoc basis, ask for help as problems arise, and interact with one another on a daily basis.

SAFEHOME



Team Structure

The scene: Doug Miller's office prior to the initiation of the *SafeHome* software project.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: Have you guys had a chance to look over the preliminary info on *SafeHome* that marketing has prepared?

Vinod (nodding and looking at his teammates): Yes. But we have a bunch of questions.

Doug: Let's hold on that for a moment. I'd like to talk about how we are going to structure the team, who's responsible for what . . .

Jamie: I'm really into the agile philosophy, Doug. I think we should be a self-organizing team.

Vinod: I agree. Given the tight time line and some of the uncertainty, and the fact that we're all really competent [laughs], that seems like the right way to go.

Doug: That's okay with me, but you guys know the drill.

Jamie (smiling and talking as if she was reciting something): We make tactical decisions, about who does what and when, but it's our responsibility to get product out the door on time.

Vinod: And with quality.

Doug: Exactly. But remember there are constraints. Marketing defines the software increments to be produced—in consultation with us, of course.

Jamie: And?

Doug: And, we're going to use UML as our modeling approach.

Vinod: But keep extraneous documentation to an absolute minimum.

Doug: Who is the liaison with me?

Jamie: We decided that Vinod will be the tech lead—he's got the most experience, so Vinod is your liaison, but feel free to talk to any of us.

Doug (laughing): Don't worry, I will.

24.3 PRODUCT

A software project manager is confronted with a dilemma at the very beginning of a software project. Quantitative estimates and an organized plan are required, but solid information is unavailable. A detailed analysis of software requirements would provide information necessary for estimates, but analysis often takes weeks or even months to complete. Worse, requirements may be fluid, changing regularly as the project proceeds. Yet, a plan is needed now!

Like it or not, you must examine the product and the problem it is intended to solve at the very beginning of the project. At a minimum, the scope of the product must be established and bounded.

24.3.1 Software Scope

The first software project management activity is the determination of *software scope.* Scope is defined by answering the following questions:

Context. How does the software to be built fit into a larger system, product, or business context, and what constraints are imposed as a result of the context?

Information objectives. What customer-visible data objects are produced as output from the software? What data objects are required for input?

Function and performance. What function does the software perform to transform input data into output? Are any special performance characteristics to be addressed?

Software project scope must be unambiguous and understandable at the management and technical levels. A statement of software scope must be bounded. That is, quantitative data (e.g., number of simultaneous users, size of mailing list, maximum allowable response time) are stated explicitly, constraints and/or limitations (e.g., product cost restricts memory size) are noted, and mitigating factors (e.g., desired algorithms are well understood and available in Java) are described. Even in the most fluid situations, the number of prototypes needs to be considered and the scope of the first prototype needs to be set.

24.3.2 Problem Decomposition

Problem decomposition, sometimes called *partitioning* or *problem elaboration,* is an activity that sits at the core of software requirements analysis (Chapters 7 and 8). During the scoping activity, no attempt is made to fully decompose the problem. Rather, decomposition is applied in two major areas: (1) the functionality and content (information) that must be delivered and (2) the process that will be used to deliver it. This can be accomplished using a list of functions or with use cases or for agile work, user stories.

Human beings tend to apply a divide-and-conquer strategy when they are confronted with a complex problem. Stated simply, a complex problem is partitioned into smaller problems that are more manageable. This is the strategy that applies as project planning begins. Software functions, described in the statement of scope, are evaluated and refined to provide more detail prior to the beginning of estimation (Chapter 25). Because both cost and schedule estimates are functionally oriented,

some degree of decomposition is often useful. Similarly, major content or data objects are decomposed into their constituent parts, providing a reasonable understanding of the information to be produced by the software.

24.4 PROCESS

The framework activities (Chapter 1) that characterize the software process are applicable to all software projects. The problem is to select the process model that is appropriate for the software to be engineered by your project team. The recommended process model in Chapter 4 may be a good starting point for many project teams to consider.

Your team must decide which process model is most appropriate for (1) the customers who have requested the product and the people who will do the work, (2) the characteristics of the product itself, and (3) the project environment in which the software team works. When a process model has been selected, the team then defines a preliminary project plan based on the set of process framework activities. Once the preliminary plan is established, process decomposition begins. That is, a complete plan, reflecting the work tasks required to populate the framework activities, must be created. We explore these activities briefly in the sections that follow and present a more detailed view in Chapter 25.

24.4.1 Melding the Product and the Process

Project planning begins with the melding of the product and the process. Each function to be engineered by your team must pass through the set of framework activities that have been defined for your software organization. The process framework establishes a skeleton for project planning. It is adapted by allocating a task set that is appropriate to the project. Assume that the organization has adopted the generic framework activities—communication, planning, modeling, construction, and deployment—discussed in Chapter 1.

The team members who work on a product function will apply each of the framework activities to it. In essence, a matrix similar to the one shown in Figure 24.1 is created. Each major product function (the figure lists functions for the fitness app software discussed in Chapter 2) or user story is listed in the left-hand column. Framework activities are listed in the top row. Software engineering work tasks (for each framework activity) would be entered in the following row.[5] The job of the project manager (and other team members) is to estimate resource requirements for each matrix cell, start and end dates for the tasks associated with each cell, and work products to be produced as a consequence of each task. These activities are considered in Chapter 25.

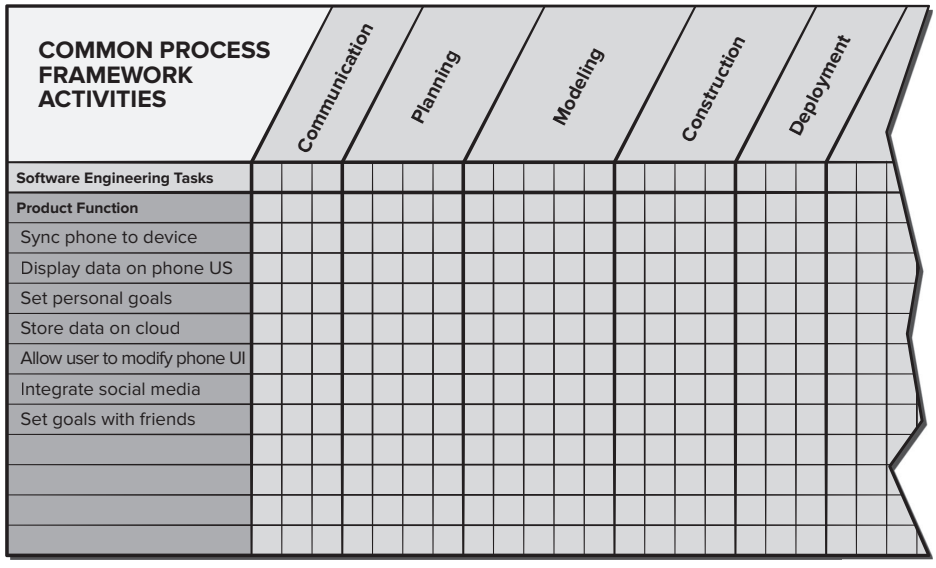
24.4.2 Process Decomposition

A software team should have a significant degree of flexibility in choosing the software process model that is best for the project and the software engineering tasks that

5 It should be noted that work tasks must be adapted to the specific needs of the project based on a number of adaptation criteria.

FIGURE 24.1

Melding the problem and the process



populate the process model once it is chosen. A relatively small project that is similar to past efforts might be best accomplished using a single sprint approach. If the deadline is so tight that full functionality cannot reasonably be delivered, an incremental strategy might be best. Similarly, projects with other characteristics (e.g., uncertain requirements, breakthrough technology, difficult customers, or potential for significant component reuse) will lead to the selection of other process models.[6]

Once the process model has been chosen, the process framework is adapted to it. In every case, the generic process framework discussed earlier can be used. It will work for linear models, for iterative and incremental models, for evolutionary models, and even for concurrent or component assembly models. The process framework is invariant and serves as the basis for all work performed by a software organization.

But actual work tasks do vary. Process decomposition commences when the project manager asks, "How do we accomplish this framework activity?" For example, a small, relatively simple project might require the following work tasks for the communication activity:

1. Develop a list of clarification issues.
2. Meet with stakeholders to address clarification issues.
3. Jointly develop a statement of scope by listing the user stories.
4. Review the statement of scope with all concerned, and determine the importance of each user story to the stakeholders.
5. Modify the statement of scope as required.

6 Recall that project characteristics also have a strong bearing on the structure of the software team (Section 24.2.3).

These events might occur over a period of less than 48 hours. They represent a process decomposition that is appropriate for the small, relatively simple project.

Now, consider a more complex project, which has a broader scope and more significant business impact. Such a project might require the following work tasks for the **communication:**

1. Review the customer request.
2. Plan and schedule a formal, facilitated meeting with all stakeholders.
3. Conduct research to specify the proposed solution and existing approaches.
4. Prepare a "working document" and an agenda for the formal meeting.
5. Conduct the meeting.
6. Jointly develop mini-specs that reflect data, function, and behavioral features of the software. This is often done by developing use cases that describe the software from the user's point of view.
7. Review each mini-spec or use case for correctness, consistency, and lack of ambiguity.
8. Assemble the mini-specs into a scoping document.
9. Review the collection of use cases with all concerned, and determine their relative importance to all stakeholders.
10. Modify the scoping document or use cases as required.

Both projects perform the framework activity that we call **communication,** but the first project team performs half as many software engineering work tasks as the second.

24.5 PROJECT

To manage a successful software project, you have to understand what can go wrong so that problems can be avoided. In an excellent paper on software projects, John Reel [Ree99] defines signs that indicate that an information systems project is in jeopardy. In some cases, software people don't understand their customer's needs. This leads to a project with a poorly defined scope. In other projects, changes are managed poorly. Sometimes the chosen technology changes or business needs shift and management sponsorship is lost. Management can set unrealistic deadlines or end users can be resistant to the new system. There are cases in which the project team simply does not have the requisite skills. And finally, there are developers who never seem to learn from their mistakes.

Jaded industry professionals often refer to the "90–90 rule" when discussing particularly difficult software projects: The first 90 percent of a system absorbs 90 percent of the allotted effort and time. The last 10 percent takes another 90 percent of the allotted effort and time [Zah94]. The seeds that lead to the 90–90 rule are contained in the signs noted in the preceding paragraph.

But enough negativity! What are the characteristics of successful software projects? Ghazi [Gha14] and her colleagues note several characteristics that are present in successful software projects and also found in most well-designed process models.

1. Clear and well-understood requirements accepted by all stakeholders
2. Active and continuous participation of users throughout the development process
3. A project manager with required leadership skills who is able to share project vision with the team
4. A project plan and schedule developed with stakeholder participation to achieve user goals
5. Skilled and engaged team members
6. Development team members with compatible personalities who enjoy working in a collaborative environment
7. Realistic schedule and budget estimates which are monitored and maintained
8. Customer needs that are understood and satisfied
9. Team members who experience a high degree of job satisfaction
10. A working product that reflects desired scope and quality

24.6 THE W^{5}HH PRINCIPLE

In an excellent paper on software process and projects, Barry Boehm [Boe96] states: "[Y]ou need an organizing principle that scales down to provide simple [project] plans for simple projects." Boehm suggests an approach that addresses project objectives, milestones and schedules, responsibilities, management and technical approaches, and required resources. He calls it the *W^{5}HH Principle,* after a series of questions that lead to a definition of key project characteristics and the resultant project plan:

Why is the system being developed? All stakeholders should assess the validity of business reasons for the software work. Does the business purpose justify the expenditure of people, time, and money?

What will be done? The task set required for the project is defined.

When will it be done? The team establishes a project schedule by identifying when project tasks are to be conducted and when milestones are to be reached.

Who is responsible for a function? The role and responsibility of each member of the software team is defined.

Where are they located organizationally? Not all roles and responsibilities reside within software practitioners. The customer, users, and other stakeholders also have responsibilities.

How will the job be done technically and managerially? Once product scope is established, a management and technical strategy for the project must be defined.

How much of each resource is needed? The answer to this question is derived by developing estimates (Chapter 25) based on answers to earlier questions.

Boehm's W^{5}HH principle is applicable regardless of the size or complexity of a software project. The questions noted provide you and your team with an excellent planning outline.

24.7 CRITICAL PRACTICES

The Airlie Council[7] has developed a list of "critical software practices for performance-based management." These practices are "consistently used by, and considered critical by, highly successful software projects and organizations whose 'bottom line' performance is consistently much better than industry averages" [Air99]. These practices are still applicable to modern performance-based management of all software projects [All14].

Critical practices[8] include: metric-based project management (Chapter 23), empirical cost and schedule estimation (Chapter 25), earned value tracking (Chapter 25), defect tracking against quality targets (Chapters 19 through 21), and people-oriented management (Chapter 24). Each of these critical practices is addressed throughout Part Four of this book.

24.8 SUMMARY

Software project management is an umbrella activity within software engineering. It begins before any technical activity is initiated and continues throughout the modeling, construction, and deployment of computer software.

Four Ps have a substantial influence on software project management—people, product, process, and project. People must be organized into effective teams, motivated to do high-quality software work, and coordinated to achieve effective communication. Product requirements must be communicated from customer to developer, partitioned (decomposed) into their constituent parts, and positioned for work by the software team. The process must be adapted to the people and the problem. A common process framework is selected, an appropriate software engineering paradigm is applied, and a set of work tasks is chosen to get the job done. Finally, the project must be organized in a manner that enables the software team to succeed.

The pivotal element in all software projects is people. Software engineers can be organized in a number of different team structures that range from traditional control hierarchies to "open paradigm" teams. A variety of coordination and communication techniques can be applied to support the work of the team. In general, technical reviews and informal person-to-person communication have the most value for practitioners.

The project management activity encompasses measurement and metrics, estimation and scheduling, risk analysis, tracking, and control. Each of these topics is considered in the chapters that follow.

---

7 The Airlie Council was comprised of a team of software engineering experts chartered by the U.S. Department of Defense to help develop guidelines for best practices in software project management and software engineering.

8 Only those critical practices associated with "project integrity" are noted here.

PROBLEMS AND POINTS TO PONDER

24.1. Based on information contained in this chapter and your own experience, develop "10 commandments" for empowering software engineers. That is, make a list of 10 guidelines that will lead to software people who work to their full potential.

24.2. The Software Engineering Institute's People Capability Maturity Model (People-CMM) takes an organized look at "key practice areas" (KPAs) that cultivate good software people. Your instructor will assign you one KPA for analysis and summary.

24.3. Describe three real-life situations in which the customer and the end user are the same. Describe three situations in which they are different.

24.4. The decisions made by senior management can have a significant impact on the effectiveness of a software engineering team. Provide five examples to illustrate that this is true.

24.5. You have been appointed a project manager within an information systems organization. Your job is to build an application that is quite similar to others your team has built, although this one is larger and more complex. Requirements have been thoroughly documented by the customer. What team structure would you choose and why? What software process model(s) would you choose and why?

24.6. You have been appointed a project manager for a small software products company. Your job is to build a breakthrough product that combines virtual reality hardware with state-of-the-art software. Because competition for the home entertainment market is intense, there is significant pressure to get the job done. What team structure would you choose and why? What software process model(s) would you choose and why?

24.7. You have been appointed a project manager for a major software products company. Your job is to manage the development of the next-generation version of its widely used mobile fitness app. Because competition is intense, tight deadlines have been established and announced. What team structure would you choose and why? What software process model(s) would you choose and why?

24.8. You have been appointed a software project manager for a company that services the genetic engineering world. Your job is to manage the development of a new software product that will accelerate the pace of gene typing. The work is R&D oriented, but the goal is to produce a product within the next year. What team structure would you choose and why? What software process model(s) would you choose and why?

24.9. You have been asked to develop a small application that analyzes each course offered by a university and reports the average grade obtained in the course (for a given term). Write a statement of scope that bounds this problem.

24.10. What, in your opinion, is the most important aspect of people management for a software project?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

25 CREATING A VIABLE SOFTWARE PLAN

Software project management begins with a set of activities that are collectively called *project planning*. Before the project can begin, the software team estimates the work to be done, the resources that will be required, and the time that will elapse from start to finish. Once these activities are accomplished, the software team should establish a project schedule that defines software engineering tasks and milestones, identifies who is responsible for conducting each task, and specifies the intertask dependencies that may have a strong bearing on progress.

KEY CONCEPTS

agile development 519
critical path 520
effort 509
estimation,
  agile projects 519
  decomposition techniques 510
  empirical models 510
  FP-based 514
  problem-based 512
  process-based 515
  reconciliation 518
  techniques 511
  use case points (UCPs) 517
people and effort 522
principles 521
project planning 504
resources 507
software scope 507
software sizing 511
task network 525
time-boxing 528
time-line charts 526
tracking 528
work breakdown 526

QUICK LOOK

What is it? Software project planning encompasses five major activities—estimation, scheduling, risk analysis, quality management planning, and change management planning.

Who does it? Software project managers and other members of the software team.

Why is it important? You need to assess the tasks to perform, and the time line for the work to be conducted. Many software engineering tasks must occur in parallel, and the result of work performed during one task may have a profound effect on work to be conducted in another task. These interdependencies are very difficult to understand without creating a schedule.

What are the steps? Software engineering activities and tasks are refined to accommodate the functions and constraints imposed by project scope. The problem is decomposed, and estimation, risk analysis, and scheduling occur.

What is the work product? An adaptable plan containing a simple table delineating the tasks to be performed, the functions to be implemented, and the cost, effort, and time involved for each is generated. A project schedule is also created based on this information.

How do I ensure that I've done it right? That's hard, because you won't really know until the project has been completed. However, if you use a systematic planning approach, you can feel confident that you've given it your best shot.

There was once a bright-eyed young engineer who was chosen to develop some code for an automated manufacturing application. The reason for his selection was simple. He was the only person in his group who knew the ins and outs of the manufacturing controller, but at the time he knew nothing about software engineering and even less about project scheduling and tracking.

His boss informed the young engineer that the project had to be completed in 2 months. He considered his approach and began writing code. After 2 weeks, the boss called him into his office and asked how things were going.

"Really great," said the young engineer with youthful enthusiasm. "This is much simpler than I thought. I'm probably close to 75 percent finished."

The boss smiled and encouraged the young engineer to keep up the good work. They planned to meet again in a week's time.

A week later the boss called the engineer into his office and asked, "Where are we?"

"Everything's going well," said the youngster, "but I've run into a few small snags. I'll get them ironed out and be back on track soon."

"How does the deadline look?" the boss asked.

"No problem," said the engineer. "I'm close to 90 percent complete."

If you've been working in the software world for more than a few years, you can finish the story. It'll come as no surprise that the young engineer[1] stayed 90 percent complete for the entire project duration and finished (with the help of others) only 1 month late.

This story has been repeated hundreds of thousands of times by software developers during the past five decades. The big question is why.

25.1 Comments on Estimation

Planning requires you to make an initial commitment, even though it's likely that this "commitment" will be proven wrong. Whenever estimates are made, you have to look into the future and accept some degree of uncertainty as a matter of course.

Estimating is as much art as it is science, and it should not be conducted in a haphazard manner. Because estimation lays a foundation for all other project planning actions, and project planning provides the road map for successful software engineering, we would be ill-advised to embark without it.

Estimation of resources, cost, and schedule for software development requires experience, access to good historical information (e.g., process and product metrics), and the courage to commit to quantitative predictions when qualitative information is all that exists. Estimation carries inherent risk,[2] and this risk leads to uncertainty. Project complexity, project size, and the degree of structural uncertainty all affect the reliability of estimates.

Project complexity has a strong effect on the uncertainty inherent in planning. Complexity, however, is a relative measure that is affected by familiarity with past efforts. The first-time developer of a sophisticated e-commerce application might

1 In case you were wondering, this story is autobiographical (RSP).

2 Systematic techniques for risk analysis are presented in Chapter 26.

consider it to be exceedingly complex. However, a Web engineering team developing its tenth e-commerce WebApp would consider such work run of the mill. A number of quantitative software complexity measures have been proposed [Zus97], but they are rarely used in real-world projects. However, other, more subjective assessments of complexity (e.g., function point complexity adjustment factors described in Section 25.6) can be established early in the planning process.

Project size is another important factor that can affect the accuracy and efficacy of estimates. As size increases, the interdependency among various elements of the software grows rapidly.[3] Problem decomposition, an important approach to estimating, becomes more difficult because the refinement of problem elements may still be formidable. To paraphrase Murphy's law: "What can go wrong will go wrong"—and if there are more things that can fail, more things will fail.

The *degree of structural uncertainty* also has an effect on estimation risk. In this context, structure refers to the degree to which requirements have been solidified, the ease with which functions can be compartmentalized, and the hierarchical nature of the information that must be processed.

The availability of historical information has a strong influence on estimation risk. By looking back, you can emulate things that worked and improve areas where problems arose. When comprehensive software metrics (Chapter 23) are available for past projects, estimates can be made with greater assurance, schedules can be established to avoid past difficulties, and overall risk is reduced.

If project scope is poorly understood or project requirements are subject to change, uncertainty and estimation risk become dangerously high. As a planner, you and the customer should recognize that variability in software requirements means instability in cost and schedule.

However, you should not become obsessive about estimation. Modern software engineering approaches (e.g., evolutionary process models) take an iterative view of development. In such approaches, it is possible to revisit estimates (as more information is known) and revise them when stakeholders make changes to requirements or schedules.

25.2 THE PROJECT PLANNING PROCESS

The objective of software project planning is to provide a framework that enables the manager to make reasonable estimates of resources, cost, and schedule. In addition, estimates should attempt to define best-case and worst-case scenarios so that project outcomes can be bounded. Although there is an inherent degree of uncertainty, the software team embarks on a plan that has been established as a consequence of a project planning task set. Therefore, the plan must be adapted and updated as the project proceeds. In the following sections, each of the activities associated with the software project planning task set is discussed.

3 Size often increases due to "scope creep" that occurs when problem requirements change. Increases in project size can have a geometric impact on project cost and schedule (Michael Mah, personal communication).

TASK SET



Task Set for Project Planning

1. Establish project scope.
2. Determine feasibility.
3. Analyze risks (Chapter 26).
4. Define required resources.
   a. Determine required human resources.
   b. Define reusable software resources.
   c. Identify environmental resources.
5. Estimate cost and effort.
   a. Decompose the problem.
   b. Develop two or more estimates using size, function points, process tasks, or use cases.
   c. Reconcile the estimates.
6. Develop an initial project schedule (Section 25.11).
   a. Establish a meaningful task set.
   b. Define a task network.
   c. Use scheduling tools to develop a time-line chart.
   d. Define schedule tracking mechanisms.
7. Repeat steps 1 to 6 to create a detailed schedule for each prototype as the scope of each prototype is defined.

25.3 SOFTWARE SCOPE AND FEASIBILITY

Software scope describes the functions and features that are to be delivered to end users; the data that are input and output; the "content" that is presented to users as a consequence of using the software; and the performance, constraints, interfaces, and reliability that *bound* the system. Scope can be defined by developing a set of use cases[4] that is developed with the end users.

Functions described in the use cases are evaluated and in some cases refined to provide more detail prior to the beginning of estimation. Because both cost and schedule estimates are functionally oriented, some degree of decomposition is often useful. Performance considerations often constrain processing and response-time requirements.

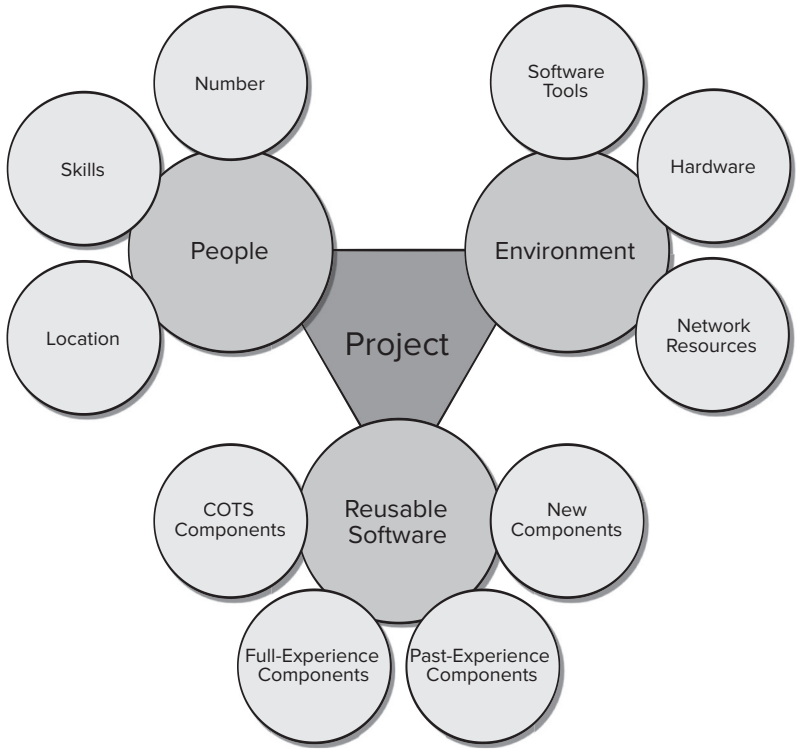
Once scope has been identified (with the concurrence of the customer), it is reasonable to ask: "Can we build software to meet this scope? Is the project feasible?" All too often, software engineers rush past these questions (or are pushed past them by impatient managers or other stakeholders), only to become mired in a project that is doomed from the onset. You must try to determine if the system can be created using available technology, dollars, time, and other resources. Project feasibility is important, but a consideration of business need is even more important. It does no good to build a high-tech system or product that no one wants.

25.4 RESOURCES

Once scope is defined, you must estimate the resources required to build software that will implement the set of use cases that describe software features and functions. Figure 25.1 depicts the three major categories of software engineering resources—people,

4 Use cases have been discussed in detail throughout Part Two of this book. A use case is a scenario-based description of the user's interaction with the software from the user's point of view.

FIGURE 25.1
Project resources



reusable software components, and the development environment (hardware and software tools). Each resource is specified with four characteristics: description of the resource, a statement of availability, time when the resource will be required, and duration of time that the resource will be applied. The last two characteristics can be viewed as a *time window*. Availability of the resource for a specified window must be established at the earliest practical time.

25.4.1 Human Resources

The planner begins by evaluating software scope and selecting the skills required to complete development. Both organizational position (e.g., manager, senior software engineer) and specialty (e.g., telecommunications, database, e-commerce) are specified. For relatively small projects (a few person-months), a single individual may perform all software engineering tasks, consulting with specialists as required. For larger projects, the software team may be geographically dispersed across a number of different locations. Hence, the location of each human resource is specified.

The number of people required for a software project can be determined only after an estimate of development effort (e.g., person-months) is made. Techniques for estimating effort are discussed later in this chapter.

25.4.2 Reusable Software Resources

Component-based software engineering (CBSE)[5] emphasizes reusability—that is, the creation and reuse of software building blocks. Such building blocks, often called *components,* must be cataloged for easy reference, standardized for easy application, and validated for easy integration.

Ironically, reusable software components are often neglected during planning, only to become a paramount concern during the development phase of the software process. It is better to specify software resource requirements early. In this way technical evaluation of the alternatives can be conducted and timely acquisition can occur. It is also important to consider whether it would be less costly to buy an existing software product (assuming it satisfies all stakeholder needs) than building a custom software product from scratch.

25.4.3 Environmental Resources

The environment that supports a software project, often called the *software engineering environment* (SEE), incorporates hardware and software. Hardware provides a platform that supports the tools (software) required to produce the work products that are an outcome of good software engineering practice.[6] Because most software organizations have multiple constituencies that require access to the SEE, you must prescribe the time window required for hardware and software and verify that these resources will be available.

When a computer-based system (incorporating specialized hardware and software) is to be engineered, the software team may require access to hardware elements being developed by other engineering teams. For example, software for a robotic device used within a manufacturing cell may require a specific robot (e.g., a robotic welder) as part of the validation test step; a software project for advanced page layout may need a high-speed digital printing system at some point during development. Each hardware element must be specified as part of planning.

25.5 Data Analytics and Software Project Estimation

Software cost and effort estimation will never be an exact science. Too many variables—human, technical, environmental, political—can affect the ultimate cost of software and effort applied to develop it. However, software project estimation can be transformed from a black art to a series of systematic steps that provide estimates with acceptable risk. To achieve reliable cost and effort estimates, a number of options arise:

1. Delay estimation until late in the project (obviously, we can achieve 100 percent accurate estimates after the project is complete!).

5 CBSE was considered briefly in Chapter 11.

6 Other hardware—the *target environment*—is the computer on which the software will execute when it has been released to the end user.

2. Base estimates on similar projects that have already been completed.
3. Use relatively simple decomposition techniques to generate project cost and effort estimates.
4. Use one or more empirical models for software cost and effort estimation.

Unfortunately, the first option, however attractive, is not practical. Cost estimates must be provided up front. However, you should recognize that the longer you wait, the more you know, and the more you know, the less likely you are to make serious errors in your estimates.

The second option can work reasonably well, if the current project is quite similar to past efforts and other project influences (e.g., the customer, business conditions, the software engineering environment, deadlines) are roughly equivalent. Unfortunately, past experience has not always been a good indicator of future results.

The remaining options are viable approaches to software project estimation. Ideally, the techniques noted for each option should be applied in tandem; each used as a cross-check for the other. Decomposition techniques take a divide-and-conquer approach to software project estimation. By decomposing a project into major functions and related software engineering activities, cost and effort estimation can be performed in a stepwise fashion.

An *empirical estimation model* for computer software uses formulas derived from existing project data to predict effort as a function of things like LOC or FP.[7] Values for LOC or FP are estimated using the approach described in Sections 25.6.3 and 25.6.4. But instead of using the tables described in those sections, the resultant values for LOC or FP are plugged into the estimation model [Whi15].

A typical empirical estimation model is derived using regression analysis on data collected from past software projects. The overall structure of such models takes the form [Mat94]

$$E = A + B \times (e_v)^C \tag{25.1}$$

where A, B, and C are empirically derived constants, E is effort in person-months, and e_v is the estimation variable (either LOC or FP). In addition to the relationship noted in Equation (25.1), the majority of estimation models have some form of project adjustment component that enables E to be adjusted by other project characteristics (e.g., problem complexity, staff experience, development environment).

Empirical estimation models can be used to complement decomposition techniques and offer a potentially valuable estimation approach in their own right. A model is based on experience (historical data) and takes the form

$$d = f(v_i)$$

where d is one of a number of estimated values (e.g., effort, cost, project duration) and v_i are selected independent parameters (e.g., estimated lines of code). The empirical data that support most software estimation models are derived from a limited

7 An empirical model using use cases as the independent variable is suggested in Section 25.6.6. However, relatively few have appeared in the literature to date.

sample of projects.[8] For this reason, no estimation model is appropriate for all classes of software and in all development environments.

Ideally, any estimation model should be calibrated to reflect local conditions. The model should be tested by applying data collected from completed projects, plugging the data into the model, and then comparing actual to predicted results. If agreement is poor, the model must be tuned and retested before it can be used.

Each of the software estimation methods is only as good as the historical data used to seed the estimate. If no historical data exist, the estimates rest on a very shaky foundation. Therefore, you should use the results obtained from such models judiciously. In Chapter 23, we examined the characteristics of some of the software metrics or data analytics that provide the basis for historical estimation data. Software data analytics concepts are discussed briefly in Appendix 2 of this book.

25.6 DECOMPOSITION AND ESTIMATION TECHNIQUES

Software project estimation is a form of problem solving, and in most cases, the problem to be solved (i.e., developing a cost and effort estimate for a software project) is too complex to be considered in one piece. For this reason, you should decompose the problem, recharacterizing it as a set of smaller (and hopefully, more manageable) problems.

In Chapter 24, the decomposition approach was discussed from two different points of view: decomposition of the problem and decomposition of the process. Estimation uses one or both forms of partitioning. But before an estimate can be made, you must understand the scope of the software to be built and generate an estimate of its "size."

25.6.1 Software Sizing

The accuracy of a software project estimate is predicated on a number of things: (1) the degree to which you have properly estimated the size of the product to be built, (2) the ability to translate the size estimate into human effort, calendar time, and dollars (a function of the availability of reliable software metrics from past projects), (3) the degree to which the project plan reflects the abilities of the software team, and (4) the stability of product requirements and the environment that supports the software engineering effort.

Because a project estimate is only as good as the estimate of the size of the work to be accomplished, *software sizing* represents your first major challenge as a planner. In the context of project planning, size refers to a quantifiable outcome of the software project. If a direct approach is taken, size can be measured in lines of code (LOC). If an indirect approach is chosen, size is represented as function points (FP). Size can be estimated by considering the type of project and its application domain, the functionality delivered (i.e., the number of function points), the number of components (or use cases) to be delivered, and the degree to which a set of existing components must be modified for the new system.

8 As an example, the COCOMO (Constructive Cost Model) was originally developed in 1981, with updated versions, COCOMO II and COCOMO III, released in later years. A presentation on the genesis of the COCOMO model can be downloaded from: http://www.psmsc.com/UG2016/Presentations/p10-Clark-COCOMO%20III%20Presentation%20v1.pdf.

25.6.2 Problem-Based Estimation

In Chapter 23, lines of code and function points were described as measures from which productivity metrics can be computed. LOC and FP data are used in two ways during software project estimation: (1) as estimation variables to "size" each element of the software and (2) as baseline metrics collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.

LOC and FP estimation are distinct estimation techniques. Yet both have a number of characteristics in common. You begin with a bounded statement of software scope and from this statement attempt to decompose the statement of scope into problem functions that can each be estimated individually. LOC or FP (the estimation variable) is then estimated for each function. Alternatively, you may choose another component for sizing, such as classes or objects, changes, or business processes affected.

Baseline productivity metrics (e.g., LOC/pm or FP/pm)[9] are then applied to the appropriate estimation variable, and cost or effort for the function is derived. Function estimates are combined to produce an overall estimate for the entire project. When collecting productivity metrics for projects, be sure to establish a taxonomy of project types. This will enable you to compute domain-specific averages, making estimation more accurate. Many modern applications reside on a network or are part of a client-server architecture. Therefore, be sure that your estimates include the effort required to develop "infrastructure" software.

25.6.3 An Example of LOC-Based Estimation

As an example of an LOC estimation technique, we consider a software package to be developed for a computer-aided design application for mechanical components. The software is to execute on a notebook computer. A preliminary statement of software scope can be developed:

> The mechanical CAD software will accept two- and three-dimensional geometric data from a designer. The designer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of devices. The software will be designed to control and interact with peripheral devices that include a touchpad, scanner, laser printer, and large-bed digital plotter.

This statement of scope is preliminary—it is not bounded. Every sentence would have to be expanded to provide concrete detail and quantitative bounding. For example, before estimation can begin, the planner must determine what "characteristics of good human/machine interface design" means or what the size and sophistication of the "CAD database" are to be.

For our purposes, assume that further refinement has occurred and that the major software functions listed in Figure 25.2 are identified. Following the decomposition technique for LOC, an estimation table (Figure 25.2) is developed. A range of LOC estimates is developed for each function. For example, the range of LOC estimates for the 3D geometric analysis function is optimistic, 4600 LOC; most likely, 6900 LOC;

9 The acronym *pm* means person-month of effort.

FIGURE 25.2

Estimation table for the LOC methods

Function	Estimated LOC
User interface and control facilities (UICF)	2300
Two-dimensional geometric analysis (2DGA)	5300
Three-dimensional geometric analysis (3DGA)	6800
Database management (DBM)	3350
Computer graphics display facilities (GCDF)	4950
Peripheral control function (PCF)	2100
Design analysis modules (DAM)	8400
Estimated lines of code	33200

and pessimistic, 8600 LOC. Applying Equation (25.1), the expected value for the 3D geometric analysis function is 6800 LOC. Other estimates are derived in a similar fashion. By summing vertically in the estimated LOC column, an estimate of 33200 lines of code is established for the CAD system.

A review of historical data indicates that the organizational average productivity for systems of this type is 620 LOC/pm. Based on a burdened labor rate of $8,000 per month, the cost per line of code is approximately $13. Based on the LOC estimate and the historical productivity data, the total estimated project cost is $431,000 and the estimated effort is 54 person-months.[10] Do not succumb to the temptation to use this result as your project estimate. You should derive another result using a different approach.

SAFEHOME



Estimating

The scene: Doug Miller's office as project planning begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: We need to develop an effort estimate for the project, and then we've got to define a micro schedule for the first increment and a macro schedule for the remaining increments.

Vinod (nodding): Okay, but we haven't defined any increments yet.

Doug: True, but that's why we need to estimate.

Jamie (frowning): You want to know how long it's going to take us?

Doug: Here's what I need. First, we need to functionally decompose the *SafeHome* software . . . at a high level . . . then we've got to estimate the number of lines of code that each function will take . . . then . . .

10 Estimates are rounded to the nearest $1,000 and person-month. Further precision is unnecessary and unrealistic, given the limitations of estimation accuracy.

Jamie: Whoa! How are we supposed to do that?

Vinod: I've done it on past projects. You begin with use cases, determine the functionality required to implement each, then guesstimate the LOC count for each piece of the function. The best approach is to have everyone do it independently and then compare results.

Doug: Or you can do a functional decomposition for the entire project.

Jamie: But that'll take forever and we've got to get started.

Vinod: No . . . it can be done in a few hours . . . this morning, in fact.

Doug: I agree . . . we can't expect exactitude, just a ballpark idea of what the size of *SafeHome* will be.

Jamie: I think we should just estimate effort . . . that's all.

Doug: We'll do that too. Then use both estimates as a cross-check.

Vinod: Let's go do it . . .

25.6.4 An Example of FP-Based Estimation

Decomposition for FP-based estimation focuses on information domain values rather than software functions. Referring to Table 25.1, you would estimate inputs, outputs, inquiries, files, and external interfaces for the CAD software. To compute the *count total* needed in the FP equation:

$$\text{FP}_{\text{estimated}} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

For the purposes of this estimate, the complexity weighting factor is assumed to be average. Table 25.1 presents the results of this estimate, and the FP count total is 320.

To compute a value for $\Sigma(F_i)$, each of the 14 complexity weighting factors listed in Table 25.2 is scored with a value between 0 (not important) and 5 (very important).

The sum of these ratings for the complexity factors $\Sigma(F_i)$ is 52. So the value of the adjustment factor is 1.17:

$$[0.65 + 0.01 \times \Sigma(F_i)] = 1.17$$

Finally, the estimated number of FP is derived:

$$\text{FP}_{\text{estimated}} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)] = 375$$

TABLE 25.1

Estimating information domain values

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	96 (24 × 4 = 96)
Number of external outputs	12	14	22	14	5	70 (14 × 5 = 70)
Number of external inquiries	16	20	28	20	5	100 (20 × 5 = 100)
Number of internal logical files	4	4	5	4	10	40 (4 × 10 = 40)
Number of external interface files	2	2	3	2	7	14 (2 × 7 = 14)
Count total						*320*

TABLE 25.2
Estimating information domain values

Complexity Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5

The organizational average productivity for systems of this type is 6.5 FP/pm. Based on a burdened labor rate of $8,000 per month, the cost per FP is approximately $1,230. Based on the FP estimate and the historical productivity data, the total estimated project cost is $461,000 and the estimated effort is 58 person-months.

25.6.5 An Example of Process-Based Estimation

The most common technique for estimating a project is to base the estimate on the process that will be used. That is, the process is decomposed into a relatively small set of activities, actions, and tasks and the effort required to accomplish each is estimated.

Like the problem-based techniques, process-based estimation begins with a delineation of software functions obtained from the project scope. A series of framework activities must be performed for each function. Functions and related framework activities[11] may be represented as part of a table similar to the one presented in Figure 25.3.

Once problem functions and process activities are melded, you estimate the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function. These data constitute the central matrix of the table in Figure 25.3. Average labor rates (i.e., cost/unit effort) are then applied to the effort estimated for each process activity.

11 The framework activities chosen for this project differ somewhat from the generic activities discussed in Chapter 2. They are customer communication (CC), planning, risk analysis, engineering, and construction/release.

FIGURE 25.3 Process-based estimation table

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function									
↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
Totals	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% effort	1%	1%	1%	8%	45%	10%	36%		

To illustrate the use of process-based estimation, we again consider the CAD software introduced in Section 25.6.3. The system configuration and all software functions remain unchanged and are indicated by project scope.

Referring to the completed process-based table shown in Figure 25.3, estimates of effort (in person-months) for each software engineering activity are provided for each CAD software function (abbreviated for brevity). The engineering and construction release activities are subdivided into the major software engineering tasks shown. Gross estimates of effort are provided for customer communication, planning, and risk analysis. These are noted in the total row at the bottom of the table. Horizontal and vertical totals provide an indication of estimated effort required for analysis, design, code, and test. It should be noted that approximately 53 percent of all effort is expended on front-end engineering tasks (requirements analysis and design), indicating the relative importance of this work.

Based on an average burdened labor rate of $8,000 per month, the total estimated project cost is $368,000 and the estimated effort is 46 person-months. If desired, labor rates could be associated with each framework activity or software engineering task and computed separately.

25.6.6 An Example of Estimation Using Use Case Points

As we have noted throughout Part Two of this book, use cases provide a software team with insight into software scope and requirements. Once use cases have been developed, they can be used to estimate the projected "size" of a software project. Use cases do not address the complexity of the functions and features that are described, and they can describe complex behavior (e.g., interactions) that involve many functions and features. Even with these constraints, it is possible to compute *use case points* (UCPs) in a manner that is analogous to the computation of functions points (Section 25.6).

Cohn [Coh05] indicates that the computation of use case points must take the following characteristics into account:

- The number and complexity of the use cases in the system.
- The number and complexity of the actors on the system.
- Various nonfunctional requirements (such as portability, performance, maintainability) that are not written as use cases.
- The environment in which the project will be developed (e.g., the programming language, the software team's motivation).

To begin, each use case is assessed to determine its relative complexity. A simple use case indicates a simple user interface, a single database, and three or fewer transactions and five or fewer class implementations. An average use case indicates a more complex UI, two or three databases, and four to seven transactions with 5 to 10 classes. Finally, a complex use case implies a complex UI with multiple databases, using eight or more transactions and 11 or more classes. Each use case is assessed using these criteria and the count of each type is weighted by a factor of 5, 10, and 15, respectively. A total *unadjusted use case weight* (UUCW) is the sum of all weighted counts [Nun11].

Next, each actor is assessed. Simple actors are automatons (another system, a machine or device) that communicate through an API. Average actors are automatons that communicate through a protocol or a data store, and complex actors are humans who communicate through a GUI or other human interface. Each actor is assessed using these criteria, and the count of each type is weighted by a factor of 1, 2, and 3, respectively. The total *unadjusted actor weight* (UAW) is the sum of all weighted counts.

These unadjusted values are modified by considering technical complexity factors (TCFs) and environment complexity factors (ECFs). Thirteen factors contribute to an assessment of the final TCF, and eight factors contribute to the computation of the final ECF [Coh05]. Once these values have been determined, the final UCP value is computed in the following manner:

$$\text{UCP} = (\text{UUCW} + \text{UAW}) \times \text{TCF} \times \text{ECF} \tag{25.2}$$

The CAD software introduced in Section 25.6.3 is composed of three subsystem groups: user interface subsystem (includes UICF), engineering subsystem group (includes the 2DGA, 3DGA, and DAM subsystems), and infrastructure subsystem group (includes CGDF and PCF subsystems). Sixteen complex use cases describe the

user interface subsystem. The engineering subsystem group is described by 14 average use cases and 8 simple use cases. And the infrastructure subsystem is described with 10 simple use cases. Therefore,

$$\text{UUCW} = (16 \text{ use cases} \times 15) + [(14 \text{ use cases} \times 10) + (8 \text{ use cases} \times 5)] + (10 \text{ use cases} \times 5) = 470$$

Analysis of the use cases indicates that there are 8 simple actors, 12 average actors, and 4 complex actors. Therefore,

$$\text{UAW} = (8 \text{ actors} \times 1) + (12 \text{ actors} \times 2) + (4 \text{ actors} \times 3) = 44$$

After evaluation of the technology and the environment,

$$\text{TCF} = 1.04$$

$$\text{ECF} = 0.96$$

Using Equation (25.2),

$$\text{UCP} = (470 + 44) \times 1.04 \times 0.96 = 513$$

Using past project data as a guide, the development group has produced 85 LOC per UCP. Therefore, an estimate of the overall size of the CAD project is 43600 LOC. Similar computations can be made for applied effort or project duration.

Using 620 LOC/pm as the average productivity for systems of this type and a burdened labor rate of $8,000 per month, the cost per line of code is approximately $13. Based on the use case estimate and the historical productivity data, the total estimated project cost is $552,000 and the estimated effort is about 70 person-months.

25.6.7 Reconciling Estimates

Any estimation technique, no matter how sophisticated, must be checked by computing at least one other estimate using a different approach. If you have created two or three estimates independently, you now have two or three estimates for cost and effort that need to be compared and reconciled. If both sets of estimates show reasonable agreement, there is good reason to believe that the estimates are reliable. If, on the other hand, the results of these decomposition techniques show little agreement, further investigation and analysis must be conducted.

When your estimates are far apart, you need to reevaluate the information used to make the estimates. Widely divergent estimates can often be traced to one of two causes: (1) the scope of the project is not adequately understood or has been misinterpreted by the planner, or (2) productivity data used for problem-based estimation techniques is inappropriate for the application or has been misapplied. You should determine the cause of divergence and then recompute these estimates.

The estimation techniques discussed in the preceding sections resulted in multiple estimates that should be reconciled to produce a single estimate of effort, project duration, or cost. The total estimated effort for the CAD software (Section 25.6.3) ranges from a low of 46 person-months (derived using a process-based estimation approach) to a high of 68 person-months (derived with use case estimation). The simple average of all four estimates is 56 person-months. But is this the best approach when the high and low estimates are 21 person-months apart?

One approach may be to compute a weighted average, based on calling a high estimate a pessimistic estimate, a low estimate an optimistic estimate, and an in-between value a most likely value. A three-point or expected value can then be computed. The *expected value* for the estimation variable (size) S can be computed as a weighted average of the optimistic (s_{opt}), most likely (s_m), and pessimistic (s_{pess}) estimates. For example,

$$S = \frac{s_{opt} + 4s_m + s_{pess}}{6} \tag{25.3}$$

gives heaviest credence to the "most likely" estimate and follows a beta probability distribution. We assume that there is a very small probability the actual size result will fall outside the optimistic or pessimistic values.

Once the expected value for the estimation variable has been determined, historical productivity data should be examined. Do our estimates seem correct? The only reasonable answer to this question is, we can't be sure. Even then, common sense and experience must prevail.

25.6.8 Estimation for Agile Development

Because the requirements for an agile project (Chapter 3) are defined by a set of user stories, it is possible to develop an estimation approach that is informal, reasonably disciplined, and meaningful within the context of project planning for each software increment. Estimation for agile projects uses a decomposition approach that encompasses the following steps:

1. Each user story (the equivalent of a mini use case created at the very start of a project by end users or other stakeholders) is considered separately for estimation purposes.

2. The user story is decomposed into the set of software engineering tasks that will be required to develop it.

3a. Each task is estimated separately. Note: Estimation can be based on historical data, an empirical model, or "experience" (e.g., using a technique like planning poker, Section 7.2.3).

3b. Alternatively, the "volume" of the user story can be estimated in LOC, FP, or some other volume-oriented measure (e.g., use case count).

4a. Estimates for each task are summed to create an estimate for the user story.

4b. Alternatively, the volume estimate for the user story is translated into effort using historical data.

5. The effort estimates for all user stories that are to be implemented for a given software increment are summed to develop the effort estimate for the increment.

Because the project duration required for the development of a software increment is quite short (typically 3 to 6 weeks), this estimation approach serves two purposes: (1) to be certain that the number of scenarios to be included in the increment conforms to the available resources, and (2) to establish a basis for allocating effort as the increment is developed.

25.7 PROJECT SCHEDULING

Software project scheduling is an activity that distributes estimated effort across the planned project duration by allocating the effort to specific software engineering tasks. It is important to note, however, that the schedule evolves over time. During early stages of project planning, a macroscopic schedule is developed. This type of schedule identifies all major process framework activities and the product functions to which they are applied. As the project gets under way, each entry on the macroscopic schedule is refined into a detailed schedule. Here, specific software actions and tasks (required to accomplish an activity) are identified and scheduled.

Although there are many reasons why software is delivered late, most can be traced to one or more of the following root causes:

- An unrealistic deadline established by someone outside the software team and forced on managers and practitioners on the group.
- Changing customer requirements that are not reflected in schedule changes.
- An honest underestimate of the amount of effort and/or the number of resources that will be required to do the job.
- Predictable and/or unpredictable risks that were not considered when the project commenced.
- Technical difficulties that could not have been foreseen in advance.
- Human difficulties that could not have been foreseen in advance.
- Miscommunication among project staff that results in delays.
- A failure by project management to recognize that the project is falling behind schedule and a lack of action to correct the problem.

Aggressive (read "unrealistic") deadlines are an unpleasant fact in the software business. Sometimes such deadlines are demanded for reasons that are legitimate, from the point of view of the person who sets the deadline. But common sense says that legitimacy must also be perceived by the people doing the work.

The estimation methods discussed in this chapter and the scheduling techniques described in this section are often implemented under the constraint of a defined deadline. If best estimates indicate that the deadline is unrealistic, a competent project manager should inform management and all stakeholders of her findings and suggest alternatives to mitigate the damage of missing the deadline.

The reality of a technical project (whether it involves building a virtual world for a video game or developing an operating system) is that hundreds of small tasks must occur to accomplish a larger goal. Some of these tasks lie outside the mainstream and may be completed without worry about the impact on the project completion date. Other tasks lie on the *critical path*. If these "critical" tasks fall behind schedule, the completion date of the entire project is put into jeopardy.

As a project manager, your objective is to define all project tasks, build a network that depicts their interdependencies, identify the tasks that are critical within the network, and then track their progress to ensure that delay is recognized "one day at a time." To accomplish this, you must have a schedule that has been defined at a degree of resolution that allows progress to be monitored and the project to be controlled. The tasks required to achieve a project manager's needs to build a schedule

and track progress should not be performed manually. There are many excellent scheduling tools. A good manager uses them.

25.7.1 Basic Principles

Scheduling for software engineering projects can be viewed from two rather different perspectives. In the first, an end date for release of a computer-based system has already (and irrevocably) been established. The software organization is constrained to distribute effort within the prescribed time frame. The second view of software scheduling assumes that rough chronological bounds have been discussed but that the end date is set by the software engineering organization. Effort is distributed to make best use of resources, and an end date is defined after careful analysis of the software. Unfortunately, the first situation is encountered far more frequently than the second.

Like all other areas of software engineering, a number of basic principles guide software project scheduling:

Compartmentalization. The project must be compartmentalized into a number of manageable activities and tasks. To accomplish compartmentalization, both the product and the process are decomposed.

Interdependency. The interdependency of each compartmentalized activity or task must be determined. Some tasks must occur in sequence, while others can occur in parallel. Some activities cannot commence until the work product produced by another is available. Other activities can occur independently.

Time allocation. Each task to be scheduled must be allocated some number of work units (e.g., person-days of effort). In addition, each task must be assigned a start date and a completion date that are a function of the interdependencies and whether work will be conducted on a full-time or part-time basis.

Effort validation. Every project has a defined number of people on the software team. As time allocation occurs, you must ensure that no more than the allocated number of people has been scheduled at any given time. For example, consider a project that has three assigned software engineers (e.g., three person-days are available per day of assigned effort).[12] On a given day, seven concurrent tasks must be accomplished. Each task requires 0.50 person-days of effort. More effort has been allocated than there are people to do the work.

Defined responsibilities. Every task that is scheduled should be assigned to a specific team member.

Defined outcomes. Every task that is scheduled should have a defined outcome. For software projects, the outcome is normally a work product (e.g., the design of a component) or a part of a work product. Work products are often combined in deliverables.

Defined milestones. Every task or group of tasks should be associated with a project milestone. A milestone is accomplished when one or more work products has been reviewed for quality (Chapter 15) and has been approved.

Each of these principles is applied as the project schedule evolves.

12 In reality, less than 3 person-days of effort are available because of unrelated meetings, sickness, vacation, and a variety of other reasons. For our purposes, however, we assume 100 percent availability.

25.7.2 The Relationship Between People and Effort

There is a common myth that is still believed by many managers who are responsible for software development work: "If we fall behind schedule, we can always add more programmers and catch up later in the project." Unfortunately, adding people late in a project often has a disruptive effect on the project, causing schedules to slip even further. The people who are added must learn the system, and the people who teach them are the same people who were doing the work. While teaching, no work is done, and the project falls further behind.

In addition to the time it takes to learn the system, more people increase the number of communication paths and the complexity of communication throughout a project. Although communication is absolutely essential to successful software development, every new communication path requires additional effort and therefore additional time. If you must add people to a late project, be sure that you've assigned them work that is highly compartmentalized.

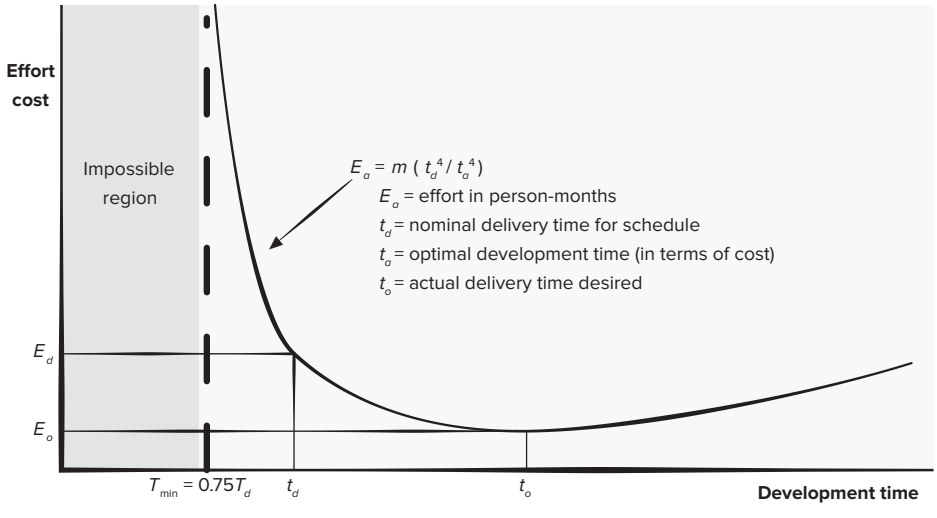
Over the years, empirical data and theoretical analysis have demonstrated that project schedules are elastic. That is, it is possible to compress a desired project completion date (by adding additional resources) to some extent. It is also possible to extend a completion date (by reducing the number of resources).

The *Putnam-Norden-Rayleigh (PNR) curve*[13] provides an indication of the relationship between effort applied and delivery time for a software project. A version of the curve, representing project effort as a function of delivery time, is shown in Figure 25.4. The curve indicates a minimum value t_o that indicates the least cost for delivery (i.e., the delivery time that will result in the least effort expended). As we move left of t_o (i.e., as we try to accelerate delivery), the curve rises nonlinearly.

As an example, we assume that a project team has estimated a level of effort E_d will be required to achieve a nominal delivery time t_d that is optimal in terms of

FIGURE 25.4

The relationship between effort and delivery time



13 Original research can be found in [Nor70] and [Put78].

schedule and available resources. Although it is possible to accelerate delivery, the curve rises very sharply to the left of t_d. In fact, the PNR curve indicates the project delivery time cannot be compressed much beyond $0.75t_d$. If we attempt further compression, the project moves into "the impossible region" and risk of failure becomes very high. The PNR curve also indicates that the lowest cost delivery option $t_o = 2t_d$. The implication here is that delaying project delivery can reduce costs significantly. Of course, this must be weighed against the business cost associated with the delay.

The *software equation* [Put92] introduced is derived from the PNR curve and demonstrates the highly nonlinear relationship between chronological time to complete a project and human effort applied to the project. The number of delivered lines of code (source statements), *L,* is related to effort and development time by the equation:

$$L = P \times E^{1/3} t^{4/3} \tag{25.4}$$

where E is development effort in person-months, P is a productivity parameter that reflects a variety of factors that leads to high-quality software engineering work (typical values for P range between 2000 and 12000), and t is the project duration in calendar months.

Rearranging this software equation, we can arrive at an expression for development effort E:

$$E = \frac{L^3}{P^3 t^4} \tag{25.5}$$

where E is the effort expended (in person-years) over the entire life cycle for software development and maintenance and t is the development time in years. The equation for development effort can be related to development cost by the inclusion of a burdened labor rate factor ($/person-year).

This leads to some interesting results. As a project deadline becomes tighter and tighter, you reach a point at which the work cannot be completed on schedule, regardless of the number of people doing the work. Face reality and define a new delivery date.

Consider also a complex, real-time software project estimated at 33000 LOC, 12 person-years of effort. If eight people are assigned to the project team, the project can be completed in approximately 1.3 years. If, however, we extend the end date to 1.75 years, the highly nonlinear nature of the model described in Equation (25.5) yields:

$$E = \frac{L^3}{P^3 t^4} \sim 3.8 \text{ person-years}$$

This implies that, by extending the end date by 6 months, we can reduce the number of people from eight to four! The validity of such results is open to debate, but the implication is clear: Benefit can be gained by using fewer people over a somewhat longer time span to accomplish the same objective.

25.8 Defining a Project Task Set

Regardless of the process model that is chosen, the work that a software team performs is achieved through a set of tasks that enable you to define, develop, and ultimately support computer software. No single task set is appropriate for all projects. The set

of tasks that would be appropriate for a large, complex system would likely be perceived as overkill for a small, relatively simple software product. Therefore, an effective software process should define a collection of task sets, each designed to meet the needs of different types of projects.

As we noted in Chapter 2, a task set is a collection of software engineering work tasks, milestones, work products, and quality assurance filters that must be accomplished to complete a particular project. The task set must provide enough discipline to achieve high software quality. But, at the same time, it must not burden the project team with unnecessary work.

To develop a project schedule, a task set must be distributed on the project time line. The task set will vary depending upon the project type and the degree of rigor with which the software team decides to do its work. Many factors influence the task set to be chosen. These include [Pre05]: size of the project, number of potential users, mission criticality, application longevity, stability of requirements, ease of customer/developer communication, maturity of applicable technology, performance constraints, embedded and nonembedded characteristics, project staff, and reengineering factors. When taken in combination, these factors provide an indication of the *degree of rigor* with which the software process should be applied.

25.8.1 A Task Set Example

A *concept development project* is initiated when the potential for some new technology must be explored. There is no certainty that the technology will be applicable, but a customer (e.g., marketing) believes that potential benefit exists. Concept development projects are approached by applying the following task set:

1.1 **Concept scoping** determines the overall scope of the project.

1.2 **Preliminary concept planning** establishes the organization's ability to undertake the work implied by the project scope.

1.3 **Technology risk assessment** evaluates the risk associated with the technology to be implemented as part of the project scope.

1.4 **Proof of concept** demonstrates the viability of a new technology in the software context.

1.5 **Concept implementation** implements the concept representation in a manner that can be reviewed by a customer and is used for "marketing" purposes when a concept must be sold to other customers or management.

1.6 **Customer reaction** to the concept solicits feedback on a new technology concept and targets specific customer applications.

A quick scan of these tasks should yield few surprises. In fact, the software engineering flow for concept development projects (and for all other types of projects as well) is little more than common sense.

25.8.2 Refinement of Major Tasks

The major tasks (i.e., software engineering actions) described in the preceding section may be used to define a macroscopic schedule for a project. However, the macroscopic schedule must be refined to create a detailed project schedule. Refinement begins by

taking each major task and decomposing it into a set of subtasks (with related work products and milestones).

As an example of task decomposition, consider Task 1.1, Concept Scoping. Task refinement can be accomplished using an outline format, but in this book, a process design language approach is used to illustrate the flow of the concept scoping activity:

```
Task definition: Task 1.1 Concept Scoping
1.1.1  Identify need, benefits and potential customers;
1.1.2  Define desired output/control and input events that drive the
       application;
       Begin Task 1.1.2
       1.1.2.1 TR: Review written description of need[14]
       1.1.2.2 Derive a list of customer visible outputs/inputs
       1.1.2.3 TR: Review outputs/inputs with customer and revise as
               required;
       endtask Task 1.1.2
1.1.3  Define the functionality/behavior for each major function;
       Begin Task 1.1.3
       1.1.3.1 TR: Review output and input data objects derived in task 1.1.2;
       1.1.3.2 Derive a model of functions/behaviors;
       1.1.3.3 TR: Review functions/behaviors with customer and revise as
               required;
       endtask Task 1.1.3
1.1.4  Isolate those elements of the technology to be implemented in software;
1.1.5  Research availability of existing software;
1.1.6  Define technical feasibility;
1.1.7  Make quick estimate of size;
1.1.8  Create a Scope Definition;
endtask definition: Task 1.1
```

The tasks and subtasks noted in the process design language refinement form the basis for a detailed schedule for the concept scoping activity.

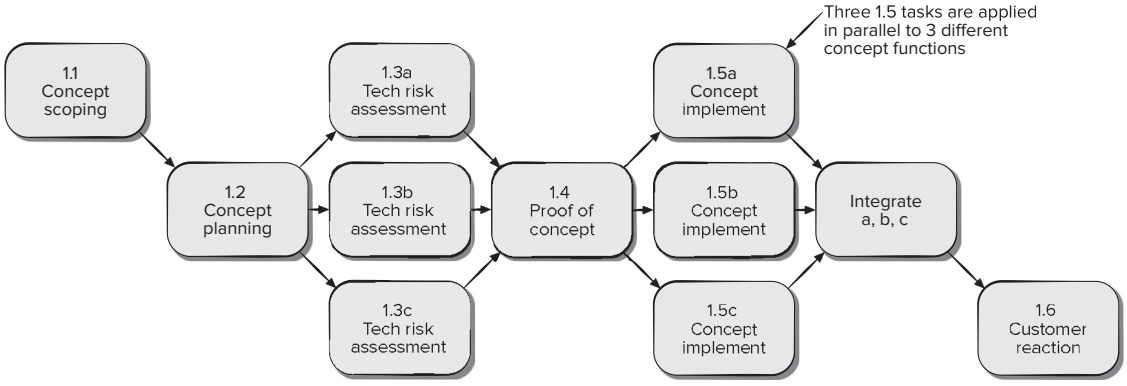
25.9 DEFINING A TASK NETWORK

Individual tasks and subtasks have interdependencies based on their sequence. In addition, when more than one person is involved in a software engineering project, it is likely that development activities and tasks will be performed in parallel. When this occurs, concurrent tasks must be coordinated so that they will be complete when later tasks require their work product(s).

A *task network,* also called an *activity network,* is a graphic representation of the task flow for a project. The task network is a useful mechanism for depicting intertask dependencies and determining the critical path. It is sometimes used as the mechanism through which task sequence and dependencies are input to an automated project scheduling tool. In its simplest form (used when creating a macroscopic schedule), the task network depicts major software engineering tasks. Figure 25.5 shows a schematic task network for a concept development project.

14 TR indicates that a technical review (Chapter 16) is to be conducted.

FIGURE 25.5 A task network for concept development



The concurrent nature of software engineering activities leads to a number of important scheduling requirements. Because parallel tasks occur asynchronously, you should determine intertask dependencies to ensure continuous progress toward completion. In addition, you should be aware of those tasks that lie on the *critical path.* That is, tasks that must be completed on schedule if the project as a whole is to be completed on schedule. These issues are discussed in more detail later in this chapter.

It is important to note that the task network shown in Figure 25.5 is macroscopic. In a detailed task network (a precursor to a detailed schedule), each activity shown in the figure would be expanded. For example, Task 1.1 would be expanded to show all tasks detailed in the refinement of Tasks 1.1 shown in Section 25.8.2.

25.10 SCHEDULING

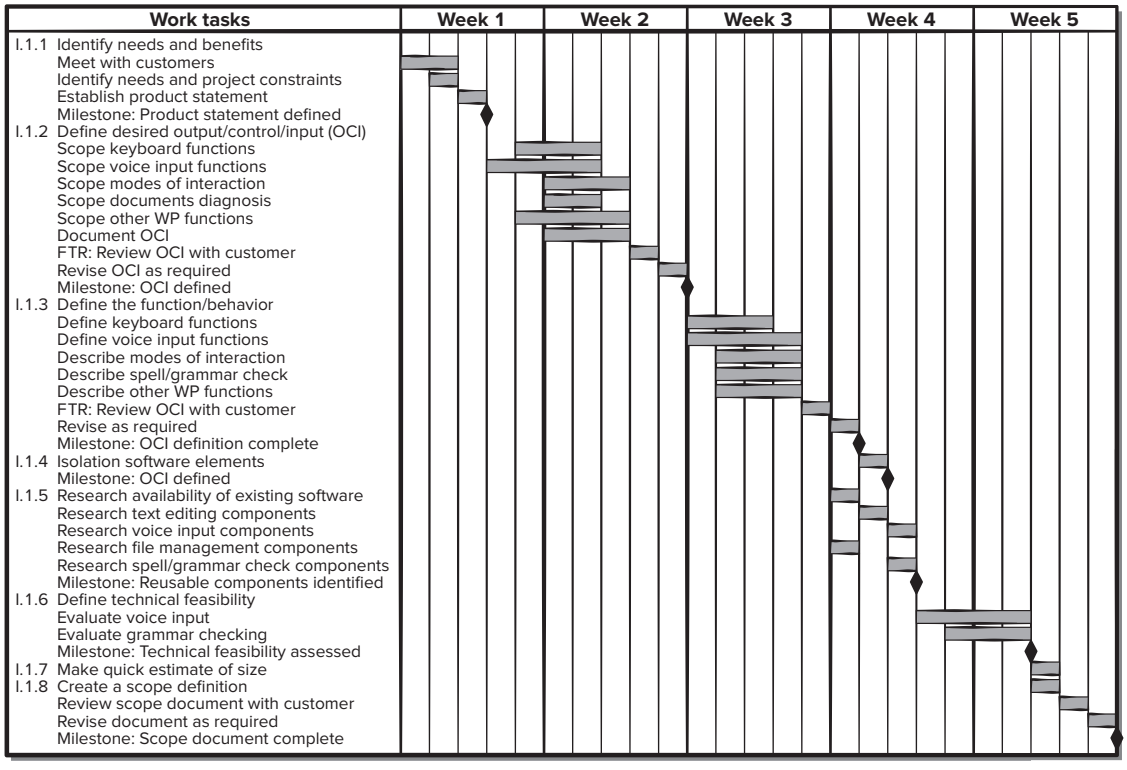
Scheduling of a software project does not differ greatly from scheduling of any multitask engineering effort. Therefore, generalized project scheduling tools and techniques can be applied with little modification for software projects [Fer14]. Interdependencies among tasks may be defined using a task network. Tasks, sometimes called the project *work breakdown structure* (WBS), are defined for the product as a whole or for individual functions.

Project scheduling tools allow you to (1) determine the critical path—the chain of tasks that determines the duration of the project, (2) establish "most likely" time estimates for individual tasks by applying statistical models, and (3) calculate "boundary times" that define a time "window" for a particular task [Ker17].

25.10.1 Time-Line Charts

When creating a software project schedule, you begin with a set of tasks (the work breakdown structure). If automated tools are used, the work breakdown is input as a

FIGURE 25.6 An example time-line chart



task network or task outline. Effort, duration, and start date are then input for each task. In addition, tasks may be assigned to specific individuals.

As a consequence of this input, a *time-line chart,* also called a *Gantt chart,* is generated. A time-line chart can be developed for the entire project. Alternatively, separate charts can be developed for each project function or for each individual working on the project [Toc18].

Figure 25.6 illustrates the format of a time-line chart. It depicts a part of a software project schedule that emphasizes the **concept scoping** task for a word-processing (WP) software product. All project tasks (for concept scoping) are listed in the left-hand column. The horizontal bars indicate the duration of each task. When multiple bars occur at the same time on the calendar, task concurrency is implied. The diamonds indicate milestones.

Once the information necessary for the generation of a time-line chart has been input, the majority of software project scheduling tools produce *project tables*—a tabular listing of all project tasks, their planned and actual start and end dates, and a variety of related information (Figure 25.7). Used in conjunction with the time-line chart, project tables enable you to track progress.

FIGURE 25.7 An example project table

Work tasks	Planned start	Actual start	Planned complete	Actual complete	Assigned person	Effort allocated	Notes
I.1.1 Identify needs and benefits							Scoping will require more effort/time
Meet with customers	wk1, d1	wk1, d1	wk1, d2	wk1, d2	BLS	2 p-d	
Identify needs and project constraints	wk1, d2	wk1, d2	wk1, d2	wk1, d2	JPP	1 p-d	
Establish product statement	wk1, d3	wk1, d3	wk1, d3	wk1, d3	BLS/	1 p-d	
Milestone: Product statement defined	wk1, d3	wk1, d3	wk1, d3	wk1, d3			
I.1.2 Define desired output/control/input (OCI)							
Scope keyboard functions	wk1, d4	wk1, d4	wk2, d2		BLS	1.5 p-d	
Scope voice input functions	wk1, d3	wk1, d3	wk2, d2		JPP	2 p-d	
Scope modes of interaction	wk2, d1		wk2, d3		MLL	1 p-d	
Scope documents diagnosis	wk2, d1		wk2, d2		BLS	1.5 pd	
Scope other WP functions	wk1, d4	wk1, d4	wk2, d3		JPP	2 p-d	
Document OCI	wk2, d1		wk2, d3		MLL	3 p-d	
FTR: Review OCI with customer	wk2, d3		wk2, d3		all	3 p-d	
Revise OCI as required	wk2, d4		wk2, d4		all	3 p-d	
Milestone: OCI defined	wk2, d5		wk2, d5				
I.1.3 Define the Function/behavior							

25.10.2 Tracking the Schedule

If it has been properly developed, the project schedule becomes a road map that defines the tasks and milestones to be tracked and controlled as the project proceeds. Tracking can be accomplished in a number of different ways:

- Conducting periodic project status meetings in which each team member reports progress and problems.
- Evaluating the results of all reviews conducted throughout the software engineering process.
- Determining whether formal project milestones (the diamonds shown in Figure 25.7) have been accomplished by the scheduled date.
- Comparing the actual start date to the planned start date for each project task listed in the resource table (Figure 25.8).
- Meeting informally with practitioners to obtain their subjective assessment of progress to date and problems on the horizon.
- Tracking the project velocity, which is a way of seeing how quickly the development team is clearing the user story backlog (Section 3.5).

In reality, all these tracking techniques are used by experienced project managers.

A software project manager employs control to administer project resources, cope with problems, and direct project staff. If things are going well (i.e., the project is on schedule and within budget, reviews indicate that real progress is being made and milestones are being reached), control is light. But when problems occur, you must exercise control to reconcile them as quickly as possible. After a problem has been diagnosed, additional resources may be focused on the problem area: staff may be redeployed or the project schedule can be redefined.

When faced with severe deadline pressure, experienced project managers sometimes use a project scheduling and control technique called *time-boxing* [Jal04]. The time-boxing strategy recognizes that the complete product may not be deliverable by the predefined deadline.

The tasks associated with each increment are then time-boxed. This means that the schedule for each task is adjusted by working backward from the delivery date for the increment. A "box" is put around each task. When a task hits the boundary of its time box (plus or minus 10 percent), work stops and the next task begins.

Time-boxing is often associated with agile incremental process models (Chapter 4), and a schedule is derived for each incremental delivery. These tasks become part of the increment schedule and are allocated over the increment development schedule. They can be input to scheduling software (e.g., Microsoft *Project*) and used for tracking and control.

The initial reaction to the time-boxing approach is often negative: "If the work isn't finished, how can we proceed?" The answer lies in the way work is accomplished. By the time the time-box boundary is encountered, it is likely that 90 percent of the task has been completed.[15] The remaining 10 percent, although important, can (1) be delayed until the next increment or (2) be completed later if required. Rather than becoming "stuck" on a task, the project proceeds toward the delivery date.

SAFEHOME



Tracking the Schedule

The scene: Doug Miller's office prior to the initiation of the *SafeHome* software project.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug (glancing at a PowerPoint slide): The schedule for the first *SafeHome* increment seems reasonable, but we're going to have trouble tracking progress.

Vinod (a concerned look on his face): Why? We have tasks scheduled on a daily basis, plenty of work products, and we've been sure that we're not over allocating resources.

Doug: All good, but how do we know when the requirements model for the first increment is complete?

Jamie: Things are iterative, so that's difficult.

Doug: I understand that, but . . . well, for instance, take "analysis classes defined." You indicated that as a milestone.

Vinod: We have.

Doug: Who makes that determination?

Jamie (aggravated): They're done when they're done.

Doug: That's not good enough, Jamie. We have to schedule TRs [technical reviews, Chapter 16], and you haven't done that. The successful completion of a review on the analysis model, for instance, is a reasonable milestone. Understand?

Jamie (frowning): Okay, back to the drawing board.

Doug: It shouldn't take more than an hour to make the corrections . . . everyone else can get started now.

15 A cynic might recall the saying: "The first 90 percent of the system takes 90 percent of the time; the remaining 10 percent of the system takes 90 percent of the time."

25.11 SUMMARY

A software project planner must estimate three things before a project begins: how long it will take, how much effort will be required, and how many people will be involved. In addition, the planner must predict the resources (hardware and software) that will be required and the risk involved.

The statement of scope helps the planner to develop estimates using one or more techniques that fall into two broad categories: decomposition and empirical modeling. Decomposition techniques require a delineation of major software functions, followed by estimates of either (1) the number of LOC, (2) selected values within the information domain, (3) the number of use cases, (4) the number of person-months required to implement each function, or (5) the number of person-months required for each software engineering activity. Empirical techniques use empirically derived expressions for effort and time to predict these project quantities. Automated tools can be used to implement a specific empirical model.

Accurate project estimates generally use at least two of the three techniques just noted. By comparing and reconciling estimates developed using different techniques, the planner is more likely to derive an accurate estimate. Software project estimation can never be an exact science, but a combination of good historical data and systematic techniques can improve estimation accuracy.

Scheduling is the culmination of a planning activity that is a primary component of software project management. When combined with estimation methods and risk analysis, scheduling establishes a road map for the project manager.

Scheduling begins with process decomposition. The characteristics of the project are used to adapt an appropriate task set for the work to be done. A task network depicts each engineering task, its dependency on other tasks, and its projected duration. The task network is used to compute the critical path, a time-line chart, and a variety of project information. Using the schedule as a guide, you can track and control each step in the software process.

PROBLEMS AND POINTS TO PONDER

25.1. Assume that you are the project manager for a company that builds software for household robots. You have been contracted to build the software for a robot that mows the lawn for a homeowner. Write a statement of scope that describes the software. Be sure your statement of scope is bounded. If you're unfamiliar with robots, do a bit of research before you begin writing. Also, state your assumptions about the hardware that will be required. Alternate: Replace the lawn-mowing robot with another problem that is of interest to you.

25.2. Do a functional decomposition of the robot software you described in Problem 25.1. Estimate the size of each function in LOC. Assuming that your organization produces 450 LOC/pm with a burdened labor rate of $7,000 per person-month, estimate the effort and cost required to build the software using the LOC-based estimation technique described in this chapter.

25.3. Develop a spreadsheet model that implements one or more of the estimation techniques described in this chapter. Alternatively, acquire one or more online models for estimation from Web-based sources.

25.4. It seems odd that cost and schedule estimates are developed during software project planning—before detailed software requirements analysis or design has been conducted. Why do you think this is done? Are there circumstances when it should not be done?

25.5. What is the difference between a macroscopic schedule and a detailed schedule? Is it possible to manage a project if only a macroscopic schedule is developed? Why?

25.6. The relationship between people and time is highly nonlinear. Using Putnam's software equation (described in Section 25.8.2), develop a table that relates number of people to project duration for a software project requiring 50000 LOC and 15 person-years of effort (the productivity parameter is 5000 and $B = 0.37$). Assume that the software must be delivered in 24 months plus or minus 12 months.

25.7. Assume that a university has contracted you to develop an online course registration system (OLCRS). First, act as the customer (if you're a student, that should be easy) and specify the characteristics of a good system. (Alternatively, your instructor will provide you with a set of preliminary requirements for the system.) Using the estimation methods discussed in this chapter, develop an effort and duration estimate for OLCRS. Suggest how you would:

a. Define parallel work activities during the OLCRS project.
b. Distribute effort throughout the project.
c. Establish milestones for the project.

25.8. Select an appropriate task set for the OLCRS project.

25.9. Define a task network for OLCRS described in Problem 25.8, or alternatively, for another software project that interests you. Be sure to show tasks and milestones and to attach effort and duration estimates to each task. If possible, use an automated scheduling tool to perform this work.

25.10. Using a scheduling tool (if available) or paper and pencil (if necessary), develop a time-line chart for the OLCRS project.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

26 RISK MANAGEMENT

In his book on risk analysis and management, Robert Charette [Cha89] notes that "risk concerns future happenings." He correctly points out that concern about today and yesterday should not be the focus, but rather poses the pivotal question: ". . . by changing our actions today, [can we] create an opportunity for a different and hopefully better situation for ourselves tomorrow." The implication is that "risk involves choice," and that introduces uncertainty in our actions.

KEY CONCEPTS

assessment 545
fuzzy logic 539
gamification 544
identification 535
proactive strategies 533
projection 538
reactive strategies 533
refinement 542
risk categories 534
risk exposure 540
risk item checklist 546
risk table 538
RMMM 546
safety and hazards 545
technical debt 533



QUICK LOOK

What is it? A risk is a potential problem for a software project—it might happen, it might not. But, regardless of the outcome, it's a really good idea to identify it, assess its probability of occurrence, estimate its impact, and establish a contingency plan.

Who does it? Everyone involved in the software process—managers, software engineers, and other stakeholders—participates in risk analysis and management.

Why is it important? Think about the Scout motto: "Be prepared." Software is a difficult undertaking. Lots of things can go wrong, and frankly, many often do. Understanding the risks and taking proactive measures to avoid or manage them—is a key element of good software project management. A project without a risk management plan can find itself in serious trouble that could have been avoided if the project team had addressed its development risks in a more systematic manner and followed its plans.

What are the steps? Recognizing what can go wrong is the first step, called "risk identification." Next, each risk is analyzed to determine the likelihood that it will occur and the damage that it will do if it does occur. Once this information is established, risks are ranked, by probability and impact. Finally, a plan is developed to manage those risks that have high probability and high impact.

What is the work product? A risk mitigation, monitoring, and management (RMMM) plan or a set of risk information sheets is produced.

How do I ensure that I've done it right? The risks that are analyzed and managed should be derived from a thorough study of the people, the product, the process, and the project. The RMMM should be revisited as the project proceeds to ensure that risks are kept up to date. Contingency plans for risk management should be realistic.

When you consider risk in the context of software engineering, Charette's conceptual underpinnings are always in evidence. The future is your concern—what risks might cause the software project to go awry? Change is your concern—how will changes in customer requirements, development technologies, target environments, and all other entities connected to the project affect timeliness and overall success? Last, you must grapple with choices—what methods and tools should you use, how many people should be involved, how much emphasis on quality is "enough"?

Technical debt is the term used to describe costs associated with putting off activities like software documentation and refactoring. Technical debt that is not paid can result in a delivered software product with inadequate functionality, erratic behavior, poor quality, insufficient documentation, and unnecessary complexity. Technical debt implies that the costs (effort, time, and resources) of dealing with technical issues can be reduced if problems are dealt with earlier rather than later during project development. Like financial interest, technical debt increases with time because unrecognized problems introduced early compound. Accumulating technical debt is mortgaging a project's future [Fai17].

Simply moving to agile software development does not remove the need to do intentional risk management. Elbanna and Sarker [Elb16] conducted a survey of several organizations that made extensive use of agile software development practices. They found several development risks that seemed to go unmanaged in agile projects. Technical debt was likely to accumulate as developers pushed for more new code and at the same time often forgot to spend time reducing this debt. In addition, inexperienced agile teams tend to produce more defects than they might have following a more controlled development model. Agile teams may be more likely to use nonstandardized project management and testing tools. These autonomous teams may not be documenting their decision-making processes adequately. That can doom developers to repeat past errors on future projects. None of these risks is impossible to control, as long as software developers are aware of them and make plans to manage them during their time-boxed sprints.

26.1 Reactive Versus Proactive Risk Strategies

Reactive risk strategies have been laughingly called the "Indiana Jones school of risk management" [Tho92]. In the movies that carried his name, Indiana Jones, when faced with overwhelming difficulty, would invariably say, "Don't worry, I'll think of something!" Never worrying about problems until they happened, Indy would react in some heroic way.

Sadly, the average software project manager is not Indiana Jones and the members of the software project team are not his trusty sidekicks. Yet, most software teams continue to rely solely on reactive risk strategies. At best, a reactive strategy monitors the project for likely risks. Resources are set aside to deal with them, should they become actual problems. More commonly, the software team does nothing about risks until something goes wrong. Then, the team flies into action trying to correct the problem rapidly. This is often called a *fire-fighting mode.* When this fails, "crisis management" [Cha92] takes over and the project is in real jeopardy.

A considerably more intelligent strategy for risk management is to be proactive. A *proactive* risk strategy begins long before technical work is initiated. Potential risks are identified, their probability and impact are assessed, and they are ranked by importance. Then, the software team establishes a plan for managing risk. The primary objective is to avoid risk, but because not all risks can be avoided, the team works to develop a contingency plan that will enable it to respond in a controlled and effective manner. Proactive risk management is one of the software engineering tools that can be used to reduce technical debt. Throughout the remainder of this chapter, we discuss a proactive strategy for risk management.

26.2 SOFTWARE RISKS

Although there has been considerable debate about the proper definition for software risk, there is general agreement that risk always involves two characteristics: *uncertainty*—the risk may or may not happen; that is, there are no 100 percent probable risks[1]—and *loss*—if the risk becomes a reality, unwanted consequences or losses will occur [Hig95]. When risks are analyzed, it is important to quantify the level of uncertainty and the degree of loss associated with each risk. To accomplish this, different categories of risks are considered.

Project risks threaten the project plan. That is, if project risks become real, it is likely that the project schedule will slip and that costs will increase. Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, stakeholder, and requirements problems and their impact on a software project. In Chapter 25, project complexity, size, and the degree of structural uncertainty were also defined as project (and estimation) risk factors.

Technical risks threaten the quality and timeliness of the software to be produced. If a technical risk becomes a reality, implementation may become difficult or impossible. Technical risks identify potential design, implementation, interface, verification, and maintenance problems. In addition, specification ambiguity, technical uncertainty, technical obsolescence, and "leading-edge" technology can also be risk factors. Technical risks occur because the problem is harder to solve than you thought it would be.

Business risks threaten the viability of the software to be built and often jeopardize the project or the product. Candidates for the top five business risks are (1) building an excellent product or system that no one really wants (market risk), (2) building a product that no longer fits into the overall business strategy for the company (strategic risk), (3) building a product that the sales force doesn't understand how to sell (sales risk), (4) losing the support of senior management due to a change in focus or a change in people (management risk), and (5) losing budgetary or personnel commitment (budget risks).

It is extremely important to note that simple risk categorization won't always work. Some risks are simply unpredictable in advance.

1 A risk that is 100 percent probable is a constraint on the software project.

Another general categorization of risks has been proposed by Charette [Cha89]. *Known risks* are those that can be uncovered after careful evaluation of the project plan, the business and technical environment in which the project is being developed, and other reliable information sources (e.g., unrealistic delivery date, lack of documented requirements or software scope, poor development environment). *Predictable risks* are extrapolated from past project experience (e.g., staff turnover, poor communication with the customer, dilution of staff effort as ongoing maintenance requests are serviced). *Unpredictable risks* are the joker in the deck. They can and do occur, but they are extremely difficult to identify in advance.

INFO



Seven Principles of Risk Management

The Software Engineering Institute (SEI) (**www.sei.cmu.edu**) identifies seven principles that "provide a framework to accomplish effective risk management." They are:

Maintain a global perspective. View software risks within the context of a system in which it is a component and the business problem that it is intended to solve.

Take a forward-looking view. Think about the risks that may arise in the future (e.g., due to changes in the software); establish contingency plans so that future events are manageable.

Encourage open communication. If someone states a potential risk, don't discount it. If a risk is proposed in an informal manner, consider it. Encourage all stakeholders and users to suggest risks at any time.

Integrate. A consideration of risk must be integrated into the software process.

Emphasize a continuous process. The team must be vigilant throughout the software process, modifying identified risks as more information is known and adding new ones as better insight is achieved.

Develop a shared product vision. If all stakeholders share the same vision of the software, it is likely that better risk identification and assessment will occur.

Encourage teamwork. The talents, skills, and knowledge of all stakeholders should be pooled when risk management activities are conducted.

26.3 RISK IDENTIFICATION

Risk identification is a systematic attempt to specify threats to the project plan (estimates, schedule, resource loading, etc.). By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.

There are two distinct types of risks for each of the categories that have been presented in Section 26.2. *Generic risks* are a potential threat to every software project. *Product-specific risks* can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the software that is to be built. Although generic risks are important to consider, it's the product-specific risks that cause the most headaches. Be certain to invest the time to identify as many product-specific risks as possible. To identify product-specific risks, you

should begin by examining the project plan and the software statement of scope and then develop an answer to the question, "What special characteristics of this product may threaten our project plan?"

One method for identifying risks is to create a risk item checklist. The checklist can be used for risk identification and focuses on some subset of known and predictable risks in the following generic subcategories:

- **Product size.** Risks associated with the overall size of the software to be built or modified.
- **Business impact.** Risks associated with constraints imposed by management or the marketplace.
- **Stakeholder characteristics.** Risks associated with the sophistication of the stakeholders and the developer's ability to communicate with stakeholders in a timely manner.
- **Process definition.** Risks associated with the degree to which the software process has been defined and is followed by the development organization.
- **Development environment.** Risks associated with the availability and quality of the tools to be used to build the product.
- **Technology to be built.** Risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- **Staff size and experience.** Risks associated with the overall technical and project experience of the software engineers who will do the work.

The risk item checklist can be organized in different ways. Questions relevant to each of the topics can be answered for each software project. The answers to these questions allow you to estimate the impact of risk. A different risk item checklist format simply lists characteristics that are relevant to each generic subcategory. Finally, a set of "risk components and drivers" [AFC88] are listed along with their probability of occurrence. Drivers for performance, support, cost, and schedule are discussed in answer to later questions.

Several comprehensive checklists for software project risk are available on the Web (e.g., [Arn11], [NAS07]). You can use these checklists to gain insight into generic risks for software projects. In addition to the use of checklists, *risk patterns* ([Mil04], [San17]) have been proposed as a systematic approach to risk identification.

26.3.1 Assessing Overall Project Risk

So how can we determine if the software project we're working on is at serious risk? The following questions have been derived from risk data obtained by surveying experienced software project managers in different parts of the world [Kei98]. The questions are ordered by their relative importance to the success of a project.

1. Have top software and customer managers formally committed to support the project?

2. Are end users enthusiastically committed to the project and the system/product to be built?
3. Are requirements fully understood by the software engineering team and its customers?
4. Have customers been involved fully in the definition of requirements?
5. Do end users have realistic expectations?
6. Is the project scope stable?
7. Does the software engineering team have the right mix of skills?
8. Are project requirements stable?
9. Does the project team have experience with the technology to be implemented?
10. Is the number of people on the project team adequate to do the job?
11. Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built?

If any one of these questions is answered negatively, mitigation, monitoring, and management steps should be instituted without fail. The degree to which the project is at risk is directly proportional to the number of negative responses to these questions.

26.3.2 Risk Components and Drivers

The U.S. Air Force [AFC88] has published a pamphlet that contains excellent guidelines for software risk identification and abatement. The Air Force approach requires that the project manager identify the risk drivers that affect software risk components—performance, cost, support, and schedule. In the context of this discussion, the risk components are defined in the following manner:

- **Performance risk.** The degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- **Cost risk.** The degree of uncertainty that the project budget will be maintained.
- **Support risk.** The degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- **Schedule risk.** The degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

The impact of each risk driver on the risk component is divided into one of four impact categories—negligible, marginal, critical, or catastrophic. Boehm [Boe89] suggests that negligible impact results only in inconvenience. The additional cost required to mitigate the impact is very low. Marginal impact might affect secondary mission objectives or requirements but would not impact overall mission success. Cost would be somewhat higher but manageable. Critical impact would directly affect system performance and some or all requirements and would bring mission success into question. Cost to mitigate would be high. Finally, catastrophic impact would result in mission failure. The cost of mitigation would be unacceptable.

26.4 Risk Projection

Risk projection, also called *risk estimation,* attempts to rate each risk in two ways—(1) the likelihood or probability that the risk is real and will occur and (2) the consequences of the problems associated with the risk, should it occur. You work along with other managers and technical staff to perform four risk projection steps:

1. Establish a scale that reflects the perceived likelihood of a risk.
2. Delineate the consequences of the risk.
3. Estimate the impact of the risk on the project and the product.
4. Assess the overall accuracy of the risk projection so that there will be no misunderstandings.

The intent of these steps is to consider risks in a manner that leads to prioritization. No software team has the resources to address every possible risk with the same degree of rigor. By prioritizing risks, you can allocate resources where they will have the most impact.

26.4.1 Developing a Risk Table

A risk table provides you with a simple technique for risk projection.[2] A sample risk table is illustrated in Figure 26.1.

You begin by listing all risks (no matter how remote) in the first column of the table. This can be accomplished with the help of the risk item checklists referenced

FIGURE 26.1 Sample risk table prior to sorting

Risk	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet exceptions	TR	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	

Impact values:
1 – catastrophic
2 – critical
3 – marginal
4 – negligible

2 The risk table can be implemented as a spreadsheet model. This enables easy manipulation and sorting of the entries.

in Section 26.3. Each risk is categorized in the second column (e.g., PS implies a project size risk, BU implies a business risk). The probability of occurrence of each risk is entered in the next column of the table. The probability value for each risk can be estimated by team members individually. One way to accomplish this is to poll individual team members in round-robin fashion until their collective assessment of risk probability begins to converge.

Another possible starting point would be to look at the risk assessment table for a previous project and see which risks apply to your current project, which do not, and identify risks that should be added. Risk assessment does not always need to be re-created from scratch. Agile developers often work on similar projects and can realize the cost savings by maintaining lists of risk management procedures shared companywide.

Next, the impact of each risk is assessed. Each risk component is assessed using the characterization presented in Figure 26.1, and an impact category is determined. The categories for each of the four risk components—performance, support, cost, and schedule—are averaged[3] to determine an overall impact value.

Once the first four columns of the risk table have been completed, the table is sorted by probability and by impact. High-probability, high-impact risks percolate to the top of the table, and low-probability risks drop to the bottom. This accomplishes first-order risk prioritization.

A project manager can then establish a cutoff line at some row in the table (Figure 26.1). All risks that lie above the cutoff line should be managed. The column labeled RMMM contains a pointer into a *risk mitigation, monitoring, and management plan* or, alternatively, a collection of risk information sheets developed for all risks that lie above the cutoff. The RMMM plan and risk information sheets are discussed in Sections 26.5 and 26.6.

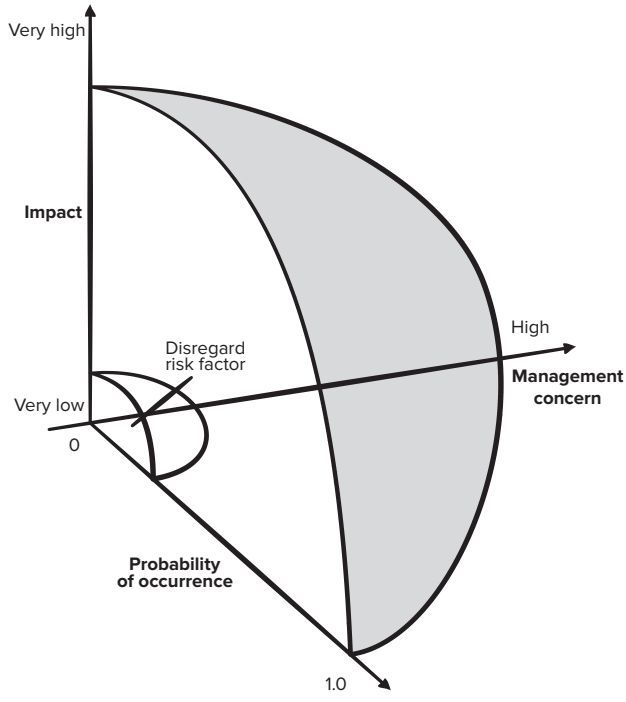
Risk probability can be determined by making individual estimates and then developing a single consensus value. Although this approach is workable, more sophisticated techniques for determining risk probability have been developed (e.g., [McC09]). Recent work makes use of *fuzzy logic*[4] to determine the characteristics that make software projects that are prone to failure. These projects often have multiple interrelated risk factors that are affected by imprecision or uncertainty and require use of expert knowledge and fuzzy logic to better understand the nature of their combined risk impact [Rod16].

Referring to Figure 26.2, risk impact and probability have a distinct influence on management concern. A risk factor that is classified as high impact but has a very low probability of occurrence should not absorb a significant amount of management time. However, high-impact risks with moderate to high probability and low-impact risks with high probability should be carried forward into the risk analysis steps that follow.

3 A weighted average can be used if one risk component has more significance for a project.

4 Fuzzy logic is a type of logic that recognizes more than simple true and false values (unlike the propositional logic you may have studied in a discrete mathematics class). With fuzzy logic, propositions can be represented with degrees of truthfulness and falsehood. For example, the statement, any animal with stripes is a tiger might only be 50% likely to be true. Fuzzy logic has proved to be particularly useful in artificial intelligence applications that involve making decisions involving incomplete or uncertain information.

FIGURE 26.2
Risk and management concern



26.4.2 Assessing Risk Impact

Three factors affect the consequences that are likely if a risk does occur: its nature, its scope, and its timing. The nature of the risk indicates the problems that are likely if it occurs. For example, a poorly defined external interface to customer hardware (a technical risk) will preclude early design and testing and will likely lead to system integration problems late in a project. The scope of a risk combines the severity (just how serious is it?) with its overall distribution (how much of the project will be affected or how many stakeholders are harmed?). Finally, the timing of a risk considers when and for how long the impact will be felt. In most cases, you want the "bad news" to occur as soon as possible, but in some cases, the longer the delay, the better.

Returning once more to the risk analysis approach proposed by the U.S. Air Force [AFC88], you can apply the following steps to determine the overall consequences of a risk: (1) determine the average probability of occurrence value for each risk component; (2) using the discussion presented in Section 26.3.2, determine the impact for each component based on the criteria shown, and (3) complete the risk table and analyze the results as described in the preceding sections.

The overall *risk exposure* (RE), is determined using the following relationship [Hal98]:

$$\text{RE} = P \times C$$

where P is the probability of occurrence for a risk, and C is the cost to the project should the risk occur.

For example, assume that the software team defines a project risk in the following manner:

Risk identification. Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Risk probability. Eighty percent (likely).

Risk impact. Sixty reusable software components were planned. If only 70 percent can be used, 18 components would have to be developed from scratch (in addition to other custom software that has been scheduled for development). Because the average component is 100 LOC and local data indicate that the software engineering cost for each LOC is $14.00, the overall cost (impact) to develop the components would be $18 \times 100 \times 14 = \$25{,}200$.

Risk exposure. $RE = 0.80 \times 25{,}200 \sim \$20{,}200$.

Risk exposure can be computed for each risk in the risk table, once an estimate of the cost of the risk is made. The total risk exposure for all risks (above the cutoff in the risk table) can provide a means for adjusting the final cost estimate for a project. It can also be used to predict the probable increase in staff resources required at various points during the project schedule.

The risk projection and analysis techniques described in Sections 26.4.1 and 26.4.2 are applied iteratively as the software project proceeds.[5] The project team should revisit the risk table at regular intervals, reevaluating each risk to determine when new circumstances cause its probability and impact to change. After completing this activity, the team may decide to add new risks to the table, remove some risks that are no longer relevant, or change the relative positions of still others. The team should compare the RE for all risks to their project cost estimate. If the total RE is greater than 50 percent of the project cost, the viability of the project must be questioned.

SAFEHOME



Risk Analysis

The scene: Doug Miller's office, prior to the initiation of the *SafeHome* software project.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: I'd like to spend some time brainstorming risks for the *SafeHome* project.

Jamie: As in what can go wrong?

Doug: Yep. Here are a few categories where things can go wrong.

5 If you have further interest, a more mathematical treatment of the cost of risk is presented in [Ben10b].

[He shows everyone the categories noted in the introduction to Section 26.3.]

Vinod: Umm . . . do you want us to just call them out, or . . .

Doug: No, here's what I thought we'd do. Everyone make a list of risks . . . right now . . .

[Ten minutes pass, everyone is writing.]

Doug: Okay, stop.

Jamie: But I'm not done!

Doug: That's okay. We'll revisit the list again. Now, for each item on your list, assign a percent likelihood that the risk will occur. Then, assign an impact to the project on a scale of 1 (minor) to 5 (catastrophic).

Vinod: So if I think that the risk is a coin flip, I specify a 50 percent likelihood, and if I think it'll have a moderate project impact, I specify a 3, right?

Doug: Exactly.

[Five minutes pass, everyone is writing.]

Doug: Okay, stop. Now we'll make a group list on the whiteboard. I'll do the writing; we'll call out one entry from your list in round-robin format.

[Fifteen minutes pass; the list is created.]

Jamie (pointing at the board and laughing): Vinod, that risk (pointing toward an entry on the board) is ridiculous. There's a higher likelihood that we'll all get hit by lightning. We should remove it.

Doug: No, let's leave it for now. We consider all risks, no matter how weird. Later we'll winnow the list.

Jamie: But we already have over 40 risks . . . how on earth can we manage them all?

Doug: We can't. That's why we'll define a cut-off after we sort these guys. I'll do that off-line and we'll meet again tomorrow. For now, get back to work . . . and in your spare time, think about any risks that we've missed.

26.5 Risk Refinement

During early stages of project planning, a risk may be stated quite generally. As time passes and more is learned about the project and the risk, it may be possible to refine the risk into a set of more detailed risks, each somewhat easier to mitigate, monitor, and manage.

One way to do this is to represent the risk in *condition-transition-consequence* (CTC) format [Glu94]. That is, the risk is stated in the following form:

> Given that <condition> then there is concern that (possibly) <consequence>.

Using the CTC format for the reuse risk noted in Section 26.4.2, you could write:

> Given that all reusable software components must conform to specific design standards and that some do not conform, then there is concern that (possibly) only 70 percent of the planned reusable modules may be integrated into the as-built system, resulting in the need to custom engineer the remaining 30 percent of components.

This general condition can be refined in the following manner:

Subcondition 1. Certain reusable components were developed by a third party with no knowledge of internal design standards.

Subcondition 2. The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.

Subcondition 3. Certain reusable components have been implemented in a language that is not supported on the target environment.

The consequences associated with these refined subconditions remain the same (i.e., 30 percent of software components must be custom engineered), but the refinement helps to isolate the underlying risks and might lead to easier analysis and response.

26.6 RISK MITIGATION, MONITORING, AND MANAGEMENT

All the risk analysis activities presented to this point have a single goal—to assist the project team in developing a strategy for dealing with risk. An effective strategy must consider three issues: risk avoidance, risk monitoring, and risk management and contingency planning.

If a software team adopts a proactive approach to risk, avoidance is always the best strategy. This is achieved by developing a plan for *risk mitigation.* For example, assume that high staff turnover is noted as a project risk r_1. Based on its history and management intuition, the likelihood l_1 of high turnover is estimated to be 0.70 (70 percent, rather high) and the impact x_1 is projected as critical. That is, high turnover will have a critical impact on project cost and schedule.

To mitigate this risk, you would develop a strategy for reducing turnover. Among the possible steps to be taken are:

- Meet with current staff to determine causes for turnover (e.g., poor working conditions, low pay, competitive job market).
- Mitigate those causes that are under your control before the project starts.
- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
- Organize project teams so that information about each development activity is widely dispersed.
- Define work product standards and establish mechanisms to be sure that all models and documents are developed in a timely manner.
- Conduct peer reviews of all work (so that more than one person is "up to speed").
- Assign a backup staff member for every critical technologist.

As the project proceeds, *risk-monitoring* activities commence. The project manager monitors factors that may provide an indication of whether the risk is becoming more likely or less likely. In the case of high staff turnover, the general attitude of team members based on project pressures, the degree to which the team has jelled, interpersonal relationships among team members, potential problems with compensation and benefits, and the availability of jobs within the company and outside it are all monitored.

In addition to monitoring these factors, a project manager should monitor the effectiveness of risk mitigation steps. For example, a risk mitigation step noted here called

for the definition of work product standards and mechanisms to be sure that work products are developed in a timely manner. This is one mechanism for ensuring continuity, should a critical individual leave the project. The project manager should monitor work products carefully to ensure that each can stand on its own and that each imparts information that would be necessary if a newcomer were forced to join the software team somewhere in the middle of the project.

Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality. Continuing the example, the project is well under way and several people announce that they will be leaving. If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team. In addition, you can temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to "get up to speed." Those individuals who are leaving are asked to stop all work and spend their last weeks in "knowledge transfer mode." This might include video-based knowledge capture, the development of "commentary documents or Wikis," and/or meeting with other team members who will remain on the project.

It is important to note that risk mitigation, monitoring, and management (RMMM) steps incur additional project cost. For example, spending the time to back up every critical technology costs money. Part of risk management, therefore, is to evaluate when the benefits accrued by the RMMM steps are outweighed by the costs associated with implementing them. You need to perform a classic cost-benefit analysis. If RE for a specific risk is less than the cost of risk mitigation, don't try to mitigate the risk but continue to monitor it. If risk aversion steps for high turnover will increase both project cost and duration by an estimated 15 percent, but the predominant cost factor is "backup," management may decide not to implement this step. On the other hand, if the risk aversion steps are projected to increase costs by 5 percent and duration by only 3 percent, management will likely put all into place.

For a large project, 30 or 40 risks may be identified. If between three and seven risk management steps are identified for each, risk management may require significant resources to manage. For this reason, you should adapt the Pareto 80–20 rule to software risk. Experience indicates that 80 percent of the overall project risk (i.e., 80 percent of the potential for project failure) can be accounted for by only 20 percent of the identified risks. The work performed during earlier risk analysis steps will help you to determine which of the risks reside in that 20 percent (e.g., risks that lead to the highest risk exposure). For this reason, some of the risks identified, assessed, and projected may not make it into the RMMM plan—they don't fall into the critical 20 percent (the risks with highest project priority).

Gamification[6] has been suggested as an approach to encouraging software developers to follow process compliance procedures in the areas like quality and risk

6 Deterding et al. defined gamification as the use of game design elements in nongame settings to increase motivation and attention on a task [Det11]. It is important to note that this definition does not refer to the *playing* of games, but rather the use of game design elements in other contexts.

management [Ped14]. A typical gamification approach might award points, badges, or other nonmonetary awards to each developer and make use of a public leader board showing each person's ranking within the development group. If such an approach can be implemented based on automatic data collection (e.g., tracking the number of commitments to the software repository), it can be a cost-effective way of ensuring all team members are monitoring the risk factor measures intended to trigger the mitigation steps required to prevent the risk from becoming a disaster [Baj11]. A word of caution: You must ensure that members of the team are *not* rewarded for doing things like injecting problems into the process, so they can earn badges for mitigating the problems they caused [Bri13].

Risk is not limited to the software project itself. Risks can occur after the software has been successfully developed and delivered to the customer. These risks are typically associated with the consequences of software failure in the field.

Software safety and hazard analysis (e.g., [Fir15], [Har12a], [Lev12]) are software quality assurance activities (Chapter 17) that focus on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software engineering process, software design features can be specified that will either eliminate or control potential hazards.

SAFEHOME



Gamification and Risk Management

The scene: Doug Miller's office, prior to the initiation of the *SafeHome* software project.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: I'd like to spend some time brainstorming ideas for getting all developers on board with monitoring and mitigating the risks we identified for the *SafeHome* project.

Jamie: I thought it was your job to keep track of the project?

Doug: It is, but the development team members are able to see potential problems much quicker than I can, since they're working in the trenches.

Vinod: Working is right. We already have a lot to do to keep this project moving forward . . .

Doug: Exactly, we don't have time to fix problems that could have been prevented by addressing risks early.

Jamie: Okay . . . so what are you thinking?

Doug: I'm thinking since you guys like games, that may be a way to make risk monitoring more like a game than a project task.

Jamie: You know, my buddy—he works at another company—told me about something called "gamification." Sounds like what you're talking about, Doug.

Doug: Yeah, gamification is growing in popularity as a compliance tool in software process areas like quality assurance and risk management.

Vinod: So how can we do this?

Doug: I was thinking that most of the triggers for our risk mitigation tasks are based on the metrics we already have in the project management dashboard. It might just be a question of encouraging developers to check the dashboard on a regular basis.

Jamie: Are we going to use "snoopware" to track the number of times people check?

Doug: Maybe not. Perhaps we can reward the first person who reports a trigger value that indicates something's wrong.

Jamie: My buddy mentioned that they use leader boards to encourage competition. We can do a simple leader board as a Google doc spreadsheet.

Vinod: Some games have badges. We might create badges for the leader board.

Doug: Would people work for badges?

Jamie: They might if they could cash badges in for rewards.

Doug: What kind of rewards?

Jamie: Maybe something like getting first pick on a user story to develop in the current sprint. Or a cash reward if a person's actions save the company a lot of rework costs.

Doug: Let me think some more on this and let you know what I come up with for rewards and a point system for the leader board. We'll meet again tomorrow. For now, think about the most important risks we should be monitoring when we start our first sprint.

26.7 The RMMM Plan

A risk management strategy can be included in the software project plan, or the risk management steps can be organized into a separate *risk mitigation, monitoring, and management plan.* The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan.

Some software teams do not develop a formal RMMM document. Rather, each risk is documented individually using a *risk information sheet* (RIS) [Wil97]. In most cases, the RIS is maintained using a cloud database system so that creation and information entry, priority ordering, searches, and other analysis may be accomplished easily. This approach might lend itself to supporting gamification of the risk management process. It would also facilitate the sharing of the risk information sheets to all the company software teams. The format of the RIS is illustrated in Figure 26.3.

Once RMMM has been documented and the project has begun, risk mitigation and monitoring steps commence. As we have already discussed, risk mitigation is a problem avoidance activity. Risk monitoring is a project tracking activity with three primary objectives: (1) to assess whether predicted risks do, in fact, occur; (2) to ensure that risk aversion steps defined for the risk are being properly applied; and (3) to collect information that can be used for future risk analysis. In many cases, the problems that occur during a project can be traced to more than one risk. Another job of risk monitoring is to attempt to allocate origin [what risk(s) caused which problems throughout the project].

FIGURE 26.3 Risk information sheet

Risk information sheet

Risk ID: P02-4-32	Date: 5 / 9 / 19	Prob: 80%	Impact: high

Description:
Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Refinement/context:
Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards.
Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.
Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.

Mitigation/monitoring:
1. Contact third party to determine conformance with design standards.
2. Press for interface standards completion; consider component structure when deciding on interface protocol.
3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.

Management/contingency plan/trigger:
RE computed to be $20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly.
Trigger: Mitigation steps unproductive as of 7 / 1 / 19.

Current status:
5 / 12 / 19: Mitigation steps initiated.

Originator: D. Gagne	Assigned: B. Laster

26.8 SUMMARY

Whenever a lot is riding on a software project, common sense dictates risk analysis. And yet, most software project managers do it informally and superficially, if they do it at all. The time spent identifying, analyzing, and managing risk pays itself back in many ways—less upheaval during the project, a greater ability to track and control a project, and the confidence that comes with planning for problems before they occur.

Risk analysis can absorb a significant amount of project planning effort. Identification, projection, assessment, management, and monitoring all take time. But the effort is worth it. To quote Sun Tzu, a Chinese general who lived 2,500 years ago, "If you know the enemy and know yourself, you need not fear the result of a hundred battles." For the software project manager, the enemy is risk.

PROBLEMS AND POINTS TO PONDER

26.1. Provide five examples from other fields that illustrate the problems associated with a reactive risk strategy.

26.2. Describe the difference between "known risks" and "predictable risks."

26.3. You're the project manager for a major software company. You have been asked to lead a team seeking to build a breakthrough product that combines virtual reality hardware with state-of-the-art software. Create a risk table for the project.

26.4. Develop a risk mitigation strategy and specific risk mitigation activities for three of the risks noted in Figure 26.1.

26.5. Develop a risk monitoring strategy and specific risk monitoring activities for three of the risks noted in Figure 26.1. Be sure to identify the factors that you'll be monitoring to determine whether the risk is becoming more likely or less likely to occur.

26.6. Develop a risk management strategy and specific risk management activities for three of the risks noted in Figure 26.1.

26.7. Attempt to refine three of the risks noted in Figure 26.1, and then create risk information sheets for each.

26.8. Recompute the risk exposure discussed in Section 26.4.2 when cost/LOC is $16 and the probability is 60 percent.

26.9. Can you think of a situation in which a high-probability, high-impact risk would not be considered as part of your RMMM plan?

26.10. Describe five software application areas in which software safety and hazard analysis would be a major concern.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

27 A Strategy for Software Support

Regardless of its application domain, its size, or its complexity, computer software will evolve over time. Change drives this process. For computer software, change occurs when errors are corrected, when the software is adapted to a new environment, when customers request new features or functions, and when the application is reengineered to provide benefit in a modern context. Software support actually begins when the developers involve stakeholders in the requirements gathering and prototype evolution process (Figure 27.1). Software support ends with the decision to retire the system from active use.

KEY CONCEPTS

document restructuring 564
forward engineering 565
inventory analysis 563
maintainability 553
maintenance tasks 554
refactoring 560
  architecture 561
  code 561
  data 561
release management 550
reverse engineering 555
  data 555
  processing 556
  user interfaces 557
software evolution 562
software maintenance 552
software reengineering 562
software support 550
supportability 552



QUICK LOOK

What is it? Software support encompasses a set of activities that correct bugs, adapt the software to changes in its environment, enhance the software based on stakeholder requests, and reengineer the software to achieve better functionality and performance. During these activities, quality must be ensured and change must be controlled.

Who does it? At an organizational level, support staff from the software engineering organization perform all support activities. User training, bug report management, performing warranty repairs, and continuing to manage customer relations may be handled by other specialized teams.

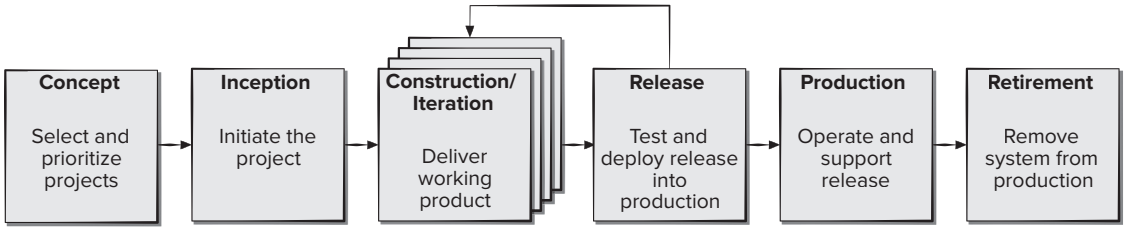
Why is it important? Software exists within a rapidly changing technology and business environment. That's why software must be maintained continually, and at the appropriate time, reengineered to keep pace.

What are the steps? Software support incorporates a maintenance function that corrects defects, adapts the software to meet a changing environment, and enhances functionality to meet the evolving needs of customers. At a strategic level, the support team works with stakeholders to examine the existing business goals for the software product and creates a revised software product to better meet the revised business goals. Software evolution using reengineering creates versions of existing programs that exhibit higher quality and better maintainability.

What is the work product? A variety of maintenance and reengineering work products (e.g., use cases, analysis and design models, test procedures) are produced. The final output is upgraded software that is easier to maintain and better meets the needs of its users.

How do I ensure that I've done it right? Use the same software quality and change management practices that are applied in every software engineering process.

FIGURE 27.1 Software prototype evolution process model



Over the past 40 years, Manny Lehman (e.g., [Leh97a]) and his colleagues have performed detailed analyses of industry-grade software and systems in an effort to develop a *unified theory for software evolution.* The details of this work are beyond the scope of this book, but a brief mention of some of these laws [Leh97b] is worthwhile:

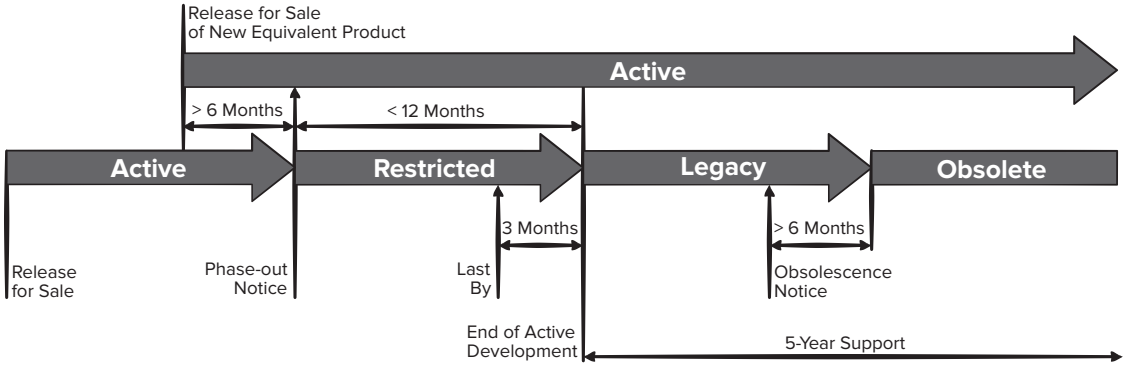
> **Law of continuing change (1974).** Software that has been implemented in a real-world computing context and will therefore evolve over time (called *E-type systems*) must be continually adapted else they become progressively less satisfactory.
>
> **Law of increasing complexity (1974).** As an E-type system evolves its complexity increases unless work is done to maintain or reduce it.
>
> **Law of conservation of familiarity (1980).** As an E-type system evolves all associated with it, developers, sales personnel, users, for example, must maintain knowledge of its content and behavior to achieve satisfactory evolution. Excessive growth diminishes that knowledge. Hence the average incremental growth remains invariant as the system evolves.
>
> **Law of continuing growth (1980).** The functional content of E-type systems must be continually increased to maintain user satisfaction over their lifetime.
>
> **Law of declining quality (1996).** The quality of E-type systems will appear to be declining unless they are rigorously maintained and adapted to operational environment changes.

The laws that Lehman and his colleagues have defined are an inherent part of a software engineer's reality. In this chapter, we'll discuss the challenges of software support including maintenance and evolution activities that are required to extend the effective life of legacy systems.

27.1 SOFTWARE SUPPORT

Software support can be considered an umbrella activity that includes many activities we have already discussed in this book: change management (Chapter 22), proactive risk management (Chapter 26), process management (Chapter 25), configuration management (Chapter 22), quality assurance (Chapter 17), and release management (Chapter 4). *Release management* is the process that brings high-quality code changes from a developer's workspace to the end user, encompassing code change integration, continuous integration, build system specifications, infrastructure-as-code, and

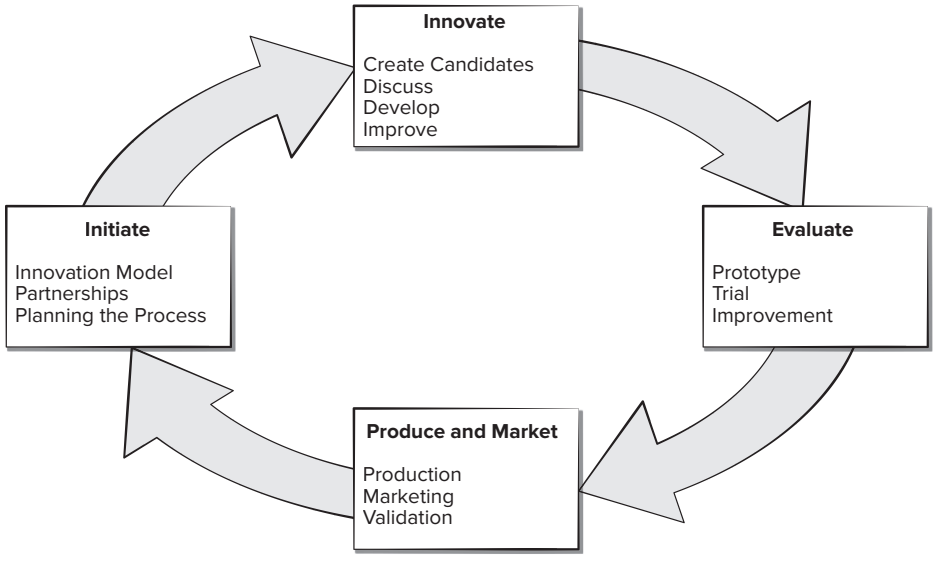
FIGURE 27.2 Software release and retirement example



deployment and release [Ada16]. Ultimately software needs to be retired. Figure 27.2 provides an example of a time line showing the release and retirement of a software product.

To effectively support industry-grade software, your organization (or its designee) must be capable of making the corrections, adaptations, and enhancements that are part of the maintenance activity. But in addition, the organization must provide other important support activities that include ongoing operational support, end-user support, and reengineering activities over the complete life of the software. Figure 27.3 shows one model for supporting software after it is released.

FIGURE 27.3
Iterative software support model



A reasonable definition of *software supportability* is

> . . . the capability of supporting a software system over its whole product life. This implies satisfying any necessary needs or requirements, but also the provision of equipment, support infrastructure, additional software, facilities, labor, or any other resource required to maintain the software operational and capable of satisfying its function. [SSO08]

In essence, supportability is one of many quality factors that should be considered during the analysis and design actions that are part of the software process. It should be addressed as part of the requirements model (or specification) and considered as the design evolves and construction commences. There should be some consideration for how long the software will be maintained before it is replaced by a new product.

For example, the need to "antibug" software at the component and code level has been discussed previously in this book. The software should contain facilities to assist support personnel when a defect is encountered in the operational environment (and make no mistake, defects *will* be encountered). In addition, support personnel should have access to a database that contains records of all defects that have already been encountered—their characteristics, cause, and cure. This will enable support personnel to examine "similar" defects and may provide a means for more rapid diagnosis and correction.

Although defects encountered in an application are a critical support issue, supportability also demands that resources be provided to support day-to-day end-user issues. The job of end-user support personnel is to answer user queries about the installation, operation, and use of the application.

27.2 SOFTWARE MAINTENANCE

Maintenance begins almost immediately. Software is released to end users, and within days, bug reports filter back to the software engineering organization. Within weeks, one class of users indicates that the software must be changed so that it can accommodate the special needs of their environment. And within months, another corporate group that wanted nothing to do with the software when it was released now recognizes that it may provide unexpected benefit. They'll need a few enhancements to make it work in their world.

The challenge of software maintenance has begun. You're faced with a growing queue of bug fixes, adaptation requests, and outright enhancements that must be planned, scheduled, and ultimately accomplished. Before long, the queue has grown long, and the work it implies threatens to overwhelm the available resources. As time passes, your organization finds that it's spending more money and time maintaining existing programs than it is engineering new applications. In fact, it's not unusual for a software organization to expend as much as 60 to 70 percent of all resources on software maintenance for products that have been in active use for several years.

As we noted in Chapter 22, the ubiquitous nature of change underlies all software work. Change is inevitable when computer-based systems are built; therefore, you must develop mechanisms for evaluating, controlling, and making modifications.

Throughout this book, we've emphasized the importance of understanding the problem (analysis) and developing a well-structured solution (design). In fact, Part 2 of the book is dedicated to the mechanics of these software engineering actions, and Part 3 focuses on the techniques required to be sure you've done them correctly. Both analysis and design lead to an important software characteristic that we call maintainability. In essence, *maintainability* is a qualitative indication[1] of the ease with which existing software can be corrected, adapted, or enhanced. Much of what software engineering is about is building systems that exhibit high maintainability.

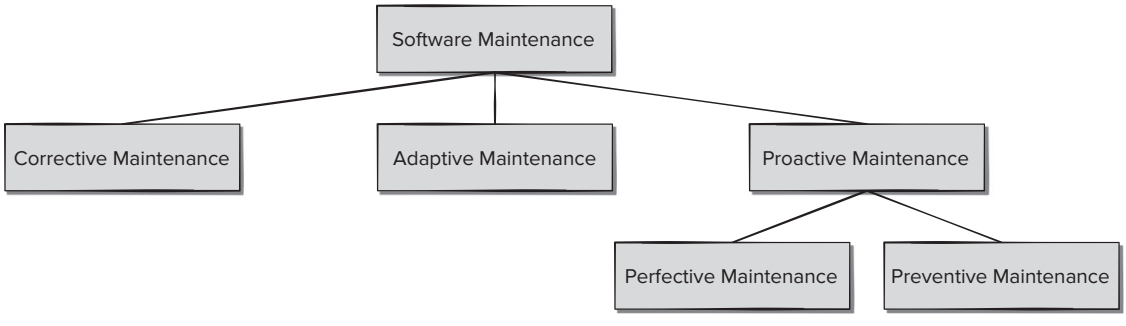
But what is maintainability? Maintainable software exhibits effective modularity (Chapter 9). It makes use of design patterns (Chapter 14) that allow ease of understanding. It has been constructed using well-defined coding standards and conventions, leading to source code that is self-documenting and understandable. It has undergone a variety of quality assurance techniques (Part 3 of this book) that have uncovered potential maintenance problems before the software is released. It has been created by software engineers who recognize that they may not be around when changes must be made. Therefore, the design and implementation of the software must "assist" the person who is making the change.

27.2.1 Maintenance Types

In Chapter 4 we discussed the four types of maintenance shown in Figure 27.4. It is clear that corrective and adaptive maintenance do not add new functionality. It is likely that new functionality will be added to the software during perfective maintenance and possibly during preventative maintenance as well.

In this chapter, we will discuss three broad classes of software maintenance that are relevant to the software support process: reverse engineering, software refactoring, and software evolution or reengineering. Reverse engineering is the process of analyzing a software system to identify the system's components and their interrelationships and to create representations of the system in another form or at a higher level of abstraction (Section 27.2.2). Often reverse engineering is used to rediscover system

FIGURE 27.4 Types of software maintenance



1 There are many quantitative measures that provide an indirect indication of maintainability (e.g., [Sch99], [SEI02]).

design elements and redocument them prior to modifying the system source code. Refactoring is the process of changing a software system in such a way that it does not alter its external behavior but improves its internal structure. Refactoring is often used to improve the quality of a software product and make it easier to understand and easier to maintain (Section 27.4). Reengineering (evolution) of software is the process of taking an existing software system and generating a new system from it that has the same quality as software created using modern software engineering practices [Osb90]. Reengineering and software evolution are discussed in Section 27.5.

27.2.2 Maintenance Tasks

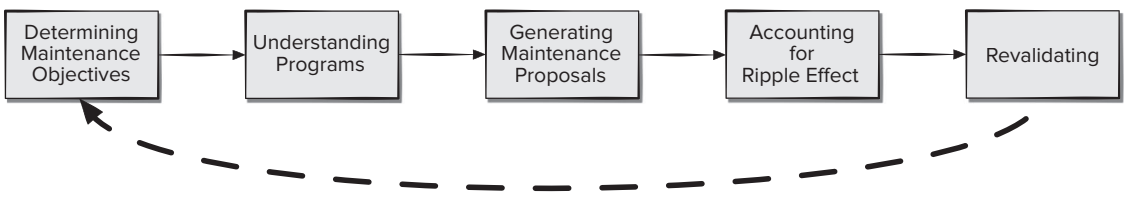
The scenario is all too common: An application has served the business needs of a company for 10 or 15 years. During this time period it has been corrected, adapted, and enhanced many times. People approached this work with the best intentions, but good software engineering practices were always shunted to the side (due to the urgency of other matters). Now the application is unstable. It still works, but every time a change is attempted, unexpected and serious side effects occur. Yet the application must continue to evolve. What to do?

Unmaintainable software is not a new problem. In fact, the broadening emphasis on software evolution and reengineering (Section 27.5) has been spawned by software maintenance problems that have been building for almost half a century. Figure 27.5 shows a set of generic tasks that should be completed as part of a controlled software maintenance process.

Agile process models similar to the one we described in Chapter 4 deliver incremental prototypes in 4-week sprints. It can be argued that agile developers are in perpetual software support mode as they add new stakeholder requested features in every software increment. However, it is important to realize that software development is not maintenance. It is advisable for separate groups of engineers to handle these two tasks. Heeager and Rose [Hee15] suggest nine heuristics to help the maintenance become more agile.

1. Use sprints to organize the maintenance work. You should balance the goal of keeping customers happy with the technical needs of the developers.
2. Allow for urgent customer requests to interrupt scheduled maintenance sprints, by including time for them during maintenance sprint planning.
3. Facilitate team learning by ensuring that more experienced developers are able to mentor less experienced team members even when working on their own discrete tasks.

FIGURE 27.5 Software maintenance tasks



4. Allow multiple team members to accept customer requests as they arise and coordinate their processing with the other maintenance team members.
5. Balance the use of written documentation with face-to-face communication to ensure planning meeting time is used wisely.
6. Write informal use cases to supplement the other documentation being used for communications with stakeholders.
7. Have developers test each other's work (both defect repairs and new feature implementations). This allows for shared learning and improves the feelings of product ownership by the team members.
8. Make sure developers are empowered to share knowledge with one another. This can motivate people to improve the skills and knowledge (allows developers to learn new things, improves their professional skills, and distributes tasks more evenly).
9. Keep planning meetings short, frequent, and focused.

27.2.3 Reverse Engineering

The first task that needs to be completed by software engineers before responding to any maintenance request is to understand the system that needs to be modified. Sadly, the system being maintained often has low quality and lacks reasonable documentation. This is what technical debt is all about. Technical debt is often caused by developers adding features without documenting them or considering their impact on the larger software system.

Reverse engineering can be used to extract design information from source code, but the abstraction level of this information, the completeness of the documentation, and the degree to which human analysts work comfortably with the available tools, are highly variable. Reverse engineering conjures an image of the "magic slot." You feed a haphazardly designed, undocumented source file into the slot and out the other end comes a complete design description (and full documentation) for the computer program. Unfortunately, the magic slot doesn't exist.

Reverse engineering requires developers to evaluate the old software system by examining its (often undocumented) source code, developing a meaningful specification of the processing being performed, the user interface that was used, and the program data structures or associated database.

Reverse Engineering to Understand Data. This occurs at different levels of abstraction and is often the first reengineering task. In some cases, the first reverse engineering activity attempts to construct a UML class diagram. At the program level, internal program data structures must often be reverse engineered as part of an overall reengineering effort. At the system level, global data structures (e.g., files, databases) are often reengineered to accommodate new database management paradigms (e.g., the move from flat file to relational or object-oriented database systems). Reverse engineering of the current global data structures sets the stage for the introduction of a new systemwide database.

Internal Data Structures. Reverse engineering techniques for internal program data focus on the definition of object classes. This is accomplished by examining the

program code with the intent of grouping related program variables together. In many cases, the data organization within the code suggests several abstract data types. For example, record structures, files, lists, and other data structures often provide initial suggestions for possible classes.

Database Structure. Regardless of its logical organization and physical structure, a database allows the definition of data objects and supports some method for establishing relationships among the objects. Therefore, reengineering one database schema into another requires an understanding of existing objects and their relationships.

The following steps [Pre94] may be used to define the existing data model as a precursor to reengineering a new database model: (1) build an initial object model, (2) determine candidate keys (the attributes are examined to determine whether they are used to point to another record or table; those that serve as pointers become candidate keys), (3) refine the tentative classes, (4) define generalizations, and (5) discover associations using techniques that are analogous to the CRC approach. Once information defined in the preceding steps is known, a series of transformations [Pre94] can be applied to map the old database structure into a new database structure.

Reverse engineering to understand processing begins with an attempt to understand and then extract procedural abstractions represented by the source code. To understand procedural abstractions, the code is analyzed at varying levels of abstraction: system, program, component, pattern, and statement.

The overall functionality of the entire application must be understood before more detailed reverse engineering work occurs. This establishes a context for further analysis and provides insight into interoperability issues among applications within a larger system. Each of the programs that make up the system represents a functional abstraction at a high level of detail. A block diagram, representing the interaction between these functional abstractions, is created. Each component performs some subfunction and represents a defined procedural abstraction. A processing narrative for each component is developed. In some situations, system, program, and component specifications already exist. When this is the case, the specifications are reviewed for conformance to existing code.[2]

Things become more complex when the code inside a component is considered. You should look for sections of code that represent generic procedural patterns. In almost every component, a section of code prepares data for processing (within the module), a different section of code does the processing, and another section of code prepares the results of processing for export from the component. Within each of these sections, you can encounter smaller patterns; for example, data validation and bounds checking often occur within the section of code that prepares data for processing.

For large systems, reverse engineering is generally accomplished using a semi-automated approach. Automated tools can be used to help you understand the semantics of existing code. The output of this process is then passed to restructuring and forward engineering tools to complete the reengineering process.

2 Often, specifications written early in the life history of a program are never updated. As changes are made, the code no longer conforms to the specification.

Reverse engineering to understand user interfaces may need to be done as part of the maintenance task. Sophisticated GUIs have become *de rigueur* for computer-based products and systems of every type. Therefore, the redevelopment of user interfaces has become one of the most common types of reengineering activity. But before a user interface can be rebuilt, reverse engineering should occur.

To fully understand an existing user interface, the structure and behavior of the interface must be specified. Merlo and his colleagues [Mer93] suggest three basic questions that must be answered as reverse engineering of the UI commences:

- What are the basic actions (e.g., keystrokes and mouse clicks) that the interface must process?
- What is a compact description of the behavioral response of the system to these actions?
- What is meant by a "replacement," or more precisely, what concept of equivalence of interfaces is relevant here?

Behavioral modeling notation (Chapter 9) can provide a means for developing answers to the first two questions. Much of the information necessary to create a behavioral model can be obtained by observing the external manifestation of the existing interface. But additional information necessary to create the behavioral model must be extracted from the code.

It is important to note that a replacement GUI may not mirror the old interface exactly (in fact, it may be radically different). It is often worthwhile to develop a new interaction metaphor. For example, an old UI that prompts the user to provide a scale factor (ranging from 1 to 10) to shrink or magnify a graphical image. A reengineered GUI might use a touch-screen gesture to accomplish the same function.

27.3 PROACTIVE SOFTWARE SUPPORT

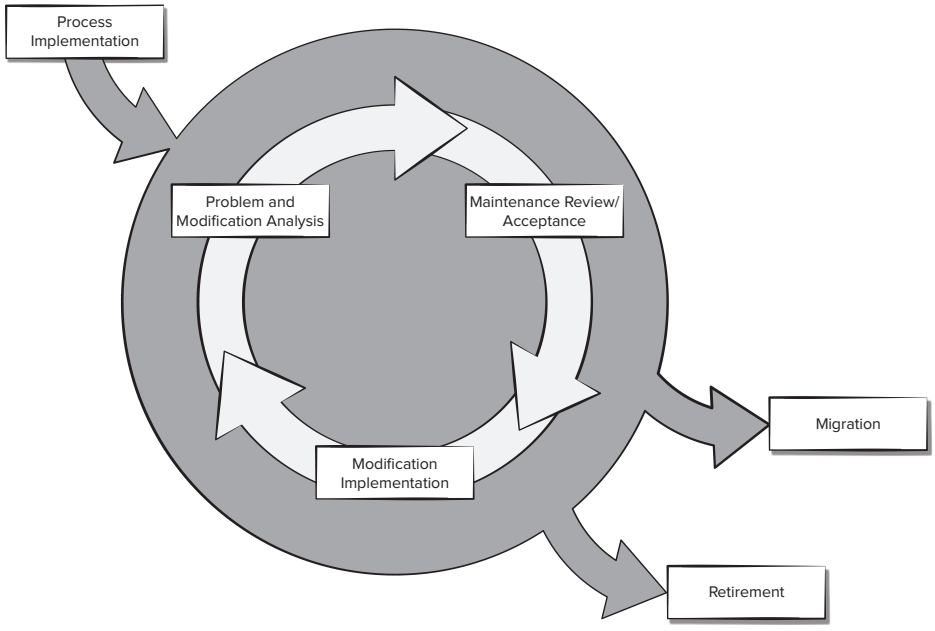
We described the differences between reactive and proactive risk management in Chapter 26. We also described preventative maintenance and perfective maintenance as being proactive maintenance activities (Section 27.2.1). If the goal of software engineering is to deliver high-quality products that meet customer needs in a timely and cost-effective manner, then software support, like other software engineering activities, requires the use of managed workflow processes so that unnecessary rework is avoided.

Supporting software means adapting it to meet changing customer demands and repairing defects reported by end users. This work may be required by law, by contractual agreement, or via product warranty. Repairing software can be a time-consuming and costly process, and it's important to anticipate problems and schedule the work required to respond to customer concerns before they become emergencies. *Proactive software support* requires software engineers to create tools and processes that can help them identify and resolve software issues before they become problems. A generic process for proactive software maintenance and support is shown in Figure 27.6.

The proactive support process is similar to risk monitoring and mitigation (Section 26.6). Developers need to search for indicators that suggest their software

FIGURE 27.6

Software maintenance and support process



product may have quality problems. Sometimes these problems can be addressed by evolving the product and migrating it to a newer version (Section 27.5). Sometimes the software can be restructured or refactored (Section 27.4) to improve its quality and make it easier to maintain. In some cases, the problems are so severe that the developer will need to make plans to retire the product and begin creating a replacement product before the customers abandon it.

27.3.1 Use of Software Analytics

There are currently three dominant uses for artificial intelligence methods in software engineering work [Har12b]: probabilistic reasoning, machine learning and prediction, and search-based software engineering. Probabilistic reasoning techniques can be used to model software reliability (Section 17.7.2). Machine learning can be used to automate the process of discovering root causes of software failures before they occur by predicting the presence of defects likely to cause these failures (Section 15.4.3). Search-based software engineering may be used to assist developers in identifying useful test cases to make regression testing more effective (Section 20.3). All of these AI applications make use of software analytics similar to those we discussed in Chapter 23.

For analytics to be useful they must be actionable, which means expending the effort to determine which measures are worth collecting because of their predictive value and which are not. Port and Tabor [Por18] suggest that analytics can be used to estimate: defect discovery rates based on estimates of yet undiscovered defects present in the product, time between defect discoveries during operation, and effort required to repair defects. Having an understanding of these can allow for better

planning in terms of the cost and time that should be allocated for maintaining the system once it is released to the end users for active use. It is important to keep in mind that even the best estimates contain elements of guessing, so unanticipated failures may still occur.

Zhang and her colleagues [Zha13] report several lessons learned when using software analytics for proactive maintenance tasks.

1. Be sure you are using analytics to identify meaningful development problems, or you will get no buy-in from the software engineers.
2. The analytics must make use of application domain knowledge to be useful to developers (this implies the use of experts to validate the analytics).
3. Developing analytics requires iterative and timely feedback from the intended users.
4. Make sure the analytics are scalable to larger problems and customizable to incorporate new discoveries made over time.
5. Evaluation criteria used needs to be correlated to real software engineering practices.

Mining of the historical information housed in software repositories is a popular way of obtaining the training information needed for the AI techniques mentioned earlier [Sun15]. Using this discovered knowledge helps developers to target their software support actions. Additional discussion of the use of analytics and data science appears in Appendix 2 to this book.

27.3.2 Role of Social Media

Many online stores such as Google Play or Apple's App Store allow users to provide feedback on the apps by posting ratings or comments. The feedback found in these reviews may contain usage scenarios, bug reports, or feature requests. Mining these reports using natural language processing and machine learning techniques can help developers identify potential maintenance and software evolution tasks [Sun15]. However, much of this information is unstructured, and there is so much of it that it is hard to make sense of it without the use of automated statistical tools to reduce the sheer volume of information and create actionable analytics to guide support decisions.

Many companies maintain Facebook pages or Twitter feeds to support their user communities. Some companies encourage their software product users to send program crash information for analysis by the support team members. Still other companies pursue the questionable practice of tracking how and where their products are being used by their customers without their knowledge. It is very easy to collect a lot of user information automatically. Software engineers must resist the temptation to make use of this information in unethical ways.

27.3.3 Cost of Support

In a perfect world, every unmaintainable program would be retired immediately, to be replaced by high-quality, reengineered applications developed using modern software engineering practices. But we live in a world of limited resources. Software evolution and maintenance tasks drain resources that can be used for other business

purposes. Therefore, before an organization attempts to modify or replace an existing application, it should perform a cost-benefit analysis.

A cost-benefit analysis model for reengineering has been proposed by Sneed [Sne95]. Nine parameters are defined:

P_1 = current annual maintenance cost for an application
P_2 = current annual operations cost for an application
P_3 = current annual business value of an application
P_4 = predicted annual maintenance cost after reengineering
P_5 = predicted annual operations cost after reengineering
P_6 = predicted annual business value after reengineering
P_7 = estimated reengineering costs
P_8 = estimated reengineering calendar time
P_9 = reengineering risk factor ($P_9 = 1.0$ is nominal)
L = expected life of the system

The cost associated with continuing maintenance of a candidate application (i.e., reengineering is not performed) can be defined as

$$C_{\text{maint}} = [P_3 - (P_1 + P_2)] \times L \tag{27.1}$$

The costs associated with reengineering are defined using the following relationship:

$$C_{\text{reeng}} = P_6 - (P_4 + P_5) \times (L - P_8) - (P_7 \times P_9) \tag{27.2}$$

Using the costs presented in Equations (27.1) and (27.2), the overall benefit of reengineering can be computed as

$$\text{Cost benefit} = C_{\text{reeng}} - C_{\text{maint}} \tag{27.3}$$

The cost-benefit analysis presented in these equations can be performed for all high-priority applications identified as candidates to evolve or retire (Section 27.5). Those applications that show the highest cost-benefit can be targeted for proactive maintenance or evolution, while work on other applications can be postponed until resources are available.

27.4 REFACTORING

Software refactoring (also known as restructuring) modifies source code and/or data in an effort to make it amenable to future changes. In general, refactoring does not modify the overall program architecture. It tends to focus on the design details of individual modules and on local data structures defined within modules. If the refactoring effort extends beyond module boundaries and encompasses the software architecture, restructuring becomes forward engineering (Section 27.5).

Refactoring occurs when the basic architecture of an application is solid, even though technical internals need work. It is initiated when major parts of the software are serviceable and only a subset of all modules and data need extensive modification.[3]

3 It is sometimes difficult to make a distinction between extensive refactoring and evolution. Both are reengineering.

27.4.1 Data Refactoring

Before data refactoring can begin, a reverse engineering activity called *source code analysis* should be conducted. All programming language statements that contain data definitions, file descriptions, I/O, and interface descriptions are evaluated. The intent is to extract data items and objects, to get information on data flow, and to understand the existing data structures that have been implemented. This activity is sometimes called *data analysis*.

Once data analysis has been completed, *data redesign* commences. In its simplest form, a *data record standardization* step clarifies data definitions to achieve consistency among data item names or physical record formats within an existing data structure or file format. Another form of redesign, called *data name rationalization,* ensures that all data naming conventions conform to local standards and that aliases are eliminated as data flow through the system.

When refactoring moves beyond standardization and rationalization, physical modifications to existing data structures are made to make the data design more effective. This may mean a translation from one file format to another, or in some cases, translation from one type of database to another.

27.4.2 Code Refactoring

Code refactoring is performed to yield a design that produces the same function but with higher quality than the original program. The objective is to take "spaghetti-bowl" code and derive a design that conforms to the quality factors discussed in Chapters 15 and 17.

Other restructuring techniques have also been proposed for use with refactoring tools. One approach might rely on the use of anti-patterns (Section 14.5), both to identify bad code design practices and suggest possible solutions to reduce coupling and improve cohesion [Bro98]. Although code refactoring can alleviate immediate problems associated with debugging or small changes, it is not reengineering. Real benefit is achieved only when data and architecture are refactored as well.

27.4.3 Architecture Refactoring

We made the point in Chapter 10 that making architectural changes to a software product already in production can be a costly and time-consuming process. However, when a program with control flow that looks like the graphic equivalent of a bowl of spaghetti, with "modules" that are 2,000 statements long, with few meaningful comment lines in 290,000 source statements and no other documentation must be modified to accommodate changing user requirements, it may be desirable to consider architectural refactoring as one of the design trade-offs. In general, for a messy program like this you have the following options:

1. You can struggle through modification after modification, fighting the ad hoc design and tangled source code to implement the necessary changes.
2. You can attempt to understand the broader inner workings of the program in an effort to make modifications more effectively.
3. You can revise (redesign, recode, and test) those portions of the software that require modification, applying a meaningful software engineering approach to all revised segments.

4. You can completely redo (redesign, recode, and test) the complete program, using reengineering tools to assist in understanding the current design.

There is no single "correct" option. Circumstances may dictate the first option even if the others are more desirable.

Rather than waiting until a maintenance request is received, the development or support organization uses the results of inventory analysis to select a program that (1) will remain in use for a preselected number of years, (2) is currently being used successfully, and (3) is likely to undergo major modification or enhancement in the near future. Then, option 2, 3, or 4 is applied. We will discuss software evolution and reengineering in Section 27.5.

27.5 SOFTWARE EVOLUTION

At first glance, the suggestion that you redevelop a large program when a working version already exists may seem quite extravagant. Reengineering takes time, it has significant cost, and it absorbs resources that might be otherwise occupied on immediate concerns. For all these reasons, reengineering is not accomplished in a few months or even a few years.

Reengineering of software systems is an activity that will absorb software engineering resources for many years. That's why every organization needs a pragmatic strategy for software reengineering. If time and resources are in short supply, you might consider applying the Pareto principle to the software that is to be reengineered and apply the reengineering process to the 20 percent of the software that accounts for 80 percent of the problems.

Before passing judgment, consider the following arguments. The cost to maintain one line of source code may be 20 to 40 times the cost of initial development of that line. In addition, redesign of the software architecture (program and/or data structure), using modern design concepts, can greatly facilitate future maintenance. Automated tools for reengineering or software evolution will make some part of the job easier. Because a prototype of the software already exists, development productivity should be much higher than average. The user now has experience with the software. Therefore, new requirements and the direction of change can be ascertained with greater ease. At the end of this evolutionary preventive maintenance, the developers will end up with a complete software configuration (documents, programs, and data).

Reengineering is a rebuilding activity. To better understand it, consider an analogous activity: the rebuilding of a house. Consider the following situation. You've purchased a house in another state. You've never actually seen the property, but you acquired it at an amazingly low price, with the warning that it might have to be completely rebuilt. How would you proceed?

- Before you can start rebuilding, it would seem reasonable to inspect the house. To determine whether it is in need of rebuilding, you (or a professional inspector) would create a list of criteria so that your inspection would be systematic.
- Before you tear down and rebuild the entire house, be sure that the structure is weak. If the house is structurally sound, it may be possible to "remodel" without rebuilding (at much lower cost and in much less time).

- Before you start rebuilding, be sure you understand how the original was built. Take a peek behind the walls. Understand the wiring, the plumbing, and the structural internals. Even if you trash them all, the insight you'll gain will serve you well when you start construction.
- If you begin to rebuild, use only the most modern, long-lasting materials. This may cost a bit more now, but it will help you to avoid expensive and time-consuming maintenance later.
- If you decide to rebuild, be disciplined about it. Use practices that will result in high quality—today and in the future.

Although these principles focus on the rebuilding of a house, they apply equally well to the evolving computer-based systems and applications.

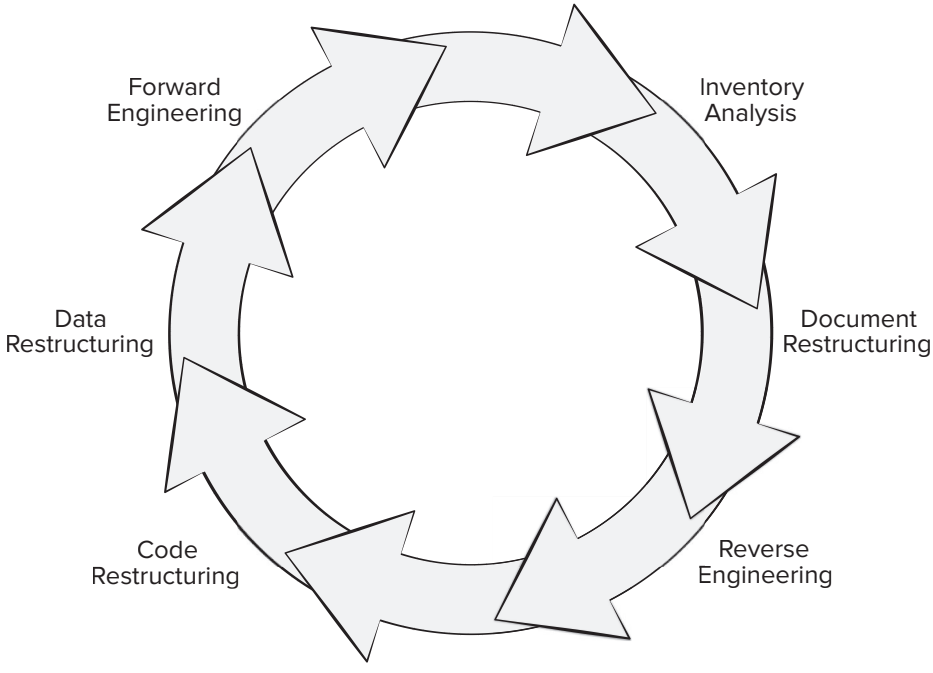
To implement these principles, you can use a cyclical process model for reengineering like the one shown in Figure 27.7. This model defines six activities. Because it is cyclical, each of the activities presented may be revisited as often as needed. For any particular cycle, the process can terminate after any one of these activities.

27.5.1 Inventory Analysis

Every software organization should have an inventory of all applications. The inventory can be nothing more than a spreadsheet model containing information that provides a detailed description (e.g., size, age, business criticality) of every active application. By sorting this information according to business criticality, longevity,

FIGURE 27.7

A software reengineering process model



current maintainability and supportability, and other locally important criteria, candidates for reengineering appear. Resources can then be allocated to candidate applications for reengineering work.

It is important to note that the inventory should be revisited on a regular basis. The status of applications (e.g., business criticality) can change as a function of time, and as a result, priorities for reengineering will shift.

27.5.2 Document Restructuring

Weak documentation is the trademark of many legacy systems. But what can you do about it? What are your options? In some cases, creating documentation when none exists is simply too costly. If the software works, let it be! In other cases, some documentation must be created, but only when changes are made. If a modification occurs, document it. Finally, there are situations in which a critical system must be fully documented, but even here, documents should achieve an essential minimum. Your software organization must choose the documentation option that is most appropriate for each case.

27.5.3 Reverse Engineering

Reverse engineering for software is the process of analyzing a program in an effort to create a representation of the program at a higher level of abstraction than source code. Reverse engineering is a process of *design recovery*. Reverse engineering tools extract data, architectural, and procedural design information from an existing program.

27.5.4 Code Refactoring

The most common type of reengineering (actually, the use of the term *reengineering* is questionable in this case) is *code refactoring*. Some legacy systems have a solid program architecture, but individual modules were coded in a way that makes them difficult to understand, test, and maintain. In such cases, the code within the suspect modules can be restructured.

To accomplish this activity, the source code is analyzed using a restructuring tool. Violations of good design practices are noted, and code is then refactored or even rewritten in a more modern programming language. The resultant refactored code is reviewed and tested to ensure that no anomalies have been introduced. Internal code documentation is updated.

27.5.5 Data Refactoring

A program with weak data architecture will be difficult to adapt and enhance. In fact, for many applications, information architecture has more to do with the long-term viability of a program than the source code itself.

Unlike code restructuring, which occurs at a relatively low level of abstraction, data restructuring is a full-scale reengineering activity. In most cases, data restructuring begins with a reverse engineering activity. Current data architecture is dissected, and necessary data models are defined. Data objects and attributes are identified, and existing data structures are reviewed for quality.

When data structure is weak (e.g., flat files are currently implemented, when a relational approach would greatly simplify processing), the data are reengineered.

Because data architecture has a strong influence on program architecture and the algorithms that populate it, changes to the data will invariably result in either architectural or code-level changes.

27.5.6 Forward Engineering

In an ideal world, applications would be rebuilt using an automated "reengineering engine." The old program would be fed into the engine, analyzed, restructured, and then regenerated in a form that exhibited the best aspects of software quality. In the short term, it is unlikely that such an "engine" will appear, but vendors have introduced tools that provide a limited subset of these capabilities that addresses specific application domains (e.g., applications that are implemented using a specific database system). More important, these reengineering tools are becoming increasingly more sophisticated.

Forward engineering not only recovers design information from existing software but uses this information to alter or reconstitute the existing system in an effort to improve its overall quality. In most cases, reengineered software re-creates the function of the existing system and also adds new functions and/or improves overall performance. In most cases, forward engineering does not simply create a modern equivalent of an older program. Rather, new user and technology requirements are integrated into the reengineering effort. The redeveloped program extends the capabilities of the older application.

27.6 SUMMARY

Software support is an ongoing activity that occurs throughout the life cycle of an application. During support, maintenance actions are initiated, defects are corrected, applications are adapted to a changing operational or business environment, enhancements are implemented at the request of stakeholders. In addition, users are supported as they integrate an application into their personal or business work flow.

Software maintenance and support activities need to be proactive in their nature. It is better to anticipate problems and remove their root causes before the customers find them and become dissatisfied with the software product. The use of software analytics may help software developers identify potential defects and maintenance issues before they become problematic.

At the software level, reengineering examines information systems and applications with the intent of restructuring or reconstructing them so that they exhibit higher quality. Software evolution or reengineering encompasses a series of activities that include inventory analysis, document restructuring, reverse engineering, program and data restructuring, and forward engineering. The intent of these activities is to create versions of existing programs that exhibit higher quality and better maintainability—programs that will be viable well into the twenty-first century.

The cost-benefit of reengineering can be determined quantitatively. The cost of the status quo, that is, the cost associated with ongoing support and maintenance of an existing application, is compared to the projected costs of reengineering and the resultant reduction in maintenance and support costs. In almost every case in which a program has a long life and currently exhibits poor maintainability or supportability, reengineering represents a cost-effective business strategy.

PROBLEMS AND POINTS TO PONDER

27.1. How does software support differ from software maintenance?

27.2. Your instructor will select one of the programs that everyone in the class has developed during this course. Exchange your program randomly with someone else in the class. Do not explain or walk through the program. Now, implement an enhancement (specified by your instructor) in the program you have received.

a. Perform all software engineering tasks including a brief walkthrough (but not with the author of the program).
b. Keep careful track of all errors encountered during testing.
c. Discuss your experiences in class.

27.3. Create a software reengineering inventory analysis checklist, and propose a quantitative software rating system that could be applied to existing programs in an effort to pick candidate programs for reengineering. Your system should extend beyond the economic analysis presented in Section 27.3.

27.4. Suggest alternatives to paper and ink or conventional electronic documentation that could serve as the basis for document restructuring. (Hint: Think of new descriptive technologies that could be used to communicate the intent of the software.)

27.5. Some people believe that artificial intelligence technology will increase the abstraction level of the reverse engineering process. Do some research on this subject (i.e., the use of AI for reverse engineering), and write a brief paper that takes a stand on this point.

27.6. Why is completeness difficult to achieve as abstraction level increases?

27.7. Why is proactive software support preferable to reactive defect repair?

27.8. Using information obtained via the Web, present characteristics of three reverse engineering tools to your class.

27.9. There is a subtle difference between refactoring and forward engineering. What is it?

27.10. How would you determine P_4 through P_7 in the cost-benefit model presented in Section 27.3.3?

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

Five

ADVANCED TOPICS

In this part of *Software Engineering: A Practitioner's Approach,* we consider several advanced topics that will extend your understanding of software engineering. The following questions are addressed in the chapters that follow:

- What is software process improvement and how can it be used to improve the state of software engineering practice?
- What emerging trends can be expected to have a significant influence on software engineering practice in the next decade?
- What is the road ahead for software engineers?

Once these questions are answered, you'll understand topics that may have a profound impact on software engineering in the years to come.

CHAPTER

28 SOFTWARE PROCESS IMPROVEMENT

Long before the phrase "software process improvement" was widely used, RSP worked with major corporations to help them improve the state of their software engineering practices. Based on his experiences, he wrote a book titled *Making Software Engineering Happen* [Pre88]. In the preface of that book he made the following comment:

> For the past ten years I have had the opportunity to help a number of large companies implement software engineering practices. The job is difficult and rarely goes as smoothly as one might like—but when it succeeds, the results are profound . . .
>
> But all is not sweetness and light. Many companies attempt to implement software engineering practice and give up in frustration. Others do it half-way and never see the benefits noted above. Still others do it in a heavy-handed fashion that results in open rebellion among technical staff and managers and subsequent loss of morale.

KEY CONCEPTS

assessment 572
CMMI 576
education and training 573
evaluation 575
installation/migration 574
justification 573
maturity models 570
People CMM 577
return on investment. 580
risk management 575
selection 573
software process improvement (SPI)
  applicability 571
  definition of 569
  frameworks 569
  process. 571



QUICK LOOK

What is it? Software process improvement (SPI) encompasses a set of activities that will lead to a better software process and higher-quality software delivered in a timely manner.

Who does it? Technical managers, software engineers, and individuals who have quality assurance responsibility.

Why is it important? As an organization works to improve its software engineering practices, it must address and correct weaknesses in its existing software process.

What are the steps? The approach to SPI is iterative and continuous, but it can be viewed in five steps: (1) assessment; (2) education and training; (3) selection and justification of process and technology; (4) implementation of the SPI plan; and (5) evaluation and tuning of the results.

What is the work product? An improved software process that leads to higher-quality software.

How do I ensure that I've done it right? The software your organization produces will be delivered with fewer defects, rework at each stage of the software process will be reduced, and on-time delivery will become much more likely.

Even though these words were written over three decades ago, they remain equally true today.

Over the years, virtually every major software engineering organization has attempted to "make software engineering happen." Some have implemented individual practices that have helped to improve the quality of the product they build and the timeliness of their delivery. Others have established a "mature" software process that guides their technical and project management activities. But others continue to struggle. Their practices are hit-and-miss, and their process is ad hoc. Occasionally, their work is spectacular, but in the main, each project is an adventure, and no one knows whether it will end badly or well.

So, which of these two cohorts needs software process improvement? The answer (which may surprise you) is *both.* Those that have succeeded in making software engineering happen cannot become complacent. They must work continually to improve their approach to software engineering. And those that struggle must begin their journey down the road toward improvement.

28.1 What Is SPI?

The term *software process improvement* (SPI) implies many things. First, it implies that elements of an effective software process can be defined in an effective manner; second, that an existing organizational approach to software development can be assessed against those elements; and third, that a meaningful strategy for improvement can be defined. The SPI strategy transforms the existing approach to software development into something that is more focused, more repeatable, and more reliable (in terms of the quality of the product produced and the timeliness of delivery).

Because SPI is not free, it must deliver a return on investment. The effort and time that is required to implement an SPI strategy must pay for itself in some measurable way. To do this, the results of improved process and practice must lead to a reduction in software "problems" that cost time and money. It must reduce the number of defects that are delivered to end users, reduce the amount of rework due to quality problems, reduce the costs associated with software maintenance and support (Chapter 27), and reduce the indirect costs that occur when software is delivered late.

28.1.1 Approaches to SPI

Although an organization can choose a relatively informal approach to SPI, the vast majority choose one of a number of SPI frameworks. An *SPI framework* defines (1) a set of characteristics that must be present if an effective software process is to be achieved, (2) a method for assessing whether those characteristics are present, (3) a mechanism for summarizing the results of any assessment, and (4) a strategy for assisting a software organization in implementing those process characteristics that have been found to be weak or missing.

An SPI framework assesses the "maturity" of an organization's software process and provides a qualitative indication of a maturity level. In fact, the term *maturity model* (Section 28.1.2) is often applied. The SPI framework encompasses a maturity model that in turn incorporates a set of process quality indicators that provide an overall measure of the process quality that will lead to product quality.

FIGURE 28.1

Elements of an SPI framework
Source: Adapted from Rout, Terry, "Software Process Assessment—Part 1: Concepts and Introductory Guide," Spice, 2002.

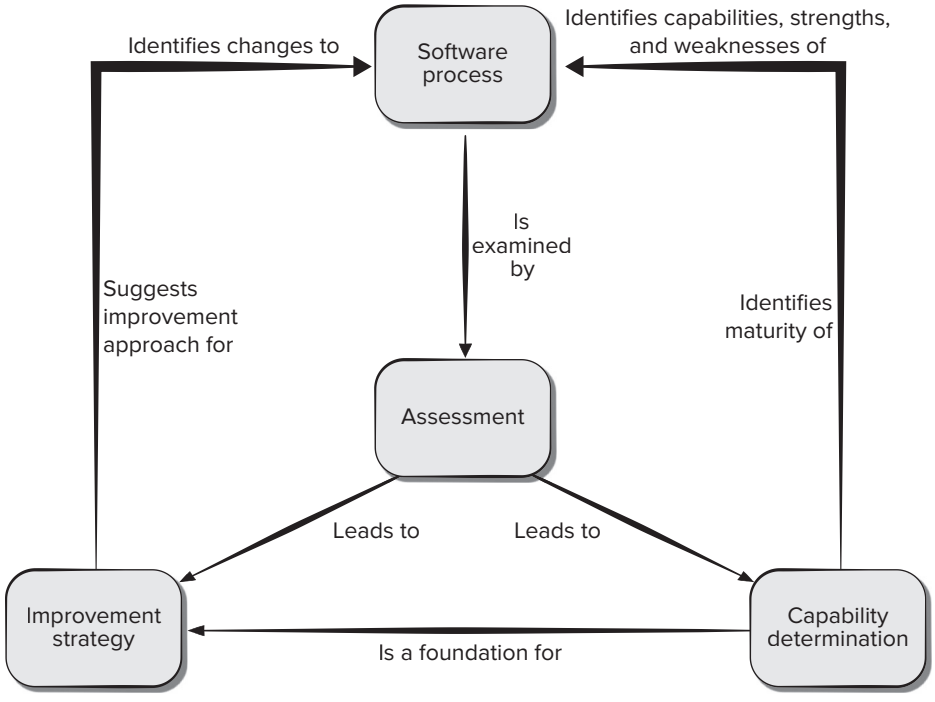


Figure 28.1 provides an overview of a typical SPI framework. The key elements of the framework and their relationship to one another are shown.

You should note that there is no universal SPI framework. In fact, the SPI framework that is chosen by an organization reflects the people in the organization that are championing the SPI effort. As an SPI framework is applied, an organization must establish mechanisms to: (1) support technology transition, (2) determine the degree to which an organization is ready to absorb process changes that are proposed, and (3) measure the degree to which changes have been adopted.

28.1.2 Maturity Models

A *maturity model* is applied within the context of an SPI framework. The intent of the maturity model is to provide an overall indication of the "process maturity" exhibited by a software organization, that is, an indication of the quality of the software process, the degree to which practitioners understand and apply the process, and the general state of software engineering practice. This is accomplished using some type of ordinal scale.

For example, the Software Engineering Institute's original *Capability Maturity Model* (Section 28.3) suggests five levels of maturity ranging from *initial* (rudimentary software process) to *optimized* (a process that leads to best practices).[1]

1 The original CMM has been updated and is discussed in Section 28.3.

The overriding question is whether maturity scales, such as the one proposed as part of the Capability Maturity Model Integration (CMMI) process framework, provide any real benefit. We think that they do. A maturity scale provides an easily understood snapshot of process quality that can be used by practitioners and managers as a benchmark from which improvement strategies can be planned.

28.1.3 Is SPI for Everyone?

For many years, SPI was viewed as a "corporate" activity—a euphemism for something that only large companies perform. But today, a significant percentage of all software development is being performed by companies that employ fewer than 100 people (or in the case of startups less than 24 people). Can a small company initiate SPI activities and do it successfully?

There are substantial cultural differences between large software development organizations and small ones. It should come as no surprise that small organizations are more informal, apply fewer standard practices, and tend to be self-organizing. They also tend to pride themselves on the "creativity" of individual members of the software organization, and initially view an SPI framework as overly bureaucratic and ponderous. Yet, process improvement is as important for a small organization as it is for a large one.

Within small organizations the implementation of an SPI framework requires resources that may be in short supply. Managers must allocate people and money to make software engineering happen. Therefore, regardless of the size of the software organization, it's reasonable to consider the business motivation for SPI. You always need to look at the process activities that are being proposed. If a specific process model or SPI approach feels like overkill for your organization, it probably is.

Quantitative analyses of many projects have shown that the agile project methodologies favored by smaller organizations can lead to greater process efficiency and increase customer satisfaction [Ser15]. It is still likely SPI will be approved and implemented only after its proponents demonstrate *financial leverage* [Bir98]. Financial leverage is demonstrated by examining technical benefits (e.g., fewer defects delivered to the field, reduced rework, lower maintenance costs, or more rapid time to market) and translating them into dollars. You must show a realistic return on investment (Section 28.6) to justify SPI costs.

28.2 The SPI Process

The hard part of SPI isn't the definition of characteristics that define a high-quality software process or the creation of a process maturity model. Those things are relatively easy. Rather, the hard part is establishing a consensus for initiating SPI and defining an ongoing strategy for implementing it across a software organization.

The Software Engineering Institute has developed IDEAL—"an organizational improvement model that serves as a road map for initiating, planning, and implementing improvement actions" [SEI08]. IDEAL is representative of many process models for SPI, defining five distinct activities—initiating, diagnosing, establishing, acting, and learning—that guide an organization through SPI activities.

In this book, we present a somewhat different road map for SPI, based on the process model for SPI originally proposed in [Pre88]. It applies a commonsense philosophy that requires an organization to (1) look in the mirror, (2) then get smarter so it can make intelligent choices, (3) select the process model (and related technology elements) that best meets its needs, (4) instantiate the model into its operating environment and its culture, and (5) evaluate what has been done. These five activities (discussed in the subsections[2] that follow) are applied in an iterative (cyclical) manner that fosters continuous process improvement.

28.2.1 Assessment and Gap Analysis

Any attempt to improve your current software process without first assessing the efficacy of current framework activities and associated software engineering practices would be like starting on a long journey to a new location with no idea where you are starting from. You'd depart with great flourish, wander around trying to get your bearings, expend lots of energy and endure large doses of frustration, and likely, decide you really didn't want to travel anyway. Stated simply, before you begin any journey, it's a good idea to know precisely where you are.

The first road map activity, called *assessment,* allows you to get your bearings. The intent of assessment is to uncover both strengths and weaknesses in the way your organization applies the existing software process and the software engineering practices that populate the process.

Assessment examines a wide range of actions and tasks that will lead to a high-quality process. For example, regardless of the process model that is chosen, the software organization must establish generic mechanisms such as defined approaches for customer communication; established methods for representing user requirements; a project management framework that includes scoping, estimation, scheduling, and project tracking; risk analysis methods; change management procedures; quality assurance and control activities including reviews; and many others. Each is considered within the context of the framework activities (Chapter 2) that have been established, and each is assessed to determine whether all the following questions have been addressed:

- Is the objective of the activity clearly defined?
- Are work products required as input and produced as output identified and described?
- Are the work tasks to be performed clearly described?
- Are the people who must perform the activity identified by role?
- Have entry and exit criteria been established?
- Have metrics for the activity been established?
- Are tools available to support the activity?
- Is there an explicit training program that addresses the activity?
- Is the activity performed uniformly for all projects?

2 Some of the content in these sections has been adapted from [Pre88] with permission.

Although the questions noted imply a *yes* or *no* answer, the role of assessment is to look behind the answer to determine whether the activity in question is being performed in a manner that would conform to best practice.

As the process assessment is conducted, you (or those who have been hired to perform the assessment) should also focus on the following issues:

Consistency. Are important activities, actions, and tasks applied consistently across all software projects and by all software teams?

Sophistication. Are management and technical actions performed with a level of sophistication that implies a thorough understanding of best practice?

Acceptance. Is the software process and software engineering practice widely accepted by management and technical staff?

Commitment. Has management committed the resources required to achieve consistency, sophistication, and acceptance?

The difference between local application and best practice represents a "gap" that offers opportunities for improvement. The degree to which consistency, sophistication, acceptance, and commitment are achieved indicates the amount of cultural change that will be required to achieve meaningful improvement.

28.2.2 Education and Training

Although few software people question the benefit of an agile, organized software process or solid software engineering practices, many practitioners and managers do not know enough about either subject.[3] As a consequence, inaccurate perceptions of process and practice lead to inappropriate decisions when an SPI framework is introduced. It follows that a key element of any SPI strategy is education and training for practitioners, technical managers, and more senior managers who have direct contact with the software organization. It is wise to try to provide "just-in-time" training targeted to the real needs of a software team. Three types of education and training should be conducted: generic software engineering concepts and methods, specific technology and tools, and communication and quality-oriented topics. In a modern context, education and training can be delivered in a variety of different ways. Everything from podcasts, to short YouTube videos, to more comprehensive Internet-based training such as Coursera,[4] to e-books, to classroom courses can be offered as part of an SPI strategy.

28.2.3 Selection and Justification

Once the initial assessment activity[5] has been completed and education has begun, a software organization should begin to make choices. These choices occur during a *selection and justification activity* in which process characteristics and specific software engineering methods and tools are chosen to populate the software process.

---

3 If you've spent time reading this book, you won't be one of them!

4 See https://www.coursera.org/.

5 In actuality, assessment is an ongoing activity. It is conducted periodically in an effort to determine whether the SPI strategy has achieved its immediate goals and to set the stage for future improvement.

First, you should choose the process model (Chapters 2 through 4) that best fits your organization, its stakeholders, and the software that you build. You should decide which of the set of framework activities will be applied, the major work products that will be produced, and the quality assurance checkpoints that will enable your team to assess progress. If the SPI assessment activity indicates that you have specific weaknesses (e.g., you have no formal SQA functions), you should focus attention on process characteristics that will address these weaknesses directly.

Next, develop an adaptable work breakdown for each framework activity (e.g., modeling), defining the task set that would be applied for a typical project. You should also consider the software engineering methods that can be applied to achieve these tasks. As choices are made, education and training should be coordinated to ensure that understanding is reinforced.

As you make your choices, be sure to consider the culture of your organization and the level of acceptance that each choice will likely elicit. Ideally, everyone works together to select various process and technology elements and moves smoothly toward the installation or migration activity (Section 28.2.4). In reality, selection can be a rocky road. It is often difficult to achieve consensus among different constituencies. If the criteria for selection are established by committee, people may argue endlessly about whether the criteria are appropriate and whether a choice truly meets the criteria that have been established.

It is true that a bad choice can do more harm than good, but "paralysis by analysis" means that little if any progress occurs and process problems remain. As long as the process characteristic or technology element has a good chance at meeting an organization's needs, it's sometimes better to pull the trigger and make a choice, rather than waiting for the perfect solution.

28.2.4 Installation/Migration

Installation is the first point at which a software organization feels the effects of changes triggered by following the SPI road map. In some cases, an entirely new process is recommended for an organization. Framework activities, software engineering actions, and individual work tasks must be defined and installed as part of a new software engineering culture. Such changes represent a substantial organizational and technological transition and must be managed very carefully.

In other cases, changes associated with SPI are relatively minor, representing small, but meaningful, modifications to an existing process model. Such changes are often referred to as *process migration.* Today, many software organizations have a "process" in place. The problem is that it doesn't work in an effective manner. Therefore, an incremental *migration* from one process (that doesn't work as well as desired) to another process is a more effective strategy.

Installation and migration are *software process redesign* (SPR) activities. Scacchi [Sca00] states that "SPR is concerned with identification, application, and refinement of new ways to dramatically improve and transform software processes." When a formal approach to SPR is initiated, three different process models are considered: (1) the existing ("as is") process, (2) a transitional ("here to there") process, and the target ("to be") process. If the target process is significantly different from the existing process, the only rational approach to installation is an incremental strategy in which the transitional process is implemented in steps. The transitional process

provides a series of waypoints that enable the software organization's culture to adapt to small changes over time.

28.2.5 Evaluation

Although it is listed as the last activity in the SPI road map, *evaluation* occurs throughout SPI. The evaluation activity assesses the degree to which changes have been instantiated and adopted, the degree to which such changes result in better software quality or other tangible process benefits, and the overall status of the process and the organizational culture as SPI activities proceed.

Both qualitative factors and quantitative metrics are considered during the evaluation activity. From a qualitative point of view, past management and practitioner attitudes about the software process can be compared to attitudes polled after installation of process changes. Quantitative metrics (Chapter 23) are collected from projects that have used the transitional or "to be" process and compared with similar metrics that were collected for projects that were conducted under the "as is" process.

28.2.6 Risk Management for SPI

SPI is a risky undertaking. SPI often fails because risks were not properly thought through and no contingency planning occurred. In fact, more than half of all SPI efforts end in failure. The reasons for failure vary greatly and are organizationally specific. Among the most common risks are: a lack of management support, cultural resistance by technical staff, a poorly planned SPI strategy, an overly formal approach to SPI, selection of an inappropriate process, a lack of buy-in by key stakeholders, an inadequate budget, a lack of staff training, and organizational instability, but there are a myriad of other factors. The role of those chartered with the responsibility for SPI is to analyze likely risks and develop an internal strategy for mitigating them [Dut15].

A software organization should manage risk at three key points in the SPI process [Ive04]: prior to the initiation of the SPI road map, during the execution of SPI activities (assessment, education, selection, installation), and during the evaluation activity that follows the instantiation of some process characteristic. In general, the following categories [Ive04] can be identified for SPI risk factors: budget and cost, content and deliverables, culture, maintenance of SPI deliverables, mission and goals, organizational management, organizational stability, process stakeholders, schedule for SPI development, SPI development environment, SPI development process, SPI project management, and SPI staff.

Within each category, several generic risk factors can be identified. For example, the organizational culture has a strong bearing on risk. The following generic risk factors[6] can be defined for the culture category [Ive04]:

- Attitude toward change, based on prior efforts to change
- Experience with quality programs, level of success
- Action orientation for solving problems versus political struggles
- Use of facts to manage the organization and business

6 Risk factors for each of the risk categories noted in this section can be found in [Ive04].

- Patience with change; ability to spend time socializing
- Tools orientation—expectation that tools can solve the problems
- Level of "planfulness"—ability of organization to plan
- Ability of organization members to participate with various levels of organization openly at meetings
- Ability of organization members to manage meetings effectively
- Level of experience in organization with defined processes

Using the risk factors and generic attributes as a guide, a risk table (Chapter 26) can be developed to isolate those risks that warrant further management attention.

28.3 The CMMI

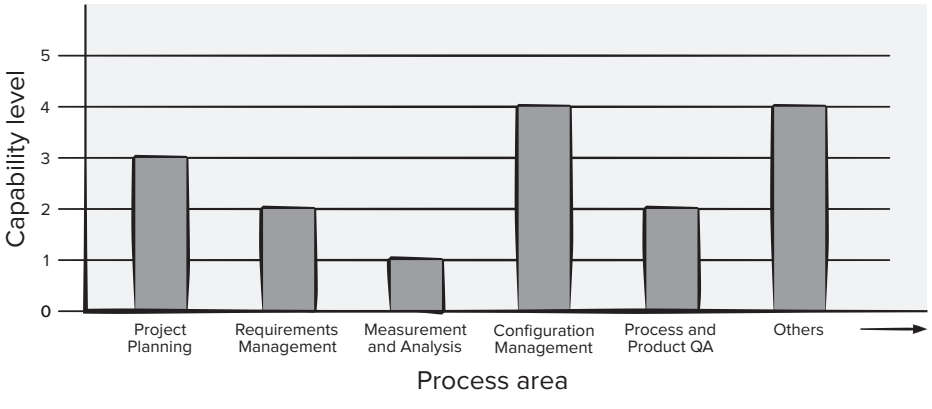
The original Capability Maturity Model (CMM) was developed and upgraded by the Software Engineering Institute throughout the 1990s as a complete SPI framework. Today, it has evolved into the *Capability Maturity Model Integration* (CMMI) [CMM18], a comprehensive process meta-model that is predicated on a set of system and software engineering capabilities that should be present as organizations reach different levels of process capability and maturity.

The CMMI represents a process meta-model in two different ways: (1) as a "continuous" model and (2) as a "staged" model. The continuous CMMI meta-model describes a process in two dimensions as illustrated in Figure 28.2. Each process area (e.g., project planning or requirements management) is formally assessed against specific goals and practices and is rated according to the following capability levels:

Level 0: *Incomplete*. The process area (e.g., requirements management) is either not performed or does not achieve all goals and objectives defined by the CMMI for level 1 capability for the process area.

FIGURE 28.2

CMMI process area capability profile
Source: Phillips, Mike, "CMMI V1.1 Tutorial," April 9, 2002.



Level 1: *Performed*. All the specific goals of the process area (as defined by the CMMI) have been satisfied. Work tasks required to produce defined work products are being conducted.

Level 2: *Managed*. All capability level 1 criteria have been satisfied. In addition, all work associated with the process area conforms to an organizationally defined policy; all people doing the work have access to adequate resources to get the job done; stakeholders are actively involved in the process area as required; all work tasks and work products are "monitored, controlled, and reviewed; and are evaluated for adherence to the process description" [CMM18].

Level 3: *Defined*. All capability level 2 criteria have been achieved. In addition, the process is "tailored from the organization's set of standard processes according to the organization's tailoring guidelines, and contributes work products, measures, and other process-improvement information to the organizational process assets" [CMM18].

Level 4: *Quantitatively managed*. All capability level 3 criteria have been achieved. In addition, the process area is controlled and improved using measurement and quantitative assessment. "Quantitative objectives for quality and process performance are established and used as criteria in managing the process" [CMM18].

Level 5: *Optimized*. All capability level 4 criteria have been achieved. In addition, the process area is adapted and optimized using quantitative (statistical) means to meet changing customer needs and to continually improve the efficacy of the process area under consideration.

The CMMI defines each process area in terms of "specific goals" and the "specific practices" required to achieve these goals. *Specific goals* establish the characteristics that must exist if the activities implied by a process area are to be effective. *Specific practices* refine a goal into a set of process-related activities.

In addition to specific goals and practices, the CMMI also defines a set of five generic goals and related practices for each process area. Each of the five generic goals corresponds to one of the five capability levels. To achieve a maturity level, the specific goals and practices associated with a set of process areas must be achieved. The relationship between maturity levels and process areas is shown in Figure 28.3.

A software process, no matter how well conceived, will not succeed without talented, motivated software people. The *People CMM* suggests practices that improve the workforce competence and culture [CMM18a]. The goal of the People CMM is to encourage continuous improvement of generic workforce knowledge (called "core competencies"), specific software engineering and project management skills (called "workforce competencies"), and process-related abilities. Like the CMMI, the People CMM defines a set of five organizational maturity levels that provide an indication of the relative sophistication of workforce practices and processes.

FIGURE 28.3

Process areas required to achieve a maturity level

Level	Focus	Process Area
Optimizing	Continuous Process Improvement	Causal analysis and resolution Organizational innovation and deployment
Quantitatively Managed	Quantitative Management	Quantitative project management Organizational process performance
Defined	Process Standardization	Technical solution Verification Organizational training Integrated project management Integrated teaming Requirements development Validation Decision analysis and resolution Organizational environment for integration Product integration Organizational process definition Integrated supplier management Risk management Organizational process focus
Managed	Basic Product Management	Supplier agreement management Process and product quality assurance Project planning Requirements management Configuration management Measurement and analysis Project monitoring and control
Performed		

INFO



The CMMI—Should We, or Shouldn't We?

The CMMI is a process meta-model. It defines (in 700+ pages) the process characteristics that should exist if an organization wants to establish a software process that is complete. The question that has been debated for almost two decades is: "Is the CMMI overkill?" Like most things in life (and in software), the answer is not a simple yes or no.

The spirit of the CMMI should always be adopted. At the risk of oversimplification, it argues that software development must be taken seriously—it must be planned thoroughly, it must be controlled uniformly, it must be tracked accurately, and it must be conducted professionally. It must focus on the needs of project stakeholders, the skills of the software engineers, and the quality of the end product. No one would argue with these ideas.

The detailed requirements of the CMMI should be seriously considered if an organization builds large, complex systems that involve dozens or hundreds of people over many months or years. It may be that the CMMI is "just right" in such situations, if the organizational culture is amenable to standard process models and management is committed to making it a success. However, in other situations, the CMMI may simply be too much for an organization to successfully assimilate. Does this mean that the CMMI is "bad" or "overly bureaucratic" or "old-fashioned"? No . . . it does not. It simply means that what is right for one organizational culture may not be right for another.

The CMMI is a significant achievement in software engineering. It provides a comprehensive discussion of the activities and actions that should be present when an organization builds computer software. Even if a software organization chooses not to adopt its details, every software team should embrace its spirit and gain insight from its discussion of software engineering process and practice.

28.4 Other SPI Frameworks

Although the SEI's CMM and CMMI are the most widely applied SPI frameworks, several alternatives[7] have been proposed and are in use. We provide a brief overview of two of these frameworks.[8]

28.4.1 SPICE

The SPICE (*Software Process Improvement and Capability dEtermination*) model [SPI99] provides an SPI assessment framework that is compliant with ISO 15504-5:2015 and ISO 12207:2017. The purpose of SPICE was to provide a framework to assess a process and provide information on the strengths, weaknesses, and capabilities to help an organization achieve its goals. [Kar12l presents an overview of the SPI framework, including a model for process management, guidelines for conducting an assessment, and rating the process under consideration.

28.4.2 TickIT Plus

The TickIT auditing method [Tic18] ensures compliance with *ISO 9001:2015 for Software*—a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies.

ISO 9001:2015 has adopted a "plan-do-check-act" cycle that is applied to the quality management elements of a software project. Within a software context, "plan" establishes the process objectives, activities, and tasks necessary to achieve high-quality software and resultant customer satisfaction. "Do" implements the software process (including both framework and umbrella activities). "Act" initiates software process improvement activities that continually work to improve the process. TickIT can be used throughout the "plan-do-check-act" cycle to ensure that SPI progress is being made. TickIT auditors assess the application of the cycle as a precursor to ISO 9001:2015 certification. For a detailed discussion of ISO 9001:2015 and TickIT, you should examine [Tic18] and [ISO15].

---

7 It's reasonable to argue that some of these frameworks are not so much "alternatives" as they are complementary approaches to SPI. A comprehensive table of many more SPI frameworks can be found at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.13.4787&rep=rep1&type=pdf.

8 If you have further interest, a wide array of print and Web-based resources is available for each.

28.5 SPI Return on Investment

SPI is hard work and requires substantial investment of dollars and people. Managers who approve the budget and resources for SPI will invariably ask the question: "How do I know that we'll achieve a reasonable return for the money we're spending?"

At a qualitative level, proponents of SPI argue that an improved software process will lead to improved software quality. They contend that an improved process will result in the implementation of better-quality filters (resulting in fewer propagated defects), better control of change (resulting in less project chaos), and less technical rework (resulting in lower cost and better time to market). But can these qualitative benefits be translated into quantitative results? The classic return on investment (ROI) equation is:

$$\text{ROI} = \frac{\Sigma(\text{benefits}) - \Sigma(\text{costs})}{\Sigma(\text{costs})} \times 100\%$$

where

benefits include the cost savings associated with higher product quality (fewer defects), less rework, reduced effort associated with changes, and the income that accrues from shorter time to market.

costs include both direct SPI costs (e.g., training, measurement) and indirect costs associated with greater emphasis on quality control and change management activities and more rigorous application of software engineering methods (e.g., the creation of a design model).

In the real world, these quantitative benefits and costs are sometimes difficult to measure with accuracy, and all are open to interpretation. But that doesn't mean that a software organization should conduct an SPI program without careful analysis of the costs and benefits that accrue. Even very small software organizations benefit from software process improvement, but they examine the ROI of the SPI activities they choose to employ [Lar16]. A comprehensive treatment of ROI for SPI can be found in a unique book by David Rico [Ric04].

28.6 SPI Trends

Over the past 35 years, many companies have attempted to improve their software engineering practices by applying an SPI framework to effect organizational change and technology transition. As we noted earlier in this chapter, over half fail in this endeavor. Regardless of success or failure, all spend significant amounts of money. David Rico [Ric04] reports that a typical application of an SPI framework such as the SEI CMMI can cost between $25,000 and $70,000 per person and take years to complete! It should come as no surprise that the future of SPI should emphasize a less costly and time-consuming approach.

To be effective in the twenty-first-century world of software development, future SPI frameworks must become significantly more agile [Bjø16]. Rather than an organizational focus (which can take years to complete successfully), contemporary SPI efforts should focus on the project level, working to improve a team process in weeks,

not months or years [Bjø16]. To achieve meaningful results (even at the project level) in a short time frame, complex framework models may give way to simpler models [Lar16]. Rather than dozens of key practices and hundreds of supplementary practices, an agile SPI framework should emphasize only a few pivotal practices (e.g., analogous to the framework activities discussed throughout this book) [Din16].

Any attempt at SPI demands a knowledgeable workforce, but education and training can be expensive and should be minimized (and streamlined). Rather than classroom courses (expensive and time consuming), future SPI efforts should rely on Web-based training that is targeted at pivotal practices. Rather than far-reaching attempts to change organizational culture (with all the political perils that ensue), cultural change should occur as it does in the real world, one small group at a time until a tipping point is reached. Jovanovic and his colleagues suggest using retrospective gaming as part of the Scrum retrospective as a means of educating and engaging agile development team members in process improvement [Jov15].

The SPI work of the past three decades has significant merit. The frameworks and models that have been developed represent substantial intellectual assets for the software engineering community. But like all things, these assets guide future attempts at SPI not by becoming a recurring dogma, but by serving as the basis for better, simpler, and more agile SPI models.

28.7 SUMMARY

A software process improvement framework defines the characteristics that must be present if an effective software process is to be achieved, an assessment method that helps determine whether those characteristics are present, and a strategy for assisting a software organization in implementing those process characteristics that have been found to be weak or missing. Regardless of who in the organization sponsors SPI, the goal is to improve process quality and to improve software quality and timeliness.

A process maturity model provides an overall indication of the "process maturity" exhibited by a software organization. It provides a qualitative feel for the relative effectiveness of the software process that is currently being used.

The SPI road map begins with assessment—a series of evaluation activities that uncover both strengths and weaknesses in the way your organization applies the existing software process and the software engineering practices that populate the process. Assessment is the tool that allows a software organization to develop an overall SPI plan.

One of the key elements of any SPI plan is education and training, an activity that focuses on improving the knowledge level of managers and practitioners. Once staff becomes well versed in current software technologies, selection and justification commence. These tasks lead to choices about the architecture of the software process, the methods that populate it, and the tools that support it. Installation and evaluation are SPI activities that instantiate process changes and assess their efficacy and impact.

To successfully improve its software process, an organization must exhibit the following characteristics: management commitment and support for SPI, staff involvement throughout the SPI process, process integration into the overall organizational culture, an SPI strategy that has been customized for local needs, and solid management of the SPI project.

Several SPI frameworks are in use today. The SEI's CMM and CMMI are widely used. The People CMM has been customized to assess the quality of the organizational culture and the people who populate it. SPICE and TickIT are additional frameworks that can lead to effective SPI.

SPI is hard work that requires substantial investment of dollars and people. To ensure that a reasonable return on investment is achieved, an organization must measure the costs associated with SPI and the benefits that can be directly attributed to it.

Problems and Points to Ponder

28.1. Why is it that software organizations often struggle when they embark on an effort to improve local software process?

28.2. Describe the concept of "process maturity" in your own words.

28.3. Do some research (check the SEI website), and determine the process maturity distribution for software organizations in the United States and worldwide.

28.4. You work for a very small software organization—only 11 people are involved in developing software. Is SPI for you? Explain your answer.

28.5. Assessment is analogous to an annual physical exam. Using a physical exam as a metaphor, describe the SPI assessment activity.

28.6. What is the difference between an "as is" process, a "here to there" process, and a "to be" process?

28.7. How is risk management applied within the context of SPI?

28.8. Do some research on the key factors for predicting success in software process improvement efforts. Pick one of them, and write a paper describing how it might be achieved in a small software development organization.

28.9. Do some research to try to determine how CMMI can be used with agile process frameworks.

28.10. Select one of the SPI frameworks discussed in Section 28.5, and write a brief paper describing it in more detail.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

29 EMERGING TRENDS IN SOFTWARE ENGINEERING

Throughout the relatively brief history of software engineering, practitioners and researchers have developed an array of process models, technical methods, and automated tools in an effort to foster fundamental change in the way we build computer software. Even though past experience indicates otherwise, there is a tacit desire to find the "silver bullet"—the magic process or transcendent technology that will allow us to build large, complex, software-based systems easily, without confusion, without mistakes, without delay—without the many problems that continue to plague software work.

KEY CONCEPTS

building blocks 591
collaborative development 595
complexity 588
crowd sourcing 596
emergent requirements 590
hype cycle 586
innovation life cycle 584
model-driven software development 596
open source 592
open-world software 589
postmodern design 601
requirements engineering 596
search-based software engineering 597
soft trends 587
technology directions 593
technology evolution 584
test-driven development 598
tools 599
variability intensive systems 590

QUICK LOOK

What is it? No one can predict the future with absolute certainty. But it is possible to assess trends in the software engineering area and from those trends to suggest possible directions for the technology. That's what we attempt to do in this chapter.

Who does it? Anyone who is willing to spend the time to stay abreast of software engineering issues can try to predict the future direction of the technology.

Why is it important? Why did ancient kings hire soothsayers? Why do major multinational corporations hire consulting firms and think tanks to prepare forecasts? Why does a substantial percentage of the public read horoscopes? We want to know what's coming so we can ready ourselves.

What are the steps? There is no formula for predicting the road ahead. We attempt to do this by collecting data, organizing it to provide useful information, examining subtle associations to extract knowledge, and from this knowledge to suggest probable trends that predict how things will be at some future time.

What is the work product? A view of the near-term future that may or may not be correct.

How do I ensure that I've done it right? Predicting the road ahead is an art, not a science. In fact, it's quite rare when a serious prediction about the future is absolutely right or unequivocally wrong (with the exception, thankfully, of predictions of the end of the world). We look for trends and try to extrapolate them. We can assess the correctness of the extrapolation only as time passes.

But history indicates that our quest for the silver bullet appears doomed to failure. New technologies are introduced regularly, hyped as "solutions" to many of the problems software engineers face, and incorporated into projects large and small. Industry pundits stress the importance of these "new" software technologies, the cognoscenti of the software community adopt them with enthusiasm, and ultimately, they do play a role in the software engineering world. But they tend not to meet their promise, and as a consequence, the quest continues.

In past editions of this book (approaching four decades), we have discussed emerging technologies and their projected impact on software engineering. Some have been widely adopted, but others never reached their potential. Our conclusion: Technologies come and go; the real trends that we should explore are softer. By this we mean that progress in software engineering will be guided by business, organizational, market, and cultural trends. Those trends lead to technology innovation.

In this chapter, we'll mention a few software engineering technology trends, but our primary emphasis will be on some of the business, organizational, market, and cultural trends that may have an important influence on software engineering technology over the next 10 or 20 years.

29.1 TECHNOLOGY EVOLUTION

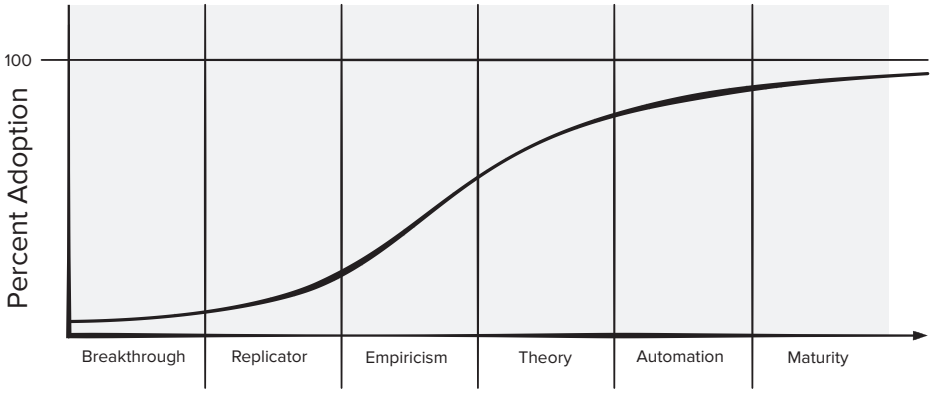
In a fascinating book that provides a compelling look at how computing (and other related) technologies will evolve, Ray Kurzweil [Kur05] argued that technological evolution is similar to biological evolution, but at a rate that is orders of magnitude faster. Evolution (whether biological or technological) occurs as a result of positive feedback—"the more capable methods resulting from one stage of evolutionary progress are used to create the next stage" [Kur05].

The big questions for the twenty-first century are: (1) How rapidly does a technology evolve? (2) How significant are the effects of positive feedback? (3) How profound will the resultant changes be?

When a successful new technology is introduced, the initial concept moves through a reasonably predictable "innovation life cycle" [Gai95], as illustrated in Figure 29.1.

FIGURE 29.1

A technology innovation cycle



In the *breakthrough* phase, a problem is recognized and repeated attempts at a viable solution are attempted. At some point, a solution shows promise. The initial breakthrough work is reproduced in the *replicator* phase and gains wider usage. *Empiricism* leads to the creation of empirical rules that govern the use of the technology, and repeated success leads to a broader *theory* of usage that is followed by the creation of automated tools during the *automation* phase. Finally, the technology matures and is used widely.

You should note that many research and technology trends never reach maturity. In fact, the vast majority of "promising" technologies in the software engineering domain receive widespread interest for a few years and then fall into niche usage by a dedicated band of adherents. This is not to say that these technologies lack merit, but rather to emphasize that the journey through the innovation life cycle is long and hard.

Computing technology is evolving at an exponential rate, and its growth may soon become explosive. Kurzweil [Kur05] agrees that computing technologies evolve through an "S-curve" that exhibits relatively slow growth during the technology's formative years, rapid acceleration during its growth period, and then a leveling-off period as the technology reaches its limits. Today, we are at the knee of the S-curve for modern computing technologies—at the transition between early growth and the explosive growth that is to follow. The implication is that over the next 20 to 40 years, we will see dramatic (even mindboggling) changes in computing capability. He suggests that within 20 years, technology evolution will accelerate at an increasingly rapid pace, ultimately leading to an era of nonbiological intelligence that will merge with and extend human intelligence in ways that are fascinating to contemplate.

And all of this, no matter how it evolves, will require software and systems that make our current efforts look infantile by comparison. By the year 2040, a combination of extreme computation, artificial intelligence and machine learning, nanotechnology, massively high bandwidth ubiquitous networks, and robotics will lead us into a different world.[1] Software—possibly in forms we cannot yet comprehend—will continue to reside at the core of this new world. Software engineering will not go away.

29.2 SOFTWARE ENGINEERING AS A DISCIPLINE

For almost 50 years, many academic researchers and industry professionals have clamored for a true engineering discipline for software. In an important follow-on to her classic 1990 paper on the subject, Mary Shaw [Sha09] comments on this continuing quest:

> Engineering disciplines typically evolve from craft practices of a technology, sufficient for local or ad hoc use. When the technology becomes economically significant, it requires stable production techniques and management control. The resulting commercial market is based on experience, rather than a deep understanding of the technology . . . an engineering profession emerges when . . . science becomes sufficiently mature to support purposeful practice and design evolution with predictable outcomes.

1 Kurzweil [Kur05] presents a reasoned technical argument that predicts a strong artificial intelligence (that will pass the Turing Test) by 2029 and suggests that the evolution of humans and machines will begin to merge by 2045. The vast majority of readers of this book will live to see whether this, in fact, takes place.

We would argue that the industry has achieved "purposeful practice," but that "predictable outcomes" have remained elusive.

As mobility begins to dominate the software landscape, Shaw identifies challenges that "emerge from the deep interdependencies between very complex systems and their users" [Sha09]. She argues that the knowledge base that leads to "purposeful practice" has been democratized by the specialized social networks that now populate the Web. For example, rather than referencing a centrally controlled software engineering handbook, a software developer can pose a problem on an appropriate forum and obtain a crowd-sourced solution that draws from the experience of many other developers. The proposed solution if often critiqued in real time, with alternatives and adaptations offered as options.

But this is not the level of discipline that many demand. As Shaw states: "[P]roblems facing software engineers are increasingly situated in complex social contexts and delineating the problem's boundaries is increasingly difficult" [Sha09]. As a consequence, isolating the scientific underpinnings of a discipline remains a challenge. At this point in the history of our field, it is reasonable to state that "the discovery of new software engineering ideas is, by now, naturally incremental and evolutionary" [Erd10].

29.3 Observing Software Engineering Trends

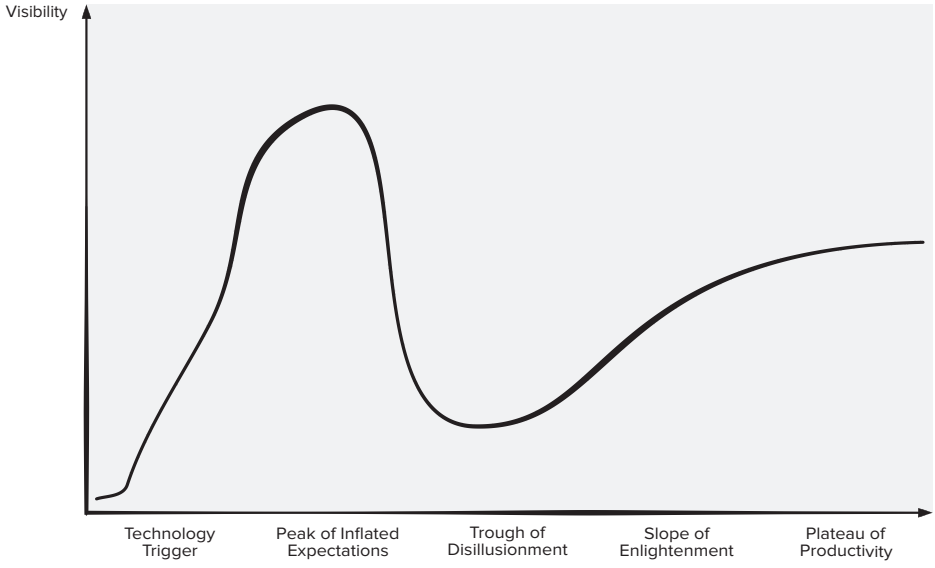
Barry Boehm [Boe08] suggests that "software engineers [will] face the formidable challenges of dealing with rapid change, uncertainty and emergence, dependability, diversity, and interdependence, but they also have opportunities to make significant contributions that will make a difference for the better." But what are the trends that will enable you to face these challenges in the years ahead?

In the introduction to this chapter, we noted that "soft trends" have a significant impact on the overall direction of software engineering. But other ("harder") research- and technology-oriented trends remain important. Research trends "are driven by general perceptions of the state of the art and the state of the practice, by researcher perceptions of practitioner needs, by national funding programs that rally around specific strategic goals, and by sheer technical interest" [Mil00b]. Technology trends occur when research trends are extrapolated to meet industry needs and are shaped by market-driven demand.

In Section 29.1, we discussed the S-curve model for technology evolution. The S-curve is appropriate for considering the long-term effects of core technologies as they evolve. But what of more modest, short-term innovations, tools, and methods? The Gartner Group [Gar08]—a consultancy that studies technology trends across many industries—has developed a *hype cycle for emerging technologies,* represented in Figure 29.2. The "hype cycle" presents a realistic view of short-term technology integration. The long-term trend, however, is exponential. Not every software engineering technology makes it all the way through the hype cycle. In some cases, the disillusionment is justified, and the technology is relegated to obscurity.

FIGURE 29.2

The Gartner Group's hype cycle for emerging technologies
Source: Linden, Alexander, Fenn, Jackie, "Understanding Gartner's Hype Cycles," Strategic Analysis Report, Gartner, Inc., May 30, 2003, 5.



29.4 IDENTIFYING "SOFT TRENDS"

Each nation with a substantial IT industry has a set of unique characteristics that define the manner in which business is conducted, the organizational dynamics that arise within a company, the distinct marketing issues that apply to local customers, and the overriding culture that dictates all human interaction. However, some trends in each of these areas are universal and have as much to do with sociology, anthropology, and group psychology (often referred to as the "soft sciences") as they do with academic or industrial research.

Connectivity and collaboration (enabled by high-bandwidth communication) has already led to software teams that do not occupy the same physical space (telecommuting and part-time employment in a local context). One team collaborates with other teams that are separated by time zones, primary language, and culture. Software engineering must respond with an overarching process model for "distributed global engineering teams" that is agile enough to meet the demands of immediacy but disciplined enough to coordinate disparate groups.

Globalization leads to a diverse workforce (in terms of language, culture, problem resolution, management philosophy, communication priorities, and person-to-person interaction). This, in turn, demands a flexible organizational structure. Different teams (in different countries) must respond to engineering problems in a way that best accommodates their unique needs, while at the same time fostering a level of uniformity that allows an overall global project to proceed. This type of organization suggests fewer levels of management and a greater emphasis

on team-level decision making. It can lead to greater agility, but only if communication mechanisms have been established so that every team can understand project and technical status (via networked groupware) at any time. Software engineering methods and tools can help achieve some level of uniformity (teams speak the same "language" implemented through specific methods and tools). Software process can provide the framework for the instantiation of these methods and tools.

In some world regions (the United States and Europe are examples), the population is aging. This undeniable demographic (and cultural trend) implies that many experienced software engineers and managers will be leaving the field over the coming decades. The software engineering community must respond with viable mechanisms that capture the knowledge of these aging managers and technologists [e.g., the use of *patterns* (Chapter 14) is a step in the right direction], so that it will be available to future generations of software workers. In other regions of the world, the number of young people available to the software industry is exploding. This provides an opportunity to mold a software engineering culture without the burden of 50 years of "old-school" prejudices.

It is estimated that over 1 billion new consumers will enter the worldwide marketplace over the next decade. Consumer spending in emerging economies is projected to grow to about $8 trillion by 2022 [Jai18]. The digitally influenced component of that spending will top $4 trillion. The implication—an increasing demand for new software. Can new software engineering technologies be developed to meet this worldwide demand? Modern market trends are often driven by the supply side.[2] In other cases, demand-side requirements drive the market. In either case, a cycle of innovation and demand progresses in a way that sometimes makes it difficult to determine which came first!

Finally, human culture itself will impact the direction of software engineering. Every generation establishes its own imprint on local culture, and yours will be no different. Faith Popcorn [Pop08], a well-known consultant who specializes in cultural trends, characterizes them in the following manner: "Our Trends are not fads. Our Trends endure. Our Trends evolve. They represent underlying forces, first causes, basic human needs, attitudes, aspirations. They help us navigate the world, understand what's happening and why, and prepare for what is yet to come." A detailed discussion of how modern cultural trends will have an impact on software engineering is best left to those who specialize in the "soft sciences."

29.4.1 Managing Complexity

When the first edition of this book was written (1982), digital consumer products as we now know them today didn't exist, and mainframe-based systems containing a million lines of source code (LOC) were considered to be quite large. Today, small digital devices typically house between 60,000 to 200,000 lines of custom software, coupled with a few million LOC for operating system features. Modern computer-based

2 Supply side adopts a "build it and they will come" approach to markets. Unique technologies are created, and consumers flock to adopt them—sometimes!

systems containing 10 to 50 million lines of code are common.[3] In the relatively near future, systems[4] requiring over 1 billion LOC will begin to emerge.[5]

Think about that for a moment!

Consider the interfaces for a billion LOC system, both to the outside world, to other interoperable systems, to the Internet (or its successor), and to the millions of internal components that must all work together to make this computing monster operate successfully. Is there a reliable way to ensure that all these connections will allow information to flow properly?

Consider the project itself. How do we manage the work flow and track progress? Will conventional approaches scale upward by orders of magnitude?

Consider the number of people (and their locations) who will be doing the work, the coordination of people and technology, the unrelenting flow of changes, the likelihood of a multiplatform, multioperating system environment. Is there a way to manage and coordinate people who are working on a monster project?

Consider the engineering challenge. How can we analyze tens of thousands of requirements, constraints, and restrictions in a way that ensures that inconsistency and ambiguity, omissions, and outright errors are uncovered and corrected? How can we create a design architecture that is robust enough to handle a system of this size? How can software engineers establish a change management system that will have to handle hundreds of thousands of changes?

Consider the challenge of quality assurance. How can we perform verification and validation in a meaningful way? How do you test a 1-billion-LOC system?

In the early days, software engineers attempted to manage complexity in what can only be described as an ad-hoc fashion. Today, we use process, methods, and tools to keep complexity under control. But tomorrow? Is our current approach up to the task?

In the future, we are likely to see wider use of artificial intelligence techniques to help software engineers manage these levels of complexity [Har12b], [Xie18]. Machine learning is one such technique that can help with testing and bug fixing [Mei18]. Data science techniques can be used to help make sense of the vast amount of software engineering data generated by these large projects [Kim16b]. Mining of these repositories is becoming an accepted research technique in the software engineering communities [Dye15].

29.4.2 Open-World Software

Concepts such as ambient intelligence,[6] context-aware applications, and pervasive/ubiquitous computing all focus on integrating software-based systems into an

3 For example, modern laptop operating systems (e.g., Linux, macOS, and Windows) have between 30 and 60 million LOC. Operating system software for mobile devices can exceed 2 million LOC.

4 In reality, this "system" will actually be a system of systems—hundreds of interoperable applications working together to achieve some overall objective, and many will be cloud based.

5 Not all complex systems are large. A relatively small application (say, less than 100,000 LOC) can still be exceedingly complex.

6 A worthwhile and quite detailed introduction to *ambient intelligence* can be found at https://www.researchgate.net/publication/220737998_Ambient_Intelligence_Basic_Concepts_and_Applications.

environment far broader than a laptop, a mobile computing device, or any other digital device. These separate visions of the near-term future of computing collectively suggest "open-world software"—software that is designed to adapt to a continually changing environment "by self-organizing its structure and self-adapting its behavior" [Bar06b].

To help illustrate the challenges that software engineers will face in the near future, consider the notion of *ambient intelligence* (amI). Ducatel [Duc01] defines amI in the following way: "People are surrounded by intelligent, intuitive interfaces that are embedded in all kinds of objects. The ambient intelligence environment is capable of recognizing and responding to the presence of different individuals [while working] in a seamless, unobtrusive way."

With the widespread use of low-cost, yet increasingly powerful smartphones, we are well on our way to ubiquitous amI systems. The challenge for software engineers is to develop apps that provide ever-increasing functionality in products of all types—functionality that adapts to user needs while at the same time protecting privacy and providing security. The rise of digital assistants and intelligent chat bots are indicative of the types of applications ahead.

The engineering of *variability intensive systems* focuses on software that needs to accommodate different usage and deployment scenarios, as well as intentional and unintentional variability in functionality or quality attributes (e.g., performance). This includes meeting the challenges posed by context-aware apps, autonomous agents, and pervasive computing as well as creating product-line software.[7] These systems can be highly variable during all software engineering activities (e.g., dynamic run-time conditions, rapidly changing requirements, configuration management), and we need to improve our understanding of how to design and manage them in a cost-effective manner [Gal17].

29.4.3 Emergent Requirements

At the beginning of a software project, there's a truism that applies equally to every stakeholder involved: "You don't know what you don't know." That means that customers rarely define "stable" requirements. It also means that software engineers cannot always foresee where ambiguities and inconsistencies lie. Requirements change—but that's nothing new.

As systems become more complex, it follows that even a rudimentary attempt to state comprehensive requirements is doomed to failure. A statement of overall goals may be possible, delineation of intermediate objectives can be accomplished, but stable requirements—not a chance! Requirements will emerge as everyone involved in the engineering and construction of a complex system learns more about it, the environment in which it is to reside, and the users who will interact with it.

This reality implies a number of software engineering trends. First, process models must be designed to embrace change and adopt the basic tenets of the agile philosophy (Chapter 3). Next, methods that yield engineering models (e.g., requirements and

7 Product-line software is a set of application programs that are built from a common set of reusable software modules designed to be easily adaptable when creating in new software products.

design models) must be used judiciously because those models will change repeatedly as more knowledge about the system is acquired. Finally, tools that support both process and methods must make adaptation and change easy.

But there is another aspect to emergent requirements. The vast majority of software developed to date assumes that the boundary between the software-based system and its external environment is stable. The boundary may change, but it will do so in a controlled manner, allowing the software to be adapted as part of a regular software maintenance cycle. This assumption is beginning to change. The growth of variability intensive systems (Section 29.4.2) demands that software "adapt and react to changes dynamically, even if they're unanticipated" [Bar06b].

By their nature, emergent requirements lead to change. How do we control the evolution of a widely used application or system over its lifetime, and what effect does this have on the way we design software?

As the number of changes grows, the likelihood of unintended side effects also grows. This should be a cause for concern as complex systems with emergent requirements become the norm. The software engineering community must develop methods that help software teams predict the impact of change across an entire system, thereby mitigating unintended side effects. Today, our ability to accomplish this is severely limited.

29.4.4 The Talent Mix

As software-based systems become more complex, as communication and collaboration among global teams becomes commonplace, as emergent requirements (with the resultant flow of changes) become the norm, the very nature of a software engineering team may change. Each software team must bring a variety of creative talent and technical skills to its part of a complex system, and the overall process must allow the output of these islands of talent to merge effectively. Using data mining for knowledge discovery in the human aspects of software engineering may help managers select the right development team before beginning a project [Gup15].

Alexandra Weber-Morales [Mor05] suggests the talent mix of a "software dream team." The *Brain* is a chief architect who is able to navigate the demands of stakeholders and map them into a technology framework that is both extensible and implementable. The *Data Grrl* is a database and data structures guru who "blasts through rows and columns with profound understanding of predicate logic and set theory as it pertains to the relational model." The *Blocker* is a technical leader (manager) who allows the team to work free of interference from other teams while at the same time ensuring that collaboration is occurring. The *Hacker* is a consummate programmer who is at home with patterns and languages and can use both effectively. The *Gatherer* "deftly discovers system requirements with . . . anthropological insight" and accurately expresses them with clarity.

29.4.5 Software Building Blocks

All of us who have fostered a software engineering philosophy have emphasized the need for reuse—of source code, object-oriented classes, components, patterns, and libraries. Although the software engineering community has made progress as it attempts to capture past knowledge and reuse proven solutions, a significant percentage

of the software that is built today continues to be built "from scratch." Part of the reason for this is a continuing desire (by stakeholders and software engineering practitioners) for "unique solutions."

In the hardware world, original equipment manufacturers (OEMs) of digital devices use application-specific standard products (ASSPs) produced by silicon vendors almost exclusively. This "merchant hardware" provides the building blocks necessary to implement everything from a smartphone to a wearable computing device. Increasingly, the same OEMs are using "merchant software"—software building blocks designed specifically for a unique application domain [e.g., Voice over Internet Protocol (VoIP) devices]. Michael Ward [War07] comments:

> One advantage of the use of software components is that the OEM can leverage the functionality provided by the software without having to develop in-house expertise in the specific functions or invest developer time on the effort to implement and validate the components. Other advantages include the ability to acquire and deploy only the specific set of functionalities that are needed for the system, as well as the ability to integrate these components into an already-existing architecture.

In addition to components packaged as merchant software, there is an increasing tendency to adopt *software platform solutions* that "incorporate collections of related functionalities, typically provided within an integrated software framework" [War07]. A software platform frees an OEM from the work associated with developing base functionality and instead allows the OEM to dedicate software effort on those features that differentiate its product.

29.4.6 Changing Perceptions of "Value"

During the last quarter of the twentieth century, the operative question that businesspeople asked when discussing software was: Why does it cost so much? That question is rarely asked today and has been replaced by: Why can't we get it (software and/or the software-based product) sooner?

When computer software is considered, the modern perception of value is changing from business value (cost and profitability) to customer values that include: speed of delivery, richness of functionality, and overall product quality.

29.4.7 Open Source

Who owns the software you or your organization uses? Increasingly, the answer is "everyone." The "open source" movement has been described in the following manner [OSO12]: "Open source is a development method for software that harnesses the power of distributed peer review and transparency of process. The promise of open source is better quality, higher reliability, more flexibility, lower cost, and an end to predatory vendor lock-in." The term *open source,* when applied to computer software, implies that software engineering work products (models, source code, test suites) are open to the public and can be reviewed and extended (with controls) by anyone with interest and permission.

If you have further interest, Weber [Web05] provides a worthwhile introduction, Feller and his colleagues [Fel07] have edited a comprehensive and objective anthology that considers the benefits and problems associated with open source, and Brown [Bro12] provides a more technical discussion.

29.5 TECHNOLOGY DIRECTIONS

We always seem to think that software engineering will change more rapidly than it does. A new "hyped" technology (it could be a new process, a unique method, or an exciting tool) is introduced, and pundits suggest that "everything" will change. But software engineering is about far more than technology—it's about people and their ability to communicate their needs and innovate to make those needs a reality. Whenever people are involved, change occurs slowly in fits and starts. It's only when a "tipping point" [Gla02] is reached that a technology cascades across the software engineering community and broad-based change truly does occur.

In this section we'll examine a few trends in process, methods, and tools that are likely to have some influence on software engineering over the next decade. Will they lead to a tipping point? We'll just have to wait and see.

29.5.1 Process Trends

It can be argued that all the business, organizational, and cultural trends discussed in Section 29.4 reinforce the need for process. But do the frameworks discussed in Chapter 28 provide a road map into the future? Will process frameworks evolve to find a better balance between discipline and creativity? Will the software process adapt to the differing needs of stakeholders who procure software, those who build it, and those who use it? Can it provide a means for reducing risk for all three constituencies at the same time?

These and many other questions remain open. In the paragraphs that follow, we have adapted six ideas proposed by Conradi and Fuggetta [Con02] to suggest possible process trends.

1. **As SPI frameworks evolve, they will emphasize "strategies that focus on goal orientation and product innovation"** [Con02]. In the fast-paced world of software development, long-term SPI strategies rarely survive in a dynamic business environment. Too much changes too quickly. This means that a stable, step-by-step road map for SPI may have to be replaced with a framework that emphasizes short-term goals that have a product orientation.
2. **Because software engineers have a good sense of where the process is weak, process changes should generally be driven by their needs and should start from the bottom up.** Conradi and Fuggetta [Con02] suggest that future SPI activities should "use a simple and focused scorecard to start with, not a large assessment." By focusing SPI efforts narrowly and working from the bottom up, practitioners will begin to see substantive changes early—changes that make a real difference in the way that software engineering work is conducted.
3. **Automated software process technology (SPT) will move away from global process management (broad-based support of the entire software process) to focus on those aspects of the software process that can best benefit from automation.** No one is against tools and automation, but in many instances, SPI has not met its promise (see Section 29.3). To be most effective, it should focus on umbrella activities (Chapter 1)—the most stable elements of the software process.

4. **Greater emphasis will be placed on the return on investment of SPI activities.** In Chapter 28 you learned that return on investment (ROI) can be defined as:

$$\text{ROI} = \frac{\Sigma(\text{benefits}) - \Sigma(\text{costs})}{\Sigma(\text{costs})} \times 100\%$$

   To date, software organizations have struggled to clearly delineate "benefits" in a quantitative manner. It can be argued [Con02] that "we therefore need a standardized market-value model . . . to account for software improvement initiatives."

5. **As time passes, the software community may come to understand that expertise in sociology and anthropology may have as much or more to do with successful SPI as other, more technical disciplines.** More than anything else SPI changes organizational culture, and cultural change involves individuals and groups of people. Conradi and Fuggetta [Con02] correctly note that "software developers are knowledge workers. They tend to respond negatively to top-level dictates on how to do work or change processes." Much can be learned by examining the sociology of groups to better understand effective ways to introduce change.

6. **New modes of learning may facilitate the transition to a more effective software process.** In this context, "learning" implies learning from successes and mistakes. A software organization that collects metrics (Chapter 23) allows itself to understand how elements of a process affect the quality of the end product.

29.5.2 The Grand Challenge

There is one trend that is undeniable—software-based systems will undoubtedly become bigger and more complex as time passes. It is the engineering of these large, complex systems, regardless of delivery platform or application domain, that poses the "grand challenge" [Bro06] for software engineers. Manfred Broy [Bro06] suggests that software engineers can meet "the daunting challenge of complex software systems development" by creating new approaches to understanding system models and using those models as a basis for the construction of high-quality next-generation software. Techniques currently being studied for variability intensive systems (continuous delivery, self-adaptive software, value-based software engineering, content aware computing) may benefit the developers of all types of software products [Gal17].

As the software engineering community develops new model-driven approaches (discussed briefly later in this section) to the representation of system requirements and design, the following characteristics [Bro06] must be addressed:

- **Multifunctionality.** As digital devices evolve, they have begun to deliver a rich set of sometimes unrelated functions. The mobile phone, once considered a straightforward communication device, has become a powerful pocket computer that performs a wide spectrum of functions that are arguably more important than making a phone call. As Broy [Bro06] notes, "[E]ngineers must describe the detailed context in which the functions will be delivered

and, most important, must identify the potentially harmful interactions between the system's different features."

- **Reactivity and timeliness.** Digital devices increasingly interact with the real world and must react to external stimuli in a timely manner. They must interface with a broad array of sensors and must respond in a period that is appropriate to the task at hand. New methods must be developed that (1) help software engineers predict the timing of various reactive features and (2) implement those features in a way that makes the feature less machine dependent and more portable.
- **New modes of user interaction.** Open-world trends for software mean that new modes of interaction must be modeled and implemented. Whether these new approaches use multitouch interfaces, voice recognition, or direct mind interfaces, new generations of software for digital devices must accommodate them.
- **Complex architectures.** A luxury automobile has over 2,000 functions controlled by software residing within a complex hardware architecture that includes multiple processors, a sophisticated bus structure, actuators, sensors, an increasingly sophisticated human interface, and many safety-rated components. Even more complex systems (e.g., autonomous vehicles) are on the immediate horizon, presenting significant challenges for software designers.
- **Heterogeneous, distributed systems.** The real-time components of any modern embedded system can be connected via an internal bus, a wireless network, or across the Internet (or all three).
- **Criticality.** Software has become the pivotal component in virtually all business-critical systems and in most safety-critical systems. Yet, the software engineering community has only begun to apply even the most basic principles of software safety.
- **Maintenance variability.** The life of software within a digital device rarely lasts beyond 3 to 5 years, but the complex avionics systems within an aircraft has a useful life of at least 20 years. Automobile software falls somewhere in between. Should this have an impact on design?

Broy [Bro06] argues that these and other software characteristics can be managed only if the software engineering community develops a more effective distributed and collaborative software engineering philosophy, better requirements engineering approaches, a more robust approach to model-driven development, and better software tools. In the sections that follow, we'll explore each of these areas briefly.

29.5.3 Collaborative Development

It seems too obvious to state, but we'll do so anyway: Software engineering is an information technology. From the onset of any software project, every stakeholder must share information—about basic business goals and objectives, about specific system requirements, about architectural design issues, about almost every aspect of the software to be built. Collaboration involves the timely dissemination of information and an effective process for communication and decision making.

Today, software engineers collaborate across time zones and international boundaries. Every one of them must share information. The same holds for open-source

projects in which hundreds or thousands of software developers work to build an open-source app. Crowd sourcing has been suggested as a means of enhancing coverage test cases generated by automated testing tools [Mao17]. Coordination of such large testing communities will be challenging. Information must be disseminated so that open collaboration can occur.

29.5.4 Requirements Engineering

Basic requirements engineering actions—elicitation, elaboration, negotiation, specification, and validation—were presented in Chapters 7 and 8. The success or failure of these actions has a very strong influence on the success or failure of the entire software engineering process. And yet, requirements engineering (RE) has been compared to "trying to put a hose clamp around jello" [Gon04]. As we've noted in many places throughout this book, software requirements have a tendency to keep changing, and with the advent of open-world systems, emergent requirements (and near-continuous change) may become the norm.

Today, most "informal" requirements engineering approaches begin with the creation of user scenarios (e.g., use cases). More formal approaches create one or more requirements models and use these as a basis for design. Formal methods enable a software engineer to represent requirements using a verifiable mathematical notation. All can work reasonably well when requirements are stable, but do not readily solve the problem of dynamic or emergent requirements.

There are a number of distinct requirements engineering research directions including natural language processing from translated textual descriptions into more structured representations (e.g., analysis classes), greater reliance on databases for structuring and understanding software requirements, the use of RE patterns to describe typical problems and solutions when requirements engineering tasks are conducted, and goal-oriented requirements engineering. However, at the industry level, RE actions remain relatively informal and surprisingly basic. To improve the manner in which requirements are defined, the software engineering community will likely implement three distinct subprocesses as RE is conducted [Gli07]: (1) improved knowledge acquisition and knowledge sharing that allows more complete understanding of application domain constraints and stakeholder needs, (2) greater emphasis on iteration as requirements are defined, and (3) more effective communication and coordination tools that enable all stakeholders to collaborate effectively.

The RE subprocesses noted in the preceding paragraph will only succeed if they are properly integrated into an evolving approach to software engineering. As pattern-based problem solving and component-based solutions begin to dominate many application domains, RE must accommodate the desire for agility (rapid incremental delivery) and the inherent emergent requirements that result. The notion of a static "software specification" is beginning to disappear, to be replaced by "value-driven requirements" [Som05] derived as stakeholders respond to features and functions delivered in early software increments.

29.5.5 Model-Driven Software Development

Software engineers grapple with abstraction at virtually every step in the software engineering process. As design commences, architectural and component-level

abstractions are represented and assessed. They must then be translated into a programming language representation that transforms the design (a relatively high level of abstraction) into an operable system with a specific computing environment (a low level of abstraction). *Model-driven software development*[8] couples domain-specific modeling languages with transformation engines and generators in a way that facilitates the representation of abstraction at high levels and then transforms it into lower levels [Sch06]. Model-driven approaches address a continuing challenge for all software developers—how to represent software at a higher level of abstraction than code.

Domain-specific modeling languages (DSMLs) represent "application structure, behavior and requirements within particular application domains" and are described with meta-models that "define the relationships among concepts in the domain and precisely specify the key semantics and constraints associated with these domain concepts" [Sch06]. The key difference between a DSML and a general-purpose modeling language such as UML (Appendix 1) is that the DSML is tuned to design concepts inherent in the application domain and can therefore represent relationships and constraints among design elements in an efficient manner.

29.5.6 Search-Based Software Engineering

Many activities in software engineering can be stated as optimization problems. *Search-based software engineering* (SBSE) applies metaheuristic search techniques such as genetic algorithms[9] to software engineering problems. Lionel Briand [Bri09] believes that evolutionary and other search techniques are more easily scaled to industrial-size problems than model-driven techniques and that there are opportunities for synergy between the two. Search-based software engineering was devised on the premise that it is often easier to check that a candidate solution solves a problem than it is to construct a solution from scratch [Kul13].

Search-based software engineering techniques can be used as the basis for genetic improvement to grow software products by grafting on new functional and nonfunctional features to existing software product line [Har14]. Genetic improvement of software has already resulted in dramatic performance improvements (e.g., execution time, energy usage, and memory consumption) in existing software products [Pet18]. Successful software products evolve continually; however, evolution, if not properly managed, may weaken the software quality and may need to be refactored to remain viable.

Search-based software engineering techniques have been used to generate and repair sequences of refactoring recommendations. It is time consuming to create refactoring recommendations manually. Using a dynamic, interactive approach to generate refactoring recommendations can improve software quality while minimizing deviations from the original design [Ali18]. Search-based software engineering techniques have been used to design test cases to evaluate developer fixes to the software following crashes [Als18]. It is possible that these techniques may lead to software systems capable of repairing themselves.

---

8 The term *model-driven engineering* (MDE) is also used.

9 Genetic algorithms can be used to generate high-quality solutions to optimization and search problems by relying on bio-inspired operators such as mutation, crossover, and selection to evolve potential solutions from a population of potential solutions.

29.5.7 Test-Driven Development

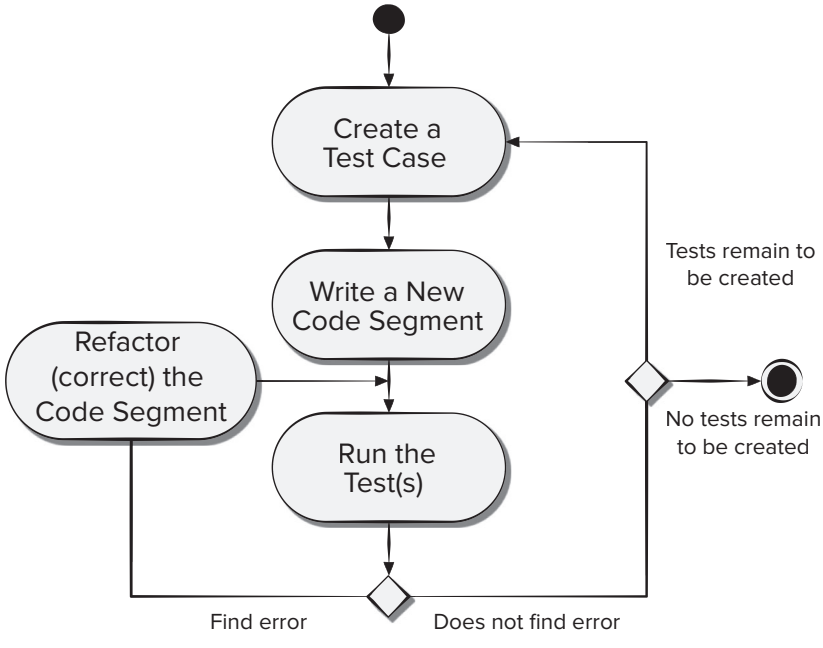
Requirements drive design, and design establishes a foundation for construction. This simple software engineering reality works reasonably well and is essential as a software architecture is created. However, a subtle change can provide significant benefit when component-level design and construction are considered.

In *test-driven development* (TDD), requirements for a software component serve as the basis for the creation of a series of test cases that exercise the interface and attempt to find errors in the data structures and functionality delivered by the component. TDD is not really a new technology but rather a trend that emphasizes the design of test cases *before* the creation of source code.[10]

The TDD process follows the simple procedural flow illustrated in Figure 29.3. Before the first small segment of code is created, a software engineer creates a test to exercise the code (to try to make the code fail). The code is then written to satisfy the test. If it passes, a new test is created for the next segment of code to be developed. The process continues until the component is fully coded and all tests execute without error. However, if any test succeeds in finding an error, the existing code is refactored (corrected) and all tests created to that point are executed again. This iterative flow continues until there are no tests left to be created, implying that the component meets all requirements defined for it.

FIGURE 29.3

Test-driven development process flow



10 Recall that Extreme Programming (Chapter 3) emphasizes this approach as part of its agile process model.

During TDD, code is developed in very small increments (one subfunction at a time), and no code is written until a test exists to exercise it. You should note that each iteration results in one or more new tests that are added to a regression test suite that is run with every change. This is done to ensure that the new code has not generated side effects that cause errors in the older code. If you have further interest in TDD, see [Bec04b], [Ste10], or [Whi12].

29.6 TOOLS-RELATED TRENDS

Hundreds of industry-grade software engineering tools are introduced each year. The majority are provided by tools vendors who claim that their tool will improve project management, or requirements analysis, or design modeling, or code generation, or testing, or change management, or any of the many software engineering activities, actions, and tasks discussed throughout this book. Other tools have been developed as open-source offerings. The majority of open-source tools focus on "programming" activities with a specific emphasis on the construction activity (particularly code generation). Still other tools grow out of research efforts at universities and government labs. Although they have appeal in very limited applications, the majority are not ready for broad industry application.

At the industry level, the most comprehensive tools packages form *software engineering environments* (SEEs)[11] that integrate a collection of individual tools around a central database (repository). When considered as a whole, a SEE integrates information across the software process and assists in the collaboration that is required for many large, complex software-based systems. But current environments are not easily extensible (it's difficult to integrate a COTS tool that is not part of the package) and tend to be general purpose (i.e., they are not application domain specific). There is also a substantial time lag between the introduction of new technology solutions (e.g., model-driven software development) and the availability of viable SEEs that support the new technology.

In the past, software tools followed two distinct paths—a *human-focused path* that responds to some of the "soft trends" discussed in Section 29.4, and a technology-centered path that addresses new technologies (Section 25.5) as they are introduced and adopted. Moving forward, software engineers are beginning to build tools that focus on the interaction of humans and technology. Machine-generated solutions do not always apply to every problem. Humans are still needed to make the decision to accept a machine recommendation or not.

The soft trends discussed in Section 29.4—the need to manage complexity, accommodate emergent requirements, establish process models that embrace change, coordinate global teams with a changing talent mix, among others—suggest a new era in which tools support for stakeholder collaboration will become as important as tools support for technology.

Agility in software engineering (Chapter 3) is achieved when stakeholders work as a team. Therefore, the trend toward collaborative SEEs will provide benefits even

11 The term *integrated development environment* (IDE) is also used.

when software is developed locally. But what of the technology tools that complement the system and components that empower better collaboration?

One of the dominant trends in technology tools is the creation of a tool set that supports model-driven development (Section 29.5.5) with an emphasis on architecture-driven design. Oren Novotny [Nov04] suggests that the model rather than the source code becomes the central software engineering focus:

> Platform independent models are created in UML and then undergo various levels of transformation to eventually wind up as source code for a specific platform. It makes sense then, that the model, not the file, should become the new unit of output. A model has many different views at different levels of abstraction. At the highest level, platform independent components can be specified in analysis; at the lowest level there is a platform specific implementation that reduces to a set of classes in code.

Novotny argues that a new generation of tools will work in conjunction with a repository to create models at all necessary levels of abstraction, establish relationships between the various models, translate models at one level of abstraction to another level (e.g., translate a design model into source code), manage changes and versions, and coordinate quality control and assurance actions against the software models. Marouane Kessentini has deployed industry-grade tools at companies like eBay and SEMA that are designed to reduce technical debt problems by automatically detecting software defects [Man17] and recommending refactoring solutions to resolve them [Ali18]. This work is showing great promise.

In addition to complete software engineering environments, point-solution tools that address everything from requirements gathering to design/code refactoring to testing will continue to evolve and become more functionally capable. In some instances, modeling and testing tools targeted at a specific application domain will provide enhanced benefit when compared to their generic equivalents. Mark Harman's group at Facebook has announced deployment of a tool that automatically designs test cases and tests developers fixes following software crashes [Als18] with the hope that production software may be able to repair itself someday.

29.7 SUMMARY

The trends that have an effect on software engineering technology often come from business, organizational, market, and cultural arenas. These "soft trends" can guide the direction of research and the technology that is derived as a consequence of research. It is likely that artificial intelligence and data science methods will continue to impact all aspects of software engineering.

As a new technology is introduced, it moves through a life cycle that does not always lead to widespread adoption, even though original expectations are high. The degree to which any software engineering technology gains widespread adoption is tied to its ability to address the problems posed by both soft and hard trends. Digital personal assistants and social media seem to be influencing the activities of individuals in many aspects of everyday life. With their rise have come concerns about the importance of both security and privacy in the development of software products.

Soft trends—the growing need for connectivity and collaboration, global projects, knowledge transfer, the impact of emerging economies, and the influence of human

culture itself—lead to a set of challenges that spans managing complexity and emergent requirements, to juggling an ever-changing talent mix among geographically dispersed software teams. Global engineering is likely here to stay.

Hard trends—the ever-accelerating pace of technology change—flow out of soft trends and affect the structure of the software and scope of the process and the manner in which a process framework is characterized. Collaborative development, new forms of requirements engineering, model-based and test-driven development, and postmodern design will change the methods landscape. Tools environments will respond to a growing need for communication and collaboration and at the same time integrate domain-specific point solutions that may change the nature of current software engineering tasks. Machine learning is likely to be one approach to automating many important software engineering tasks.

PROBLEMS AND POINTS TO PONDER

29.1. Get a copy of the best-selling book *The Tipping Point* by Malcolm Gladwell (available via Google Book Search), and discuss how his theories apply to the adoption of new software engineering technologies.

29.2. Why does open-world software present a challenge to conventional software engineering approaches?

29.3. Review the Gartner Group's *hype cycle for emerging technologies.* Select a well-known technology product, and present a brief history that illustrates how it traveled along the curve. Select another well-known technology product that did not follow the path suggested by the hype curve.

29.4. What is a "soft trend"?

29.5. You're faced with an extremely complex problem that will require a lengthy solution. How would you go about addressing the complexity and crafting a solution?

29.6. What are "emergent requirements," and why do they present a challenge to software engineers?

29.7. Select an open-source development effort (other than Linux), and present a brief history of its evolution and relative success.

29.8. Describe how you think the software process will change over the next decade.

29.9. You're based in Los Angeles and are working on a global software engineering team. You and colleagues in London, Mumbai, Hong Kong, and Sydney must edit a 245-page requirements specification for a large system. The first editing pass must be completed in three days. Describe the ideal online tool set that would enable you to collaborate effectively.

29.10. Describe model-driven software development in your own words. Do the same for test-driven development.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

30 CONCLUDING COMMENTS

In the 29 chapters that have preceded this one, we've explored a process for software engineering that encompasses management procedures and technical methods, basic concepts and principles, specialized techniques, people-oriented activities and tasks that are amenable to automation, paper-and-pencil notation, and software tools. We have argued that measurement, discipline, and an overriding focus on agility and quality will result in software that meets the customer's needs, software that is reliable, software that is supportable, software that is better. Yet, we have never promised that software engineering is a panacea.

KEY CONCEPTS

artificial intelligence 606
change 603
communication 604
ethics 607
future 606
genetic algorithms 606
information spectrum 605
knowledge 605
knowledge discovery 605
machine learning 606
people 603
responsibility 607
software revisited 603



QUICK LOOK

What is it? As we come to the end of a relatively long journey through software engineering, it's time to put things into perspective and make a few concluding comments.

Who does it? Authors like us. When you come to the end of a long and challenging book, it's nice to wrap things up in a meaningful way.

Why is it important? It's always worthwhile to remember where we've been and to consider where we're going.

What are the steps? We'll consider where we've been and address some of the core issues and some directions for the future.

What is the work product? A discussion that will help you understand the big picture.

How do I ensure that I've done it right? That's difficult to accomplish in real time. It's only after a number of years that any of us can tell whether the software engineering concepts, principles, methods, and techniques discussed in this book have helped you to become a better software engineer.

Software and systems technologies remain a challenge for every software professional and every company that builds computer-based systems. Although he wrote these words with a twentieth-century outlook, Max Hopper [Hop90] accurately describes the current state of affairs:

> Because changes in information technology are becoming so rapid and unforgiving, and the consequences of falling behind are so irreversible, companies will either master the technology or die . . . Think of it as a technology treadmill. Companies will have to run harder and harder just to stay in place.

Changes in software engineering technology are indeed "rapid and unforgiving," but at the same time real progress is often quite slow. By the time a decision is made to adopt a new process, method, or tool; conduct the training necessary to understand its application; and introduce the technology into the software development culture, something newer (and even better) has come along, and the process begins anew.

One thing we've learned over our years in this field is that software engineering practitioners are "fashion conscious." The road ahead will be littered with the carcasses of exciting new technologies (the latest fashion) that never really made it (despite the hype). It will be shaped by more modest technologies that somehow modify the direction and width of the thoroughfare. We discussed a few of those in Chapter 29.

In this concluding chapter, we'll take a broader view and consider where we've been and where we're going from a more philosophical perspective.

30.1 The Importance of Software—Revisited

The importance of computer software can be stated in many ways. In Chapter 1, software was characterized as a differentiator. The function delivered by software differentiates products, systems, and services and provides competitive advantage in the marketplace. But software is more than a differentiator. When taken as a whole, software engineering work products generate the most important commodity that any individual, business, or government can acquire—information.

In Chapter 29, we briefly discussed open-world computing—a technology that is fundamentally changing our perception of computers, the things that we do with them (and they do for us), and our perception of information as a guide, a commodity, and a necessity. We also noted that software required to support open-world computing will present dramatic new challenges for software engineers. But far more important, the growing pervasiveness of computer software will present even more dramatic challenges for society as a whole. Whenever a technology has a broad impact—an impact that can save lives or endanger them, build businesses or destroy them, inform government leaders or mislead them—it must be handled with care.

30.2 People and the Way They Build Systems

The software required for high-technology systems becomes more complex with each passing year, and the size of resultant programs increases proportionally. The rapid growth in the size of the "average" program would present us with few problems if it wasn't for one simple fact: As program size increases, the number of people who must work on the program must also increase.

Experience indicates that as the number of people on a software project team increases, the overall productivity of the group may suffer. One way around this problem is to create a number of software engineering teams, thereby compartmentalizing people into individual working groups. However, as the number of software engineering teams grows, communication between them becomes as difficult and time consuming. Worse, communication (between individuals or teams) tends to be inefficient—that is, too much time is spent transferring too little information content, and all too often, important information "falls into the cracks."

If the software engineering community is to deal effectively with the communication dilemma, the road ahead for software engineers must include radical changes in the way individuals and teams communicate with one another. In Chapter 27, we discussed the use of social media in supporting software release management that may provide dramatic improvements in the ways developers and customers communicate.

Finally, communication is the transfer of knowledge, and the acquisition (and transfer) of knowledge is changing in profound ways. As search engines become increasingly sophisticated, social networking and crowd-sourcing morph into development tools, and mobile applications provide better synergy, the speed and quality of knowledge transfer will grow exponentially.

If past history is any indication, it is fair to say that people themselves will not change. However, the ways in which they communicate, the environment in which they work, the manner in which they acquire knowledge, the methods and tools that they use, the discipline that they apply, and therefore, the overall culture for software development will change in significant and even profound ways.

SAFEHOME



Conclusion?

The scene: Doug Miller's office.

The players: Doug Miller, manager of the *SafeHome* software engineering group, and Vinod Raman, a member of the product software engineering team.

The conversation:

Doug: I'm really pleased that we got it done without too much drama.

Vinod (sighing and leaning back in his chair): Yeah, but the project grew, didn't it?

Doug: And you're surprised? When we started *SafeHome*, marketing thought a desktop app would do the trick, and then . . .

Vinod (smiling): And then, context aware mobility took over and we dabbled with VR.

Doug: But we all learned a lot.

Vinod: We did. The tech stuff was interesting, but the software engineering stuff is probably what allowed us to get it done close to schedule.

Doug: Yeah, that and hard work by all of you guys. What are you seeing from customer support? How's quality in the field?

Vinod: There are a few issues, but nothing really serious. We're on it. In fact, I gotta meet with Jamie on one of them in 5 minutes.

Doug: Before you go . . .

Vinod (on his way out the door): I know, more work, right?

Doug: Engineering has developed a new sensor . . . very high tech . . . we'll need to integrate it in *SafeHome II.*

Vinod: *SafeHome II?*

Doug: Yeah, *SafeHome II.* We'll begin planning next week.

30.3 KNOWLEDGE DISCOVERY

Over the history of computing, a subtle transition has occurred in the terminology that is used to describe software development work performed for the business community. Fifty years ago, the term *data processing* was the operative phrase for describing the use of computers in a business context. Today, data processing has given way to another phrase—*information technology*—that implies the same thing but presents a subtle shift in focus. The emphasis is not merely to process large quantities of data but rather to extract meaningful information from this data. Obviously, this was always the intent, but the shift in terminology reflects a far more important shift in management philosophy.

When software applications are discussed today, the words *data, information,* and *content* occur repeatedly. We encounter the word *knowledge* in many artificial intelligence applications. Virtually no one discusses *wisdom* in the context of software applications.

Data is raw information—collections of facts that must be processed to be meaningful. Information is derived by associating facts within a given context. Knowledge relates information obtained in one context with other information obtained in a different context. Finally, wisdom occurs when generalized principles are derived from disparate bits of knowledge.

To date, the vast majority of all software has been built to process data or information. Software engineers are now equally concerned with systems that process knowledge.[1] Information collected on a variety of related and unrelated topics is connected to form a body of facts that we call *knowledge*. The key is our ability to associate information from a variety of different sources that may not have any obvious connection and combine it in a way that provides us with some distinct benefit.[2]

To illustrate the progression from data to knowledge, consider census data indicating that the birthrate in 1996 in the United States was 4.9 million. This number represents a data value. Relating this piece of data with birthrates for the preceding 40 years, we can derive a useful piece of information—aging baby boomers of the 1950s and early 1960s made a last-gasp effort to have children prior to the end of their child-bearing years. In addition, gen-Xers began their childbearing years. The census data can then be connected to other seemingly unrelated pieces of information, for example, the current number of elementary school teachers who will retire during the next decade, the number of college students graduating with degrees in primary and secondary education, and the pressure on politicians to hold down taxes and therefore limit pay increases for teachers.

All these pieces of information can be combined to formulate a representation of knowledge—there will be significant pressure on the education system in the United States in the early twenty-first century, and this pressure will continue for a number of decades. Using this knowledge, a business opportunity may emerge. There may be significant opportunity to develop new modes of learning that are more effective and less costly than current approaches.

1 The rapid growth of data mining and data warehousing technologies reflect this growing trend.

2 The semantic Web (Web 2.0) allows the creation of "mashups" that may provide a facile mechanism for achieving this.

The road ahead for software leads to systems that both discover and process knowledge. We have been processing data using computers for more than 70 years and extracting information for more than three decades. One of the most significant challenges facing the software engineering community is to build systems that take the next step along the spectrum—systems that extract knowledge from data and information in a way that is practical and beneficial. *Knowledge discovery* is an interdisciplinary area focusing upon methodologies for extracting useful relationships from data.

Mark Harman (currently Facebook's manager for software engineering research) was one of the first people to recognize the value of using data mining and machine learning to solve difficult software engineering problems [Har12b]. The availability of several public software engineering data repositories (e.g., *Bugzilla, GitHub, SourceForge*) make it possible to use search-based software engineering techniques to discover insights into software development artifacts and processes [Dye15] [Gup15]. This is not an easy task and suggests that it may be desirable to include data scientists as members of large software engineering projects [Kim16b]. The knowledge discovered while mining public repositories may suggest practice improvements to software engineers working on smaller proprietary projects or may yield techniques that can be applied on their own software engineering data repositories.

Machine learning has been used in many areas of software engineering ranging from behavior extraction, design pattern recognition, program generation, test-case generation, and defect detection [Mei18]. This work cannot be done without access to large sets of software engineering data and access to domain experts to help shape the concepts learned by machines. *Genetic algorithms*[3] make use of automated search and can be used to grow an improved software product or process by heuristically combining elements of existing software products and processes. Genetics have been used to improve software performance for a diverse set of properties such as execution time, memory consumption, as well as defect repair and existing system functionality extensions [Pet18].

Intelligent software engineering is emerging as an academic area of study that combines artificial intelligence (AI) and software engineering. Intelligent software engineering techniques explore software engineering solutions to improve the productivity of developing AI software and the dependability of AI software. It also seeks to address some of the problems encountered when attempting to automate software engineering processes [Xie18]. As AI techniques become more powerful and easier to use, they are increasingly deployed as key components of modern software systems. Although this allows the creation of products better able to adapt to user needs, it also creates additional problems for software engineers and exposes companies to new risks [Fel18].

30.4 The Long View

In Section 30.3, we suggested that the road ahead leads to systems that "discover knowledge." But the future of computing in general and software-based systems in particular may lead to events that are considerably more profound.

3 Genetic algorithms are used to search a population of computer-generated potential solutions to a problem with the intent of finding the best solution while maintaining the diversity of the set of candidate solutions.

In a fascinating book that is must reading for every person involved in computing technologies, Ray Kurzweil [Kur05] suggests that we have reached a time when "the pace of technological change will be so rapid, its impact so deep, that human life will be irreversibly transformed." Kurzweil[4] makes a compelling argument that we are currently at the "knee" of an exponential growth curve that will lead to enormous increases in computing capacity over the next few decades. When coupled with equivalent advances in nanotechnology, genetics, and robotics, we may approach a time in the middle part of this century when the distinction between humans (as we know them today) and machines begins to blur—a time when human evolution accelerates in ways that are both frightening (to some) and spectacular (to others).

Kurzweil argues that sometime in the coming decade computing capacity and the requisite software will be sufficient to model every aspect of the human brain, including all the physical connections, analog processes, and chemical overlays [Kur13]. When this occurs, human beings will take the first step toward achieving "strong AI (artificial intelligence)," and as a consequence, machines that truly do think (using today's conventional use of the word). But there will be a fundamental difference. Human brain processes are exceedingly complex and only loosely connected to external informal sources. They are also computationally slow, even in comparison to today's computing technology. When full human brain emulation occurs, "thought" will occur at speeds thousands of times more rapid than its human counterpart with intimate connections to a sea of information (think of the present-day Web as a primitive example). The result is . . . well . . . so fantastical that it's best left to Kurzweil to describe.

It's important to note that not everyone believes that the future Kurzweil describes is a good thing. In a now-famous essay titled "The Future Doesn't Need Us," Bill Joy [Joy00], one of the founders of Sun Microsystems, argues that "robotics, genetic engineering, and nanotech are threatening to make humans an endangered species." His arguments, along with commentary by luminaries such Bill Gates, Elon Musk, and the late Steven Hawking, predicting a potential technology dystopia represent a counterpoint to Kurzweil's predicted utopian future. Both should be seriously considered as software engineers take one of the lead roles in defining the long view for the human race.

30.5 THE SOFTWARE ENGINEER'S RESPONSIBILITY

Software engineering has evolved into a respected, worldwide profession. As professionals, software engineers should abide by a code of ethics that guides the work they do and the products they produce. An ACM/IEEE-CS Joint Task Force has produced

4 It's important to note that Kurzweil is not a run-of-the mill science fiction writer or a futurist without portfolio. He is a serious technologist who (from Wikipedia) "has been a pioneer in the fields of optical character recognition (OCR), text-to-speech synthesis, speech recognition technology, and electronic keyboard instruments."

a *Software Engineering Code of Ethics and Professional Practices* (Version 5.1). The code [ACM12] states:

> Software engineers shall commit themselves to making the analysis, specification, design, development, testing, and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:
>
> 1. PUBLIC—Software engineers shall act consistently with the public interest.
> 2. CLIENT AND EMPLOYER—Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
> 3. PRODUCT—Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
> 4. JUDGMENT—Software engineers shall maintain integrity and independence in their professional judgment.
> 5. MANAGEMENT—Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
> 6. PROFESSION—Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
> 7. COLLEAGUES—Software engineers shall be fair to and supportive of their colleagues.
> 8. SELF—Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Although each of these eight principles is equally important, an overriding theme appears: A software engineer should work in the public interest. On a personal level, a software engineer should abide by the following rules:

- Never steal data for personal gain.
- Never distribute or sell proprietary information obtained as part of your work on a software project.
- Never maliciously destroy or modify another person's programs, files, or data.
- Never violate the privacy of an individual, a group, or an organization.
- Never hack into a system for sport or profit.
- Never create or promulgate a computer virus or worm.
- Never use computing technology to facilitate discrimination or harassment.

Over the past decade, certain members of the software industry have lobbied for protective legislation that (1) allows companies to release software without disclosing known defects, (2) exempts developers from liability for any damages resulting from these known defects, (3) constrains others from disclosing defects without permission from the original developer, (4) allows the incorporation of "self-help" software within a product that can disable (via remote command) the operation of the product, and (5) exempts developers of software with "self-help" from damages should the software be disabled by a third party.

Like all legislation, debate often centers on issues that are political, not technological. However, many people (including us) feel that protective legislation, if

improperly drafted, conflicts with the software engineering code of ethics by indirectly exempting software engineers from their responsibility to produce high-quality software. Given the sizeable social media data breaches that occurred in 2018, there will be more demands for enhanced security protections by companies storing large amounts of confidential customer data.

The growing decision-making capabilities of autonomous systems and the influence of AI in our daily lives make us want to consider the values embedded in these systems [Vak18]. The software engineering profession needs to examine ways to measure bias in search algorithms and social networks [Pit18]. The revised ACM Code of Ethics and Professional Conduct [ACM18] adopts several new principles that address issues in specific computing technologies such as AI, machine learning, and autonomous machines making ethically significant decisions [Got18]. It is likely that the ACM and IEEE will consider revising their Software Engineering Code of Ethics to reflect these new areas as well.

30.6 A Final Comment from RSP

It has been almost four decades since work on the first edition of this book began. I [RSP] can still recall sitting at my desk as a young professor, writing the manuscript for a book on a subject that few people cared about and even fewer really understood. I remember the rejection letters from publishers, who argued (politely, but firmly) that there would never be a market for a book on "software engineering." Luckily, McGraw-Hill decided to try it,[5] and the rest, as they say, is history.

Since the first edition, this book has changed dramatically—in scope, in size, in style, and in content. Like software engineering, it has grown and matured over the years.

An engineering approach to the development of computer software is now conventional wisdom. Debate continues on the "right paradigm," the importance of agility, the degree of automation, and the most effective methods. But the underlying principles of software engineering are now accepted throughout the industry. Why, then, have we seen their broad adoption only recently?

The answer, I think, lies in the difficulty of technology transition and the cultural change that accompanies it. Even though most of us appreciate the need for an engineering discipline for software, we struggle against the inertia of past practice and face new application domains (and the developers who work in them) that appear ready to repeat the mistakes of the past. To ease the transition, we need many things—an agile, adaptable, and sensible software process; more effective methods; more powerful tools; better acceptance by practitioners and support from managers; and no small dose of education.

You may not agree with every approach described in this book. Some of the techniques and opinions are controversial; others must be tuned to work well in different

5 Actually, credit should go to Peter Freeman and Eric Munson, who convinced McGraw-Hill that it was worth a shot, and 3 million copies later, it's fair to say they made a good decision.

software development environments. It is my sincere hope, however, that *Software Engineering: A Practitioner's Approach* has delineated the problems we face, demonstrated the strength of software engineering concepts, and provided a framework of methods and tools.

As we move further into the twenty-first century, software continues to be the most important product and the most important industry on the world stage. Its impact and importance have come a long, long way. And yet, a new generation of software developers must meet many of the same challenges faced by earlier generations. Let us hope that the people who meet the challenge—software engineers—will have the wisdom to develop systems that improve the human condition.

Design element: Quick Look icon magnifying glass: © Roger Pressman

APPENDIX

1 AN INTRODUCTION TO UML[1]

KEY CONCEPTS

activity diagram 622
class diagram 612
communication diagram 621
dependency 614
deployment diagram 615
generalization 613
interaction frames 620
multiplicity 614
sequence diagram 618
state diagram 625
stereotype 613
swimlanes 624
use-case diagram 616

The *Unified Modeling Language* (UML) is "a standard language for writing software blueprints. UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system" [Boo05]. In other words, just as building architects create blueprints to be used by a construction company, software architects create UML diagrams to help software developers build the software. If you understand the vocabulary of UML (the diagrams' pictorial elements and their meanings), you can much more easily understand and specify a system and explain the design of that system to others.

Grady Booch, Jim Rumbaugh, and Ivar Jacobson developed UML in the mid-1990s with much feedback from the software development community. UML merged a number of competing modeling notations that were in use by the software industry at the time. In 1997, UML 1.0 was submitted to the Object Management Group, a nonprofit consortium involved in maintaining specifications for use by the computer industry. UML 1.0 was revised to UML 1.1 and adopted later that year. The current standard is UML 2.5.1[2] and is now an ISO standard. Because this standard is new, many older references, such as [Gam95] do not use UML notation.

UML 2.5.1 provides 13 different diagrams for use in software modeling. In this appendix, we will discuss only *class, deployment, use-case, sequence, communication, activity,* and *state diagrams.* These diagrams are used in this edition of *Software Engineering: A Practitioner's Approach.*

You should note that there are many optional features in UML diagrams. The UML language provides these (sometimes arcane) options so that you can express all the important aspects of a system. At the same time, you have the flexibility to suppress those parts of the diagram that are not relevant to the aspect being modeled to avoid cluttering the diagram with irrelevant details. Therefore, the omission of a feature does not mean that the feature is absent; it may mean that the feature was suppressed. In

1 This appendix has been contributed by Dale Skrien and has been adapted from his book, *An Introduction to Object-Oriented Design and Design Patterns in Java* (McGraw-Hill, 2008). All content is used with permission.

2 See https://www.omg.org/spec/UML/2.5.1/. This document contains the current specification for the UML 2.5.1 modeling language.

this appendix, we will not present exhaustive coverage of all the features of the UML diagrams. Instead, we focus on the standard options, especially those options that have been used in this book.

Class Diagrams

To model classes, including their attributes, operations, and their relationships and associations with other classes, UML provides a *class diagram.* A class diagram provides a static or structural view of the system. It does not show the dynamic nature of the communications between the objects of the classes in the diagram.

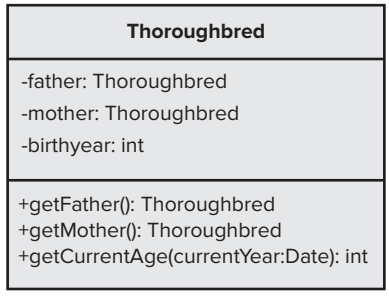
The main elements of a class diagram are boxes, which are the icons used to represent classes and interfaces. Each box is divided into horizontal parts. The top part contains the name of the class. The middle section lists the attributes of the class. Attributes can be values that the class can compute from its instance variables or values that the class can get from other objects of which it is composed. For example, an object might always know the current time and be able to return it to you whenever you ask, in which case it would be appropriate to list the current time as an attribute of that class of objects. However, the object would most likely not have that time stored in one of its instance variables, because it would need to continually update that field. Instead, the object would likely compute the current time (e.g., through consultation with objects of other classes) at the moment when the time is requested. The third section of the class diagram contains the operations or behaviors of the class. An *operation* refers to what objects of the class can do. It is usually implemented as a *method* of the class.

Figure A1.1 presents a simple example of a **Thoroughbred** class that models thoroughbred horses. It has three attributes displayed—`mother`, `father`, and `birth year`. The diagram also shows three operations: *getCurrentAge(), getFather(),* and *getMother()*. There may be other suppressed attributes and operations not shown in the diagram.

Each attribute can have a name, a type, and a level of visibility. The type and visibility are optional. The type follows the name and is separated from the name by a colon. The visibility is indicated by a preceding −, #, ~, or +, indicating, respectively, *private, protected, package,* or *public* visibility. In Figure A1.1, all attributes have private visibility, as indicated by the leading minus sign (−). You can also specify that

FIGURE A1.1

A class diagram for a Thoroughbred class



an attribute is a static or class attribute by underlining it. Each operation can also be displayed with a level of visibility, parameters with names and types, and a return type.

An abstract class or abstract method is indicated by the use of italics for the name in the class diagram. See the **Horse** class in Figure A1.2, for an example. An interface is indicated by adding the phrase "«interface»" (called a *stereotype*) above the name. See the **OwnedObject** interface in Figure A1.2. An interface can also be represented graphically by a hollow circle.

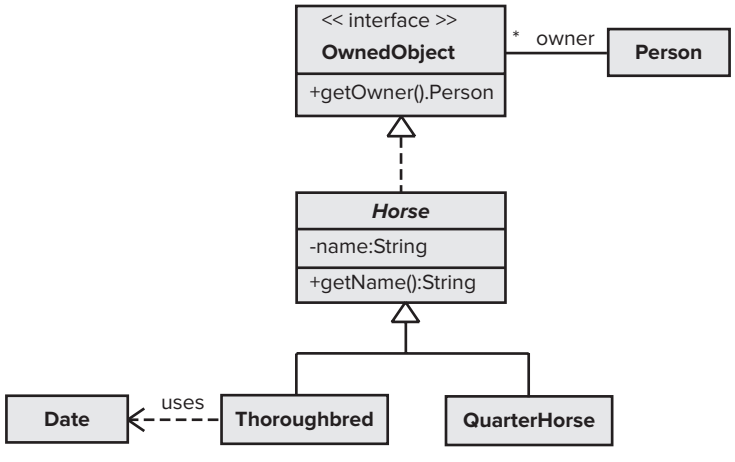
It is worth mentioning that the icon representing a class can have other optional parts. For example, a fourth section at the bottom of the class box can be used to list the responsibilities of the class. This section is particularly useful when transitioning from CRC cards (Chapter 8) to class diagrams in that the responsibilities listed on the CRC cards can be added to this fourth section in the class box in the UML diagram before creating the attributes and operations that carry out these responsibilities. This fourth section is not shown in any of the figures in this appendix.

Class diagrams can also show relationships between classes. A class that is a subclass of another class is connected to it by an arrow with a solid line for its shaft and with a triangular hollow arrowhead. The arrow points from the subclass to the superclass. In UML, such a relationship is called a *generalization*. For example, in Figure A1.2, the **Thoroughbred** and **QuarterHorse** classes are shown to be subclasses of the **Horse** abstract class. An arrow with a dashed line for the arrow shaft indicates implementation of an interface. In UML, such a relationship is called a *realization*. For example, in Figure A1.2, the **Horse** class implements or realizes the **OwnedObject** interface.

An *association* between two classes means that there is a structural relationship between them. Associations are represented by solid lines. An association has many optional parts. It can be labeled, as can each of its ends, to indicate the role of each class in the association. For example, in Figure A1.2, there is an association between **OwnedObject** and **Person** in which the **Person** plays the role of owner. Arrows on either or both ends of an association line indicate navigability. Also, each end of the association line can have a multiplicity value displayed. Navigability and multiplicity

FIGURE A1.2

A class diagram regarding horses



are explained in more detail later in this section. An association might also connect a class with itself, using a loop. Such an association indicates the connection of an object of the class with other objects of the same class.

An association with an arrow at one end indicates one-way navigability. The arrow means that from one class you can easily access the second associated class to which the association points, but from the second class, you cannot necessarily easily access the first class. Another way to think about this is that the first class is aware of the second class, but the second class is not necessarily directly aware of the first class. An association with no arrows usually indicates a two-way association, which is what was intended in Figure A1.2, but it could also just mean that the navigability is not important and so was left off.

It should be noted that an attribute of a class is very much the same thing as an association of the class with the class type of the attribute. That is, to indicate that a class has a property called "name" of type String, one could display that property as an attribute, as in the **Horse** class in Figure A1.2. Alternatively, one could create a one-way association from the **Horse** class to the **String** class with the role of the **String** class being "name." The attribute approach is better for primitive data types, whereas the association approach is often better if the property's class plays a major role in the design, in which case it is valuable to have a class box for that type.

A *dependency* relationship represents another connection between classes and is indicated by a dashed line (with optional arrows at the ends and with optional labels). One class depends on another if changes to the second class might require changes to the first class. An association from one class to another automatically indicates a dependency. No dashed line is needed between classes if there is already an association between them. However, for a transient relationship (i.e., a class that does not maintain any long-term connection to another class but does use that class occasionally) we should draw a dashed line from the first class to the second. For example, in Figure A1.2, the **Thoroughbred** class uses the **Date** class whenever its *getCurrentAge()* method is invoked, and so the dependency is labeled "uses."

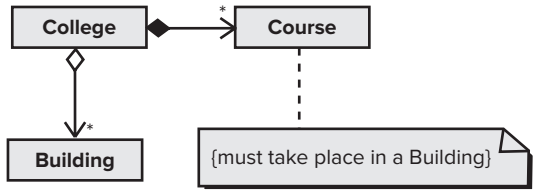
The *multiplicity* of one end of an association means the number of objects of that class associated with the other class. A multiplicity is specified by a nonnegative integer or by a range of integers. A multiplicity specified by "0..1" means that there are 0 or 1 objects on that end of the association. For example, each person in the world has either a Social Security number or no such number (especially if they are not U.S. citizens), and so a multiplicity of 0..1 could be used in an association between a **Person** class and a **SocialSecurityNumber** class in a class diagram. A multiplicity specified by "1..*" means one or more, and a multiplicity specified by "0..*" or just "*" means zero or more. An * was used as the multiplicity on the **OwnedObject** end of the association with class **Person** in Figure A1.2 because a **Person** can own zero or more objects.

If one end of an association has multiplicity greater than 1, then the objects of the class referred to at that end of the association are probably stored in a collection, such as a set or ordered list. One could also include that collection class itself in the UML diagram, but such a class is usually left out and is implicitly assumed to be there due to the multiplicity of the association.

An *aggregation* is a special kind of association indicated by a hollow diamond on one end of the icon. It indicates a "whole/part" relationship, in that the class to which

FIGURE A1.3

The relationship between Colleges, Courses, and Buildings



the arrow points is considered a "part" of the class at the diamond end of the association. A *composition* is an aggregation indicating strong ownership of the parts. In a composition, the parts live and die with the owner because they have no role in the software system independent of the owner. See Figure A1.3 for examples of aggregation and composition.

A **College** has an aggregation of **Building** objects, which represent the buildings making up the campus. The college also has a collection of courses. If the college were to fold, the buildings would still exist (assuming the college wasn't physically destroyed) and could be used for other things, but a **Course** object has no use outside of the college at which it is being offered. If the college were to cease to exist as a business entity, the **Course** object would no longer be useful and so it would also cease to exist.

Another common element of a class diagram is a *note*, which is represented by a box with a dog-eared corner and is connected to other icons by a dashed line. It can have arbitrary content (text and graphics) and is similar in a programming language comment. It might contain information about the role of a class or constraints that all objects of that class must satisfy. If the contents are a constraint, braces surround the contents. Note the constraint attached to the **Course** class in Figure A1.3.

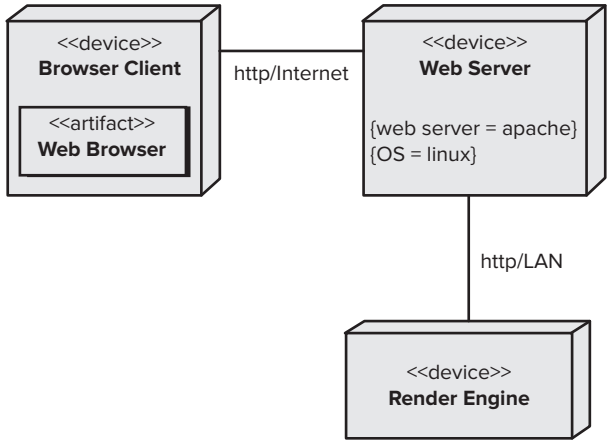
DEPLOYMENT DIAGRAMS

A UML *deployment diagram* focuses on the structure of the software system and is useful for showing the physical distribution of a software system among hardware platforms and execution environments. Suppose, for example, you are developing a Web-based graphics-rendering package. Users of your package will use their Web browser to go to your website and enter rendering information. Your website would render a graphical image according to the user's specification and send it back to the user. Because graphics rendering can be computationally expensive, you decide to move the rendering itself off the Web server and onto a separate platform. Therefore, there will be three hardware devices involved in your system: the Web client (the users' computer running a browser), the computer hosting the Web server, and the computer hosting the rendering engine.

Figure A1.4 shows the deployment diagram for such a package. In such a diagram, hardware components are drawn in boxes labeled with "«device»". Communication paths between hardware components are drawn with lines with optional labels. In Figure A1.4, the paths are labeled with the communication protocol and the type of network used to connect the devices.

FIGURE A1.4

A deployment diagram



Each node in a deployment diagram can also be annotated with details about the device. For example, in Figure A1.4, the browser client is depicted to show that it contains an artifact consisting of the Web browser software. An artifact is typically a file containing software running on a device. You can also specify tagged values, as is shown in Figure A1.4 in the Web server node. These values define the vendor of the Web server and the operating system used by the server.

Deployment diagrams can also display execution environment nodes, which are drawn as boxes containing the label "«execution environment»". These nodes represent systems, such as operating systems, that can host other software programs.

USE-CASE DIAGRAMS

Use cases (Chapters 7 and 8) and the UML *use-case diagram* help you determine the functionality and features of the software from the user's perspective. To give you a feeling for how use cases and use-case diagrams work, we'll create some for a software application for managing an online digital music store. Some of the things the software might do include:

- Download an MP3 music file and store it in the application's library.
- Capture streaming music and store it in the application's library.
- Manage the application's library (e.g., delete songs or organize them in playlists).
- Burn a list of the songs in the library onto a CD.
- Load a list of the songs in the library onto an iPod or MP3 player.
- Convert a song from MP3 format to AAC format and vice versa.

This is not an exhaustive list, but it is sufficient to understand the role of use cases and use-case diagrams.

A *use case* describes how a user interacts with the system by defining the steps required to accomplish a specific goal (e.g., burning a list of songs onto a CD).

Variations in the sequence of steps describe various scenarios (e.g., what if all the songs in the list don't fit on one CD?).

A UML use-case diagram is an overview of all the use cases and how they are related. It provides a big picture of the functionality of the system. A use-case diagram for the digital music application is shown in Figure A1.5.

In this diagram, the stick figure represents an *actor* (Chapter 8) that is associated with one category of user (or other interaction element). Complex systems typically have more than one actor. For example, a vending machine application might have three actors representing customers, repair personnel, and vendors who refill the machine.

In the use-case diagram, the use cases are displayed as ovals. The actors are connected by lines to the use cases that they carry out. Note that none of the details of the use cases are included in the diagram and instead need to be stored separately. Note also that the use cases are placed in a rectangle but the actors are not. This rectangle is a visual reminder of the system boundaries and that the actors are outside the system.

Some use cases in a system might be related to each other. For example, there are similar steps in burning a list of songs to a CD and in loading a list of songs to an iPod or smartphone. In both cases, the user first creates an empty list and then adds songs from the library to the list. To avoid duplication in use cases, it is usually better to create a new use case representing the duplicated activity and then let the other

FIGURE A1.5

A use-case diagram for the music system

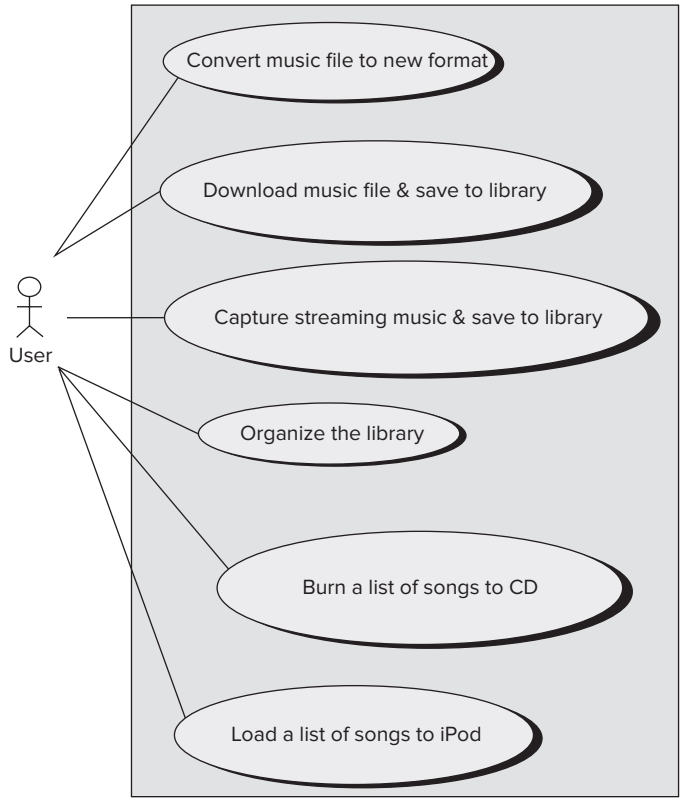
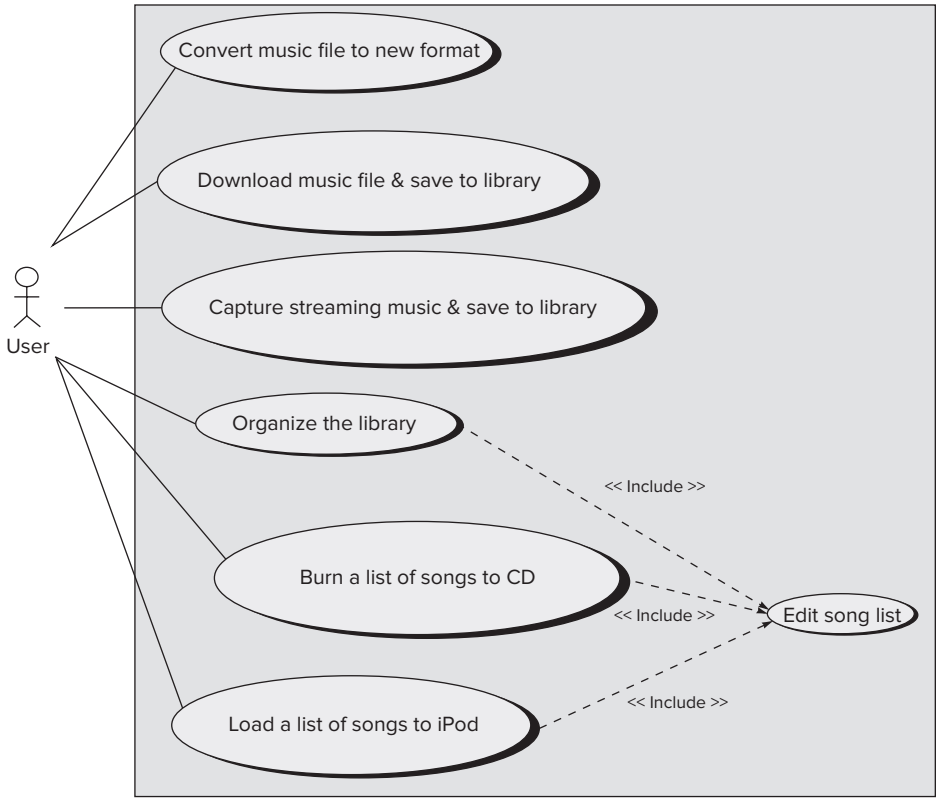


FIGURE A1.6

A use-case diagram with included use cases



use cases include this new use case as one of their steps. Such inclusion is indicated in use-case diagrams, as in Figure A1.6, by means of a dashed arrow labeled «include» connecting a use case with an included use case.

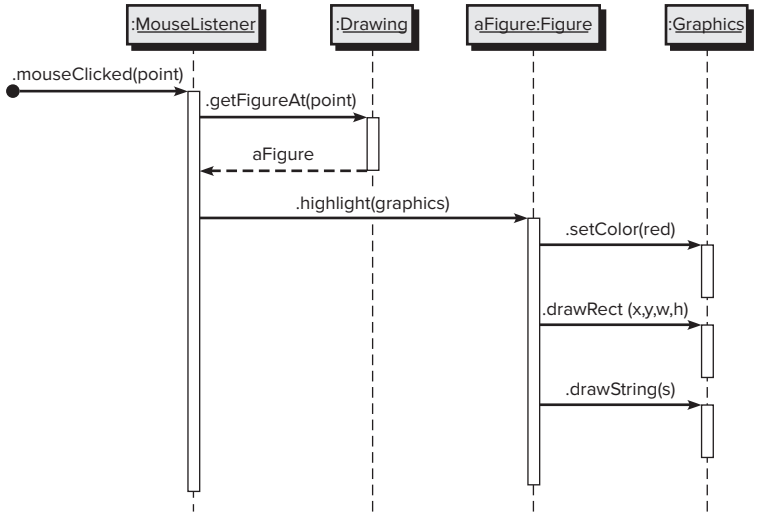
A use-case diagram, because it displays all use cases, is a helpful aid for ensuring that you have covered all the functionality of the system. In our digital music organizer, we would surely want more use cases, such as a use case for playing a song in the library. But keep in mind that the most valuable contribution of use cases to the software development process is the textual description of each use case, not the overall use-case diagram [Fow04]. It is through the descriptions that you are able to form a clear understanding of the goals of the system you are developing.

SEQUENCE DIAGRAMS

In contrast to class diagrams and deployment diagrams, which show the static structure of a software component, a *sequence diagram* is used to show the dynamic communications between objects during execution of a task. It shows the temporal order in which messages are sent between the objects to accomplish that task. One might use a sequence diagram to show the interactions in one use case or in one scenario of the software system.

FIGURE A1.7

A sample sequence diagram



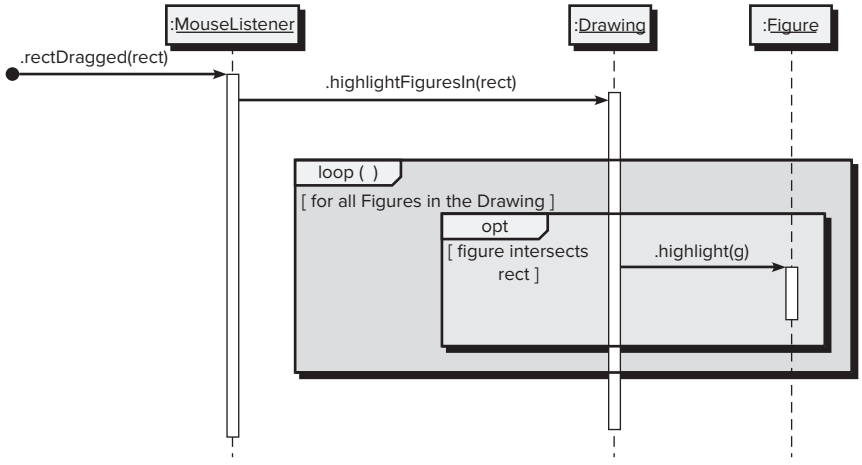
In Figure A1.7, you see a sequence diagram for a drawing program. The diagram shows the steps involved in highlighting a figure in the drawing when it has been clicked. Each box in the row at the top of the diagram usually corresponds to an object, although it is possible to have the boxes model other things, such as classes. If the box represents an object (as is the case in all our examples), then inside the box you can optionally state the type of the object preceded by the colon. You can also precede the colon and type by a name for the object, as shown in the third box in Figure A1.7. Below each box there is a dashed line called the *lifeline* of the object. The vertical axis in the sequence diagram corresponds to time, with time increasing as you move downward.

A sequence diagram shows method calls using horizontal arrows from the *caller* to the *callee*, labeled with the method name and optionally including its parameters, their types, and the return type. For example, in Figure A1.7, the **MouseListener** calls the **Drawing**'s *getFigureAt()* method. When an object is executing a method (that is, when it has an activation frame on the stack), you can optionally display a white bar, called an *activation bar*, down the object's lifeline. In Figure A1.7, activation bars are drawn for all method calls. The diagram can also optionally show the return from a method call with a dashed arrow and an optional label. In Figure A1.7, the *getFigureAt()* method call's return is shown labeled with the name of the object that was returned. A common practice, as we have done in Figure A1.7, is to leave off the return arrow when a void method has been called, since it clutters up the diagram while providing little information of importance. A black circle with an arrow coming from it indicates a *found message* whose source is unknown or irrelevant.

You should now be able to understand the task that Figure A1.7 is displaying. An unknown source calls the *mouseClicked()* method of a **MouseListener**, passing in the point where the click occurred as the argument. The **MouseListener** in turn calls the *getFigureAt()* method of a **Drawing**, which returns a **Figure.** The **MouseListener** then calls the highlight method of **Figure**, passing in a **Graphics** object as an argument. In response, **Figure** calls three methods of the **Graphics** object to draw the figure in red.

FIGURE A1.8

A sequence diagram with two interaction frames



The diagram in Figure A1.7 is very straightforward and contains no conditionals or loops. If logical control structures are required, it is probably best to draw a separate sequence diagram for each case. That is, if the message flow can take two different paths depending on a condition, then draw two separate sequence diagrams, one for each possibility.

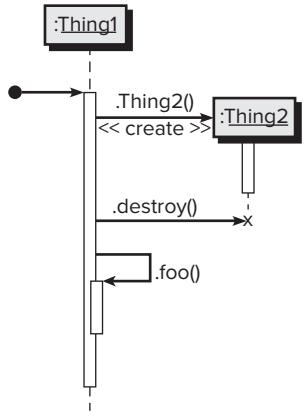
If you insist on including loops, conditionals, and other control structures in a sequence diagram, you can use *interaction frames*, which are rectangles that surround parts of the diagram and that are labeled with the type of control structures they represent. Figure A1.8 illustrates this, showing the process involved in highlighting all figures inside a given rectangle. The **MouseListener** is sent the `rectDragged` message. The **MouseListener** then tells the drawing to highlight all figures in the rectangle by calling the method *highlightFiguresIn()*, passing the rectangle as the argument. The method loops through all **Figure** objects in the **Drawing** object and, if the **Figure** intersects the rectangle, the **Figure** is asked to highlight itself. The phrases in square brackets are called *guards*, which are Boolean conditions that must be true if the action inside the interaction frame is to continue.

There are many other special features that can be included in a sequence diagram. For example:

1. You can distinguish between synchronous and asynchronous messages. Synchronous messages are shown with solid arrowheads, while asynchronous messages are shown with stick arrowheads.
2. You can show an object sending itself a message with an arrow going out from the object, turning downward, and then pointing back to the same object.
3. You can show object creation by drawing an arrow appropriately labeled (for example, with a «create» label) to an object's box. In this case, the box will appear lower in the diagram than the boxes corresponding to objects already in existence when the action begins.

FIGURE A1.9

Creation, destruction, and loops in sequence diagrams



4. You can show object destruction by a big X at the end of the object's lifeline. Other objects can destroy an object, in which case an arrow points from the other object to the X. An X is also useful for indicating that an object is no longer usable and so is ready for garbage collection.

The last three features are all shown in the sequence diagram in Figure A1.9.

COMMUNICATION DIAGRAMS

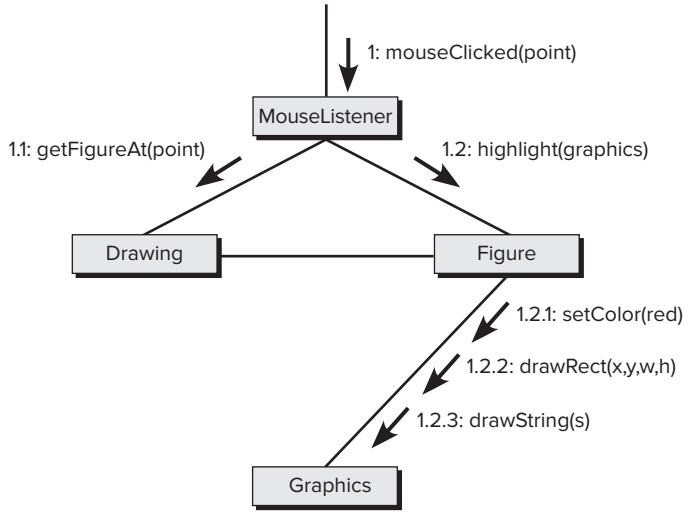
The UML *communication diagram* (known as a "collaboration diagram" in UML 1.X) provides another indication of the temporal order of the communications, but emphasizes the relationships among the objects and classes instead of the temporal order. A communication diagram is illustrated in Figure A1.10, which displays the same actions shown in the sequence diagram in Figure A1.7.

In a communication diagram the interacting objects are represented by rectangles. Associations between objects are represented by lines connecting the rectangles. There is typically an incoming arrow to one object in the diagram that starts the sequence of message passing. That arrow is labeled with a number and a message name. If the incoming message is labeled with the number 1 and if it causes the receiving object to invoke other messages on other objects, then those messages are represented by arrows from the sender to the receiver along an association line and are given numbers 1.1, 1.2, and so forth, in the order they are called. If those messages in turn invoke other messages, another decimal point and number are added to the number labeling these messages, to indicate further nesting of the message passing.

In Figure A1.10, you see that the **mouseClicked** message invokes the methods *getFigureAt()* and then *highlight()*. The *highlight()* message invokes three other messages: *setColor(), drawRect(),* and *drawString()*. The numbering in each label shows the nesting as well as the sequential nature of each message.

There are many optional features that can be added to the arrow labels. For example, you can precede the number with a letter. An incoming arrow could be labeled `A1: mouseClicked(point)`. indicating an execution thread, A. If other messages are

FIGURE A1.10
A UML communication diagram



executed in other threads, their label would be preceded by a different letter. For example, if the *mouseClicked()* method is executed in thread A but it creates a new thread B and invokes *highlight()* in that thread, then the arrow from **MouseListener** to **Figure** would be labeled `1.B2: highlight(graphics)`.

If you are interested in showing the relationships among the objects in addition to the messages being sent between them, the communication diagram is probably a better option than the sequence diagram. If you are more interested in the temporal order of the message passing, then a sequence diagram is probably better.

ACTIVITY DIAGRAMS

A UML *activity diagram* depicts the dynamic behavior of a system or part of a system through the flow of control between actions that the system performs. It is similar to a flowchart except that an activity diagram can show concurrent flows.

The main component of an activity diagram is an *action* node, represented by a rounded rectangle, which corresponds to a task performed by the software system. Arrows from one action node to another indicate the flow of control. That is, an arrow between two action nodes means that after the first action is complete the second action begins. A solid black dot forms the *initial node* that indicates the starting point of the activity. A black dot surrounded by a black circle is the *final node,* indicating the end of the activity.

A *fork* represents the separation of activities into two or more concurrent activities. It is drawn as a horizontal black bar with one arrow pointing to it and two or more arrows pointing out from it. Each outgoing arrow represents a flow of control that can be executed concurrently with the flows corresponding to the other outgoing arrows. These concurrent activities can be performed on a computer using different threads or even using different computers.

FIGURE A1.11

A UML activity diagram showing how to bake a cake

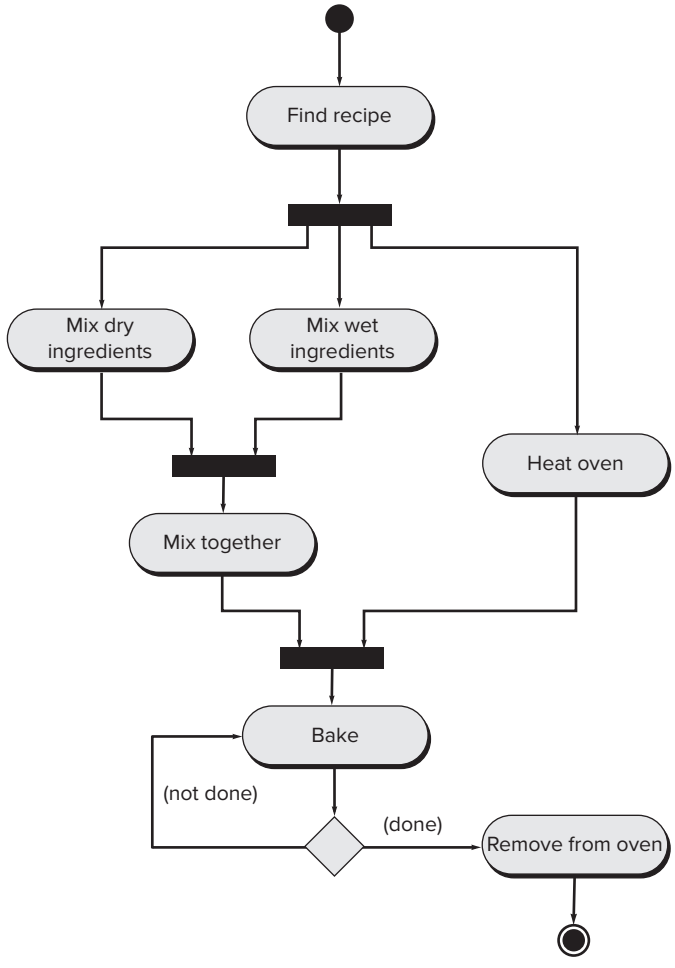


Figure A1.11 shows a sample activity diagram involving baking a cake. The first step is finding the recipe. Once the recipe has been found, the dry ingredients and wet ingredients can be measured and mixed and the oven can be preheated. The mixing of the dry ingredients can be done in parallel with the mixing of the wet ingredients and the preheating of the oven.

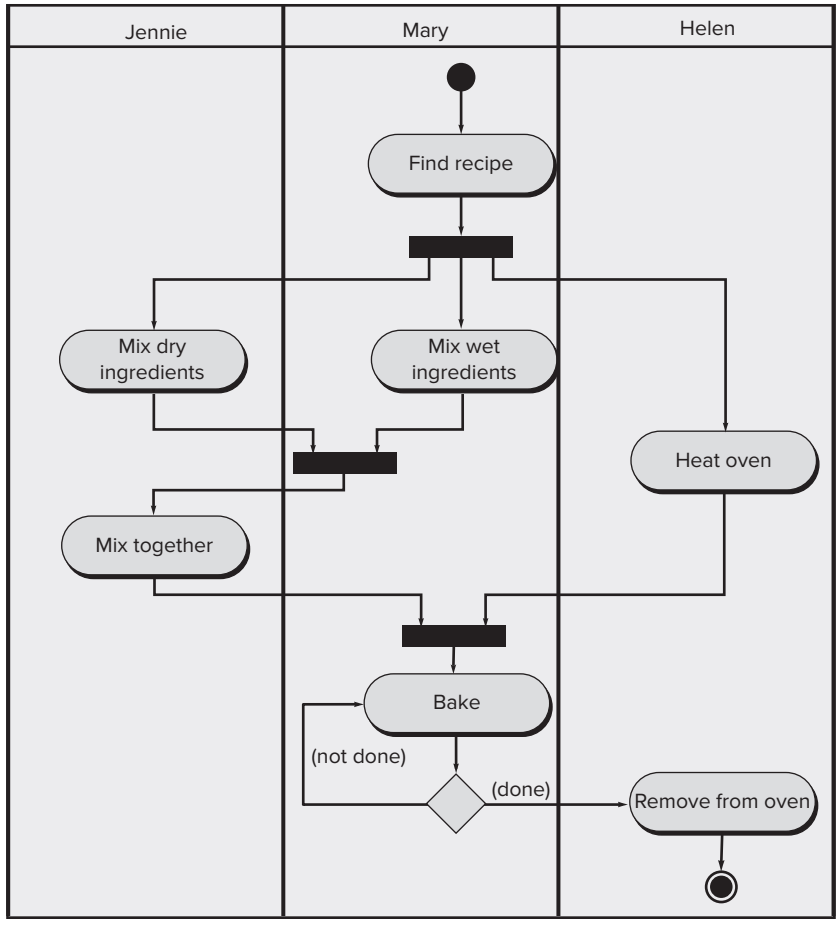
A *join* is a way of synchronizing concurrent flows of control. It is represented by a horizontal black bar with two or more incoming arrows and one outgoing arrow. The flow of control represented by the outgoing arrow cannot begin execution until all flows represented by incoming arrows have been completed. In Figure A1.11, we have a join before the action of mixing together the wet and dry ingredients. This join indicates that all dry ingredients must be mixed and all wet ingredients must be mixed before the two mixtures can be combined. The second join in the figure indicates that, before the baking of the cake can begin, all ingredients must be mixed together, and the oven must be at the right temperature.

A *decision* node corresponds to a branch in the flow of control based on a condition. Such a node is displayed as a white triangle with an incoming arrow and two or more outgoing arrows. Each outgoing arrow is labeled with a guard (a condition inside square brackets). The flow of control follows the outgoing arrow whose guard is true. It is advisable to make sure that the conditions cover all possibilities so that exactly one of them is true every time a decision node is reached. Figure A1.11 shows a decision node following the baking of the cake. If the cake is done, then it is removed from the oven. Otherwise, it is baked for a while longer.

One of the things the activity diagram in Figure A1.11 does not tell you is who or what does each of the actions. Often, the exact division of labor does not matter. But if you do want to indicate how the actions are divided among the participants, you can decorate the activity diagram with swimlanes, as shown in Figure A1.12. *Swimlanes*, as the name implies, are formed by dividing the diagram into strips or "lanes," each of which corresponds to one of the participants. All actions in one lane are done by the corresponding participant. In Figure A1.12, Jennie is responsible for

FIGURE A1.12

The cake-baking activity diagram with swimlanes added



mixing the dry ingredients and then mixing the dry and wet ingredients together, Helen is responsible for heating the oven and taking the cake out, and Mary is responsible for everything else.

STATE DIAGRAMS

The behavior of an object at a particular point in time often depends on the state of the object, that is, the values of its variables at that time. As a trivial example, consider an object with a Boolean instance variable. When asked to perform an operation, the object might do one thing if that variable is *true* and do something else if it is *false*.

A UML *state diagram* models an object's states, the actions that are performed depending on those states, and the transitions between the states of the object.

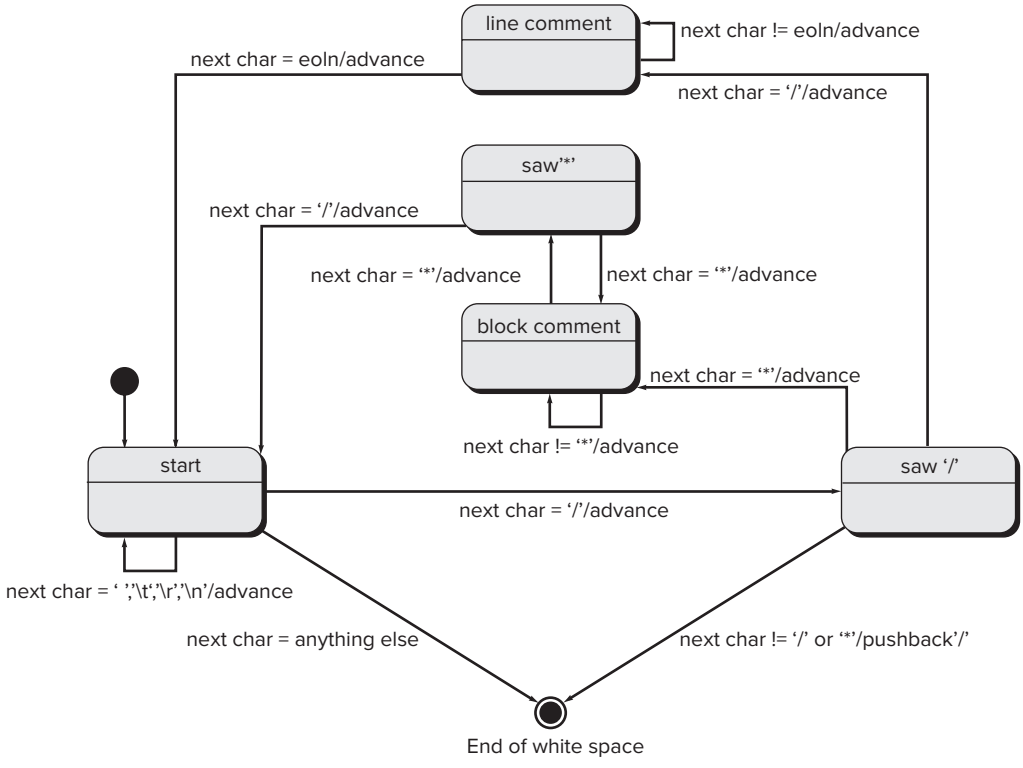
As an example, consider the state diagram for a part of a Java compiler. The input to the compiler is a text file, which can be thought of as a long string of characters. The compiler reads characters one at a time and from them determines the structure of the program. One small part of this process of reading the characters involves ignoring "white-space" characters (e.g., the *space, tab, newline,* and *return* characters) and characters inside a comment.

Suppose that the compiler delegates to a **WhiteSpaceAndCommentEliminator** the job of advancing over white-space characters and characters in comments. That is, this object's job is to read input characters until all white-space and comment characters have been read, at which point it returns control to the compiler to read and process non-white-space and noncomment characters. Think about how the **WhiteSpaceAndCommentEliminator** object reads in characters and determines whether the next character is white space or part of a comment. The object can check for white space by testing the next character against " ", "\t", "\n", and "\r". But how does it determine whether the next character is part of a comment? For example, when it sees a "/" for the first time, it doesn't yet know whether that character represents a division operator, part of the /= operator, or the beginning of a line or block comment. To make this determination, **WhiteSpaceAndCommentEliminator** needs to make a note of the fact that it saw a "/" and then move on to the next character. If the character following the "/" is another "/" or an "*", then **WhiteSpaceAndCommentEliminator** knows that it is now reading a comment and can advance to the end of the comment without processing or saving any characters. If the character following the first "/" is anything other than a "/" or an "*", then **WhiteSpaceAndCommentEliminator** knows that the "/" represents the division operator or part of the /= operator and so it stops advancing over characters.

In summary, as **WhiteSpaceAndCommentEliminator** reads in characters, it needs to keep track of several things, including whether the current character is white space, whether the previous character it read was a "/", whether it is currently reading characters in a comment, whether it has reached the end of comment, and so forth. These all correspond to different states of the **WhiteSpaceAndCommentEliminator** object. In each of these states, **WhiteSpaceAndCommentEliminator** behaves differently with regard to the next character read in.

To help you visualize all the states of this object and how it changes state, you can use a UML state diagram as shown in Figure A1.13. A state diagram displays states using rounded rectangles, each of which has a name in its upper half. There is also a

FIGURE A1.13 **A state diagram for advancing past white space and comments in Java**



black circle called the "initial pseudostate," which isn't really a state and instead just points to the initial state. In Figure A1.13, the **start** state is the initial state. Arrows from one state to another state indicate transitions or changes in the state of the object. Each transition is labeled with a trigger event, a slash (/), and an activity. All parts of the transition labels are optional in state diagrams. If the object is in one state and the trigger event for one of its transitions occurs, then that transition's activity is performed and the object takes on the new state indicated by the transition. For example, in Figure A1.13, if the **WhiteSpaceAndCommentEliminator** object is in the **start** state and the next character is "/", then **WhiteSpaceAndCommentEliminator** advances past that character and changes to the **saw '/'** state. If the character after the "/" is another "/", then the object advances to the **line comment** state and stays there until it reads an end-of-line character. If instead the next character after the "/""is a "*", then the object advances to the **block comment** state and stays there until it sees another "*" followed by a "/", which indicates the end of the block comment. Study the diagram to make sure you understand it. Note that, after advancing past white space or a comment, **WhiteSpaceAndCommentEliminator** goes back to the **start** state and starts over. That behavior is necessary since there might be several successive comments or white-space characters before any other characters in the Java source code.

An object may transition to a final state, indicated by a black circle with a white circle around it, which indicates there are no more transitions. In Figure A1.13, the

WhiteSpaceAndCommentEliminator object is finished when the next character is not white space or part of a comment. Note that all transitions except the two transitions leading to the final state have activities consisting of advancing to the next character. The two transitions to the final state do not advance over the next character because the next character is part of a word or symbol of interest to the compiler. Note that if the object is in the **saw '/'** state but the next character is not "/" or "*", then the "/" is a division operator or part of the /= operator and so we don't want to advance. In fact, we want to back up one character to make the "/" into the next character, so that the "/" can be used by the compiler. In Figure A1.13, this activity of backing up is labeled as pushback '/'.

A state diagram will help you to uncover missed or unexpected situations. That is, with a state diagram, it is relatively easy to ensure that all possible trigger events for all possible states have been accounted for. For example, in Figure A1.13, you can easily verify that every state has included transitions for all possible characters.

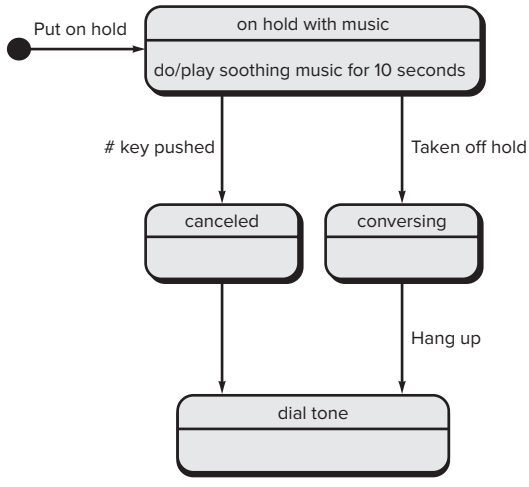
UML state diagrams can contain many other features not included in Figure A1.13. For example, when an object is in a state, it usually does nothing but sit and wait for a trigger event to occur. However, there is a special kind of state, called an *activity state*, in which the object performs some activity, called a *do-activity*, while it is in that state. To indicate that a state is an activity state in the state diagram, you include in the bottom half of the state's rounded rectangle the phrase "do/" followed by the activity that is to be done while in that state. The do-activity may finish before any state transitions occur, after which the activity state behaves like a normal waiting state. If a transition out of the activity state occurs before the do-activity is finished, then the do-activity is interrupted.

Because a trigger event is optional when a transition occurs, it is possible that no trigger event may be listed as part of a transition's label. In such cases for normal waiting states, the object will immediately transition from that state to the new state. For activity states, such a transition is taken as soon as the do-activity finishes.

Figure A1.14 illustrates this situation using the states for a business telephone. When a caller is placed on hold, the call goes into the **On hold with music** state

FIGURE A1.14

A state diagram with an activity state and a triggerless transition



(soothing music is played for 10 seconds). After 10 seconds, the do-activity of the state is completed and the state behaves like a normal nonactivity state. If the caller pushes the # key when the call is in the **On hold with music** state, the call transitions to the **Canceled** state and then transitions immediately to the **dial tone** state. If the # key is pushed before the 10 seconds of soothing music has completed, the do-activity is interrupted and the music stops immediately.

APPENDIX

DATA SCIENCE FOR SOFTWARE ENGINEERS

2

Contributed by: William Grosky and Terry Ruas[1]

KEY CONCEPTS

classification problems 634
computational intelligence 638
data science 629
dimensional reduction 637
machine learning 631
regression problems 634
search-based software engineering 638
statistical models 633

DATA SCIENCE—THE BIG PICTURE

Data science incorporates the work of many different disciplines to transform raw data into information, knowledge, and, hopefully, into wisdom. Data science has a long history that incorporates concepts from computer science, mathematics, statistics, data visualization, along with algorithms and their implementations. It is beyond the scope of this appendix to exhaustively detail all concepts under the rubric of "data science." Instead, we hope to provide a concise summary of the most important topics while connecting them to software engineering.

Figure A2.1 indicates that data science is the intersection of three major areas: computer science, mathematics and statistics, and domain knowledge [Con10].

A data scientist must be interested in more than the data itself. Using knowledge of mathematics and statistics along with domain-specific knowledge, the data scientist develops necessary skills to evaluate whether data, experiments, and evaluation are properly designed for a given problem. However, to bring these capabilities to different scenarios requires a certain flexibility, and good computing skills can be the way to accomplish it.

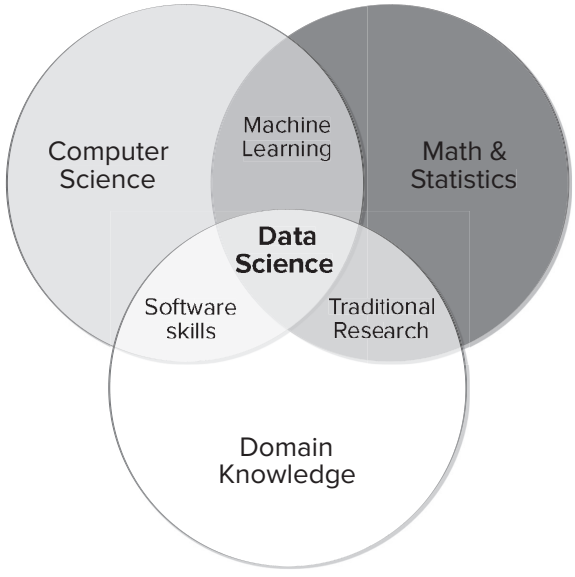
Popular Languages, APIs, and Tools

One of the great things about data science is that you can use it in virtually any environment that allows you to manipulate data. But to accomplish this, you'll need programming languages, APIs, and tools to make your lives easier. We'll provide an overview of these in the following sections.

Languages When it comes to programming languages, we all have our biases toward the one we like the most. For practitioners in data science, this is no different. However, you should keep in mind that one size does not fit all, and the proper approach to language selection for data science applications is to choose the right tool for the right job, considering its constraints, contexts, and goals.

1 Computer and Information Science Department, University of Michigan, Dearborn.

FIGURE A2.1
Data science Venn diagram



For data science application, fast prototyping is a strongly desired characteristic, one that allows us to produce interesting projects with a simplified and intuitive syntax. The resources available with respect to purpose and performance also play a crucial role in the adoption of a programming language. Thus, the less used a language is, the less attractive it will be for our daily tasks. Data science is closely related with *data munging*,[2] which is "the process of transforming and mapping data from one 'raw' data form into another format with the intent of making it more appropriate and valuable for a variety of downstream purposes, such as analytics." Data munging is a time-consuming activity and community support through well-documented APIs and libraries can make a significant difference, especially when looking for use examples, details about specific methods, constraints, and other technical aspects. Thus, a programming language used for data science applications should be broadly adopted, supported, and documented.

Therefore, it should come as no surprise that Python is widely used in the data science community. Other promising programming languages rising among data scientists are Scala and Julia, both more concerned with high performance and scalability. R is another interesting choice for data manipulation, specialized in statistical functions and data visualization libraries. Because its architecture is focused mainly on statistical analysis, data cleaning, and data visualization, R should not be your first choice for general-purpose programming. In other words, R is highly effective if used to solve the right problems.

Java's popularity is indisputable in data science and many other areas of software development. Following the recent trends in data science and big data, Java also has

2 See https://en.wikipedia.org/wiki/Data_wrangling.

dedicated frameworks, such as Hive,[3] Spark,[4] and Hadoop.[5] Considering its nonspecific architecture and verbosity, Java should not be the first option for advanced statistical analysis or data munging, especially for machine-learning algorithms. For these cases, Python and R offer dynamic scripting and huge dedicated libraries that might be more interesting. Other strong programming languages, but not that popular among data scientists, are C/C++, F#, SQL.

Libraries and Tools

It is impossible to talk about data science without referring to the artifacts that assist you in the process of extracting knowledge out of data. In this section, we'll note some of the most popular ones offered in Python [Van16].

NumPy is designed especially to efficiently manipulate *n*-dimensional arrays and handle scientific tasks. You can reshape the number of rows and columns, slice matrices, perform linear algebra operations, sort, search, and perform many other useful tasks. NumPy is used by a vast number of other libraries, and it is part of the SciPy stack.

There are two kinds of SciPy, the library itself and the scientific stack, composed of several open-source ecosystems, including the former. The sub-libraries that form the scientific stack are: NumPy, SciPy, Matplotlib, IPython, Sympy, and Pandas. The library, which is built on top of NumPy, is designed to provide efficient methods to deal with optimization, integration, and several other useful operations [Nun17], [Sci18].

As part of the scientific ecosystem, Pandas helps you with data structures and analysis through easy manipulations. It allows you to shape your data intuitively, providing easy adaptability from unstructured data to structured DataFrames. Some useful functions in Pandas include: indexing, labeling, fixing missing data records, and easy integration with different data structures [McK17], [Num18].

Particularly important for data science, Python also has a large portfolio of machine-learning libraries, in which Scikit-learn, TensorFlow, and Keras have a special place in the spotlight. Scikit-learn is probably one of the most known off-the-shelf machine-learning libraries in Python, featuring several algorithm types, such as clustering, regression, classification, and dimensionality reduction [Ger17]. TensorFlow, originally developed by the Google Brain team (part of Google's AI division), proposes an open-source machine-learning and deep-learning framework for everyone.

Aside from the presented libraries, Python also has several other specialized tools that are used in data science. Examples of visualization tools are Matplotlib, Seaborn, Bokeh, and Plotly; examples of natural language processing (NLP) tools are Natural Language Toolkit (NLTK), Gensim, spaCy, and Scrapy [Act18].

Data Science and Machine Learning

Data science is an umbrella term for a collection of data-driven approaches for finding approximate solutions to very difficult problems. The main enabling technology of data science is *machine learning*, a suite of statistics-based techniques that use an

3 https://hive.apache.org/.
4 https://spark.apache.org/.
5 https://hadoop.apache.org/.

inductive approach that attempts to generalize from a set of known exemplars to unknown exemplars.

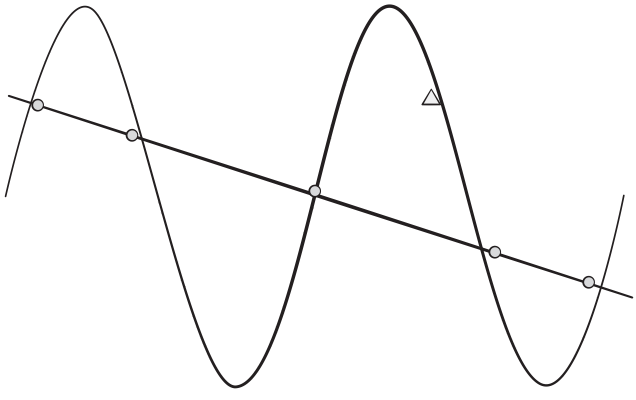
For example, our environment can consist of a set of multiple readings of various meteorological conditions, such as high temperature, low temperature, humidity, as well as several other values. We then have a small subset of these examples, our known exemplars, that are labeled; that is, for each example in this small set, the system is told whether it rained the following day or not. From this *training set* of known exemplars, the system then constructs a mathematical model to categorize an unknown daily example as to whether or not it will rain the following day. Another example from software engineering would be to predict whether an individual piece of code has a fault, based on a training set of faulty and nonfaulty programs. Alternatively, we might try to predict the cost of developing a new version of a program, based on the history of costs of previous versions of that same program.

In model building, there is an inherent tension between trying to construct a model that generalizes the training data, along with following the commonly held principle of Occam's razor, which is that the model should be as simple as possible to explain the current training set data. Figure A2.2 illustrates this conundrum. If the training set consists of only the circles, Occam's razor would choose the linear model, but with the addition of the triangle into the training set, perhaps Occam's razor would choose the sine curve. So, which is the appropriate model: straight line, sine curve, or some, as yet, unknown curve?

In the meteorological example, the system tries to discover commonalities and, at the same time, differences among the meteorological readings, both in the training set for the days it rained the following day and for the days it didn't rain the following day. Then it uses this metadata to determine whether or not it will rain tomorrow. Similarly, in the first software engineering example, the system tries to discover commonalities, in the training set, among programs with faults and those with no faults, as well as how faulty and nonfaulty programs differ, eventually being able to determine whether an unknown program is faulty or not. In the second software engineering example, the system tries to find a general rule that connects the cost of succeeding versions of programs in the training set, using this rule to predict the cost of succeeding versions not in the training set.

FIGURE A2.2

Linear versus nonlinear versus nonlinear model



The process which is followed during a data analytics project consists of (1) collecting the appropriate data, (2) *cleaning* the data, (3) *transforming* the data, (4) *analyzing* the data, and then (5) *fabricating* a training set.

1. **Data collection.** Thinking about what data to collect is quite important, as it depends on the goal of your project. Questions that should be answered include what type of data is needed and how much data should be collected. For example, for software engineering data collection, what type of artifacts are needed? Do we need source code, object code, bug logs? What volume of data do we need to do the appropriate analytics?
2. **Data cleaning.** Once collected, the data must be cleaned. This is a process that consists of eliminating problems in the data that would cause problems in further processing. For example, missing data should be filled in, and corrupt data should be found and corrected.
3. **Data transformation.** After cleaning, data should be transformed so they are more suitable for the downstream analytics tasks. This process is called *data munging* or *data wrangling*. An example of this activity might be changing the format in which the data appears, eliminating punctuation in a text data file, and doing parts-of-speech analysis for text data.
4. **Data analysis.** After all this, the data are ready to be analyzed and processed by various data analytics tools. But, before this happens, we generally use visualization tools for various tasks. For example, it may help us determine which features to use to predict the value of other features. It is only after this that we can determine the best analytical approach to be used for predictive or inferential purposes.
5. **Training set fabrication.** Choosing an appropriate training set is important. The generalizations produced from different training sets might differ among themselves, but the hope is that the downstream answers produced are still correct. It is important not to *overfit* the training set, which means that the approach predicts items in the training set with close to 100 percent accuracy but largely fails to predict the correct results for unknown items. This can happen quite easily if one is not careful to try to avoid this outcome. All of this is determined by testing the derived statistical model on a *test set* of data to determine its error rate.

For the above process to work, objects must be represented by some mathematical structure that can be manipulated easily and compared among themselves. A common way of associating a mathematical structure with an individual object is to use *feature vectors*. A *feature* is a given property of an object. A feature vector is a vector of values for multiple features of an object class, so that the feature vectors for objects in the same object class have the same feature ordering. For example, the meteorological features of a day may have the following structure: low-temperature, high-temperature, low-humidity, high-humidity, prevailing-cloud-type, overall-wind-strength. The variables of low-temperature, high-temperature, low-humidity, high humidity are *continuous* variables, while prevailing-cloud-type is an *unordered categorical variable* and overall-wind-strength is an *ordered categorical variable* (assuming the possible values are weak, average, strong).

In the software engineering environment, a feature vector corresponding to a piece of code could be a vector of values of several software metrics, such as number of lines of code, average program execution time, cohesion, and coupling. It is often challenging to choose the appropriate feature vector, and a new area of study, *feature engineering,* has evolved to help guide the process.

Machine-Learning Approaches

Machine learning is an integral part of data science. The process discussed earlier in this section establishes the data set that is used to drive learning. *Supervised learning* consists of approaches where the user is in the loop and interacts with the learning system, mainly by providing certain types of meta-information, such as labeled data for training sets. *Unsupervised learning* does not have the user in the loop to provide categorical information. These techniques are purely data driven and find ways of labeling the data from the data itself.

Within supervised learning approaches, there are two main types of problems: *classification* problems and *regression* problems.

Classification problems are those whose aim is to determine which of several classes an entity belongs; in other words, to predict a class label. A problem with two possible labels is called a *binary* classification problem, while a problem with more than two classes is called a *multiclass* classification problem. If an entity can fall into several classes, we have a *multilabel* classification problem. In this case, it often happens that the membership of an entity in an individual class is associated with a number between 0 and 1. This number can be interpreted as the strength of membership or the probability of membership. In this case, for a given entity, the sum of all its associated membership strengths or probabilities is equal to 1. A classic example of a binary classification problem is to classify e-mail as *spam* or *nonspam*. An example of a multiclass classification problem would be to classify the contents of an e-mail to various topic classes.

Regression problems are those whose aim is to predict the value of an output variable given the values of several input variables. The value predicted can be real valued or discrete valued. Suppose one had many feature vectors consisting of vita-related information for a prospective new hire. An example of a regression problem would be to predict the length of time that person will stay with your company before looking for a new job.

The boundary between classification problems and regression problems is imprecise. A regression problem where the values predicted are from a finite set can be couched as a classification problem where each class corresponds to a given value in the finite set of predicted values. Similarly, a classification problem can be couched as a regression problem where the output values predicted correspond to the set of class labels.

Popular techniques used for supervised learning include *linear regression*, *logistic regression*, *linear discriminant analysis*, *decision trees*, *k-nearest neighbor*, and *neural networks*. Popular techniques for unsupervised learning approaches include *neural networks*, *clustering*, and *dimensional reduction*. We consider only a small sampling of these techniques in this appendix.

Decision Trees Decision tree learning is a predictive technique that uses data-derived observations contained in the branches of the tree to develop conclusions about a target value contained in the leaves of the tree. Based on the input variable

values, a set of hierarchical decisions are made. The output variable value is found by following a tree from the root to a leaf, based on answers to questions asked along the way.

In general, decision trees can be binary or nonbinary and the questions asked can be arbitrary, as long as they conform to the number of children at an individual node. For this appendix, we will consider only binary trees with Boolean questions of the form $x < a$ or $x \leq b$, for some input variable x and constants a and b. If the answer to a question is TRUE, we would choose the left child to continue our walk down the tree, while if it is FALSE, we would choose the right child.

Given a training set of input and output variable values, we construct the tree choosing the type of Boolean question to ask at each internal node. This is usually done in a greedy fashion, simply asking, at a given node, which decision minimizes the sum of the squared errors. For visualization purposes, let us assume we have two input variables, x_1 and x_2, both of which are continuous. Suppose the training set is of the form (y, x_1, x_2) and is $t_1 = (5.7, 2.3, 9.6)$, $t_2 = (3.5, 1.1, 10)$, $t_3 = (0.55, 3.6, 17.5)$. We first must decide whether the first split will be on x_1 or x_2. We choose the input variable giving us the lowest error.

Now, each node of the tree is associated with a subset of the training set. For example, the root is associated with the entire training set. If the question at the root is $x_1 < 1$, then the root's left child is associated with the empty set and the root's right child is associated with the entire training set. However, if the question at the root is $x_1 < 1.8$, the root's left child is associated with t_2 and the root's right child is associated with the training tuples t_1 and t_3. Note that if we change 1.8 to 2.2, we would have the same association. However, even if the tree is the same, the choice of the split point would affect the results for input value pairs which are not in the training set.

So, what is the error produced by the $x_1 < 1.8$ split? If we stopped at this point, the tree would be used as follows. For an input value pair (c, d), if $c < 1.8$, we would predict the output value of 3.5, while if $c \geq 1.8$, we would predict the output value of 3.125, the average of 5.7 and 0.55. For this example, the squared error produced from the left child is 0, while the squared error produced from the right child is $(3.125 - 5.7)^2 + (3.125 - 0.55)^2 \cong 13.26$. We could stop here, or perform a further refinement on the right-hand side, producing a division of the plane into three regions, each associated with a single training set tuple. See Figure A2.3 for an illustration of the trees, associated regions, and errors for two different splits.

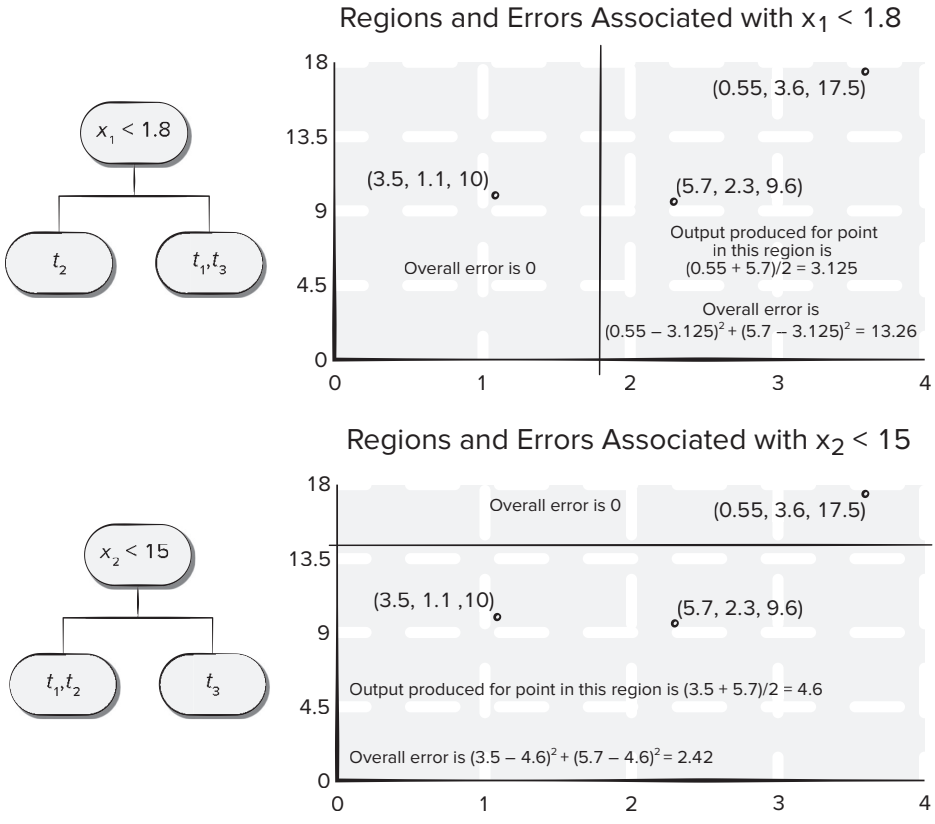
There are many efficient approaches to finding the best tree, which include when and how a region should be split and when to cease splitting a region containing more than one training set tuple. Some of these approaches can be found in [Jam13].

In Young et al. [You18], the use of decision trees in software engineering research is illustrated for the problem of just-in-time defect prediction. This technique predicts defects at small granularities. Code changes are predicted which are more likely to introduce defects. In this paper, it is demonstrated that the decision tree methodology is better than many other learning techniques for this problem. Decision trees are used in an *ensemble learning* environment. This type of learning combines many parallel learners in such a way that the final results are much improved.

Nearest Neighbor The technique of k-nearest neighbor is an approach to estimate the probability of membership of an unclassified input variable tuple, v, in a finite set

FIGURE A2.3

Decision trees examples for different splits (a) Decision tree split $x_1 < 1.8$ (b) Decision tree split $x_2 < 15$



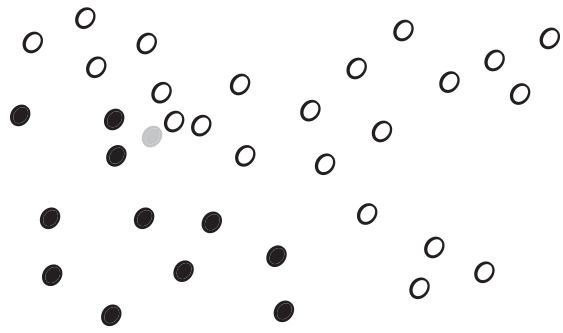
of classes. It is quite simple conceptually but can be very powerful. One finds the closest k points in the training set to the given point v. For a given class, c, suppose there are n points among these closest k training set points that belong to class c. Then the probability that v belongs to c would be n/k. If we had to label v with a single class, it would be the class with the highest probability. The value of k certainly affects the results. It has been found that values of k that are too small or too large don't perform well. As k increases to its sweet spot, the error decreases, but as k further increases, the error gets larger. See Figure A2.4 for an example: with 1-nearest neighbor, the gray point is classified as black; with 3-nearest neighbors, it is classified as white; and with 5-nearest neighbors, it is classified as white.

In Huang et al. [Hua17], nearest neighbor is used to develop an improved approach for missing data in the software quality area. Missing data causes many problems for machine learning. There are many approaches to intelligently estimate values for this data.

Neural Networks Neural networks embody *connectionism*, an architecture consisting of connections of multiple simple processors (i.e., brain cells) together in a massively parallel environment, supporting many concurrent processes, that can be used

FIGURE A2.4

***k*-Nearest neighbor example for $k = 1,3,5$**



to solve many problems. The power of neural networks results from the fact that this approach models the output variable as a nonlinear function of several linear combinations of the input variables. The more powerful neural networks have some form of feedback. To specify a neural network, we must specify the connectivity of the nodes, the way that a given node transforms all its inputs into an output, and how the final output is generated. In general, a neural network uses feedback through what is called a backpropagation technique (steepest descent or following the gradient direction) to train the network (estimate the parameters needed to carry out the regression). A weakness of neural networks is that it is hard to determine how particular parameters correlate with the parameters of the problem, leading to a weakness in explanatory power as to why a neural net has made a given decision.

Clustering Clustering is a general data-driven approach to find groups of any entities that are similar in some sense. A group of *clusters* are found, each cluster containing a set of entities. The idea is that two entities in the same cluster are highly similar, while two entities, each in different clusters, are not as similar. It is generally up to the investigator to figure out what exactly is meant by similarity. In the context of this appendix, the entities will be feature vectors and similarity is defined using a distance function between vectors. Vectors having a smaller distance between them will be more similar. There are hundreds of clustering algorithms, and there is no guarantee that you will get the same clustering from each of them. Clusters can have different shapes, and some algorithms work better with convex shapes, while others can relax this condition. Some algorithms are specifically designed for high-dimensional spaces, while others aren't.

Dimensional Reduction By dimensional reduction, we mean decreasing the lengths of the input feature vectors. In many important environments, the size of these vectors can get quite large. In natural language processing, for example, each vocabulary word has its own position in the vector. It is quite common, therefore, for these vectors to have from 5,000 to 50,000 elements. Reducing the dimensionality would thus speed up the learning process. Initially, in several disciplines of computer science, this was the sole reason for dimensionality reduction. However, it was soon discovered that reducing the dimensionality of the input feature vectors also improved the performance of many of the underlying algorithms used in downstream applications.

Computational Intelligence and Search-Based Software Engineering

Computational intelligence generally refers to the ability of a system (hardware and software) to learn a specific task from a set of data collected about that task. Some classify computational computing as a combination of granular computing (fuzzy sets, rough sets, probabilistic reasoning), neuro-computing (neural nets), evolutionary computing (genetic programming, genetic algorithms, and swarm intelligence), and artificial life (artificial immune systems). These approaches can be used for optimization, classification, search, and regression.

In search-based software engineering, computational intelligence–based techniques have been used for various sorts of optimizations. For example, there are papers [Oun17] using genetic algorithms for multi-objective optimization that quickly search the space of all possible code refactorings and recommend the best options, each option illustrating some particular trade-offs among the input variables, but still locally optimal. It has been shown that these approaches can also be used to build predictive models [Mal17].

The merger of data science, machine learning, and search-based software engineering may lead to exciting breakthroughs in the way software is specified, designed, coded, and tested. Time will tell.

References

[Abb83] Abbott, R., "Program Design by Informal English Descriptions," *CACM,* vol. 26, no. 11, November 1983, pp. 892–894.

[Abd16] Abdessalem, R., et al., "Testing Advanced Driver Assistance Systems Using Multi-Objective Search and Neural Networks," *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering,* ACM, 2016, pp. 63–74.

[Abr17] Abreu, L., "UX Design Patterns for Mobile Apps: Which and Why," 2017, available at https://www.raywenderlich.com/167174/design-patterns-mobile-apps-which-why.

[ACM12] ACM/IEEE-CS Joint Task Force, *Software Engineering Code of Ethics and Professional Practice,* 2012, available at https://ethics.acm.org/code-of-ethics/software-engineering-code.

[ACM18] *ACM Code of Ethics and Professional Conduct,* 2018, available at https://ethics.acm.org/.

[Act18] ActiveWizards, "Top 20 Python Libraries for Data Science in 2018," February 13, 2018. Retrieved October 2018 from ActiveWizards: https://activewizards.com/blog/top-20-python-libraries-for-data-science-in-2018/.

[Ada16] Adams, B., and S. McIntosh, "Modern Release Engineering in a Nutshell—Why Researchers Should Care," *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER),* March 2016, pp. 78–90.

[AFC88] Software Risk Abatement, AFCS/AFLC Pamphlet 800-45, U.S. Air Force, September 30, 1988.

[Agi17] Agile Alliance home page, available at https://www.agilealliance.org.

[Air99] Airlie Council, "Performance Based Management: The Program Manager's Guide Based on the 16-Point Plan and Related Metrics," Draft Report, March 8, 1999.

[Alb10] Alberts, C., et al., "Integrated Measurement and Analysis Framework for Software Security," CMU/SEI-2010-TN-025. Software Engineering Institute, Carnegie Mellon University, 2010, available at http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=9369.

[Ale02] Alexander, I., "Misuse Cases Help to Elicit Non-Functional Requirements," 2002, available at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.88.9670&rep=rep1&type=pdf.

[Ale03] Alexander, I., "Misuse Cases: Use Cases with Hostile Intent," *IEEE Software,* vol. 20, no. 1, 2003, pp. 58–66.

[Ale11] Alexander, I., "Gore, Sore, or What?" *IEEE Software,* vol. 28, no. 1, January–February 2011, pp. 8–10.

[Ale17] Alebrahim, A., "Phase 1: Context Elicitation & Problem Analysis," *Bridging the Gap between Requirements Engineering and Software Architecture,* Springer Vieweg, Wiesbaden, 2017, available at https://doi.org/10.1007/978-3-658-17694-5_4.

[Ale77] Alexander, C., *A Pattern Language,* Oxford University Press, 1977.

[Ale79] Alexander, C., *The Timeless Way of Building,* Oxford University Press, 1979.

[Alh13] Alhusain, S., "Towards Machine Learning Based Design Pattern Recognition," *Proceedings of 13th UK Workshop on Computational Intelligence,* September 2013, pp. 244–251.

[Ali14] Alice, G., and Mead, N., "Using Malware Analysis to Tailor SQUARE for Mobile Platforms," CMU/SEI-2014-TN-018. Software Engineering Institute, Carnegie Mellon University, 2014, available at http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=425994.

[Ali18] Alizadeh, V., M. Kessentini, W. Mkaouer, M. Ocinneide, A. Ouni, and Y. Cai, "An Interactive and Dynamic Search-Based Approach to Software Refactoring Recommendations," *IEEE Transactions on Software Engineering,* 2018, available at https://ieeexplore.ieee.org/document/8477161.

[All08] Allen, J. H., et al., *Software Security Engineering: A Guide for Project Managers,* Addison-Wesley Professional, 2008.

[All14] Alleman, G., *Performance-Based Project Management: Increasing the Probability of Project Success,* AMACOM, 2014.

[Als18] Alshahwan, N., et al., "Deploying Search Based Software Engineering with Sapienz at Facebook," *Search-Based Software Engineering SSBSE 2018.* Lecture Notes in Computer Science, T. Colanzi and P. McMinn (eds.), vol. 11036, Springer, 2018.

[Amb02] Ambler, S., and R. Jeffries, *Agile Modeling,* Wiley, 2002.

[Amb04] Ambler, S., "Examining the Cost of Change Curve," in *The Object Primer,* 3rd ed., Cambridge University Press, 2004.

[Amb95] Ambler, S., "Using Use-Cases," *Software Development,* July 1995, pp. 53–61.

[Amb98] Ambler, S., *Process Patterns: Building Large-Scale Systems Using Object Technology,* Cambridge University Press/SIGS Books, 1998.

[Amp13] Ampatzoglou, A., et al., "Building and Mining a Repository of Design Pattern Instances: Practical and Research Benefits," *Entertainment Computing,* vol. 4, April 2013, pp. 131–142.

[And05] Andreou, A., et al., "Key Issues for the Design and Development of Mobile Commerce Services and Applications," *International Journal of Mobile Communications,* vol. 3, no. 3, March 2005, pp. 303–323.

[And06] Andrews, M., and J. Whittaker, *How to Break Web Software: Functional and Security Testing of Web Applications and Web Services,* Addison-Wesley, 2006.

[And16] Anderson, D., and A. Carmichael, *Essential Kanban Condensed,* Lean Kanban University Press, 2016, available at leankanban.com/guide.

[ANS87] ANSI/ASQC A3-1987, *Quality Systems Terminology,* 1987.

[App00] Appleton, B., "Patterns and Software: Essential Concepts and Terminology," February 2000, available at www.cmcrossroads.com/bradapp/docs/patterns-intro.html.

[App13] Apple Computer, *Accessibility,* 2013, available at www.apple.com/accessibility/.

[Arl02] Arlow, J., and I. Neustadt, *UML and the Unified Process,* Addison-Wesley, 2002.

[Arn11] Arnuphaptrairong, T., "Top Ten Lists of Software Project Risks: Evidence from the Literature Survey," *Proceedings International Multi-Conference of Engineers and Computer Scientists*, vol. I, IMECS 2011, March 2011.

[ISO17] ISO/IEC/IEEE 24765:2017(E), ISO/IEC/IEEE International Standard: Systems and Software Engineering—Vocabulary, available at https://standards.ieee.org/findstds/standard/24765-2017.html.

[Ast04] Astels, D., *Test Driven Development: A Practical Guide,* Prentice Hall, 2004.

[Baa10] Baaz, A., et al., "Appreciating Lessons Learned," *IEEE Software,* vol. 27, no. 4, July–August, 2010, pp. 72–79.

[Bab09] Babar, M., and I. Groton, "Software Architecture Review: The State of Practice," *IEEE Computer,* vol. 42, no. 6, June 2009, pp. 1–8.

[Bab86] Babich, W. A., *Software Configuration Management,* Addison-Wesley, 1986.

[Bae98] Baetjer, Jr., H., *Software as Capital,* IEEE Computer Society Press, 1998, p. 85.

[Baj11] Bajdor, P., and L. Dragolea, "The Gamification as a Tool to Improve Risk Management in the Enterprise," *Annales Universitatis Apulensis Series Oeconomica,* vol. 2, no. 13, 2011, available at http://www.oeconomica.uab.ro/upload/lucrari/1320112/38.pdf.

[Bar06] Baresi, L., E. DiNitto, and C. Ghezzi, "Toward Open-World Software: Issues and Challenges," *IEEE Computer,* vol. 39, no. 10, October 2006, pp. 36–43.

[Bas12] Bass, L., P. Clements, and R. Kazman, *Software Architecture in Practice,* 3rd ed., Addison-Wesley, 2012.

[Bat18] Batarseh, F., and A. Gonzalez, "Predicting Failures in Agile Software Development through Data Analytics," *A Software Quality Journal,* vol. 26, no. 1, March 2018, pp. 49–66.

[Bec99] Beck, K., *Extreme Programming Explained: Embrace Change,* Addison-Wesley, 1999.

[Bec01] Beck, K., et al., "Manifesto for Agile Software Development," 2001, available at www.agilemanifesto.org/.

[Bec04a] Beck, K., *Extreme Programming Explained: Embrace Change,* 2nd ed., Addison-Wesley, 2004.

[Bec04b] Beck, K., *Test-Driven Development: By Example,* 2nd ed., Addison-Wesley, 2004.

[Bee99] Beedle, M., et al., "SCRUM: An Extension Pattern Language for Hyperproductive Software Development," included in *Pattern Languages of Program Design 4,* Addison-Wesley Longman, 1999, available at http://jeffsutherland.com/scrum/scrum_plop.pdf.

[Beg10] Begel, A., R. DeLine, and T. Zimmermann, "Social Media for Software Engineering," *Proceedings FoSER 2010,* ACM, November 2010.

[Bei90] Beizer, B., *Software Testing Techniques,* 2nd ed., Van Nostrand-Reinhold, 1990.

[Bei95] Beizer, B., *Black-Box Testing,* Wiley, 1995.

[Bel17] Bell, L., et al., *Agile Application Security,* O'Reilly Media, 2017.

[Bel14] Bellomo, S., P. Krutchen, R. Nord, and I. Ozkaya, "How to Agilely Architect an Agile Architecture," *Cutter IT Journal,* February 2014, pp. 10–15.

[Ben10a] Bennett, S., S. McRobb, and R. Farmer, *Object-Oriented Analysis and Design Using UML,* 4th ed., McGraw-Hill, 2010.

[Ben10b] Benaroch, M., and A. Appari, "Financial Pricing of Software Development Risk Factors," *IEEE Software,* vol. 27, no. 3, September–October 2010, pp. 65–73.

[Ber80] Bersoff, E., V. Henderson, and S. Siegel, *Software Configuration Management,* Prentice Hall, 1980.

[Ber93] Berard, E., *Essays on Object-Oriented Software Engineering,* vol. 1, Addison-Wesley, 1993.

[Bin94] Binder, R., "Testing Object-Oriented Systems: A Status Report," *American Programmer,* vol. 7, no. 4, April 1994, pp. 23–28.

[Bin99] Binder, R., *Testing Object-Oriented Systems: Models, Patterns, and Tools,* Addison-Wesley, 1999.

[Bir98] Biró, M., and T. Remzsö, "Business Motivations for Software Process Improvement," *ERCIM News,* no. 32, January 1998, available at www.ercim.org/publication/Ercim_News/enw32/biro.html.

[Bis02] Bishop, M., *Computer Security: Art and Science*, Addison-Wesley Professional, 2002.

[Bjø16] Bjørnson, F., and K. Vestues, "Knowledge Sharing and Process Improvement in Large-Scale Agile Development," *Proceedings Scientific Workshop of XP2016 (XP '16 Workshops).* ACM, New York, NY, 2016, Article 7.

[Bla10] Blair, S., et al., "Responsibility-Driven Architecture," *IEEE Software,* vol. 27, no. 3, March–April 2010, pp. 26–32.

[Boe01a] Boehm, B., "The Spiral Model as a Tool for Evolutionary Software Acquisition," *CrossTalk,* May 2001, available at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.642.8250&rep=rep1&type=pdf.

[Boe01b] Boehm, B., and V. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer,* vol. 34, no. 1, January 2001, pp. 135–137.

[Boe08] Boehm, B., "Making a Difference in the Software Century," *IEEE Computer,* vol. 41, no. 3, March 2008, pp. 32–38.

[Boe81] Boehm, B., *Software Engineering Economics,* Prentice Hall, 1981.

[Boe88] Boehm, B., "A Spiral Model for Software Development and Enhancement," *Computer,* vol. 21, no. 5, May 1988, pp. 61–72.

[Boe89] Boehm, B. W., *Software Risk Management,* IEEE Computer Society Press, 1989.

[Boe96] Boehm, B., "Anchoring the Software Process," *IEEE Software,* vol. 13, no. 4, July 1996, pp. 73–82.

[Boe98] Boehm, B., "Using the WINWIN Spiral Model: A Case Study," *Computer,* vol. 31, no. 7, July 1998, pp. 33–44.

[Boh00] Bohl, M., and M. Rynn, *Tools for Structured Design: An Introduction to Programming Logic,* 5th ed., Prentice Hall, 2000.

[Boh66] Bohm, C., and G. Jacopini, "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules," *CACM,* vol. 9, no. 5, May 1966, pp. 366–371.

[Boi04] Boiko, B., *Content Management Bible,* 2nd ed., Wiley, 2004.

[Boo05] Booch, G., J. Rumbaugh, and I. Jacobsen, *The Unified Modeling Language User Guide*, 2nd ed., Addison-Wesley, 2005.

[Boo11a] Booch, G., "Dominant Design," *IEEE Software,* vol. 28, no. 2, January–February 2011, pp. 8–9.

[Boo11b] Booch, G., "Draw Me a Picture," *IEEE Software,* vol. 28, no. 1, January–February 2011, pp. 6–7.

[Bos00] Bosch, J., *Design & Use of Software Architectures,* Addison-Wesley, 2000.

[Bos11] Bose, B., et al., "Morphing Smartphones into Automotive Application Platforms," *IEEE Computer,* vol. 44, no. 5, May 2011, pp. 28–29.

[Bre02] Breen, P., "Exposing the Fallacy of 'Good Enough' Software," informit.com, February 1, 2002, available at http://www.informit.com/articles/article.aspx?p=25141&seqNum=3.

[Bri09] Briand, L., "Model-Driven Development and Search-Based Software Engineering: An Opportunity for Research Synergy," *2009 1st International Symposium on Search Based Software Engineering,* Windsor, 2009.

[Bri13] Briers, B., "The Gamification of Project Management." PMI® Global Congress 2013—North America, New Orleans, LA. Newtown Square, PA: Project Management Institute, 2013, available at https://www.pmi.org/learning/library/gamification-project-management-5949.

[Bro06] Broy, M., "The 'Grand Challenge' in Informatics: Engineering Software Intensive Systems," *IEEE Computer,* vol. 39, no. 10, October 2006, pp. 72–80.

[Bro10a] Brown, N., R. Nord, and I. Ozkaya, "Enabling Agility through Architecture," *Crosstalk,* November–December 2010, available at https://apps.dtic.mil/dtic/tr/fulltext/u2/a552111.pdf.

[Bro10b] Broy, M., and R. Reussner, "Software Architecture Review: The State of Practice," *IEEE Computer,* vol. 43, no. 10, June 2010, pp. 88–91.

[Bro12] Brown, A., *The Architecture of Open Source Applications,* lulu.com, 2012.

[Bro98] Brown, W. J., et al., *AntiPatterns—Refactoring Software, Architectures, and Projects in Crisis,* Wiley, 1998.

[Bud96] Budd, T., *An Introduction to Object-Oriented Programming,* 2nd ed., Addison-Wesley, 1996.

[Bur16] Burns, A., B. Umbaugh, and C. Dunn, "Introduction to the Mobile Software Development Lifecycle," 2016, available at https://docs.microsoft.com/en-us/xamarin/cross-platform/get-started/introduction-to-mobile-sdlc.

[Bus07] Buschmann, F., et al., *Pattern-Oriented Software Architecture, A System of Patterns,* Wiley, 2007.

[Bus10b] Buschmann, F., and K. Henley, "Five Considerations for Software Architecture, Part 1," *IEEE Software,* vol. 27, no. 3, May–June 2010, pp. 63–65.

[Bus10c] Buschmann, F., and K. Henley, "Five Considerations for Software Architecture, Part 2," *IEEE Software,* vol. 27, no. 4, July–August 2010, pp. 12–14.

[Cac02] Cachero, C., et al., "Conceptual Navigation Analysis: A Device and Platform Independent Navigation Specification," *Proceedings of the 2nd International Workshop on Web-Oriented Technology,* June 2002, available at http://gplsi.dlsi.ua.es/iwad/ooh_project/papers/iwwost02.pdf.

[Car08] Carrasco, M., "7 Key Attributes of High Performance Software Development Teams," June 30, 2008, available at http://www.realsoftwaredevelopment.com/7-key-attributes-of-high-performance-software-development-teams/.

[Car90] Card, D., and R. Glass, *Measuring Software Design Quality,* Prentice Hall, 1990.

[Cav78] Cavano, J., and J. McCall, "A Framework for the Measurement of Software Quality," *Proceedings ACM Software Quality Assurance Workshop,* November 1978, pp. 133–139.

[CMM18a] *Capability Maturity Model Integration (CMMI),* Software Engineering Institute, 2018, available at https://cmmiinstitute.com/cmmi.

[CMM18b] *People Capability Maturity Model Integration (People CMM),* Software Engineering Institute, 2018, available at https://cmmiinstitute.com/cmmi/pm.

[Cha89] Charette, R., *Software Engineering Risk Analysis and Management,* McGraw-Hill/Intertext, 1989.

[Cha92] Charette, R., "Building Bridges over Intelligent Rivers," *American Programmer,* vol. 5, no. 7, September 1992, pp. 2–9.

[Cha93] de Champeaux, D., D. Lea, and P. Faure, *Object-Oriented System Development,* Addison-Wesley, 1993.

[Chi94] Chidamber, S., and C. Kemerer, "A Metrics Suite for Object-Oriented Design," *IEEE Transactions on Software Engineering,* vol. SE-20, no. 6, June 1994, pp. 476–493.

[Cho16] Choma, J., et al., "Interaction Patterns for User Interface Design of Large Web Applications," *Proceedings 11th Latin-American Conference on Pattern Languages of Programming (SugarLoafPLoP '16),* 2016, The Hillside Group, Article 8, 11 pages.

[Chr17] Christensen, E., "How to Create a Customer Journey Map," 2017, available at https://www.lucidchart.com/blog/how-to-build-customer-journey-maps.

[Chu09] Chung, L., and J. Leite, "On Non-Functional Requirements in Software Engineering," in A. T. Borgida et al. (eds.), *Conceptual Modeling: Foundations and Applications,* Springer-Verlag, 2009.

[Cig07] Cigital, Inc., "Case Study: Finding Defects Earlier Yields Enormous Savings," 2007.

[Cla05] Clark, S., and E. Baniasaad, *Aspect-Oriented Analysis and Design,* Addison-Wesley, 2005.

[Cle03] Clements, P., R. Kazman, and M. Klein, *Evaluating Software Architectures: Methods and Case Studies,* Addison-Wesley, 2003.

[Cle10] Clements, P., and L. Bass, "The Business Goals Viewpoint," *IEEE Software,* vol. 27, no. 6, November–December 2010, pp. 38–45.

[CMM07] *Capability Maturity Model Integration (CMMI),* Software Engineering Institute, 2007, available at www.sei.cmu.edu/cmmi/.

[Coa91] Coad, P., and E. Yourdon, *Object-Oriented Analysis,* 2nd ed., Prentice Hall, 1991.

[Coc01a] Cockburn, A., and J. Highsmith, "Agile Software Development: The People Factor," *IEEE Computer,* vol. 34, no. 11, November 2001, pp. 131–133.

[Coc01b] Cockburn, A., *Writing Effective Use-Cases,* Addison-Wesley, 2001.

[Coc02] Cockburn, A., *Agile Software Development,* Addison-Wesley, 2002.

[Coh05] Cohn, M., "Estimating with Use Case Points," *Methods & Tools,* Fall 2005, available at http://www.mountaingoatsoftware.com/articles/estimating-with-use-case-points.

[Con02] Conradi, R., and A. Fuggetta, "Improving Software Process Improvement," *IEEE Software,* July–August 2002, pp. 2–9, available at http://citeseer.ist.psu.edu/conradi02improving.html.

[Con10] Conway, D., *The Data Science Venn Diagram,* 2010, available at http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram.

[Con95] Constantine, L., "What DO Users Want? Engineering Usability in Software," *Windows Tech Journal,* December 1995, available at http://www.wytsg.org:88/reslib/400/180/110/020/030/050/060/L000000000240585.pdf.

[Cro79] Crosby, P., *Quality Is Free,* McGraw-Hill, 1979.

[Cro07] Cross, M., *Developer's Guide to Web Application Security,* Syngress, 2007.

[Cur09] Curtis, B., and W. Heflley, *The People CMM: A Framework for Human Capital Management*, 2nd ed., Addison-Wesley, 2009.

[Cur90] Curtis, B., and D. Walz, "The Psychology of Programming in the Large: Team and Organizational Behavior," *Psychology of Programming,* Academic Press, 1990.

[DAC03] "An Overview of Model-Based Testing for Software," Data and Analysis Center for Software, CR/TA 12, June 2003.

[Dah72] Dahl, O., E. Dijkstra, and C. Hoare, *Structured Programming,* Academic Press, 1972.

[Dak14] Daka, E., and G. Fraser, "A Survey on Unit Testing Practices and Problems," *2014 IEEE 25th International Symposium on Software Reliability Engineering,* Naples, 2014, pp. 201–211.

[Dam17] Dam, R., and T. Siang, "Test Your Prototype: How to Gather Feedback and Maximize Learning," October 2017, available at https://www.interaction-design.org/literature/article/test-your-prototypes-how-to-gather-feedback-and-maximise-learning.

[Dar01] Dart, S., *Spectrum of Functionality in Configuration Management Systems,* Software Engineering Institute, 2001, available at www.sei.cmu.edu/legacy/scm/tech_rep/TR11_90/TOC_TR11_90.html.

[Dar91] Dart, S., "Concepts in Configuration Management Systems," *Proceedings Third International Workshop on Software Configuration Management,* ACM SIGSOFT, 1991, available at https://dl.acm.org/citation.cfm?id=111063.

[Das15] Dasanayake, S., et al., "Software Architecture Decision-Making Practices and Challenges: An Industrial Case Study," *Proceedings of the 24th Australasian Software Engineering Conference,* SA, Australia, 2015. pp. 88–97.

[Dav93] Davis, A., et al., "Identifying and Measuring Quality in a Software Requirements Specification," *Proceedings of the. First International Software Metrics Symposium,* IEEE, Baltimore, MD, May 1993, pp. 141–152.

[Dav95a] Davis, M., "Process and Product: Dichotomy or Duality," *Software Engineering Notes,* ACM Press, vol. 20, no. 2, April 1995, pp. 17–18.

[Dav95b] Davis, A., *201 Principles of Software Development,* McGraw-Hill, 1995.

[Day99] Dayani-Fard, H., et al., "Legacy Software Systems: Issues, Progress, and Challenges," IBM Technical Report: TR-74.165-k, April 1999.

[Dem86] Deming, W., *Out of the Crisis,* MIT Press, 1986.

[DeM95] DeMarco, T., *Why Does Software Cost So Much?* Dorset House, 1995.

[DeM98] DeMarco, T., and T. Lister, *Peopleware,* 2nd ed., Dorset House, 1998.

[Den73] Dennis, J., "Modularity," in *Advanced Course on Software Engineering,* F. L. Bauer (ed.), Springer-Verlag, 1973, pp. 128–182.

[Des08] de Sáa, M., and L. Carriocço, "Lessons from Early Stages Design of Mobile Applications," *Proceedings of 10th International Conference on Human Computer Human with Mobile Services and Devices,* 2008, pp. 127–136.

[Des09] de Souza, C., H. Sharp, G. Venolia, and L. Cheng, "Guest Editors' Introduction: Cooperative and Human Aspects of Software Engineering," *IEEE Software,* vol. 26, no. 6, 2009, pp. 17–19.

[Det11] Deterding, S., et al., "From Game Design Elements to Gamefulness: Defining Gamification," *Proceedings of the 2011 Annual Conference Extended Abstracts on Human Factors in Computing Systems—CHI EA '11,* 2011, p. 2425.

[Dia14] Díaz-Bossini, J., and L. Moreno, "Accessibility to Mobile Interfaces for Older People," *Procedia Computer Science,* vol. 27, 2014, pp. 57–66.

[Dij65] Dijkstra, E., "Programming Considered as a Human Activity," *Proceedings 1965 IFIP Congress,* North-Holland Publishing Co., 1965.

[Dij72] Dijkstra, E., "The Humble Programmer," 1972 ACM Turing Award Lecture, *CACM,* vol. 15, no. 10, October 1972, pp. 859–866.

[Dij76a] Dijkstra, E., "Structured Programming," in *Software Engineering, Concepts and Techniques*, J. Buxton et al. (eds.), Van Nostrand-Reinhold, 1976.

[Dij76b] Dijkstra, E., *A Discipline of Programming,* Prentice Hall, 1976.

[Dij82] Dijksta, E., "On the Role of Scientific Thought," in *Selected Writings on Computing: A Personal Perspective,* Springer-Verlag, 1982.

[Din16] Dingsøyr, T., and C. Lassenius, "Emerging Themes in Agile Software Development: Introduction to the Special Section on Continuous Value Delivery," *Information and Software Technology,* vol. 77, 2016, pp. 56–60.

[Dix99] Dix, A., "Design of User Interfaces for the Web," *Proceedings User Interfaces to Data Systems Conference,* September 1999, available at www.comp.lancs.ac.uk/computing/users/dixa/topics/webarch/.

[Don99] Donahue, G., S. Weinschenck, and J. Nowicki, "Usability Is Good Business," Compuware Corp., July 1999, available at www.compuware.com.

[Duc01] Ducatel, K., et al., Scenarios for Ambient Intelligence in 2010, ISTAG-European Commission, 2001, available at https://www.researchgate.net/publication/262007900_Scenarios_for_ambient_intelligence_in_2010.

[Dun82] Dunn, R., and R. Ullman, *Quality Assurance for Computer Software,* McGraw-Hill, 1982.

[Dut15] Dutra, E., and G. Santos, "Software Process Improvement Implementation Risks: A Qualitative Study Based on Software Development Maturity Models Implementations in Brazil," in *Product-Focused Software Process Improvement,* PROFES 2015 Lecture Notes in Computer Science, P. Abrahamsson et al. (eds.), vol. 9459, Springer, 2015.

[DXL18] DX Lab Design Sprint, "Make Your UX Design Process Agile Using Google's Methodology," available at https://www.interaction-design.org/literature/article/make-your-ux-design-process-agile-using-google-s-methodology, 2018.

[Dye15] Dyer, R., et al., "Boa: Ultra-Large-Scale Software Repository and Source-Code Mining," *ACM Transactions on Software Engineering Methodology,* vol. 25, no. 1, Article 7, December 2015.

[Ebe14] Ebert, C. "Software Product Management," *IEEE Software,* vol. 31, no. 3, May–June 2014, pp. 21–24.

[Edg95] Edgemon, J., "Right Stuff: How to Recognize It When Selecting a Project Manager," *Application Development Trends,* vol. 2, no. 5, May 1995, pp. 37–42.

[Eis01] Eisenstein, J., et al., "Applying Model-Based Techniques to the Development of UIs for Mobile Computers," *Proceedings of Intelligent User Interfaces,* January 2001.

[Eji91] Ejiogu, L., *Software Engineering with Formal Metrics,* QED Publishing, 1991.

[Elb16] Elbanna, A., and S. Sarker, "The Risks of Agile Development: Learning from Adopters," *IEEE Software,* vol. 33, no. 5, September–October 2016, pp. 72–79.

[Era09] Erasmus, H., "The Seven Traits of Superprofessionals," *IEEE Software,* vol. 26, no. 4, July–August 2009, pp. 4–6.

[Erd10] Erdogmus, H., "Déjà vu: The Life of Software Engineering Ideas," *IEEE Software,* vol. 27, no. 1, January–February 2010, pp. 2–3.

[Eri15] Ericson, C., *Hazard Analysis Techniques for System Safety,* 2nd ed., Wiley, 2015.

[Eve09] Everett, G., and B. Meyer, "Point/Counterpoint," *IEEE Software,* vol. 26, no. 4, July–August 2009, pp. 62–65.

[Fag86] Fagan, M., "Advances in Software Inspections," *IEEE Transactions on Software Engineering,* vol. 12, no. 6, July 1986.

[Fai17] Fairley, R., and M. J. Willshire, "Better Now Than Later: Managing Technical Debt in Systems Development," *Computer,* vol. 50, no. 5, May 2017, pp. 80–87.

[Fal10] Falessi, D., et al., "Peaceful Coexistence: Agile Developer Perspectives on Software Architecture," *IEEE Software,* vol. 27, no. 3, March–April 2010, pp. 23–25.

[Fel07] Feller, J., et al. (eds.), *Perspectives on Free and Open Source Software,* The MIT Press, 2007.

[Fel18] Feldt, R., et al., "Ways of Applying Artificial Intelligence in Software Engineering," In *Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE '18).* ACM, New York, NY, 2018, pp. 35–41.

[Fel89] Felican, L., and G. Zalateu, "Validating Halstead's Theory for Pascal Programs," *IEEE Transactions on Software Engineering,* vol. SE-15, no. 2, December 1989, pp. 1630–1632.

[Fen91] Fenton, N., *Software Metrics,* Chapman and Hall, 1991.

[Fen94] Fenton, N., "Software Measurement: A Necessary Scientific Basis," *IEEE Transactions on Software Engineering,* vol. SE-20, no. 3, March 1994, pp. 199–206.

[Fer14] Ferrucci F., M. Harman, and F. Sarro, "Search-Based Software Project Management," in *Software Project Management in a Changing World,* G. Ruhe and C. Wohlin (eds.), Springer, 2014.

[Fir13] Firesmith, D., *Security and Safety Requirements for Software-Intensive Systems,* Auerbach, 2013.

[Fow00] Fowler, M., et al., *Refactoring: Improving the Design of Existing Code,* Addison-Wesley, 2000.

[Fow01] Fowler, M., and J. Highsmith, "The Agile Manifesto," *Software Development Magazine,* August 2001.

[Fow02] Fowler, M., "The New Methodology," available at www.martinfowler.com/articles/newMethodology.html#N8B, June 2002.

[Fow03] Fowler, M., et al., *Patterns of Enterprise Application Architecture,* Addison-Wesley, 2003.

[Fow04] Fowler, M., *UML Distilled,* 3rd ed., Addison-Wesley, 2004.

[Fow16] Fowler, M., and Sutherland, J., *The Scrum Guide,* 2016, available at, http://www.scrumguides.org/.

[Fow97] Fowler, M., *Analysis Patterns: Reusable Object Models,* Addison-Wesley, 1997.

[Fra93] Frankl, P., and S. Weiss, "An Experimental Comparison of the Effectiveness of Branch Testing and Data Flow," *IEEE Transactions on Software Engineering,* vol. 19, no. 8, August 1993, pp. 770–787.

[Fre90] Freedman, D., and G. Weinberg, *Handbook of Walkthroughs, Inspections and Technical Reviews,* 3rd ed., Dorset House, 1990.

[Fri10] Fricker, S., "Handshaking with Implementation Proposals: Negotiating Requirements Understanding," *IEEE Software,* vol. 27, no. 2, March–April 2010, pp. 72–80.

[Fug14] Fuggetta, A., and E. Di Nitto, "Software Process," in *Proceedings of the on Future of Software Engineering (FOSE 2014),* ACM, New York, NY, 2014, pp. 1–12.

[Gag04] Gage, D., and J. McCormick, "We Did Nothing Wrong," *Baseline Magazine,* March 4, 2004, available at http://www.baselinemag.com/c/a/Projects-Processes/We-Did-Nothing-Wrong.

[Gai95] Gaines, B., "Modeling and Forecasting the Information Sciences," Technical Report, University of Calgary, September 1995, downloadable at https://www.lri.fr/~mbl/ENS/FundHCI/2017/papers/Gaines-BRETAM99.pdf.

[Gal16] Galster, M., et al., "Variability and Complexity in Software Design," ACM SIGSOFT Software Engineering Notes, vol. 41, no. 6, November 2016, pp. 27–30.

[Gal17] Galster, M., et al., "Variability and Complexity in Software Design: Towards a Research Agenda," SIGSOFT Software Engineering Notes, vol. 41 no. 6, January 2017, pp. 27–30.

[Gam95] Gamma, E., et al., *Design Patterns: Elements of Reusable Object-Oriented Software,* Addison-Wesley, 1995.

[Gar08] GartnerGroup, "Understanding Hype Cycles," 2008, available at https://www.gartner.com/en/documents/3887767.

[Gar09a] Garlan, D., et al., "Architectural Mismatch: Way Reuse Is Still So Hard," *IEEE Software,* vol. 26, no. 4, July–August 2009, pp. 66–69.

[Gar10a] Garcia-Crespo, A., et al., "A Qualitative Study of Hard Decision Making in Managing Global Software Development Teams," *Journal of Management Information Systems,* vol. 27, no. 3, June 2010, pp. 247–252.

[Gar10b] Garrett, J. J., *The Elements of User Experience: User-Centered Design for the Web and Beyond,* 2nd ed., New Riders Publishing, 2010.

[Gar84] Garvin, D., "What Does 'Product Quality' Really Mean?" *Sloan Management Review,* Fall 1984, pp. 25–45.

[Gar95] Garlan, D., and M. Shaw, "An Introduction to Software Architecture," *Advances in Software Engineering and Knowledge Engineering,* vol. I, V. Ambriola and G. Tortora (eds.), World Scientific Publishing Company, 1995.

[Gas17] Gasparic, M., et al., "GUI Design for IDE Command Recommendations," *Proceedings of the 22nd International Conference on Intelligent User Interfaces (IUI '17),* 2017, ACM, New York, NY, pp. 595–599.

[Gau89] Gause, D., and G. Weinberg, *Exploring Requirements: Quality Before Design,* Dorset House, 1989.

[Ger17] Géron, A., *Hands-On Machine Learning with Scikit-Learn and TensorFlow,* O'Reilly Media, 2017.

[Gey01] Geyer-Schulz, A., and M. Hahsler, "Software Engineering with Analysis Patterns," Technical Report 01/2001, Institut für Informationsverarbeitung undwirtschaft, Wirschaftsuniversität Wien, November 2001, available at https://www.researchgate.net/publication/250241449_Software_Engineering_with_Analysis_Patterns.

[Gha14] Ghazi, P., A. M. Moreno, and L. Peters, "Looking for the Holy Grail of Software Development," *IEEE Software,* vol. 31, no. 1, January–February 2014, pp. 96.

[Gil06] Gillis, D., "Pattern-Based Design," tehan + lax blog, September 14, 2006.

[Gil88] Gilb, T., *Principles of Software Project Management,* Addison-Wesley, 1988.

[Gil95] Gilb, T., "What We Fail to Do in Our Current Testing Culture," *Testing Techniques Newsletter* (online edition, ttn@soft.com), Software Research, January 1995.

[Gla02] Gladwell, M., *The Tipping Point,* Back Bay Books, 2002.

[Gla98] Glass, R., "Defining Quality Intuitively," *IEEE Software,* May–June 1998, pp. 103–104, 107.

[Gli07] Glinz, M., and R. Wieringa, "Stakeholders in Requirements Engineering," *IEEE Software,* vol. 24, no. 2, March–April 2007, pp. 18–20.

[Glu94] Gluch, D., "A Construct for Describing Software Development Risks," CMU/SEI-94-TR-14, Software Engineering Institute, 1994.

[Gna99] Gnaho, C., and F. Larcher, "A User-Centered Methodology for Complex and Customizable Web Engineering," *Proceedings of the First ICSE Workshop on Web Engineering,* ACM, Los Angeles, May 1999.

[Goa14] Gao, J., et al., "Mobile Application Testing: A Tutorial," *Computer,* vol. 47, no. 2, 2014, pp. 46–55.

[Gon04] Gonzales, R., "Requirements Engineering," Sandia National Laboratories, 2004, a slide presentation.

[Gon17] Gonzalez, D., et al., "A Large-Scale Study on the Usage of Testing Patterns That Address Maintainability Attributes: Patterns for Ease of Modification, Diagnoses, and Comprehension," *Proceedings of the 14th International Conference on Mining Software Repositories (MSR '17).* IEEE Press, Piscataway, NJ, 2017, pp. 391–401.

[Goo18] Google, Design Sprint Kit, 2018, available at https://designsprintkit.withgoogle.com/.

[Gor06] Gorton, I., *Essential Software Architecture,* Springer, 2006.

[Got11] Gotel, O., and S. Morris, "Requirements Tracery," *IEEE Software,* vol. 28, no. 5, September–October 2011, pp. 92–94.

[Got18] Gotterbarn, D., et al., "Thinking Professionally: The Continual Evolution of Interest in Computing Ethics," *ACM Inroads,* vol. 9, no. 2, April 2018, pp. 10–12.

[Gra18] Gray, C., et al., "The Dark (Patterns) Side of UX Design," *Proceedings 2018 CHI Conference on Human Factors in Computing Systems (CHI '18).* ACM, New York, NY, Paper 534, 2018.

[Gra92] Grady, R. G., *Practical Software Metrics for Project Management and Process Improvement,* Prentice Hall, 1992.

[Gru00] Gruia-Catalin, R., et al., "Software Engineering for Mobility: A Roadmap," *Proceedings of the 22nd International Conference on the Future of Software Engineering,* 2000.

[Gup15] Gupta, S., and V. Suma, "Data Mining: A Tool for Knowledge Discovery in Human Aspect of Software Engineering," *2015 2nd International Conference on Electronics and Communication Systems (ICECS),* Coimbatore, 2015, pp. 1289–1293.

[Gut15] Gutiérrez, J., M. Escalona, and M. Mejías, "A Model-Driven Approach for Functional Test Case Generation," *Journal of Systems and Software,* vol. 109, 2015, pp. 214–228.

[Hac98] Hackos, J., and J. Redish, *User and Task Analysis for Interface Design,* Wiley, 1998.

[Hal77] Halstead, M., *Elements of Software Science,* North-Holland, 1977.

[Hal98] Hall, E. M., *Managing Risk: Methods for Software Systems Development,* Addison-Wesley, 1998.

[Har11] Harris, N., and P. Avgeriou, "Pattern-Based Architecture Reviews," *IEEE Software,* vol. 28, no. 6, November–December 2011, pp. 66–71.

[Har12a] Hardy, T., *Software and System Safety,* Authorhouse, 2012.

[Har12b] Harman, M., "The Role of Artificial Intelligence in Software Engineering," *Proceedings First International Workshop on Realizing AI Synergies in Software Engineering (RAISE '12).* IEEE Press, Piscataway, NJ, 2012, pp. 1–6.

[Har14] Harman, M., et al., "Search Based Software Engineering for Product Line Engineering: A Survey and Directions for Future Work," *Proceedings 18th International Software Product Line Conference,* SPL14, Florence, Italy, vol. 1, September 2014, pp. 5–18.

[Har98] Harrison, R., S. Counsell, and R. Nithi, "An Evaluation of the MOOD Set of Object-Oriented Software Metrics," *IEEE Transactions on Software Engineering,* vol. SE-24, no. 6, June 1998, pp. 491–496.

[Hee15] Heeager, L., and J. Rose, "Optimising Agile Development Practices for the Maintenance Operation: Nine Heuristics," *Journal of Empirical Software Engineering,* vol. 20, no. 6, 2015, pp. 1762–1784.

[Hei02] Heitmeyer, C., "Software Cost Reduction," in *Encyclopedia of Software Engineering,* J. J. Marciniak (ed.), 2 vols., John Wiley & Sons, 2002, pp. 1374–1380.

[Hel18] Helfrich, J., *Security for Software Engineers,* Chapman and Hall/CRC, 2018.

[Her06] Hernan, S., et al., "Uncover Security Design Flaws Using the STRIDE Approach," *MSDN Magazine,* November 2006.

[Het84] Hetzel, W., *The Complete Guide to Software Testing,* QED Information Sciences, 1984.

[Hig02a] Highsmith, J. (ed.), "The Great Methodologies Debate: Part 2," *Cutter IT Journal,* vol. 15, no. 1, January 2002.

[Hig95] Higuera, R., "Team Risk Management," *CrossTalk,* U.S. Dept. of Defense, January 1995, pp. 2–4.

[Hil17] Hill, C., et al., "Gender-Inclusiveness Personas vs. Stereotyping: Can We Have It Both Ways?" *Proceedings 2017 CHI Conference on Human Factors in Computing Systems (CHI '17).* ACM, New York, NY, 2017, pp. 6658–6671.

[Hne11] Hneif, M., and S. Lee, "Using Guidelines to Improve Quality in Software Nonfunctional Attributes," *IEEE Software,* vol. 28, no. 5, November–December 2011, pp. 72–73.

[Hoe16] Hoegl, M., and M. Muethel, "Enabling Shared Leadership in Virtual Project Teams: A Practitioners' Guide," *Project Management Journal,* vol. 47, no. 1, February/March 2016, pp. 7–12.

[Hog04] Hoglund, G., and G. McGraw, *Exploiting Software: How to Break Code,* Addison-Wesley Professional, 2004.

[Hol06] Holzner, S., *Design Patterns for Dummies,* For Dummies Publishers, 2006.

[Hoo12] Hoober, S., and E. Berkman, *Designing Mobile Interfaces,* O'Reilly Media, 2012.

[Hoo96] Hooker, D., "Seven Principles of Software Development," September 1996, a summary of these principles is available at https://lingualeo.com/pt/jungle/seven-principles-of-software-development-by-david-hooker-48432#/page/1.

[Hop90] Hopper, M., "Rattling SABRE, New Ways to Compete on Information," *Harvard Business Review,* May–June 1990.

[Hor03] Horch, J., *Practical Guide to Software Quality Management,* 2nd ed., Artech House, 2003.

[Hua17] Huang, J., et al., "Cross-Validation Based K Nearest Neighbor Imputation for Software Quality Datasets: An Empirical Study," *Journal of Systems and Software,* 2017, pp. 226–252.

[Hub99] Hubbard, R., "Design, Implementation, and Evaluation of a Process to Structure the Collection of Software Project Requirements," PhD dissertation, Colorado Technical University, 1999.

[Hus15] Hussain, A., et al., "Usability Evaluation of Mobile Game Applications: A Systematic Review," *International Journal of Computer and Information Technology,* vol. 4, no. 3, May 2015, pp. 547–551.

[Hya96] Hyatt, L., and L. Rosenberg, "A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality," NASA SATC, 1996, available at http://articles.adsabs.harvard.edu/cgi-bin/nph-iarticle_query?1996ESASP.377..209H&data_type=PDF_HIGH&whole_paper=YES&type=PRINTER&filetype=.pdf.

[IBM13] IBM, Web Services Globalization Model, 2013, downloadable from ftp://public.dhe.ibm.com/software/globalization/.../webservicesglobalizationmodel.pdf.

[IBM81] "Implementing Software Inspections," course notes, IBM Systems Sciences Institute, IBM Corporation, 1981.

[IEE05] IEEE Std. 982.1-2005, *IEEE Standard Dictionary of Measures of the Software Aspects of Dependability,* 2005.

[IEE11] IEEE-Std-42010:2011(E), *Systems and Software Engineering-Architectural Description,* 2011, available at https://ieeexplore.ieee.org/document/6129467.

[IEE17] ISO/IEC/IEEE 24765:2017(E), *ISO/IEC/IEEE International Standard: Systems and Software Engineering—Vocabulary,* available at https://standards.ieee.org/findstds/standard/24765-2017.html.

[ISO08] ISO SPICE 2008, an earlier description is available at https://www.cs.helsinki.fi/u/paakki/Pyhajarvi.pdf.

[ISO14] ISO/IEC 90003:2014, Second Edition: *Software Engineering—Guidelines for the Application of ISO 9001:2008 to Computer Software,* International Organization for Standardization, 2014.

[ISO18] ISO/IEC/IEEE 90003:2018, *Software Engineering—Guidelines for the Application of ISO 9001:2015 to Computer Software,* International Organization for Standardization, 2018.

[ISO15] *Plain English Summary of ISO 9001: 2015,* 2015, available at http://praxiom.com/iso-9001.htm.

[ISO11] ISO/IEC 25010:2011, *Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*, 2011, available at https://www.iso.org/standard/35733.html.

[Ive04] Iversen, J., L. Mathiassen, and P. Nielsen, "Managing Risk in Software Process Improvement: An Action Research Approach," *MIS Quarterly,* vol. 28, no. 3, September 2004, pp. 395–433.

[Ivo01] Ivory, M., R. Sinha, and M. Hearst, "Empirically Validated Web Page Design Metrics," ACM SIGCHI'01, March 31–April 4, 2001, available at http://webtango.berkeley.edu/papers/chi2001/.

[Jac02a] Jacobson, I., "A Resounding 'Yes' to Agile Processes—But Also More," *Cutter IT Journal,* vol. 15, no. 1, January 2002, pp. 18–24.

[Jac02b] Jacyntho, D., D. Schwabe, and G. Rossi, "An Architecture for Structuring Complex Web Applications," 2002, available at https://www.semanticscholar.org/paper/A-Software-Architecture-for-Structuring-Complex-Web-Jacyntho-Schwabe/2809668ede5034ed8d65e7669c6b0b463e9ee464.

[Jac04] Jacobson, I., and P. Ng, *Aspect-Oriented Software Development,* Addison-Wesley, 2004.

[Jac92] Jacobson, I., *Object-Oriented Software Engineering,* Addison-Wesley, 1992.

[Jac98] Jackman, M., "Homeopathic Remedies for Team Toxicity," *IEEE Software,* July 1998, pp. 43–45.

[Jac99] Jacobson, I., G. Booch, and J. Rumbaugh, *The Unified Software Development Process,* Addison-Wesley, 1999.

[Jai18] Jain, N., et al., "Digital Consumers, Emerging Markets, and the $4 Trillion Future," September 18, 2018, available at https://www.bcg.com/en-us/publications/2018/digital-consumers-emerging-markets-4-trillion-dollar-future.aspx.

[Jal04] Jalote, P., et al., "Timeboxing: A Process Model for Iterative Software Development," *Journal of Systems and Software,* vol. 70, no. 2, 2004, pp. 117–127.

[Jam13] James, G., et al., *An Introduction to Statistical Learning with Applications in R,* Springer, 2013.

[Jan16] Jan, S., et al., "An Innovative Approach to Investigate Various Software Testing Techniques and Strategies," *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET),* vol. 2, no. 2, March–April 2016, pp. 682–689.

[Jon04] Jones, C., "Software Project Management Practices: Failure Versus Success," *CrossTalk,* October 2004, available at http://www.pauldee.org/se-must-have/jones-failure-success.pdf.

[Jon86] Jones, C., *Programming Productivity,* McGraw-Hill, 1986.

[Jon91] Jones, C., *Systematic Software Development Using VDM,* 2nd ed., Prentice Hall, 1991.

[Jov15] Jovanovic M., A. Mesquida, and A. Mas, "Process Improvement with Retrospective Gaming in Agile Software Development," *Systems, Software and Services Process Improvement,* EuroSPI 2015 Communications in Computer and Information Science, R. O'Connor, et al. (eds.), vol. 543, Springer, 2015.

[Joy00] Joy, B., "The Future Doesn't Need Us," *Wired,* vol. 8, no. 4, April 2000.

[Kan01] Kaner, C., "Pattern: Scenario Testing" (draft), 2001, available at http://www.exampler.com/testing-com/test-patterns/patterns/pattern-scenario-testing-kaner.html.

[Kan93] Kaner, C., J. Falk, and H. Q. Nguyen, *Testing Computer Software,* 2nd ed., Van Nostrand-Reinhold, 1993.

[Kan95] Kaner, C., "Lawyers, Lawsuits, and Quality Related Costs," 1995, available at www.badsoftware.com/plaintif.htm.

[Kap15] Kapyaho, M., and M. Kauppinen, "Agile Requirements Engineering with Prototyping: A Case Study," *Proceedings IEEE 23rd International Requirements Engineering Conference,* August 2015, ON, Canada, pp. 334–343.

[Kar12] Kar S., S. Das, A. Kumar Rath, and S. K. Kar, "Self-assessment Model and Review Technique for SPICE: SMART SPICE," in *Software Process Improvement and Capability Determination,* A. Mas et al. (eds.), SPICE 2012, Communications in Computer and Information Science, vol. 290, 2012, Springer, Berlin, Heidelberg.

[Kar94] Karten, N., *Managing Expectations,* Dorset House, 1994.

[Kau11] Kaur, A., and S. Goel, "COTS Components Usage Risks in Component Based Software Development." *International Journal of Information Technology and Knowledge Management,* vol. 4, no. 2, July–December 2011, pp. 573–575.

[Kaz98] Kazman, R., et al., *The Architectural Tradeoff Analysis Method,* Software Engineering Institute, CMU/SEI-98-TR-008, July 1998, summarized at http://www.sei.cmu.edu/architecture/tools/evaluate/atam.cfm.

[Kea07] Keane, "Testing Mobile Business Applications," a white paper, 2007. A 40-point checklist that complements this white paper is available at https://softcrylic.com/blogs/40-point-checklist-testing-mobile-applications/.

[Kei18] Keith, J., "10 Great Sites for UI Design Patterns," 2018, available at https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns.

[Kei98] Keil, M., et al., "A Framework for Identifying Software Project Risks," *CACM,* vol. 41, no. 11, November 1998, pp. 76–83.

[Ker17] Kerzner, H., *Project Management: A Systems Approach to Planning, Scheduling, and Controlling,* 12th ed., Wiley, 2017.

[Kho12] Khode, A., "Getting Started with Mobile Apps Testing," 2012, available at http://www.mobileappstesting.com/getting-started-with-mobile-apps-testing/.

[Kim16a] Kim, G., et al., *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations,* Revolution Press, 2016.

[Kim16b] Kim, M., et al., "The Emerging Role of Data Scientists on Software Development Teams," *Proceedings 38th International Conference on Software Engineering (ICSE '16).* ACM, New York, NY, 2016, pp. 96–107.

[Kir94] Kirani, S., and W. Tsai, "Specification and Verification of Object-Oriented Programs," Technical Report TR 94-64, Computer Science Department, University of Minnesota, December 1994.

[Kiz05] Kizza, J., *Computer Network Security,* Springer, 2005.

[Kna16] Knapp, J., J. Zeratsky, and B. Kowitz, *Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days,* Simon and Schuster, 2016.

[Koe12] Koester, J., "The Seven Deadly Sins of MobileApp Design," *Venture Beat/Mobile,* May 31, 2012, available at http://venturebeat.com/2012/05/31/the-7-deadly-sins-of-mobile-app-design/.

[Kor03] Korpipaa, P., et al., "Managing Context Information in Mobile Devices," *IEEE Pervasive Computing,* vol. 2, no. 3, July–September 2003, pp. 42–51.

[Kou14] Kouzes, J., *Five Practices of Exemplary Leadership—Technology,* Wiley, 2014.

[Kra88] Krasner, G., and S. Pope, "A Cookbook for Using the Model-View Controller User Interface Paradigm in Smalltalk-80," *Journal of Object-Oriented Programming,* vol. 1, no. 3, August–September 1988, pp. 26–49.

[Kra95] Kraul, R., and L. Streeter, "Coordination in Software Development," *CACM,* vol. 38, no. 3, March 1995, pp. 69–81.

[Kru05] Krutchen, P., "Software Design in a Postmodern Era," *IEEE Software,* vol. 22, no. 2, March–April 2005, pp. 16–18.

[Kru06] Kruchten, P., H. Obbink, and J. Stafford (eds.), "Software Architectural" (special issue), *IEEE Software,* vol. 23, no. 2, March–April 2006.

[Kru09] Kruchten, P., et al., "The Decision View's Role in Software Architecture Practice," *IEEE Software,* vol. 26, no. 2, March–April 2009, pp. 70–72.

[Kub17] Kubat, M., *An Introduction to Machine Leaning,* 2nd ed., Springer, 2017.

[Kul13] Kulkarni, V., "Model Driven Software Development," *Modelling Foundations and Applications,* ECMFA 2013. Lecture Notes in Computer Science, Van Gorp, et al. (eds.), vol. 7949, Springer, 2013.

[Kur05] Kurzweil, R., *The Singularity Is Near,* Penguin Books, 2005.

[Kur13] Kurzweil, R., *How to Create a Mind,* Viking, 2013.

[Kyb84] Kyburg, H., *Theory and Measurement,* Cambridge University Press, 1984.

[Laa00] Laakso, S., et al., "Improved Scroll Bars," In *CHI '00 Extended Abstracts on Human Factors in Computing Systems (CHI EA '00).* ACM, New York, NY, 2000, pp. 97–98.

[Lag10] Lago, P., et al., "Software Architecture: Framing Stakeholders' Concerns," *IEEE Software,* vol. 27, no. 6, November–December 2010, pp. 20–24.

[Lai02] Laitenberger, A., "A Survey of Software Inspection Technologies," in *Handbook on Software Engineering and Knowledge Engineering,* World Scientific Publishing Company, 2002.

[Lam09] Lamsweerde, A., "Goal-Oriented Requirements Engineering: A Guided Tour," *Proceedings of 5th IEEE International Symposium on Requirements Engineering,* Toronto, August 2009, pp. 249–263.

[Lan01] Lange, M., "It's Testing Time! Patterns for Testing Software," June 2001, available at http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.116.5064&rep=rep1&type=pdf.

[Lar16] Larrucea, X., et al., "Software Process Improvement in Very Small Organizations," *IEEE Software,* vol. 33, no. 2, March–April 2016, pp. 85–89.

[Laz11] Lazzaroni, M., et al., *Reliability Engineering,* Springer, 2011.

[Leh97a] Lehman, M., and L. Belady, *Program Evolution: Processes of Software Change,* Academic Press, 1997.

[Leh97b] Lehman, M., et al., "Metrics and Laws of Software Evolution—The Nineties View," *Proceedings 4th International Software Metrics Symposium (METRICS '97),* IEEE, 1997, available at www.ece.utexas.edu/~perry/work/papers/feast1.pdf.

[Let04] Lethbridge, T., and R. Lagraniere, *Object-Oriented Software Engineering; Practical Software Development Using UML and Java,* 2nd ed., McGraw-Hill, 2004.

[Lev01] Levinson, M., "Let's Stop Wasting $78 billion a Year," *CIO Magazine,* October 15, 2001, available at https://www.cio.com/article/2441228/software-development---let-s-stop-wasting--78-billion-a-year.html.

[Lev12] Leveson, N., *Engineering a Safer World: Systems Thinking Applied to Safety* (Engineering Systems), MIT Press, 2012.

[Li16] Li, W., Z. Huang, and Q. Li, "Three-Way Decisions Based Software Defect Prediction," *Knowledge-Based Systems,* vol. 91, 2016, pp. 263–274.

[Lin16] Lin, Y., et al., "Interactive and Guided Architectural Refactoring with Search-Based Recommendation," *Proceedings 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering,* November 2016, pp. 535–546.

[Lin79] Linger, R., H. Mills, and B. Witt, *Structured Programming,* Addison-Wesley, 1979.

[Lis88] Liskov, B., "Data Abstraction and Hierarchy," *SIGPLAN Notices,* vol. 23, no. 5, May 1988.

[Maa07] Maassen, O., and S. Stelting, "Creational Patterns: Creating Objects in an OO System," 2007, available at www.informit.com/articles/article.asp?p=26452&rl=1.

[Mac10] Maciel, C., et al., "An Integration Testing Approach Based on Test Patterns and MDA Techniques," *Proceedings 8th Latin American Conference on Pattern Languages of Programs (SugarLoafPLoP '10),* ACM, New York, NY, Article 14, 2010.

[Mal16] Malhotra, R., "An Empirical Framework for Defect Prediction Using Machine Learning Techniques with Android Software," *Applied Software Computing,* vol. 49, December 2016, pp. 1034–1050.

[Mal17] Malhotra, R., M. Khanna, and R. Raje, "On the Application of Search-Based Techniques for Software Engineering Predictive Modeling: A Systematic Review and Future Directions," *Swarm and Evolutionary Computation,* vol. 32, February 2017, pp. 85–109.

[Man17] Mansoor, U., et al., "Multi-view Refactoring of Class and Activity Diagrams Using a Multi-Objective Evolutionary Algorithm," *Software Quality Journal,* vol. 25, no. 2, 2017, pp. 529–552.

[Man81] Mantai, M., "The Effect of Programming Team Structures on Programming Tasks," *CACM,* vol. 24, no. 3, March 1981, pp. 106–113.

[Man97] Mandel, T., *The Elements of User Interface Design,* Wiley, 1997.

[Mao17] Mao, K., M. Harman, and Y. Jia, "Crowd Intelligence Enhances Automated Mobile Testing," *Proceedings 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017),* IEEE Press, Piscataway, NJ, 2017, pp. 16–26.

[Mar00] Martin, R., "Design Principles and Design Patterns," 2000, available at https://fi.ort.edu.uy/innovaportal/file/2032/1/design_principles.pdf.

[Mar02] Marick, B., "Software Testing Patterns," 2002.

[Mar94] Marick, B., *The Craft of Software Testing,* Prentice Hall, 1994.

[Mat94] Matson, J., et al., "Software Cost Estimation Using Function Points," *IEEE Transactions on Software Engineering,* vol. 20, no. 4, April 1994, pp. 275–287.

[Max16] Maxim, B. R., and M. Kessentini, "An Introduction to Modern Software Quality Assurance," in *Software Quality Assurance,* I. Mistrik et al. (eds.), Morgan Kaufman, 2016, pp. 19–46.

[McC09] McCaffrey, J., "Analyzing Risk Exposure and Risk Using PERIL," *MSDN Magazine,* January 2009, available at http://msdn.microsoft.com/en-us/magazine/dd315417.aspx.

[McC76] McCabe, T., "A Software Complexity Measure," *IEEE Transactions on Software Engineering,* vol. SE-2, December 1976, pp. 308–320.

[McC77] McCall, J., P. Richards, and G. Walters, "Factors in Software Quality," three volumes, NTIS AD-A049-014, 015, 055, November 1977.

[McC96] McConnell, S., "Best Practices: Daily Build and Smoke Test," *IEEE Software,* vol. 13, no. 4, July 1996, pp. 143–144.

[McG06] McGraw, G., *Software Security: Building Security In,* Addison-Wesley Professional, 2006.

[McG91] McGlaughlin, R., "Some Notes on Program Design," *Software Engineering Notes,* vol. 16, no. 4, October 1991, pp. 53–54.

[McG94] McGregor, J., and T. Korson, "Integrated Object-Oriented Testing and Development Processes," *CACM,* vol. 37, no. 9, September, 1994, pp. 59–77.

[McK17] McKinney, W., *Python for Data Analysis Data Wrangling with Pandas, NumPy, and IPython,* O'Reilly Media, 2017.

[McT16] McTear, M., Z. Callejas, and D. Griol, *The Conversational Interface: Talking to Smart Devices,* Springer, 2016.

[Mea05] Mead, N., E. Hough, and T. Stehney, "Security Quality Requirements Engineering (SQUARE) Methodology" (CMU/SEI-2005-TR-009, ADA452453), Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2005, available at http://www.sei.cmu.edu/publications/documents/05.reports/05tr009.html.

[Mea16] Mead, N., and C. Woody, *Cyber Security Engineering: A Practical Approach for Systems and Software Assurance,* Addison-Wesley, 2016.

[Mea18] Mead, N., et al., "A Hybrid Threat Modeling Method," Software Engineering Institute CMU/SEI Report Number: CMU/SEI-2018-TN-002, March 2018, downloadable at https://resources.sei.cmu.edu/asset_files/TechnicalNote/2018_004_001_516627.pdf.

[Mei09] Meier, J., et al., *Microsoft Application Architecture Guide,* 2nd ed., Microsoft Press, 2009, available at http://msdn.microsoft.com/en-us/library/ff650706.

[Mei12] Meier, J., et al., "Chapter 19: Mobile Applications," *Application Architecture Guide, 2.0,* 2012, available at https://guidanceshare.com/wiki/Application_Architecture_Guide_-_Chapter_19_-_Mobile_Applications.

[Mei18] Meinke, K., and A. Bennaceur, "Machine Learning for Software Engineering: Models, Methods, and Applications," *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeding (ICSE '18).* ACM, 2018.

[Mel06] Mellado, D., et al., *Applying a Security Requirements Engineering Process,* Springer, 2006, available at https://pdfs.semanticscholar.org/379f/941eaf2878341948fbc30f6da246d90702ab.pdf.

[Men01] Mendes, E., N. Mosley, and S. Counsell, "Estimating Design and Authoring Effort," *IEEE Multimedia,* vol. 8, no. 1, January–March 2001, pp. 50–57.

[Men13] Menzies, T., and T. Zimmermann, "Software Analytics: So What?" *IEEE Software,* vol. 30, no. 4, July 2013, pp. 31–37.

[Mer93] Merlo, E., et al., "Reengineering User Interfaces," *IEEE Software,* January 1993, pp. 64–73.

[Mic04] Microsoft, "Prescriptive Architecture: Integration and Patterns," *MSDN,* May 2004, available at http://msdn2.microsoft.com/en-us/library/ms978700.aspx.

[Mic09] Microsoft Patterns & Practices Team, *Microsoft Application Architecture Guide,* 2nd ed., Microsoft Press, 2009.

[Mic10] Microsoft Security Development Lifecycle Version 5.0, 2010, available at http://download.microsoft.com/download/F/2/0/F205C451-C59C-4DC7-8377-9535D0A208EC/Microsoft%20SDL_Version%205.0.docx.

[Mic13a] Microsoft Accessibility Technology for Everyone, 2013, available at www.microsoft.com/enable/.

[Mic13b] Microsoft, "Patterns and Practices," *MSDN,* 2013, available at http://msdn.microsoft.com/en-us/library/ff647589.aspx.

[Mic17] Microsoft Corporation, "SDL Threat Modeling Tool," *Security Development Lifecycle,* November 10, 2017, available at https://www.microsoft.com/en-us/sdl/adopt/threatmodeling.aspx.

[Mic18] *The Microsoft Security Development Lifecycle (SDL)*, available at https://www.microsoft.com/en-us/securityengineering/sdl/, Microsoft, 2018.

[Mil00] Mili, A., and R. Cowan, "Software Engineering Technology Watch," April 6, 2000, available at https://www.researchgate.net/publication/222828018_Software_engineering_technology_watch.

[Mil04] Miler, J., and J. Gorski, "Risk Identification Patterns for Software Projects," *Foundations of Computing and Decision Sciences,* vol. 29, no. 1, 2004, pp. 115–131, available at http://iag.pg.gda.pl/RiskGuide/papers/Miler-Gorski_Risk_Identification_Patterns.pdf.

[Mil72] Mills, H., "Mathematical Foundations for Structured Programming," Technical Report FSC 71-6012, IBM Corp., Federal Systems Division, Gaithersburg, MD, 1972.

[Mit14] Mitre Corp., "Common Weakness Ennumeration: A Community-Developed Dictionary of Software Weakness Types," 2014, available at http://cwe.mitre.org.

[Mob12] "Mobile UI Patterns," 2012, available at http://mobile-patterns.com/.

[Mof04] Moffett, J., et al., "Core Security Requirements Artefacts," Technical Report 2004/23. Milton Keynes, UK: Department of Computing, The Open University, June 2004, available at http://computing.open.ac.uk.

[Mol12] Molitor, M., "Software Configuration Management and Continuous Integration," 2012, available at https://sewiki.iai.uni-bonn.de/_media/teaching/labs/xp/2012b/seminar/6-scm.pdf.

[Mor05] Morales, A., "The Dream Team," Dr. Dobbs Portal, March 3, 2005, available at www.ddj.com/dept/global/184415303.

[Mor81] Moran, T., "The Command Language Grammar: A Representation for the User Interface of Interactive Computer Systems," *International Journal of Man-Machine Studies,* vol. 15, 1981, pp. 3–50.

[Mun17] Munaiah, N., et al., "Do Bugs Foreshadow Vulnerabilities? An In-depth Study of the Chromium Project," *Empirical Software Engineering,* vol. 22, 2017, pp. 1305–1347.

[Mus87] Musa, J., A. Iannino, and K. Okumoto, *Engineering and Managing Software with Reliability Measures,* McGraw-Hill, 1987.

[Mye78] Myers, G., *Composite Structured Design,* Van Nostrand, 1978.

[Mye79] Myers, G., *The Art of Software Testing,* Wiley, 1979.

[Nan14] Ramanathan, N., A. Lal, and R. Parmar, "State of the Art in Software Quality Assurance," *ACM SIGSOFT Software Engineering Notes,* vol. 39, 2014, pp. 1–6.

[NAS07] NASA, "Software Risk Checklist, Form LeR-F0510.051," March 2007, available at https://www.scribd.com/document/250436/Software-Risk-Checklist-Department-of-Defense-NASA-USA.

[Nat15] National Instruments, "7 Steps in Creating a Functional Prototype," November 2015, available at http://www.ni.com/white-paper/10590/en/.

[Nei14] Neil, T., *Mobile Design Pattern Gallery: UI Patterns for Smartphone Apps,* 2nd ed., O'Reilly Media, 2014.

[Nei93] Nielsen, J., *Usability Engineering.* Morgan Kaufmann Publishers Inc., 1993.

[Net18] Neto, F., et al., "Improving Continuous Integration with Similarity-Based Test Case Selection," *Proceedings 13th International Workshop on Automation of Software Test (AST '18),* ACM, New York, NY, 2018, pp. 39–45.

[Nie00] Nielsen, J., *Designing Web Usability,* New Riders Publishing, 2000.

[Nie10] Nielsen, D., "Successfully Building a Software Prototype," July 2010, http://www.nuwavetech.com/it-project-blog/bid/43839/successfully-building-a-software-prototype (downloaded January 16, 2018).

[Nie13] Nielsen, L., "Personas," in *The Encyclopedia of Human-Computer Interaction*, 2nd ed., M. Soegaard (ed.), The Interaction Design Foundation, 2013.

[Nie94] Nielsen, J., and J. Levy, "Measuring Usability: Preference vs. Performance," *CACM,* vol. 37, no. 4, April 1994, pp. 65–75.

[Nie96] Nielsen, J., and A. Wagner, "User Interface Design for the WWW," *Proceedings CHI '96 Conf. on Human Factors in Computing Systems,* ACM Press, 1996, pp. 330–331.

[Nor13] Norman, D. A., *The Design of Everyday Things,* Revised Expanded Edition, Basic Books, Inc., 2013.

[Nor70] Norden, P., "Useful Tools for Project Management," in *Management of Production,* M. K. Starr (ed.), Penguin Books, 1970.

[Nor88] Norman, D., *The Design of Everyday Things,* Doubleday, 1988.

[Nov05] Novotny, O., "Next Generation Tools for Object-Oriented Development," *The Architecture Journal,* January 2005, available at http://msdn2.microsoft.com/en-us/library/aa480062.aspx.

[Num18] NumFOCUS, "Python Data Analysis Library," 2018. Retrieved from pandas: http://pandas.pydata.org/.

[Nun11] Nunes, N., L. Constantine, and R. Kazman, "iUCP: Estimating Interactive Software Project Size with Enhanced Use Case Points," *IEEE Software,* vol. 28, no. 4, July–August 2011, pp. 64–73.

[Nun17] Nunez-Iglesias, J., S. Walt, and H. Dashnow, *Elegant SciPy,* O'Reilly Media, 2017.

[Nyg11] Nygard, M., "Documenting Architecture Decisions," 2011, available at http://thinkrelevance.com/blog/2011/11/15/documenting-architecture-decisions.

[Off02] Offutt, J., "Quality Attributes of Web Software Applications," *IEEE Software,* March–April 2002, pp. 25–32.

[Ols99] Olsina, L., et al., "Specifying Quality Characteristics and Attributes for Web Sites," *Proceedings 1st ICSE Workshop on Web Engineering,* ACM, Los Angeles, May 1999.

[OMG03] Object Management Group, *OMG Unified Modeling Language Specification,* version 1.5, March 2003, available at www.rational.com/uml/resources/documentation/.

[Oth17] Othmane, B. L., et al., "Time for Addressing Software Security Issues: Prediction Models and Impacting Factors," *Data Science Engineering,* vol. 2, no. 2, 2017, pp. 107–124.

[Osb90] Osborne, W. M., and E. J. Chikofsky, "Fitting Pieces to the Maintenance Puzzle," *IEEE Software,* January 1990, pp. 10–11.

[OSO12] OpenSource.org, 2012, available at www.opensource.org/.

[Oun17] Ouni, A., M. Kessentini, and M. Cinneide, "MORE: A Multi-Objective Refactoring Recommendation Approach to Introducing Design Patterns and Fixing Code Smells," *Journal of Software: Evolution and Process,* 2017, available at https://doi.org/10.1002/smr.1843.

[OWA16] "Open Web Application Security Project—Buffer Overflow," 2016, available at https://www.owasp.org/index.php/Buffer_Overflow.

[OWA18] "Open Web Application Security Project—Attack Surface Cheat Sheet," 2018, available at https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.md.

[Pad18] Padhy, N., et al., "Software Reusability Metrics Estimation: Algorithms, Models and Optimization Techniques," *Computers and Electrical Engineering,* vol. 69, July 2018, pp. 653–668.

[Pag85] Page-Jones, M., *Practical Project Management,* Dorset House, 1985.

[Par11] Pardo, C., et al., "Harmonizing Quality Assurance Processes and Product Characteristics," *IEEE Computer,* June 2011, pp. 94–96.

[Par15] Parunak, H., and S. Brueckner, "Software Engineering for Self-Organizing Systems," *The Knowledge Engineering Review,* vol. 30, no. 4, September 2015, pp. 419–434.

[Par72] Parnas, D., "On Criteria to Be Used in Decomposing Systems into Modules," *CACM,* vol. 14, no. 1, April 1972, pp. 221–227.

[Par96b] Park, R. E., W. B. Goethert, and W. A. Florac, *Goal Driven Software Measurement—A Guidebook,* CMU/SEI-96-BH-002, Software Engineering Institute, Carnegie Mellon University, August 1996.

[Pas10] Passos, L., et al., "Static Architecture-Conformance Checking: An Illustrative Overview," *IEEE Software,* vol. 27, no. 5, September–October 2010, pp. 82–89.

[Pau94] Paulish, D., and A. Carleton, "Case Studies of Software Process Improvement Measurement," *Computer,* vol. 27, no. 9, September 1994, pp. 50–57.

[Ped15] Pedreira, O., et al., "Gamification in Software Engineering—A Systematic Mapping," *Information and Software Technology,* vol. 57, 2015, pp. 157–168.

[Per74] Persig, R., *Zen and the Art of Motorcycle Maintenance,* Bantam Books, 1974.

[Pet18] Petke, J., et al., "Genetic Improvement of Software: A Comprehensive Survey," in *IEEE Transactions on Evolutionary Computation,* vol. 22, no. 3, June 2018, pp. 415–432.

[Pew18] PEW Research Center, "Mobile Fact Sheet," 2018, available at http://www.pewinternet.org/fact-sheet/mobile/.

[Phi02] Phillips, M., "CMMI V1.1 Tutorial," April 2002, available at www.sei.cmu.edu/cmmi/.

[Pit18] Pitoura, E., et al., "On Measuring Bias in Online Information," *SIGMOD,* vol. 46, no. 4, February 2018, pp. 16–21.

[Pol45] Polya, G., *How to Solve It,* Princeton University Press, 1945.

[Pop08] Popcorn, F., *Faith Popcorn's Brain Reserve,* 2008, available at www.faithpopcorn.com/.

[Por18] Port, D., and B. Taber, "Actionable Analytics for Strategic Maintenance of Critical Software: An Industry Experience Report," *IEEE Software,* vol. 35, no. 1, January–February 2018, pp. 58–63.

[Pre05] Pressman, R., "Adaptable Process Model, Version 2.0," R. S. Pressman & Associates, 2005, available at www.rspa.com/apm/index.html.

[Pre08] Pressman, R., and D. Lowe, *Web Engineering: A Practitioner's Approach,* McGraw-Hill, 2008.

[Pre88] Pressman, R., *Making Software Engineering Happen,* Prentice Hall, 1988.

[Pre94] Premerlani, W., and M. Blaha, "An Approach for Reverse Engineering of Relational Databases," *CACM,* vol. 37, no. 5, May 1994, pp. 42–49.

[Pri10] Prince, B., "10 Most Dangerous Web App Security Flaws," *eWeek.com,* April 19, 2010, available at https://www.eweek.com/security/10-most-dangerous-web-app-security-risks.

[Pun17] Punchoojit, L., and N. Hongwarittorrn, "Usability Studies on Mobile User Interface Design Patterns: A Systematic Literature Review," *Advances in Human-Computer Interaction,* available at https://www.hindawi.com/journals/ahci/2017/6787504/

[Put78] Putnam, L., "A General Empirical Solution to the Macro Software Sizing and Estimation Problem," *IEEE Transactions on Software Engineering,* vol. SE-4, no. 4, July 1978, pp. 345–361.

[Put92] Putnam, L., and W. Myers, *Measures for Excellence,* Yourdon Press, 1992.

[Pyz14] Pyzdek, T., and P. Keller, *The Six Sigma Handbook,* 4th ed., McGraw-Hill, 2014.

[Rad02] Radice, R., *High-Quality Low Cost Software Inspections,* Paradoxicon Publishing, 2002.

[Raj14] Rajagoplan, S., "Review of the Myths on Original Development Model," *International Journal of Software Engineering and Applications,* vol. 5, no. 6, November 2014, pp. 103–111.

[Ray12] Raymond P., L. Buse, and T. Zimmermann. "Information Needs for Software Development Analytics," *Proceedings 34th International Conference on Software Engineering (ICSE '12),* IEEE Press, Piscataway, NJ, 2012, pp. 987–996.

[Ree99] Reel, J., "Critical Success Factors in Software Projects," *IEEE Software,* May 1999, pp. 18–23.

[Rem14] Rempel, P., et al., "Mind the Gap: Assessing the Conformance of Software Traceability to Relevant Guidelines," *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014).* ACM, New York, NY, 2014, pp. 943–954.

[Reu12] Reuveni, D., "Crowdsourcing Provides Answer to App Testing Dilemma," 2012, available at https://www.ecnmag.com/article/2010/02/crowdsourcing-provides-answer-app-testing-dilemma.

[Ric04] Rico, D., *ROI of Software Process Improvement,* J. Ross Publishing, 2004, available at http://davidfrico.com/rico03a.pdf.

[Rob10] Robinson, W., "A Roadmap for Comprehensive Requirements Monitoring," *IEEE Computer,* vol. 43, no. 5, May 2010, pp. 64–72.

[Rod16] Rodríguez, A., F. Ortega, and R. Concepción, "A Method for the Evaluation of Risk in IT Projects," *Expert Systems with Applications,* vol. 45, 2016, pp. 273–285.

[Rod17] Rodríguez, P., et al., "Continuous Deployment of Software Intensive Products and Services: A Systematic Mapping Study," *Journal of Systems and Software,* vol. 123, 2017, pp. 263–291.

[Rod98] Rodden, T., et al., "Exploiting Context in HCI Design for Mobile Systems," *Proceedings of Workshop on Human Computer Interaction with Mobile Devices,* 1998.

[Roo09] Rooksby, J., et al., "Testing in the Wild: The Social and Organizational Dimensions of Real World Practice," *Journal of Computer Supported Work,* vol. 18, no. 5–6, December 2009, pp. 559–580.

[Ros17] Rosa, W., and C. Wallshein, "Software Effort Estimation Models for Contract Cost Proposal Evaluation," *Proceedings 2017 ICEAA Professional Development & Training Workshop,* June 2017, pp. 1–8.

[Ros75] Ross, D., J. Goodenough, and C. Irvine, "Software Engineering: Process, Principles and Goals," *IEEE Computer,* vol. 8, no. 5, May 1975.

[Rot02] Roth, J., "Seven Challenges for Developers of Mobile Groupware," *Proceedings of Computer Human Interaction Workshop on Mobile Ad Hoc Collaboration,* 2002.

[Rou02] Rout, T. (project manager), *SPICE: Software Process Assessment—Part 1: Concepts and Introductory Guide,* 2002, downloadable at http://www.noginfo.com.br/arquivos/SPICE.pdf.

[Roy70] Royce, W., "Managing the Development of Large Software Systems: Concepts and Techniques," *Proceedings WESCON,* August 1970.

[Roz11] Rozanski, N., and E. Woods, *Software Systems Architecture,* 2nd ed., Addison-Wesley, 2011.

[Rya11] Ryan, T., *Statistical Methods for Quality Improvement,* Wiley, 2011.

[San17] Sancetta, G., et al., "Risk Patterns, Structural Characteristics, and Organizational Configurations," *Strategic Change,* 2017, available at https://doi.org/10.1002/jsc.2138.

[Sce02] Sceppa, D., *Microsoft ADO.NET,* Microsoft Press, 2002.

[Sca15] Scandariato, R., K. Wuyts, and W. Joosen, "A Descriptive Study of Microsoft's Threat Modeling Technique," *Requirements Engineering,* vol. 20, no. 2, June 2015, pp. 163–180.

[Sch01a] Schneider, G., and J. Winters, *Applying Use Cases,* 2nd ed., Que, 2001.
[Sch01b] Schwaber, K., and M. Beedle, *Agile Software Development with SCRUM,* Prentice Hall, 2001.
[Sch06] Schmidt, D., "Model-Driven Engineering," *IEEE Computer,* vol. 39, no. 2, February 2006, pp. 25–31.
[Sch09] Schumacher, R. (ed.), *Handbook of Global User Research,* Morgan-Kaufmann, 2009.
[Sch11] Schilit, B., "Mobile Computing: Looking to the Future," *IEEE Computer,* vol. 44, no. 5, May 2011, pp. 28–29.
[Sch15] Schell, M., and J. O'Brien, *Communicating the UX Vision: 13 Anti-Patterns That Block Ideas,* Morgan Kaufman, 2015.
[Sch98] Schneider, G., and J. Winters, *Applying Use Cases,* Addison-Wesley, 1998.
[Sch99] Schneidewind, N., "Measuring and Evaluating Maintenance Process Using Reliability, Risk, and Test Metrics," *IEEE Transactions,* SE, vol. 25, no. 6, November–December 1999, pp. 768–781.
[Sci18] SciPy Developers, *SciPy Library,* 2018, available at https://scipy.org/scipylib/index.html.
[SEI02] SEI, "Maintainability Index Technique for Measuring Program Maintainability," 2002.
[SEI08] Software Engineering Institute, "The Ideal Model," 2008, available at https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=20208.
[Ser15] Serrador, P., and J. Pinto, "Does Agile Work?—A Quantitative Analysis of Agile Project Success," *International Journal of Project Management,* vol. 33, no. 5, 2015, pp. 1040–1051.
[Sha05] Shalloway, A., and J. Trott, *Design Patterns Explained,* 2nd ed., Addison-Wesley, 2005.
[Sha09] Shaw, M., "Continuing Prospects for an Engineering Discipline of Software," *IEEE Software,* vol. 26, no. 8, November–December 2009, pp. 64–67.
[Sha15] Shaw, M., and D. Garla, *Software Architecture: Perspectives on an Emerging Discipline,* Pearson, 2015.
[Sha17] Sharma, S., and B. Coyne, *DevOps for Dummies,* 3rd ed., Wiley, 2017.
[Shn09] Shneiderman, B., et al., *Designing the User Interface,* 5th ed., Addison-Wesley, 2009.
[Shn16] Shneiderman, B., et al., *Designing the User Interface: Strategies for Effective Human-Computer Interaction,* 6th ed., Pearson, 2016.
[Sho14] Shostack, A., *Threat Modeling: Designing for Security.* John Wiley & Sons, 2014.
[Shu12] Shull, F., "Designing a World at Your Fingertips: A Look at Mobile User Interfaces," *IEEE Software,* vol. 29, no. 4, July–August 2012, pp. 4–7.
[Shu13] Shunn, A., et al., *Strengths in Security Solutions, Software Engineering Institute,* Carnegie Mellon University, 2013, available at http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=77878.
[Shu16] Shull, F., *Evaluation of Threat Modeling Methodologies,* Software Engineering Institute, Carnegie Mellon University, 2016, available at http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=474197.
[Sin00] Sindre, G., and A. Opdahl, "Eliciting Security Requirements by Misuse Cases," *Proceedings 37th International Conference on Technology of Object-Oriented Languages and Systems (TOOLS-37'00),* New York, NY: IEEE Press, 2000, pp. 120–131.
[Sin01] Sindre, G., and A. Opdahl, "Templates for Misuse Case Description," *Seventh International Workshop on Requirements Engineering: Foundation for Software Quality,* 2001, available at http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.9.8190.
[Sla12] Slattery, K., "Study Shows the Importance of Localization Testing," 2012. A related article can be found at https://www.welocalize.com/6-reasons-localization-qa-testing-important/.
[Smo08] Smolander, K., et al., "Software Architectures: Blueprint, Literature, Language, or Decision?" *European Journal of Information Systems,* vol. 17, 2008, pp. 575–588.
[Sne18] Snee, R., and R. Hoerl, *Leading Six Sigma,* 2nd ed., Pearson, 2018.
[Sne95] Sneed, H., "Planning the Reengineering of Legacy Systems," *IEEE Software,* January 1995, pp. 24–25.
[Soa10] Soares, G., et al., "Making Program Refactoring Safer," *IEEE Software,* vol. 37, no. 4, July–August 2010, pp. 52–57.
[Soa11] SOASTA, White Paper: "Five Strategies for Performance Testing Mobile Applications," 2011, available at http://hosteddocs.ittoolbox.com/whitepapersoastamobile.pdf.
[Som97] Somerville, I., and P. Sawyer, *Requirements Engineering,* Wiley, 1997.
[Som05] Somerville, I., "Integrating Requirements Engineering: A Tutorial," *IEEE Software,* vol. 22, no. 1, January–February 2005, pp. 16–23.

[Sou08] de Sousa, C., and D. Redmiles, "An Empirical Study of Software Developer's Management of Dependencies and Changes," *ICSE Proceedings,* May 2008, available at www.ics.uci.edu/~redmiles/publications/C078-deSR08.pdf.

[Spa11] Spagnolli, B., et al., "Eco-Feedback on the Go: Motivating Energy Awareness," *IEEE Computer,* vol. 44, no. 5, May 2011, pp. 38–45.

[SPI99] "SPICE: Software Process Assessment, Part 1: Concepts and Introduction," Version 1.0, ISO/IEC JTC1, 1999.

[SSO08] Software-Supportability.org, 2008, available at www.software-supportability.org/.

[Ste10] Stephens, M., and D. Rosenberg, *Design Driven Testing,* Apress, 2010.

[Ste16] Steed, S., et al., "An 'In the Wild' Experiment on Presence and Embodiment Using Consumer Virtual Reality Equipment," *IEEE Transactions on Visualization and Computer Graphics,* vol. 22, no. 4, April 2016, pp. 1406–1414.

[Ste18] Steffens, A., H. Lichter, and J. Döring, "Designing a Next-Generation Continuous Software Delivery System: Concepts and Architecture," *Proceedings 4th International Workshop on Rapid Continuous Software Engineering (RCoSE '18),* ACM, New York, NY, 2018, pp. 1–7.

[Ste74] Stevens, W., G. Myers, and L. Constantine, "Structured Design," *IBM Systems Journal,* vol. 13, no. 2, 1974, pp. 115–139.

[Sto05] Stone, D., et al., *User Interface Design and Evaluation,* Morgan Kaufman, 2005.

[Sul11] Sullivam, B., and V. Liu, *Web Application Security, A Beginner's Guide,* McGraw-Hill, 2011.

[Sun15] Sun X., et al., "What Information in Software Historical Repositories Do We Need to Support Software Maintenance Tasks? An Approach Based on Topic Model," in *Computer and Information Science,* Studies in Computational Intelligence, R. Lee (ed.), vol. 566, Springer, 2015.

[SWE14] *Software Engineering Body of Knowledge,* version 3, 2014, available at https://www.computer.org/web/swebok (accessed December 9, 2018).

[Tai12] Taivalsaari, A., and K. Systa, "Mobile Content as a Service: A Blueprint for a Vendor-Neutral Cloud of Mobile Devices," *IEEE Software,* vol. 29, no. 4, July–August 2012, pp. 28–33.

[Tai89] Tai, K., "What to Do Beyond Branch Testing," *ACM Software Engineering Notes,* vol. 14, no. 2, April 1989, pp. 58–61.

[Tan01] Tandler, P., "Aspect-Oriented Model-Driven Development for Mobile Context-Aware Computing," *Proceedings of UbiComp 2001: Ubiquitous Computing,* 2001.

[Tao17] Tao, H., and J. Gao, "On Building a Cloud-Based Mobile Testing Infrastructure Service System," *Journal of Systems and Software,* vol. 124, 2017, pp. 39–55.

[Tay09] Taylor, R., N. Medvidovic, and E. Dashofy, *Software Architecture,* Wiley, 2009.

[Tho04] Thomas, J., et al., *Java Testing Patterns,* Wiley, 2004.

[Tho92] Thomsett, R., "The Indiana Jones School of Risk Management," *American Programmer,* vol. 5, no. 7, September 1992, pp. 10–18.

[Tic18] *TickIT plus,* 2018, available at http://www.tickitplus.org/.

[Tid11] Tidwell, J., *Designing Interfaces: Patterns for Effective Interaction Design,* 2nd ed., O'Reilly, 2011.

[Til00] Tillman, H., "Evaluating Quality on the Net," Babson College, May 30, 2000, available at http://www.dronet.org/lineeguida/ligu_pdf/evelqual.pdf.

[Toc18] Tonchia, S., "Project Time Management," in *Industrial Project Management. Management for Professionals,* Springer, 2018.

[Tog01] Tognozzi, B., "First Principles," askTOG, 2001, available at www.asktog.com/basics/firstPrinciples.html.

[Tos17] Tosun, A., A. Bener, and S. Akbarinasaji, "A Systematic Literature Review on the Applications of Bayesian Networks to Predict Software Quality," *Software Quality Journal,* vol. 25, no. 1, March 2017, pp. 273–305.

[Tri03] Trivedi, R., *Professional Web Services Security,* Wrox Press, 2003.

[Tyr05] Tyree, J., and A. Akerman, "Architectural Decisions: Demystifying Architecture," *IEEE Software,* vol. 22, no. 2, March–April 2005.

[Uni03] Unicode, Inc., *The Unicode Home Page,* 2003, available at www.unicode.org/.

[USA87] U.S. Air Force, "Management Quality Insight," AFCSP 800-14, January 20, 1987.

[Ute12] UTest, E-book: *Essential Guide to Mobile App Testing,* 2012, available at http://go.applause.com/rs/539-CKP-074/images/The-Essential-Guide-to-Mobile-App-Testing.pdf.

[Vac06] Vacca, J., *Practical Internet Security,* Springer, 2006.

[Vak18] Vakkuri, V., and P. Abrahamsson, "The Key Concepts of Ethics of Artificial Intelligence," *2018 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC),* Stuttgart, 2018, pp. 1–6, doi:10.1109/ICE.2018.8436265.

[Van16] VanderPlas, J., *Python Data Science Handbook Essential Tools for Working with Data,* O'Reilly Media, 2016.

[Vel16] Veloso, A., and L. Costa, "Heuristics for Designing Digital Games in Assistive Environments: Applying the Guidelines to an Ageing Society," *2016 1st International Conference on Technology and Innovation in Sports, Health and Wellbeing (TISHW),* Vila Real, 2016, pp. 1–8.

[Ven03] Venners, B., "Design by Contract: A Conversation with Bertrand Meyer," *Artima Developer,* December 8, 2003, available at www.artima.com/intv/contracts.html.

[Vit03] Vitharana, P., "Risks and Challenges of Component-Based Software Development," *CACM,* vol. 46, no. 8, August 2003, pp. 67–72.

[Vit17] Vitharana, P., "Defect Propagation at the Project-Level: Results and a Post-Hoc Analysis on Inspection Efficiency," *Empirical Software Engineering,* vol. 22, no. 1, February 2017, pp. 57–79.

[Voa12] Voas, J., et al., "Mobile Software App Takeover," *IEEE Software,* vol. 29, no. 4, July–August 2012, pp. 25–27.

[Voe14] Voehl, F., H. Harrington, C. Mignosa, and R. Charron, *The Lean Six Sigma Black Belt Handbook,* Productivity Press, 2014, https://doi.org/10.1201/b15163.

[W3C18] *W3C Web Content Accessibility Guidelines (WCAG 2.1),* 2018, available at https://www.w3.org/TR/WCAG21/.

[Wal12] Walker, J., "Computer Programmers Learn Tough Lesson in Sharing," *The Wall Street Journal,* vol. 260, no. 48, August 27, 2012, p. 1.

[War07] Ward, M., "Using VoIP Software Building zBlocks—A Look at the Choices," TMNNet, 2007, available at www.tmcnet.com/voip/0605/featurearticle-using-voip-software-building-blocks.htm.

[Was10] Wasserman, A., "Software Engineering Issues for Mobile Application Development," *Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research,* 2010.

[Web05] Weber, S., *The Success of Open Source,* Harvard University Press, 2005.

[Web13] Web Application Security Consortium, 2013, available at http://www.webappsec.org/.

[Wee11] Weevers, I., "Seven Guidelines for Designing High Performance Mobile User Experiences," *Smashing Magazine,* July 18, 2011, available at http://uxdesign.smashingmagazine.com/2011/07/18/seven-guidelines-for-designing-high-performance-mobile-user-experiences/.

[Wel01] vanWelie, M., "Interaction Design Patterns," 2001. A related article can be found at https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns.

[Whi08] White, J., "Start Your Engines: Mobile Application Development," April 22, 2008, available at http://www.devx.com/SpecialReports/Article/37693.

[Whi12] Whittaker, J., et al., *How Google Tests Software,* Addison-Wesley, 2012.

[Whi15] Whigham, P. A., C. A. Owen, and S. G. MacDonell, "A Baseline Model for Software Effort Estimation," *ACM Transactions on Software Engineering and Methodology,* vol. 24, no. 3, 2015, pp. 1–11.

[Whi97] Whitmire, S., *Object-Oriented Design Measurement,* Wiley, 1997.

[Wie02] Wiegers, K., *Peer Reviews in Software,* Addison-Wesley, 2002.

[Wil05] Willoughby, M., "Q&A: Quality Software Means More Secure Software," *Computerworld,* March 21, 2005, available at https://www.computerworld.com/article/2563708/q-a--quality-software-means-more-secure-software.html.

[Wil97] Williams, R., J. Walker, and A. Dorofee, "Putting Risk Management into Practice," *IEEE Software,* May 1997, pp. 75–81.

[Wir71] Wirth, N., "Program Development by Stepwise Refinement," *CACM,* vol. 14, no. 4, 1971, pp. 221–227.

[Wir90] Wirfs-Brock, R., B. Wilkerson, and L. Weiner, *Designing Object-Oriented Software,* Prentice Hall, 1990.

[WMT02] *Web Mapping Testbed Tutorial,* 2002. A related presentation can be found at http://proceedings.esri.com/library/userconf/devsummit17/papers/dev_int_114.pdf.

[Woo04] Woody, C., *Eliciting and Analyzing Quality Requirements: Management Influences on Software Quality Requirements* (CMU/SEI-2005-TN-010, ADA441310). Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2004, available at http://www.sei.cmu.edu/publications/documents/05.reports/05tn010.html.

[Woo14] Woody, C., R. Ellison, and W. Nichols, *Predicting Software Assurance Using Quality and Reliability Measures,* CMU/SEI-2014-TN-026, Software Engineering Institute, Carnegie Mellon University, 2014, http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=428589.

[Woo89] Wood, J., and D. Silver, *Joint Application Design: How to Design Quality Systems in 40% Less Time,* John Wiley & Sons, 1989.

[Wri11] Wright, A., "Lessons Learned: Architects Are Facilitators, Too!" *IEEE Software,* vol. 28, no. 2, January–February 2011, pp. 70–72.

[Xia16] Xiao, L., et al., "Identifying and Quantifying Architectural Debt," *Proceedings of 38th ACM International Conference on Software Engineering,* May 2016, pp. 488–498.

[Xie18] Xie, T., "Intelligent Software Engineering: Synergy between AI and Software Engineering," *Proceedings of the 11th Innovations in Software Engineering Conference (ISEC '18).* ACM, New York, NY, Article 1, 2018.

[Yad17] Yadav, H., and D. Yadav, "Early Software Reliability Analysis Using Reliability Relevant Software Metrics," *International Journal of System Assurance Engineering and Management*, vol. 8, suppl., December 2017, pp. 2097–2108.

[Yau11] Yau, S., and H. An, "Software Engineering Meets Services and Cloud Computing," *IEEE Computer,* vol. 44, no. 10, October 2011, pp. 47–53.

[Yoo13] Yoo, S., and M. Harman, "Regression Testing Minimization, Selection and Prioritization: A Survey," *Journal of Software: Testing, Verification, and Reliability,* vol. 22, no. 2, 2013, pp. 67–120.

[You01] Young, R., *Effective Requirements Practices,* Addison-Wesley, 2001.

[You18] Young, S., T. Abdou, and A. Bener, "A Replication Study: Just-in-Time Defect Prediction with Ensemble Learning," *Proceedings of the ACM/IEEE Sixth International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering,* ACM, 2018, pp. 42–47.

[You75] Yourdon, E., *Techniques of Program Structure and Design,* Prentice Hall, 1975.

[Zah90] Zahniser, R., "Building Software in Groups," *American Programmer,* vol. 3, no. 7–8, July–August 1990.

[Zah94] Zahniser, R., "Timeboxing for Top Team Performance," *Software Development,* March 1994, pp. 35–38.

[Zan15] Zanoni, M., Fontana, F., and Stella, F. "On Applying Machine Learning Techniques for Pattern Detection," *Journal of Systems and Software,* vol. 103, May 2015, pp. 102–117.

[Zan18] Zancan, B. A. G., et al., "Accessibility Guidelines for Virtual Environments," in M. Antona and C. Stephanidis (eds.), *Universal Access in Human-Computer Interaction. Virtual, Augmented, and Intelligent Environments,* UAHCI 2018. Lecture Notes in Computer Science, vol. 10908, 2018.

[Zha13] Zhang, D., et al., "Software Analytics in Practice," *IEEE Software,* September–October 2013, vol. 30, no. 5, pp. 30–37.

[Zim11] Zimmermann, O., "Architectural Decisions as Reusable Design Assets," *IEEE Software,* vol. 28, no. 1, January–February 2011, pp. 64–69.

[Zus90] Zuse, H., *Software Complexity: Measures and Methods,* DeGruyter, 1990.

[Zus97] Zuse, H., *A Framework of Software Measurement,* DeGruyter, 1997.

INDEX

Abstraction, 87, 163, 168
Abstraction dimension, 171–172
Abuse cases, 363–364
Acceptance testing, 48, 68–69, 95, 430
Access control, 450
Accessibility, 233, 237, 259–261, 432–433
Action, defined, 10
Active state, 150
Activity, defined, 9–10
Activity diagrams, 146–148, 151–154, 223, 622–625
Activity networks, 525–526
Activity state, 627
Actors, 114–115, 131
Adapters, 230
Adaptive maintenance, 70, 553
Aesthetic design, 237, 277
Aesthetic metrics, 472
Aggregation, 614–615
Agile Alliance, 40–41
Agile manifesto, 37, 38, 42
Agile spirit, 41
Agile teams, 40, 41, 78, 495
Agility
  application of, 38–39
  in change management, 453–458
  characteristics of, 38, 40
  comparison of techniques for, 52
  cost of change and, 39–40
  in design, 60, 174, 185–186
  development of, 38, 41
  DevOps approach to, 50–51
  estimation and, 519
  Kanban method for, 48–50, 56
  principles of, 40–41
  process models for, 42–51, 56, 57
  in requirements engineering, 58, 104
  in reviews, 336
  Scrum framework for, 42–45, 56
  testing and, 95
  XP framework for, 46–48
Algorithms, 62–63, 352, 606
Alpha tests, 430
Ambient intelligence, 590
Analysis
  attack surface, 367
  boundary value, 389–390
  data, 633
  errors/defects, 342
  gap, 573
  hazards, 545
  impact, 443
  inventory, 563–564
  source code, 561
  static, 368
  task, 243, 247–248
  threats, 363
  user interface, 243–249
  work environment, 248–249
Analysis classes
  attributes for, 140
  defined, 104–105
  identifying, 137–140
  manifestations of, 138
  selection characteristics for, 139–140
  state diagrams for, 150–151
Analysis models and modeling. *see also* Requirements models
  building, 118–122
  domains of, 93
  elements of, 119–121
  patterns in, 122
  principles of, 129–130
  purpose of, 118
  rules of thumb for, 128–129
  terminology for, 126
Analysis patterns, 122
Analytics, 462–463, 509–511, 558–559
Anchor point milestones, 29
Antibugging, 382
Anti-patterns, 302–304
Anti-requirements, 383
Application domains, 7
Application objects, 251
Application software, 7
Appraisal costs, 317
Archetypes, 196–198
Architectural context diagram (ACD), 196
Architectural decision record (ADR), 184
Architectural decisions, 184–185, 194–196
Architectural description (AD), 183–185
Architectural description language (ADL), 164
Architectural design
  agility and, 60, 185–186
  alternatives in, 201–204
  archetypes in, 196–198
  economy of, 193
  elements of, 175
  emergence in, 194
  functions of, 158, 182
  instantiations of system in, 200
  metrics for, 466–467
  organization and refinement of, 193
  patterns, 187, 192–193, 203–204, 291, 299–300
  preliminary, 59–60
  properties of, 164
  refining into components, 198–200
  representing system in context, 196–197
  spacing in, 194
  styles of, 186–193
  symmetry in, 194
  visibility in, 194
  of WebApps, 278–280
Architectural mismatch, 229–230
Architectural patterns, 187, 192–193, 203–204, 291, 299–300
Architectural reviews, 202–204
Architecture
  call-and-return, 189
  conformance checking, 204
  content, 279
  data-centered, 187–188
  data-flow, 188
  defined, 163–164, 173, 182
  erosion of, 204
  functional, 226
  importance of, 183
  information, 235, 279
  layered, 189–192
  mobile, 273–274
  MVC, 189, 191–192, 279–280
  object-oriented, 189
  refactoring, 561–562
  responsibility-driven, 186
  testing, 375, 376
Architecture trade-off analysis method (ATAM), 201–202
Artificial intelligence (AI)
  applications involving, 7
  for decision making, 229
  integration-level testing and, 403
  machine learning and, 294–295, 322
  to model reliability, 351–352
  software engineering and, 589, 606
  software support and, 558
  testing, 428–429
Assessment. *see also* Evaluation; Risk assessment
  architectural, 202
  of quality, 314–315
  in software improvement process, 572–573
  of software process, 24–25
Assessment effort, 328
Associations, 144, 154, 613–615

Mc Graw Hill
501-1111

Atomic modules, 399
Attack patterns, 363–364
Attack surface, 366–367
Attributes
  defining, 140–141
  of metrics, 462
  in software quality assurance, 346
Audits, 342, 452, 458, 579
Audit trails, 445
Authoring, 456
Automated testing tools, 50, 413, 415, 422, 430
Automation phase of technology innovation, 585
Availability, 351

Backlog, 43–45
Backward impact management, 451
Baselines, 441, 442, 450, 481
Basis path testing, 384–386
Bayesian inference, 351–352
Behavioral elements, 120–121
Behavioral modeling
  activity diagrams in, 151–154
  characteristics of, 127
  identifying events with use case in, 149–150
  state diagrams in, 150–151
  steps for creation of, 149
Behavioral patterns, 292–293
Behavioral testing, 388–390, 392–393
Beta tests, 430
Big bang approach to integration testing, 398
Binary classification problems, 634
Black box metrics, 466
Black-box testing, 388–390, 397
Blueprint metaphor, 183
Bottom-up integration, 399–400
Boundary testing, 381–382
Boundary value analysis (BVA), 389–390
Breadth-first integration, 398
Breakthrough phase of technology innovation, 585
Buffer overflow, 367
Bugs, 326. *see also* Defects
Building blocks, 591–592
Building Security in Maturity Model (BSIMM), 370
Business activity monitoring, 123
Business goals, 485
Business risks, 534

Call-and-return architectures, 189
Capability Maturity Model Integration (CMMI), 370, 576–579
Capture/playback tools, 403
Casual meetings, 331
Casual reviews, 326, 331
Categorical variables, 633
Certification testing, 414
Change, sources of, 439
Change control, 441, 446, 448–450, 453–455
Change control authority (CCA), 448, 450
Change costs, 39–40
Change descriptions, 455
Change management
  agile, 453–458
  process for, 447–452
  in SCM, 445
  in SQA, 342
Change reports, 448
Change requests, 448
Change sets, 446
Changes to legacy systems, 8
Checklists
  mobile product quality, 285
  for reviews, 331
  risk item, 536
  validation requirements, 106
Chunking, 227–228
CK metrics suite, 469–471
Class-based design metrics, 469–470
Class-based elements, 120
Class-based modeling
  characteristics of, 127
  defining attributes and operations in, 140–141
  identifying analysis classes in, 137–140
  UML, 141–144
Class coupling, 218
Class diagrams, 120, 141, 143, 612–615
Classes. *see also* Analysis classes
  attributes, 140
  collaborating, 207
  defined, 120
  dependent, 404
  design, 169–171
  equivalence, 389
  hierarchies, 469, 470
  independent, 404
  operations, 141
  server, 404
Classification problems, 634
Class-responsibility-collaborator (CRC) model, 47, 144–146
Class testing, 390–391
Cloud-based testing, 428
Cloud computing, 7, 273–274
Clustering technique, 637
Cluster testing, 404
CMMI-DEV, 24
Code quality, 345
Code refactoring, 561, 564
Coding activity, 48, 95, 96, 367–368
Cognitive walkthroughs, 246, 253
Cohesion, 167, 170, 216–218, 468
Collaboration
  benefits of, 89
  in development, 595–596
  in requirements gathering, 110–113
  among stakeholders, 108
  teams and, 587
Collaboration diagrams, 220, 406, 621
Collaborators, in CRC modeling, 144
Collection subsystem, 455–456
Command labeling, 260
Common closure principle (CCP), 215
Common reuse principle (CRP), 215
Communication
  effectiveness of modes of, 89
  by interface, 257
  principles of, 88–90
  in process framework, 10
  in prototyping paradigm, 27
  task set for, 23, 24, 500
  in teams, 496, 604
Communicational cohesion, 216
Communication diagrams, 189, 190, 621–622
Compartmentalization, 521, 604
Compatibility testing, 414
Completeness, 468
Complexity, 468, 588–589
Component-based software engineering (CBSE), 228–230, 509
Component diagrams, 177
Component elements of SCM, 440
Component-level design
  cohesion in, 216–218
  coupling in, 218–219
  elements of, 176–177
  example of, 210–211
  functions of, 158, 206
  guidelines for, 215–216
  for MobileApps, 226–227
  patterns in, 300–301
  principles of, 173, 212–215
  specialized, 225–230
  steps for, 219–225
  for traditional components, 227–228
  for WebApps, 226, 282
Component-level testing
  black-box, 388–390
  elements of, 372
  object-oriented, 390–393
  planning and recordkeeping in, 378–380
  strategic approach to, 373–378
  test case design in, 381–383
  white-box, 383–387
Components
  class-based, 212–219
  defined, 207
  elaboration of, 207–209
  naming, 215–216
  object-oriented view of, 207–209
  process-related view of, 211–212
  refactoring, 230–231
  reuse of, 214–215, 228, 229
  traditional view of, 209–211
Composition, 615
Computational intelligence, 638

Computer-aided software engineering, 9
Concept development projects, 524–526
Concerns, 165. *see also* Separation of concerns
Condition testing, 386
Condition-transition-consequence (CTC) format, 542
Configuration audits, 452
Configuration management, 11, 437, 445. *see also* Software configuration management
Configuration objects, 441–443, 445–446, 450, 452, 457–458
Configuration reviews, 408
Configuration status reporting, 452
Connectionism, 636–637
Connectivity and connectivity testing, 414, 587
Consistency, 87, 240–241, 257
Construction
  of interface, 243
  as phase of Unified Process, 33
  principles of, 95–98
  in process framework, 10
Construction elements of SCM, 440
Constructive cost model (COCOMO), 511
Content architecture, 279
Content coupling, 218
Content design, 277–278
Content management, 455, 456
Content metrics, 473
Content objects, 277–278
Content repository, 444
Content testing, 420–421
Context-aware apps, 274–275
Context-free questions, 108
Context variables, 427
Contingency planning, 86, 92, 544
Continuous integration (CI), 400–402, 446–447
Continuous process improvement, 341
Continuous variables, 633
Contracted software, 342
Control coupling, 218
Control structure testing, 386–387
Convergence, 157
Coordination of teams, 496
Corrective maintenance, 69–70, 553
Cost of change, 39–40
Cost of quality, 317–318
Coupling, 167, 170, 174, 218–219, 468
Coupling between object classes (CBO), 469–470
Creational patterns, 292
Critical path, 520, 526
Critical practices for project management, 502
Crowdsourcing, 423
Customer journey maps, 244–245
Customers, defined, 88
Customer satisfaction, 40–41
Customer testing. *see* Acceptance testing
Cyclomatic complexity, 385–386

Data abstraction, 163
Data analysis, 633
Data analytics, 509–511
Data-centered architectures, 187–188
Data cleaning, 633
Data collection, 633
Data complexity, 467
Data design, 173, 174
Data-flow architectures, 188
Data flow diagrams (DFDs), 365, 366
Data flow testing, 381, 386
Data models, 127
Data munging, 630, 633
Data name rationalization, 561
Data patterns, 291
Data processing, 605
Data record standardization, 561
Data redesign, 561
Data refactoring, 561, 564–565
Data science
  analysis of data in, 633
  characteristics of, 461, 629
  cleaning data in, 633
  clustering technique in, 637
  collecting data in, 633
  computational intelligence and, 638
  decision trees in, 634–635
  dimensional reduction in, 637
  language selection for, 629–631
  libraries and tools for, 631
  machine learning and, 631–637
  nearest neighbor technique in, 635–637
  neural networks in, 636–637
  search-based software engineering and, 638
  training set fabrication in, 633
  transformation of data in, 633
Data transformation, 633
Debugging, 123
Decision metaphor, 184
Decision trees, 634–635
Decision view, 195
Decomposition, 497–500, 511–519
Defect removal efficiency (DRE), 482–484, 486
Defects
  amplification of, 327
  collection and analysis of, 342
  cost impact of, 326–327
  defined, 478
  prediction of, 322
  propagation of, 327
  tracking, 446
Deficiency lists, 408
Degree of rigor, 524
Degree of structural uncertainty, 506
Dependencies, 154, 216, 442–443, 445, 614
Dependency inversion principle (DIP), 214
Dependent classes, 404
Deployment
  in DevOps approach, 50
  in mobile development life cycle, 269
  principles of, 98–100
  in process framework, 11
  prototyping and, 63
Deployment diagrams, 177, 178, 225, 615–616
Deployment-level design, 177–178
Depth-first integration, 398
Depth of the inheritance tree (DIT), 469, 475
Descriptor form of deployment diagrams, 178
Design
  agility in, 60, 174, 185–186
  architectural (*see* Architectural design)
  component-level (*see* Component-level design)
  concepts of (*see* Design concepts)
  content, 277–278
  in context of software engineering, 157–159
  convergence in, 157
  of data, 173, 174
  defined, 181
  deployment-level, 177–178
  diversification in, 157
  dominant, 184
  evaluation of, 160, 161, 253–256
  evolution of, 161–162
  importance of, 3, 159
  interface (*see* Interface design)
  level, 456
  metrics for, 466–473, 475
  mobile (*see* MobileApps)
  in mobile development life cycle, 268
  models (*see* Design models)
  navigation, 280–282
  object-oriented approach to, 161, 173
  pattern-based (*see* Pattern-based design)
  practice, 156
  principles, 156
  process for, 159–162
  quality of, 159–161, 282–285, 311, 345
  refactoring, 48, 168, 225
  refinement of, 167–168
  task set for, 162
  test case, 381–383, 405–407
  user interaction, 236
  visual, 237, 277
  in XP framework, 47–48
Design classes, 169–171

Design concepts
abstraction, 163, 168
architecture, 163–164
classes, 169–171
functional independence, 167
importance of, 156
information hiding, 166
modularity, 165–166
patterns, 164–165
refactoring, 48, 168, 225
separation of concerns, 165
stepwise refinement, 167–168
Design models
architectural, 175
characteristics represented by, 93
component-level, 176–177
for data, 174
deployment-level, 177–178
dimensions of, 171–172
for interfaces, 175–176
principles for, 173–174
translation from requirements models, 158
Design patterns. *see* Pattern-based design
Design recovery, 564
Design structure quality index (DSQI), 468
Desk checks, 331–332
Determinate software, 7
Developer notes, 195
Development teams, 43–45, 67
Device compatibility testing, 414
DevOps approach, 50–51
Diagrams
activity, 146–148, 151–154, 223, 622–625
architectural context, 196
class, 120, 141, 143, 612–615
collaboration, 220, 406, 621
communication, 189, 190, 621–622
component, 177
data flow, 365, 366
deployment, 177, 178, 225, 615–616
sequence, 148–149, 618–621
state, 120–121, 150–151, 392, 625–628
swimlane, 151, 153–154
use case, 118, 119, 136, 137, 616–618
Dimensional reduction, 637
Direct measures, 479–480
Distributed debugging, 123
Diversification, 157
Do-activity, 627
Documentation testing, 434
Document restructuring, 564
Domain-specific modeling languages (DSMLs), 597
Dominant design, 184
Drivers, 379, 399
Dynamic models, 164
Dynamic testing, 429

Education and training, 342, 573
Efficiency, 257
Effort validation, 521
Egoless programming, 64–65
Elaboration
in communication activity, 23
of components, 207–209
as phase of Unified Process, 33
problem, 497–498
refinement as process of, 167–168, 248
in requirements engineering, 104–105
Elicitation. *see also* Requirements gathering
agile, 104
in communication activity, 23, 24
in requirements engineering, 104, 109
of security needs, 362
work products produced during, 114
Embedded software, 7
Empirical estimation models, 510–511
Empiricism phase of technology innovation, 585
Encapsulation, 475
End users, defined, 88
Engineering. *see also* Software engineering
algorithms, 62–63
feature, 634
forward, 565
reengineering, 554, 562–565
reverse, 553–557, 564
security, 357, 360–363
Engineering change order (ECO), 448, 450, 452
Engineering software, 7
Ensemble learning environment, 635
Environmental resources, 509
Equivalence classes, 389
Equivalence partitioning, 389
Errors
collection and analysis of, 342
cost of, 317–318
defined, 326, 478
density of, 328, 329
handling systems, 260
Estimation. *see also* Project planning
for agile development, 519
complexity of projects and, 505–506
data analytics and, 509–511
decomposition techniques, 511–519
empirical models for, 510–511
FP-based, 514–515
LOC-based, 512–513
objectives of, 92
problem-based, 512
process-based, 515–516
quality and, 321
reconciling estimates, 518–519
of resources, 60–61, 505
of risk, 538–541
size of projects and, 506
software sizing, 511
use case, 517–518
Ethics, 607–609
Ethnographic observation, 272
Evaluation. *see also* Assessment
of design, 160, 161, 253–256
of interface design, 253–256
post-mortem, 336
in prototype development, 64–65, 68–69, 253–254
in software process improvement, 575
Evolutionary process flow, 22, 23
Evolutionary process models, 29–31, 56–57, 67
Exceptions, 134–135
Exhaustive testing, 380
Exposure to risk, 540–541
Extensible Markup Language (XML), 273
External coupling, 218
External failure costs, 317
Extra-functional properties, 164
Extreme Programming (XP), 46–48, 67

Failure costs, 317
Failure curves, 5–6
Failures in time (FIT), 351
Families of related systems, 164
Fan-in (FIN), 475
Fault-based testing, 405–406
Faults, 326. *see also* Defects
Feature engineering, 634
Feature vectors, 633
Filters, 188
Financial leverage, 571
Fire-fighting mode, 533
First law of system engineering, 438–439
Flexibility, 258
Flow graphs, 384
Flow-oriented models, 127
Formal change control, 450
Formal technical reviews (FTRs), 327, 332–335
Formal use cases, 127, 135
Forward engineering, 565
Forward impact management, 451
FP-based estimation, 514–515
Frameworks
models, 164
pattern-based design, 293
risk management, 364–365
Scrum, 42–45, 56, 66–67
software process, 10–11, 21, 23
software process improvement, 569–570
Functional architecture, 226
Functional cohesion, 216
Functional independence, 167, 173
Functional modeling, 146–149, 164
Functional testing, 97, 388–390, 397
Function-oriented metrics, 481
Function point (FP), 481, 514–515
Fuzzy logic, 539

Gamification, 544–545
Gantt charts, 527
Gap analysis, 573
Generalizations, 613
Generative patterns, 291
Generic process models, 21–23
Generic risks, 535
Genetic algorithms, 352, 606
Gesture testing, 415–416
Glass-box testing, 383–387, 397
Globalization, 587–588
Global software development (GSD) teams, 80–81
Go, no-go decisions, 65–67
Goal in context, 135
Goals
  business, 485
  defined, 104
  of interface design, 243
  of requirements gathering, 24
  security, 362
  of software quality assurance, 345–346
"Good enough" software, 316
Grammatical parse, 137, 138, 141
Granularity, 92
Graphic design, 237, 277
Graphic design metrics, 472
Graphic icons, 276
Graphic images, 276

Handshaking, 122
Hardware, failure curve for, 5–6
Hazards, 352–353, 545
Help facilities, 260, 434
Heuristic evaluations, 253
Heuristics, 7, 157, 162, 554
High-granularity plans, 92
High-order testing, 377
Historical data, 481
Hooks, 293
Human elements of SCM, 440
Human-focused path, 599
Human interface objects, 258
Human resources, 508
Hype cycle for emerging technologies, 586–587

Identification
  of analysis classes, 137–140
  of events with use cases, 149–150
  of risk, 535–537
  of stakeholders, 107
Idioms, 292
Impact analysis, 443
Impact management, 443, 451
Impact of risk, 540–541
Implementation model, 242
Inception
  in communication activity, 23
  in mobile development life cycle, 268
  as phase of Unified Process, 32
  in requirements engineering, 104
Incremental development strategies, 40
Incremental process models, 55–56
Increments. *see* Software increments
Independent classes, 404
Independent paths, 384–385
Independent test groups (ITGs), 375
Indeterminate software, 7
Indicators, defined, 462
Informal change control, 450
Information architecture, 235, 279
Information delivery, 4
Information domain, 129
Information hiding, 166
Information objectives, 497
Information technology, 605
Inheritance, 216, 475
Inspections, 326, 332
Installation, 574–575
Instance form of deployment diagrams, 178
Integrability, 187
Integration
  bottom-up, 399–400
  breadth-first, 398
  continuous, 400–402, 446–447
  depth-first, 398
  in DevOps approach, 50
  top-down, 398–399
Integration-level testing
  of architecture, 375, 376
  artificial intelligence and, 403
  black-box, 397
  bottom-up, 399–400
  challenges of, 395
  in construction activity, 95
  fundamentals of, 396–397
  objectives of, 398
  object-oriented, 404–407
  patterns, 409
  regression, 403–404
  top-down, 398–399
  validation, 407–408
  white-box, 397
  work products, 402
Integrity, 483
Intelligent software engineering, 606
Interaction design, 236
Interaction frames, 620
Interaction mechanisms, 234
Interdependency, 521
Interface analysis, 243
Interface construction, 243
Interface design. *see also* User experience design
  elements of, 175–176
  evaluation of, 253–256
  functions of, 158
  goals of, 243
  golden rules of, 234, 238–241
  guidelines for, 216, 259
  metrics for, 471–473
  for MobileApps, 261, 270–271
  models for, 241–242, 270–271
  patterns in, 252–253, 291, 304–305
  for placing user in control, 238–239
  principles of, 173
  process for, 242–243
  for reducing user's memory load, 239–240
  steps for, 251–252
  for WebApps, 275–276
Interface segregation principle (ISP), 214
Interface testing, 388, 421
Interface validation, 243
Internal failure costs, 317
Internationalization, 260–261, 423
Inventory analysis, 563–564
ISO 9001:2015, 353–354
Issues lists, 112
Issues tracking, 69, 446
Iterative planning, 91, 92
Iterative process flow, 22, 23

Jelled teams, 76–77, 494
Justification activity, 573–574

Kanban method, 48–50, 56, 67
Key performance indicators (KPIs), 462
Knowledge discovery, 605–606
Known risks, 535

Lack of cohesion in methods (LCOM), 470, 475
Language metaphor, 183
Languages
  architectural description, 164
  for data science, 629–631
  domain-specific modeling, 597
  pattern, 297
  UML (*see* Unified modeling language)
Latency reduction, 258
Law of Demeter, 170
Layer cohesion, 216, 217
Layered architectures, 189–192
Layered behavioral model, 75–76
Layout, 277, 431
Leadership, 493–494
Learnability, 258
Legacy software, 8
Level design, 456
Liability, 320
Libraries, 631
Life-cycle security models, 357–359
Linear process flow, 22, 23
Linear sequential model. *see* Waterfall model
Liskov substitution principle (LSP), 214
Listening, 46, 89
Literature metaphor, 184
Load testing, 425–426
Localization, 260, 423
LOC-based estimation, 512–513
LOC-based metrics, 481

Loop testing, 387
Low-granularity plans, 92

Machine learning, 294–295, 322, 558, 606, 631–637
Main program/subprogram architectures, 189
Maintainability, 483, 553
Maintenance
  adaptive, 70, 553
  challenges of, 552
  corrective, 69–70, 553
  defined, 69
  importance of, 3, 71
  metrics for, 476
  perfective, 70, 553
  practice principles and, 88
  preventive, 70, 553
  tasks related to, 554–555
Make facilities, 446
Management. *see also* Project management; Risk management; Software configuration management
  change, 342, 445, 447–452
  of complexity, 588–589
  configuration, 11, 437, 445
  content, 455, 456
  impact, 443, 451
  quality, 9, 340
  release, 550–551
  in requirements engineering, 106
  reusability, 11
  security, 342
  vendor, 342
  version, 446
Management subsystem, 456–457
Man-in-the-middle attacks, 267
Manufacturer view of quality, 311
Maturity levels, 577, 578
Maturity models, 570–571
Maturity phase of technology innovation, 585
Mean time between failure (MTBF), 350–351
Mean time to change (MTTC), 483
Measurement. *see also* Metrics
  of availability, 351
  defined, 461–462
  direct, 479–480
  as management tool, 461
  of productivity, 480, 482
  of reliability, 350–351
  of security, 368–369
  as umbrella activity, 11
  usefulness of, 460, 461
Measures, defined, 461
Meetings
  casual, 331
  Kanban, 50
  question-and-answer format for, 109
  requirements gathering, 112
  review, 332–333
  Scrum, 44–45, 56, 66–67
  sprint, 44, 45
Melding product and process, 498, 499
Mental model, 241–242
Menu labeling, 260
Meta-model, 443–444
Metaphors, 258, 276
Meta-questions, 109
Methods, 9
Metrics
  aesthetic, 472
  attributes of, 462
  black-box, 466
  class-based, 469–470
  content, 473
  for conventional software, 465–468
  defined, 462
  design, 466–473, 475
  function-oriented, 481
  for interface design, 471–473
  key performance indicators, 462
  LOC-based, 481
  for maintenance, 476
  for MobileApps, 465–466
  morphology, 467, 468
  navigation, 473
  for object-oriented software, 468–470
  private, 480, 482
  process, 476–478
  product, 461, 463–464, 469, 480, 482
  productivity, 481
  program establishment, 485–487
  project, 476–478
  public, 480
  quality, 482–484
  for requirements model, 464–466
  for reviews, 327–330
  size-oriented, 480–481
  for small organizations, 486
  for software quality assurance, 346
  for source code, 473–474
  for testing, 474–475
  usefulness of, 461
  for WebApps, 472, 473
Middleware, 266
Migration, 574–575
Milestones, 521
Mini-specifications, 112
Misuse cases, 363–364
MobileApps (mobile applications)
  agile change management for, 453–458
  alerts and extraordinary conditions, 417
  architecture for, 273–274
  best practices for design, 285–287
  challenges related to, 265–268
  component-level design for, 226–227
  context-aware, 274–275
  development considerations, 265
  gestures and, 415–416
  guidelines for design, 271–272
  interface design for, 261, 270–271
  internationalization of, 423
  metrics for, 465–466
  mistakes in design of, 272
  patterns for, 292, 305–306
  performance of, 424–426
  prevalence of, 264
  quality of, 282–285
  real-time applications, 426–428
  security of, 424
  stages of development, 268–272
  technical considerations, 266–268
  testing, 413–417, 424–428
  tools for, 265, 267
  types of, 7
  usability of, 257, 258, 415–417
  virtual keyboard input and, 416
  voice input and recognition, 416–417
Model-based testing (MBT), 429–430
Model-driven software development, 596–597
Models and modeling. *see also* Analysis models and modeling; Requirements models and modeling
  behavioral, 127, 149–154
  class-based, 127, 137–146
  constructive cost, 511
  CRC, 47, 144–146
  data, 127
  design, 93, 158, 171–178
  dynamic, 164
  empirical estimation, 510–511
  flow-oriented, 127
  framework, 164
  functional, 146–149, 164
  for interface design, 241–242, 270–271
  maturity, 570–571
  principles of, 92–94
  process (*see* Process models)
  in process framework, 10
  product quality, 313–314
  quality in use, 313
  scenario-based, 127, 128, 130–137
  security, 357–359
  structural, 164
  system approach, 446
  threats, 357, 365–366
  user, 241, 245–246
Model-View-Controller (MVC) architecture, 189, 191–192, 279–280
Modularity, 87, 90, 165–166
Modules, 165, 209. *see also* Components
Morphology metrics, 467, 468
Multiclass classification problems, 634
Multilabel classification problems, 634
Multiplicity, 614

Navigation
  design of, 280–282
  menus for, 276
  syntax of, 280
  visible, 258

Navigational nodes (NN), 280
Navigation metrics, 473
Navigation semantic links (NSLs), 281
Navigation semantic units (NSUs), 280–282
Navigation testing, 421–423
Nearest neighbor technique, 635–637
Negative test cases, 383
Negligence, 320
Negotiation, 23, 90, 105, 122–123
Nested loops, 387
Neural networks, 636–637
90–90 rule, 500
Nomadic networks, 266
Nonfunctional requirements (NFRs), 109
Nongenerative patterns, 291
Notes, 615
Number of children (NOC), 469, 475
Number of root classes (NOR), 475

Object-oriented architectures, 189
Object-oriented design metrics, 468–470
Object-oriented testing, 390–393, 404–407
OO design metrics, 469–470, 475
Open-closed principle (OCP), 212–213
Open source movement, 592
Open-world software, 589–590
Operations, 141
Ordered Categorical variables, 633

Pair programming, 48, 332
Paper prototypes, 62, 272
Parallel process flow, 22, 23
Pareto principle, 97–98
Partitioning, 130, 389, 497–498
Passive state, 150
Pattern-based architecture review (PBAR), 203–204
Pattern-based design
  common mistakes in, 298–299
  in context, 290, 295–296
  defined, 164, 290
  describing, 293–294
  framework for, 293
  machine learning in, 294–295
  objectives of, 165
  organizing tables in, 298, 299
  repository for, 295
  tasks in, 297–298
  template for, 293–294
  thinking in, 296–297
  types of, 291–293
  use of, 87
  for user interfaces, 252–253
Pattern languages, 297
Pattern-organizing tables, 298, 299
Patterns. *see also* Pattern-based design
  analysis of, 122
  anti-patterns, 302–304
  architectural, 187, 192–193, 203–204, 291, 299–300
  attack, 363–364
  behavioral, 292–293
  component-level, 300–301
  creational, 292
  data, 291
  of defects, 98
  generative, 291
  interface design, 252–253, 291, 304–305
  mobile, 292
  for MobileApps, 292, 305–306
  nongenerative, 291
  pipe-and-filter, 188
  process, 24
  risk, 536
  structural, 292
  testing, 409
  for WebApps, 291–292
Peer reviews, 326
People Capability Maturity Model, 577
People Capability Maturity Model (People-CMM), 491
Percent public and protected (PAP), 475
Perfective maintenance, 70, 553
Performance testing, 414, 424–426
Pipe-and-filter pattern, 188
Planning. *see also* Project planning
  contingency, 86, 92, 544
  iterative, 91, 92
  principles of, 91–92
  in process framework, 10
  for software quality assurance, 343–344, 354
  for testing, 378–380
  in XP framework, 46–47
Platform model, 270
Playability testing, 433–434
Plug points, 293
Postconditions, 214
Postmortem evaluations (PMEs), 336
Practice (software engineering), 12–15, 85–88
Preconditions, 135, 214
Predictable risks, 535
Predictive technologies, 416
Preparation effort, 328
Prescriptive process models, 25–33
Presentation model, 270
Prevention costs, 317
Preventive maintenance, 70, 553
Primary actors, 115
Primary scenarios, 134
Primitiveness, 468
Priority points, 108
Private metrics, 480, 482
Proactive risk strategies, 534
Proactive software support, 557–560
Probabilistic reasoning, 558
Problem-based estimation, 512
Problem decomposition, 497–498
Problem elaboration, 497–498
Problem solving, 12–13
Procedural abstraction, 163
Process. *see* Software process
Process adaptation, 11–12
Process-based estimation, 515–516
Process dimension, 171–172
Process elements of SCM, 440
Process flow, 22–23, 25, 31
Process framework, 10–11, 21, 23
Process improvement. *see* Software process improvement
Processing narratives, 138, 141
Process maturity, 570
Process metrics, 476–478
Process migration, 574
Process models
  agile, 42–51, 56, 57
  comparison of, 33–34
  criticisms of, 38
  in design, 164
  evolutionary, 29–31, 56–57, 67
  generic, 21–23
  guidelines for use of, 55
  incremental, 55–56
  prescriptive, 25–33
  prototyping, 26–29
  reengineering, 563
  selection of, 28, 31
  spiral, 29–30, 56–57
  unified, 31–33
  waterfall, 25–26, 55
Process patterns, 24
Producers, 161, 333
Product, relationship to process, 34–35
Product backlog, 43–45
Production phase of Unified Process, 33
Productivity measures, 480, 482
Productivity metrics, 481
Product-line software, 7, 164, 590
Product metrics, 461, 463–464, 469, 480, 482
Product quality model, 313–314
Product requests, 110–111
Product scope, 491–492, 497
Product-specific risks, 535–536
Product view of quality, 311
Progressive sign-off, 186
Project complexity, 505–506
Project databases, 441, 442, 445–446
Project feasibility, 507
Projection, risk, 538–541
Project level change control, 450
Project management
  characteristics for success, 500–501
  critical practices for, 502
  people in, 491, 493–496
  process and, 492, 498–500
  product issues and, 491–492, 497–498
  for quality, 322
  spectrum of, 491–492
  W^{5}HH principle for, 501

Project metrics, 476–478
Project planning. *see also* Estimation
  data analytics in, 509–511
  objectives of, 506
  overview, 504
  in process framework, 10
  resources in, 507–509
  scheduling in, 520–523, 526–529
  scope and feasibility in, 507
  task networks in, 525–526
  task sets in, 507, 523–525
Project risks, 534, 536–537
Project size, 506
Project tables, 527–528
Project velocity, 47
Prototype development
  in agile process models, 56
  architectural design for, 59–60
  evaluation of, 64–65, 68–69, 253–254
  evolutionary model of, 67–68
  first prototype construction, 61–63
  go, no-go decisions in, 65–67
  incremental model for, 55–56
  for mobile devices, 272
  release stage of, 68–69
  scope determination in, 67
  spiral model for, 56–57
  stakeholders in, 58–59, 61–62
  in user experience design, 250
Prototype user interface, 62
Prototyping paradigm, 26–29
Public access to data members (PAD), 475
Public metrics, 480
Publishing subsystem, 457
Putnam-Norden-Rayleigh (PNR) curve, 522–523

Quality
  checklist for, 285
  of codes, 345
  of conformance, 311
  cost of, 317–318
  definitions of, 311, 312
  of design, 159–161, 282–285, 311, 345
  dilemmas related to, 315–321
  Garvin's dimensions of, 311
  of "good enough" software, 316
  guidelines for achievement of, 321–323
  importance of, 3
  of legacy software, 8
  management actions and impact on, 321
  McCall's factors for, 313
  metrics for, 482–484
  of MobileApps, 282–285
  planning for, 92
  qualitative assessment of, 314–315
  quantitative assessment of, 315
  of requirements model, 345
  reviews for, 341–342
  risk and, 319–320
  security and, 320
  standards for, 313–314, 341, 353–354
  of WebApps, 282–285
Quality assurance, 11, 323, 353. *see also* Software quality assurance
Quality control, 322–323, 346
Quality dilemma, 315–321
Quality in use model, 313
Quality management, 9, 340, 349. *see also* Software quality assurance
Quality requirements tree, 283
Question-and-answer sessions, 109

Reactive risk strategies, 533
Readability, 258, 431
Real-time testing, 426–428
Reconciliation of estimates, 518–519
Recorders, 161, 333
Recordkeeping, 333–334, 378–380
Recovery testing, 417
Reengineering, 554, 562–565
Refactoring
  architecture, 561–562
  code, 561, 564
  components, 230–231
  data, 561, 564–565
  defined, 48, 554, 560
  design, 48, 168, 225
  interfaces, 221
  tools for, 230, 561
Refinement
  of risk, 542–543
  stepwise, 167–168, 248
  of task sets, 524–525
Regression problems, 634
Regression testing, 68, 403–404
Release management, 550–551
Release reuse equivalency principle (REP), 214–215
Reliability, 350–353
Remote procedure call architectures, 189
Replicator phase of technology innovation, 585
Reporting, 333–334, 402, 448, 452, 458
Repositories, 295, 441, 443–446
Representation State Transfer (REST), 273
Requirements
  analysis of, 127–130
  architectural design and, 59–60
  collaboration on, 108
  emergent, 590–591
  monitoring, 123
  multiple viewpoints on, 107–108
  negotiating, 122–123
  nonfunctional, 109
  test cases for, 382–383
  tracing, 445
  understanding, 102–103
  validation of, 123–124
Requirements engineering
  agile, 58, 104
  defined, 57, 103
  first questions in, 108–109
  goal-oriented, 104
  for security, 360–363
  specifications in, 105, 407
  tasks in, 104–106
  for technology, 596
  traceability in, 109–110
Requirements gathering, 24, 110–114. *see also* Elicitation
Requirements models and modeling. *see also* Analysis models and modeling
  behavioral, 127, 149–154
  class-based, 127, 137–146
  domains of, 93
  functional, 146–149
  metrics for, 464–466
  objectives and philosophy of, 128
  principles of, 129–130
  quality of, 345
  rules of thumb for, 128–129
  scenario-based, 127, 128, 130–137
  terminology for, 126
  translation into design model, 158
  types of, 127
Requirements quality, 345
Requirements tasks
  elaboration, 104–105
  elicitation, 104, 109
  inception, 104
  management, 106
  negotiation, 105, 122–123
  specification, 105
  validation, 105–106, 123–124
Resources
  environmental, 509
  estimation of, 60–61, 505
  human, 508
  in project planning, 507–509
  reusable, 509
Response for a class (RFC), 470
Response time, 259
Responsibilities, in CRC modeling, 144
Responsibility-driven architecture (RDA), 186
Restructuring, 564
Return on investment (ROI), 580
Reusability management, 11
Reusable software resources, 509
Reuse, 214–215, 228, 229
Reverse engineering, 553–557, 564
Review effort, 328
Review issues list, 333–334
Review leaders, 161, 333
Review meetings, 332–333
Reviews. *see also* Technical reviews
  agile, 336
  architectural, 202–204
  casual, 326, 331

checklists for, 331
desk checks, 331–332
formal, 327, 332–335
guidelines for, 334–335
informal, 331–332
inspections, 326, 332
metrics for, 327–330
peer, 326
purpose of, 325
for quality, 341–342
reporting and recordkeeping for, 333–334
walkthroughs, 326, 332
Rework effort, 328
Risk. *see also* Risk assessment; Risk management
business, 534
categorization of, 534–535
characteristics of, 534
in component-based software engineering, 229
components of, 537
drivers of, 537
estimation of, 538–541
exposure to, 540–541
generic, 535
identification of, 535–537
impact of, 540–541
known, 535
low-quality software and, 319–320
patterns of, 536
predictable, 535
product-specific, 535–536
project, 534, 536–537
refinement of, 542–543
technical, 534
unpredictable, 535
Risk assessment
collective, 539
contingency plans and, 86
in prototype development, 66
security, 362, 364–365
in spiral model, 30, 56, 57
Risk information sheet (RIS), 546, 547
Risk item checklists, 536
Risk management
framework for, 364–365
gamification and, 544–545
intentional, 533
principles of, 535
proactive strategies, 534
reactive strategies, 533
RMMM plan for, 546
for software process improvement, 575–576
in software quality assurance, 342
as umbrella activity, 11
Risk Mitigation, Monitoring, and Management (RMMM), 539, 543–547
Risk-oriented decisions, 321
Risk projection, 538–541
Risk tables, 538–539
Run-time validation, 123
Run-time verification, 123

SafeHome
activity diagram, 147, 152
applying CK metrics, 471
applying patterns, 301–302
archetypes, 197–198
architectural assessment, 202
architectural context diagram, 197
behavioral modeling, 121
choosing an architectural style, 192
class models, 142
class testing, 391
cohesion in action, 217–218
communication mistakes, 90
conclusion, 604
considering agile software development, 43
considering first prototype, 63
control panel, 115–117
coupling in action, 219
CRC models, 145–146
debating product metrics, 464
design classes, 171
design concepts, 168–169
designing unique tests, 382
design patterns, 292–293
design vs. coding, 159
establishing a metrics approach, 479
estimating, 513–514
evaluating architectural decisions, 194–195
evaluating first prototype, 65
formulating mobile device requirements, 269–270
gamification and risk management, 545–546
grammatical parse, 138
graphic design, 237
instantiations of, 200
interface design review, 254–255
metrics-based quality approach, 484
mini-specifications, 112
negotiation, 122–123
open-closed principle in action, 213
overall architectural structure, 199–200
preparing for testing, 377
preparing for validation, 408
product request, 111
quality assurance, 345
quality issues, 319, 335
regression testing, 403–404
requirements gathering meeting, 112
risk analysis, 541–542
SCM issues, 451
selecting process model, 28, 31
sequence diagram, 148
starting projects, 16
state diagram, 151
swimlane diagram, 153
team structure, 79, 496
tracking the schedule, 529
use case diagram, 118, 119, 137
use cases for user interface design, 247
use case template, 135–136
user scenario, 113–114, 132–133
using cyclomatic complexity, 386
violating interface golden rules, 240
WebApp testing, 419–420
Safety considerations, 342, 352–353, 545
Scaffolding, 64, 379
Scenario-based elements, 119
Scenario-based modeling
actors and user profiles in, 131
characteristics of, 127
overview, 130
popularity of, 128
use cases in, 131–137
Scenario-based testing, 407
Scheduling, 321, 520–523, 526–529
Scientific software, 7
Scope
of products, 491–492, 497
of projects, 91, 114
in prototype development, 67
of software, 497, 507
of use cases, 135
of user experience design, 234
Scrum framework, 42–45, 56, 66–67
Search-based software engineering (SBSE), 161, 558, 597, 638
Secondary actors, 115
Secondary scenarios, 134
Secure coding, 367
Security
attack patterns and, 363–364
attack surface and, 366–367
coding and, 367–368
importance of, 356, 357
in integrity measurement, 483
life-cycle models for, 357–359
management of, 342
measurement of, 368–369
misuse and abuse cases, 363–364
of MobileApps, 424
process improvement and maturity models, 370
quality and, 320
requirements engineering for, 360–363
risk assessment, 362, 364–365
testing, 414, 423–424
threat modeling methods, 365–366
tools for, 358, 365
touchpoints for, 359
of WebApps, 424
Security Development Lifecycle (SDL), 357–359, 365
Selection and justification activity, 573–574

Selective testing, 381
Self-organizing teams, 41, 45, 78, 86, 495
Semantic testing, 420–421
Sensitivity points, 201
Separation of concerns, 87, 130, 165, 194
Sequence diagrams, 148–149, 618–621
Server classes, 404
Service computing, 273
Service-oriented architecture decision (SOAD), 195–196
Shrink-wrapped packages, 342
Simple Object Access Protocol (SOAP), 273
Six Sigma, 9, 349
Size-oriented metrics, 480–481
Slots, 293
Smoke testing, 400–401
Social media, 79–80, 559, 604
Social networking tools, 79
Software
  application domains, 7
  building blocks for, 591–592
  contracted, 342
  defined, 5
  failure curves for, 6
  "good enough," 316
  importance of, 2–3, 603
  key questions regarding, 4
  legacy, 8
  model-driven development of, 596–597
  nature of, 4–8, 74
  open-world, 589–590
  product-line software, 7, 164, 590
  realities related to, 3
  reliability of, 350–353
  safety of, 342, 352–353, 545
Software analytics, 462–463, 558–559
Software architecture. *see* Architecture
Software Assurance Maturity Model (SAMM), 370
Software capital, 20
Software components. *see* Components
Software configuration, defined, 438
Software configuration audits, 452
Software configuration items (SCIs), 438, 441–442
Software configuration management (SCM)
  baselines in, 441, 442, 450
  continuous integration and, 446–447
  of dependencies and changes, 442–443
  elements of, 440
  repository for, 441, 443–446
  scenario for, 439–440
  tasks involved in, 447–448
  version control systems, 445–446
  work flow for, 438
Software defects. *see* Defects
Software design. *see* Design
Software engineering. *see also* Trends in software engineering
  component-based, 228–230, 509
  computer-aided, 9
  defined, 8
  as discipline, 585–586
  ethics and, 607–609
  goals of, 438
  grand challenge for, 594–595
  intelligent, 606
  layers of, 9
  long view of, 606–607
  practice, 12–15, 85–88
  principles of, 14–15, 85–100
  psychology of, 75–76
  search-based, 161, 558, 597, 638
  social media and, 79–80
  start of projects, 15–16
  video games, 1–2
Software engineering environment (SEE), 509, 599–600
Software engineers, 75, 84
Software equation, 523
Software evolution, 550, 562–565
Software increments, 11, 32, 40, 58
Software maintenance. *see* Maintenance
Software maturity index (SMI), 476
Software measurement. *see* Measurement
Software platform solutions, 592
Software process
  adaptation of, 11–12
  agility and (*see* Agility)
  assessment and improvement of, 24–25
  components of, 9–10
  decomposition of, 498–500
  defined, 9, 20, 21
  flow of, 22–23, 25, 31
  framework for, 10–11, 21, 23
  models for (*see* Process models)
  principles guiding, 85–86
  project management and, 492, 498–500
  recommended steps, 71
  relationship to product, 34–35
  schematic representation of, 21
  trends related to, 593–594
Software process improvement (SPI)
  assessment and gap analysis in, 572–573
  continuous, 341
  education and training in, 573
  evaluation in, 575
  frameworks for, 569–570, 576–579
  installation/migration in, 574–575
  maturity models for, 570–571
  return on investment, 580
  risk management for, 575–576
  security and, 370
  selection and justification in, 573–574
  for small organizations, 571
  steps for, 571–576
  trends in, 580–581
Software process redesign (SPR), 574
Software product lines, 7, 164
Software project plans. *see* Project planning
Software quality assurance (SQA)
  attributes in, 346
  background issues related to, 341
  elements of, 340–343
  formal approaches to, 347
  goals of, 345–346
  metrics for, 346
  plans for, 343–344, 354
  processes and product characteristics, 343
  reliability, 350–353
  safety, 342, 352–353
  Six Sigma for, 9, 349
  statistical, 347–349, 378
  tasks of, 343–344
Software reengineering, 554, 562–565
Software reviews. *see* Reviews
Software scope, 497, 507
Software security. *see* Security
Software sizing, 511
Software support. *see also* Maintenance
  analytics in, 558–559
  cost of, 559–560
  elements of, 549, 550
  iterative model of, 551
  proactive, 557–560
  social media and, 559
Software teams. *see* Teams
Source code analysis, 561
Source code metrics, 473–474
Source objects, 251
Spacing, 194
Specifications
  in communication activity, 23
  mini-specifications, 112
  in requirements engineering, 105, 407
  template for, 105
  test, 402
SPICE model, 579
Spiral model, 29–30, 56–57
Sprints, 42–45, 61, 186
SQA groups, 341, 343–344
SQUARE process, 360–363
Stakeholders
  collaboration among, 108
  communication with, 23, 24
  defined, 10, 107
  feedback from, 27, 45, 55
  identifying, 107
  negotiation with, 122
  in planning, 91
  in project management, 493
  in prototype development, 58–59, 61–62

Standards for quality, 313–314, 341, 353–354
State, defined, 120
Statecharts, 224
State diagrams, 120–121, 150–151, 392, 625–628
Static analysis tools, 368
Static architecture-conformance analysis (SACA), 204
Static testing, 429
Statistical quality assurance, 347–349, 378
Statistical software process improvement (SSPI), 478
Status reporting, 452
Stepwise refinement, 167–168, 248
Stereotypes, 613
Storyboarding, 186
Stress testing, 425–426
STRIDE model, 365, 366
Structural complexity, 467
Structural models, 164
Structural patterns, 292
Structural properties, 164
Structural testing, 97, 383–387, 397
Structural uncertainty, 506
Structure charts, 210
Structured analysis, 128
Stubs, 379
Sufficiency, 468
Supervised learning, 634
Supportability, 552. *see also* Software support
Swimlane diagrams, 151, 153–154
Symmetry, 194
Synchronization control, 450
System complexity, 467
System coupling, 129
System modeling, 446
System of forces, 290
System software, 7
System testing, 376, 377, 396

Tailored shell, 342
Talent mix, 591
Target environment, 509
Target objects, 251
Task, defined, 10
Task analysis, 243, 247–248
Task model, 270
Task networks, 525–526
Task sets
  communication, 23, 24, 500
  concept development projects, 524
  defined, 21
  design, 162
  project planning, 507, 523–525
  refinement of, 524–525
Teams
  agile, 40, 41, 78, 495
  collaboration and, 587
  coordination and communication, 496, 604
  development, 43–45, 67
  global, 80–81
  jelled, 76–77, 494
  Kanban, 48–50
  leaders of, 493–494
  in project management, 494–496
  Scrum, 43, 44
  self-organizing, 41, 45, 78, 86, 495
  structure of, 78–79, 496
  talent mix and, 591
  toxicity of, 77, 495
Technical debt, 157, 230, 327, 533
Technical reviews (TRs)
  cost effectiveness of, 329
  criteria for, 330–331
  of design quality, 160, 161
  formal, 327, 332–335
  informal, 331–332
  purpose of, 326
  for quality, 341–342
  reference model for, 330
  for requirements validation, 105–106
  as umbrella activity, 11
Technical risks, 534
Technology
  collaborative development of, 595–596
  innovations in, 584–585
  predictive, 416
  process trends, 593–594
  requirements engineering for, 596
Templates
  for pattern-based design, 293–294
  for publication, 457
  for software testing, 373
  for specifications, 105
  for system modeling, 446
  use case, 135–136
Test case design, 381–383, 405–407
Test-driven development (TDD), 598–599
Test frames, 62
Testing. *see also* Component-level testing; Integration-level testing
  acceptance, 48, 68–69, 95, 430
  accessibility, 432–433
  in agile processes, 95
  alerts and extraordinary conditions, 417
  alpha, 430
  architecture, 375, 376
  artificial intelligence systems, 428–429
  basis path, 384–386
  behavioral, 388–390, 392–393
  beta, 430
  black-box, 388–390, 397
  boundary, 381–382
  certification, 414
  class, 390–391
  cloud-based, 428
  cluster, 404
  condition, 386
  connectivity, 414
  content, 420–421
  control structure, 386–387
  cost-effective, 379–380
  criteria for "done," 377–378
  data flow, 381, 386
  documentation, 434
  dynamic, 429
  exhaustive, 380
  fault-based, 405–406
  functional, 97, 388–390, 397
  gesture, 415–416
  glass-box, 383–387, 397
  help facilities, 434
  high-order, 377
  interface, 388, 421
  loop, 387
  metrics for, 474–475
  MobileApps, 413–417, 424–428
  model-based, 429–430
  navigation, 421–423
  object-oriented, 390–393, 404–407
  organizing for, 374–375
  patterns, 409
  performance, 414, 424–426
  playability, 433–434
  preparing for, 377
  principles of, 96–98
  prototypes, 63–65, 68–69
  for quality, 342
  real-time, 426–428
  recovery, 417
  regression, 68, 403–404
  scenario-based, 407
  security, 414, 423–424
  selective, 381
  semantic, 420–421
  smoke, 400–401
  spiral illustration of, 375–376
  static, 429
  steps for, 376–377
  structural, 97, 383–387, 397
  system, 376, 377, 396
  thread-based, 404
  tools for, 50, 413, 415, 422, 430
  unit, 48, 95, 376, 378–381
  usability, 415–417, 430–432
  use-based, 404
  user, 255–256
  user experience, 414–417
  validation, 95, 373–377, 407–408
  verification in, 373–374
  virtual environments, 430–434
  virtual keyboard input, 416
  voice input and recognition, 416–417
  WebApps, 418–426
  white-box, 383–387, 397
Testing in the wild, 414, 426–427
Test reports, 402
Test sets, 633
Test specifications, 402

Theory phase of technology innovation, 585
Thread-based testing, 404
Threats
  analysis of, 363
  in integrity measurement, 483
  mitigation of, 357, 358
  modeling, 357, 365–366
TickIT auditing method, 579
Time allocation, 521
Time-boxes, 42, 44, 45, 528–529
Time-line charts, 527
Time window, 508
Tools
  capture/playback, 403
  data science, 631
  function of, 9
  interface design, 261
  for MobileApps, 265, 267
  reengineering, 562
  refactoring, 230, 561
  restructuring, 564
  reverse engineering, 555, 564
  scheduling, 521
  for security, 358, 365
  social networking, 79
  static analysis, 368
  testing, 50, 413, 415, 422, 430
  trends in, 599–600
  version control, 445
Top-down integration, 398–399
Total quality management (TQM), 9, 349
Touchpoints for security, 359
Toxic team environment, 77, 495
Traceability, 109–110, 383, 442
Traceability matrix, 109–110, 442
Tracking
  dependencies, 445
  issues, 69, 446
  progress, 92
  schedules, 528–529
  state of user, 258
  as umbrella activity, 11
Training and education, 342, 573
Training set, 632, 633
Transcendental view of quality, 311
Transition phase of Unified Process, 33
Trends in software engineering
  changing perceptions of value, 592
  connectivity and collaboration, 587
  emergent requirements, 590–591
  globalization, 587–588
  managing complexity, 588–589
  observation of, 586–587
  open source movement, 592
  open-world software, 589–590
  soft trends, 587–592
  software building blocks, 591–592
  talent mix, 591
  technological, 584–585, 593–599
  tools-related, 599–600
Triggers, 135, 150
Trustworthiness, 431

Umbrella activities, 10, 11, 21–22
UML notation
  activity diagrams, 146–148, 151–154, 223, 622–625
  archetypes, 198
  class diagrams, 120, 612–615
  communication diagrams, 189, 190, 621–622
  component diagrams, 177
  deployment diagrams, 177, 178, 615–616
  sequence diagrams, 148–149, 618–621
  state diagrams, 121, 150–151, 625–628
  swimlane diagrams, 151, 153–154
  use case diagrams, 119, 616–618
Unadjusted actor weight (UAW), 517, 518
Unadjusted use case weight (UUCW), 517, 518
Uncertainty, 534
Unified modeling language (UML)
  actors in, 131
  associations in, 144, 154
  class models, 141–144
  dependencies in, 154
  development of, 32, 611
  for multiple viewpoints, 88
  notation (*see* UML notation)
  profiles in, 131
Unified Process (UP), 31–33
Unified theory for software evolution, 550
Unit testing, 48, 95, 376, 378–381
Unordered Categorical variables, 633
Unpredictable risks, 535
Unsupervised learning, 634
Usability
  definitions of, 233, 256
  engineering, 236–237
  guidelines for, 257–259
  software quality metric, 483
  testing, 415–417, 430–432
Usage scenarios, 113–114, 271
Use-based testing, 404
Use case diagrams, 118, 119, 136, 137, 616–618
Use case points (UCPs), 517
Use cases
  creating, 131–135
  defined, 31, 616
  development of, 114–118
  documenting, 135–137
  estimation and, 517–518
  exceptions, 134–135
  formal, 127, 135
  identifying events with, 149–150
  questions to be answered by, 115
  refinement of, 135
  in requirements gathering, 113
  scope of, 135
  for testing requirements, 382–383
  in Unified Process, 31–33
  for user interface design, 247
User experience testing, 414–417
User experience (UX) design. *see also* Interface design
  defined, 234
  elements of, 175, 234–237
  information architecture in, 235
  interaction design in, 236
  scope of, 234
  steps for, 249–250
  usability engineering in, 236–237
  visual design in, 237
User interaction design, 236
User interface. *see also* Interface design
  accessibility of, 233, 237, 259–261
  analysis of, 243–249
  consistency of, 240–241, 257
  construction of, 243
  interaction mechanisms in, 234
  prototype of, 62
  usability of, 233, 236–237, 256–259
  validation of, 243
User modeling, 241, 245–246
User-observable functionality, 146
User personas, 245–246
User profiles, 131
User research, 244–245
User stories, 46, 48, 56, 58, 61–63, 67–68
User testing, 255–256
User view of quality, 311

Validation
  in communication activity, 23
  of effort, 521
  of interface, 243
  in requirements engineering, 105–106, 123–124
  run-time, 123
  in user experience design, 250
Validation testing, 95, 373–377, 407–408
Value, changing perceptions of, 592
Value-based view of quality, 311
Variability-intensive systems, 162, 590
Vendor management, 342
Verification, 123, 373–374
Version control, 445–446, 457–458
Versioning, 445
Version management, 446
Viewpoints, multiple, 87–88, 107–108
Virtual environments, 430–434
Virtual keyboard input, 416
Visual design, 237, 277
Voice input, 416–417
Voice recognition, 416–417
Volatility, 118, 468

Walking skeleton, 203
Walkthroughs, 326, 332
Waterfall model, 25–26, 55
Ways of navigating (WoN), 280–282
WebApps (Web applications)
  accessibility of, 259
  aesthetic design of, 277
  agile change management for, 453–458
  architectural design of, 192, 278–280
  component-level design for, 226, 282
  content design of, 277–278
  design pyramid for, 275–282
  graphic design of, 237
  interface design for, 275–276
  metrics for, 472, 473
  navigation design of, 280–282
  patterns for, 291–292
  performance of, 424–426
  quality of, 282–285
  security of, 424
  testing, 418–426
  types of, 7
  usability of, 257–259
Web Services Description Language (WSDL), 273
Weighted device platform matrix (WDPM), 427
Weighted methods per class (WMC), 469
W^{5}HH principle, 501
White-box testing, 383–387, 397
Work breakdown structure (WBS), 526
Work environment analysis, 248–249
Work flow. *see* Process flow
Work in progress (WIP), 49
Work products, 11, 86, 114, 402
Work product size (WPS), 328
Work tasks. *see* Task sets